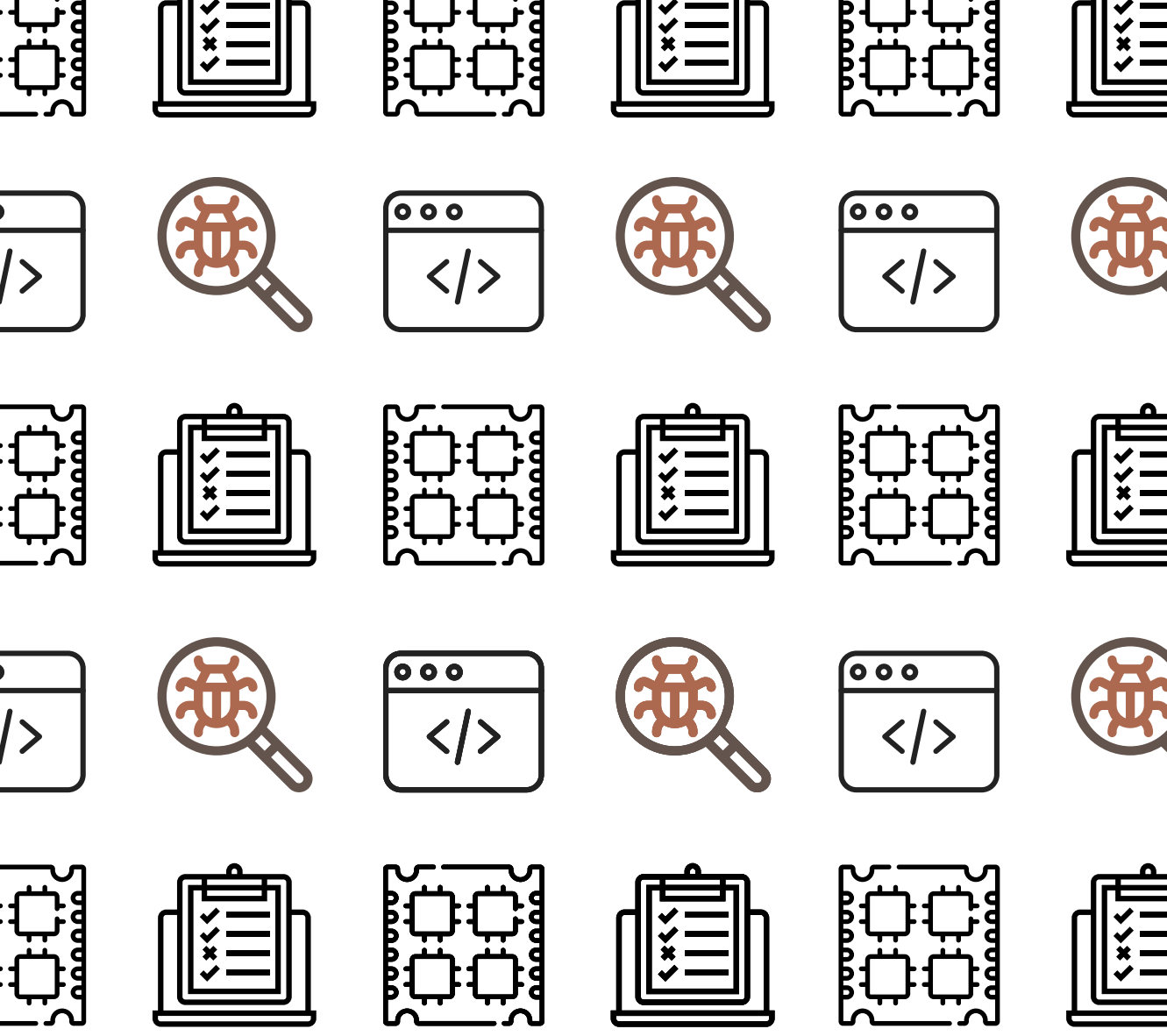




ADVANCED SE233 PROGRAMMING

COURSEBOOK

PASSAKORN PHANNACHITTA

SE233 – ADVANCED PROGRAMMING

Preface

THIS is the sixth edition of the coursebook for the Advanced Programming course (SE233), which encompasses 30 lecture hours and 30 laboratory hours. This course is a core component of the Bachelor of Science program in Software Engineering at the College of Arts, Media and Technology, Chiang Mai University, Thailand.

Building upon the foundation established in the Object-Oriented Programming (OOP) course, this course explores more advanced programming concepts, providing students with a comprehensive toolkit of techniques for effective problem-solving. This coursebook is structured into six chapters, each addressing a fundamental aspect of software development. The first chapter introduces the event-driven programming paradigm using JavaFX, the primary technology employed throughout this coursebook. The second chapter examines exception handling techniques for enhancing the robustness of software applications. The third chapter illustrates multithreaded programming, a technique for improving the utilization of computational resources and for understanding the standard operation of graphical user interfaces (GUIs). The fourth chapter covers error localization and correction through debugging techniques. The fifth chapter provides an introduction to software testing, with an emphasis on unit testing. Finally, the sixth chapter addresses topics related to software build automation and dependency management.

A defining addition to this sixth edition is the continuous evaluation of Artificial Intelligence (AI)-assisted development tools. Each chapter features a dedicated AI advisory section that critically examines the appropriate application and potential risks of AI generation within the corresponding technical domain. The instruc-

tional sequence emphasizes mastering fundamental concepts and understanding failure modes before introducing automated tools. This framework ensures that AI assistance serves to amplify existing technical capabilities rather than substitute for foundational reasoning.

This coursebook is intended to serve as a comprehensive resource for students and other individuals interested in the theoretical and practical application of the topics covered within this course.

Passakorn Phannachitta
June 2026

Contents

Contents	v
List of Figures	xi
List of Tables	xv
Course Syllabus	xvii
1 Event Driven Development	1
1.1 GUI	3
1.1.1 JavaFX	4
1.2 Events	9
1.3 Event Driven Programming	10
1.3.1 Events and GUI	12
More examples	13
1.3.2 Mouse events	18
Drag and drop	18
Dragging and dropping items in a ListView	26
1.3.3 Keyboard events	29
1.4 Case Study	31
1.4.1 Model components	36
1.4.2 View components	41
1.4.3 Controller components and programming logic	44
1.4.4 Flexible-size container	49
	v

1.4.5	Drag-and-drop functionality	51
1.4.6	Attributes propagation	57
1.5	Exercise	61
1.6	Discussion and Summary	62
2	Exception Handling	67
2.1	Categorization of Exceptions	69
2.1.1	Checked exceptions	69
2.1.2	Unchecked exceptions	69
2.2	Exception Handling	70
2.2.1	Catching the exceptions	70
	Try-catch block	70
	The finally block	76
2.2.2	Throwing the exceptions	77
2.2.3	Customized exception	81
2.3	Extra Topics: Lambda Expressions and Streams in Java	84
2.3.1	What is a Lambda expression?	84
2.3.2	Illustrative example: String sorting	86
2.3.3	Lambda syntax	87
2.3.4	Utilizing Lambda expressions	88
2.3.5	Introducing streams	89
	Stream sources	90
	Intermediate stream operations	90
	Stream parallelism	93
	Terminal stream operations	95
	Example: Prime number generation	96
	Example: Estimating area between curves	97
2.3.6	Map and Reduce paradigm	98
	Example: Word count using Map and Reduce	99
2.4	Case Study	101
2.4.1	Currency exchange API	104
	Parameters	104
	Constraints	105
	Supported currencies	105

	Return value	105
	Example query and response	106
2.4.2	Extracting information from the API	107
2.4.3	Adding exception handlers	110
2.4.4	The view components	113
2.4.5	The controller components	116
2.4.6	Event handlers	120
2.4.7	Requirement change handling	122
2.4.8	Background process	130
2.5	Exercise	136
2.6	Discussion and Summary	137
3	Concurrency	143
3.1	Purpose	146
3.1.1	Concurrency and GUI programming	146
3.1.2	Concurrency and accelerated computing	147
	Synchronization	148
3.2	The JavaFX Concurrent Framework	151
3.2.1	Components	151
	The Worker interface	151
	The Task class	152
	The Service and ScheduledService class	152
3.2.2	Example: JavaFX with concurrent threads	153
3.3	Example project: Download manager	155
3.4	Case Study	164
3.4.1	Constructing the GUI with fxml	168
3.4.2	The drop-file functionality	170
3.4.3	Pdf file extraction	173
3.4.4	Map and ListView	182
3.4.5	Multithreaded indexing implementation	184
3.4.6	Incorporating a Loading Indicator	188
3.5	Exercise	190
3.6	Discussion and Summary	190
	References	194

4	Debugging	195
4.1	Debugging Methodologies	197
4.1.1	Probing	197
4.1.2	Logging	199
	Logging frameworks	200
	Logging levels	202
	Loggers	203
	Appender	203
	Layout	204
	Configuration	205
4.1.3	Debugging	206
4.2	Extra Topic: Sprite Animation	209
4.2.1	Drawing	211
4.2.2	Clock	211
4.2.3	Movement	211
4.2.4	Collision handling	212
4.2.5	Interpolation	214
4.3	Case Study	216
4.3.1	Game loop	221
4.3.2	Keys	225
4.3.3	Collision checking	230
4.3.4	Jumping	231
4.3.5	Interpolation	239
4.3.6	Acceleration	242
4.3.7	Logging	244
4.4	Exercise	248
4.5	Discussion and Summary	249
	References	253
5	Unit Testing	255
5.1	Basic Software Testing	257
5.1.1	Importance	257
5.1.2	Sources of bugs	258
5.1.3	Testing methods	258

5.1.4	Testing objectives	261
5.2	Unit Testing	262
5.2.1	Objective of unit testing	263
5.2.2	Example scenario	263
5.2.3	JUnit 5	264
	Assertion methods	265
	JUnit naming conventions	267
	Unit testing tips	269
	JUnit test suites	274
	Mocking	276
	Reflection	278
5.3	Test-Driven Development (TDD)	281
5.3.1	Benefits	283
5.3.2	Red-Green-Refactor: A Worked Example	283
5.4	Case Study (Part 1)	287
5.4.1	Character collision	291
5.4.2	Death scene rendering	293
5.4.3	Scoreboard	294
5.4.4	Unit testing	298
5.4.5	Unit testing with reflection	300
5.5	Case Study (Part 2)	302
5.5.1	The snake's head, body, and tail	303
5.5.2	Food collision and snake growth	306
5.5.3	Implementing snake death condition	310
5.5.4	The game loop	312
5.5.5	The JUnit test suite	318
5.6	Exercise (2 parts)	321
5.6.1	Part 1	321
5.6.2	Part 2	322
5.7	Discussion and Summary	322
	References	326
6	Build Management System	327
6.1	Built Tool and Dependency Management	329

6.1.1	Build tool	329
6.1.2	Dependency management	330
6.2	Java Build Management System	331
6.2.1	Apache Maven	332
	Maven build lifecycle	332
	Maven standard directory layout	333
	Project object model (POM)	334
6.3	Continuous Integration (CI)	337
6.4	Case Study	340
6.4.1	Managing Maven project	342
6.4.2	Packaging with Maven	347
6.5	AI-Assisted Development and GitHub Copilot	350
6.5.1	What is GitHub Copilot?	351
6.5.2	Integration with IntelliJ IDEA	351
6.5.3	Appropriate Use Cases in Build Configuration	352
6.5.4	Limitations and Verification Habits	353
6.5.5	Lessons from This Course	353
6.6	Exercise	354
6.7	Discussion and Summary	355
	References	359
A	Appendix A	361
B	Appendix B	367
	References	388
	Bibliography	389

List of Figures

1.1	A Simple GUI with a Button	3
1.2	JavaFX Embedded System Development Example (Arduino-er, 2015) .	5
1.3	The Rendered GUI of HelloApplication.java	6
1.4	Result of Adding a Button to HelloApplication.java	7
1.5	JavaDoc Specifications for the getChildren() Method and the ObservableList Interface (Oracle, 2026a)	8
1.6	Conceptual Model of a GUI System in Java (Wong, 2017)	12
1.7	Event Handling Process for a Button Click in Java (Wong, 2017) . . .	13
1.8	GUI Output of customizedEvent.java	15
1.9	GUI Output of the Drag-and-Drop Application	22
1.10	Drag-and-Drop Item Interaction between Lists	26
1.11	GUI Output of the Case Study Application	33
1.12	Project Parameter Configuration Example	35
1.13	An Alternative GUI Output of the Case Study Application	48
1.14	Generation of Accessor and Mutator Methods in IntelliJ	56
1.15	GUI Output of the Case Study Application with Weapon and Armor Equipped	57
2.1	IDE Suggestion to Convert an Anonymous Class to a Lambda Expression	89
2.2	Area Estimation Between Two Curves	97
2.3	GUI of the Currency Watcher Case Study Application	104
2.4	Reloading the Maven Configuration	110
2.5	Generation of Accessor and Mutator Methods for Currency Class . . .	113

2.6	GUI of the Currency Watcher Case Study Application After Completing the Refresh Button Functionality	122
2.7	GUI Output of the Completed Currency Watcher Application	135
3.1	A JavaFX Application Demonstrating Concurrency Principles	146
3.2	Graphical Representation of the FxConcurrentExample Application	153
3.3	Comparison of Download Manager GUIs	156
3.4	Inverted Index Creation Application Interface	166
3.5	GUI Structure Tree Generated by SceneBuilder	168
3.6	Summary of GUI Parameters for Configured Components	168
3.7	GUI Layout After Parameter Configuration	170
3.8	Declaration of the Main Application Controller	170
3.9	Application Interface with Added <i>Start Indexing</i> Button	178
4.1	Establishing a Breakpoint in IntelliJ IDEA	206
4.2	Configuring Program Arguments for Debugging in IntelliJ IDEA	207
4.3	Debugger Tool Window in IntelliJ IDEA	208
4.4	Sprite Sheet Example (Armstead, 2022)	214
4.5	Initial Static GUI of the Case Study Application	221
4.6	Rendered GUI after Initiating the GameLoop Thread	225
4.7	Character Movement without Boundary Collision Detection	229
4.8	Character Movement with Jumping Mechanism Implementation	238
4.9	Rendered GUI After Interpolation Implementation	242
5.1	The V-Model of Software Testing (Java T Point 2020)	259
5.2	Execution Results of an Aggregated Test Suite	275
5.3	GUI of the Initialized Project for the Chapter 5 (Part 1) Case Study	289
5.4	The Completed 2D Platformer Game Case Study	297
5.5	Test Results for Chapter5part1 Project	302
5.6	Rendered Snake Game	318
5.7	GUI of the Completed Snake Game Application	321
6.1	Example Query Results from a Maven Dependency Search	337
6.2	Example CI Workflow	339
6.3	Renaming Packages in a Project (Step 1)	343

6.4	Packaging a Project using IntelliJ's Maven Toolset	347
6.5	Error Message Encountered When Executing chapter6-1.0-SNAPSHOT.jar	348
A.1	New Project Configuration in IntelliJ IDEA	363
A.2	Default GUI of the Generated JavaFX Application	364
B.1	Data Processing Architecture Diagram	368
B.2	AWS Lambda Function Creation	371
B.3	Creating an Access Key for S3	372
B.4	Binding Access Keys with Environment Variables	372
B.5	Creating Layers for Lambda Dependencies	376
B.6	Adding Layers to the Lambda Function	377
B.7	Lambda Function General Configuration	377
B.8	Configuring Amazon EventBridge	378
B.9	API Endpoint Architecture Diagram	379
B.10	Creating a New API in AWS API Gateway	382
B.11	Creating a New Resource Path in API Gateway	383
B.12	Linking the API with the Created Lambda Function	384
B.13	Selecting Lambda Proxy Integration in API Gateway	384
B.14	Example Output of an API Test	385
B.15	Example Output in a Web Browser	385

List of Tables

- 2.1 Syntactic Forms of Java Lambda Expressions with Examples 89
- 2.2 Supported Currency Codes and Names 105
- 4.1 Relationship between curIndex, curColumnIndex, and curRowIndex
for the Sprite Sheet in Figure 4.4. 215
- B.1 Comparison of Storage Options for AWS Lambda 387

Course Syllabus

University	Chiang Mai University
Program	Bachelor of Science (Software Engineering)
Course	SE 233 (953233) Advanced Programming 3(2-2-5) credits
Prerequisites	SE 231 (953231) Object-Oriented Programming
Course Description	Event Driven Development. Exceptions handling. Multi-Thread Programming. Debugging. Unit test. Build management tools.

- Course Objective**
- Students will be able to develop the software which contains the exception handlers, with multiple threading and event driven.
 - Students will be able to test the software with unit test and debugging error.
 - Students will be able to manage the software using the build management tools.

Course Contents	Topics	Lecture (Hrs)	Lab (Hrs)
	Event driven development	6	6
	Exceptions handling	4	4
	Multi-thread programming	6	6
	Debugging	6	6
	Unit test	6	6
	Build management tools	2	2
	Total	30	30

- Course Work**
- Lectures
 - Hand-ons
 - Assignments
 - Projects
 - Paper-based examinations

Grading Criteria	10%	Lab tutorial
	10%	Lab exercise
	30%	Two term projects
	50%	Paper-based examinations

Course Policies

- The evaluation is based on **criteria**. The letter grades will be A, B+, B, C+, C, D+, D, or F.
- The following letter grades may also be given
 - **I** for incomplete
 - **P** for in progress
 - **W** for withdrawn
- The lecturers have the right to adjust the grading system and course content depends on how the students perform.
- Use of cell phone or laptop in the classroom is permitted only for taking note and working on the course-related activities.

Cheating Policies

Academic misconduct, a.k.a., cheating, is considered as a severe offence in any classes. It is mainly due to that; it devalues the effort of honest students. Therefore, all suspected cases will be aggressively pursued, and the confirmed cases will be punished to the full extent permitted by the university.

Policy for cheating in the courses:

- First offence will result in zero scores for the particular cheated assignment or exam.
- Multiple instances of cheating of any kind will result in receiving an F for the course.

CHAPTER 1

Event Driven Development

Prerequisites

- A solid understanding of Java object-oriented programming (OOP): classes, inheritance, interfaces, and polymorphism.
- A working development environment with a current Java Development Kit (JDK) and IntelliJ IDEA (or an equivalent Integrated Development Environment).
- The JavaFX Software Development Kit (SDK) installed and configured as described in the Appendix.
- Familiarity with basic Java project structure, including packages, source files, and the compilation process.

Expected Outcomes

- Construct a simple JavaFX application with a Stage, a Scene, and a node-based scene graph.
- Implement event handlers for button clicks, mouse interactions, and keyboard input.
- Apply the Model-View-Controller (MVC) pattern to structure a GUI application.
- Implement drag-and-drop interactions between visual components using mouse event sequences.
- Use state variables and flags to manage application behaviour across multiple events.

WITH the widespread adoption of graphical user interfaces (GUIs) in modern software, programming languages are equipped with comprehensive libraries and toolsets to facilitate GUI development. A fundamental principle of GUI development is the prompt response to user-initiated signals. This responsiveness is enabled by functionality that continuously monitors for such events, records relevant information upon their occurrence, and invokes corresponding user-defined procedures. Fundamentally, a GUI application is required to perform three primary functions: accept user input, process this input, and deliver the resulting output in accordance with the specified software requirements. A typical GUI application incorporates control elements, such as buttons, which await user interaction (e.g., a click). Upon the activation of a control element, the application executes a predefined process and presents the output to the user. This output is commonly presented through various visual components, including text or dialogue boxes.

The programming paradigm characterized by the specification of input types and their corresponding handling procedures is formally known as Event-Driven Development.

1.1 GUI

Prior to examining the specifics of GUI development in Java, it is necessary to establish the fundamental role of OOP in GUI design. The significance of OOP can be illustrated by considering the task of constructing the GUI depicted in Figure 1.1, including the implementation of its visual elements. This process requires rendering the message box, two text labels, a text field, and an *Ok* button.

For the button component alone, a rounded rectangle must be drawn, assigned a specific color (e.g., blue), positioned with defined coordinates within the message box, and inscribed with text (e.g., *Ok* in white). In addition to its graphical representation, the implementation of the button's behavior is a critical task. This involves defining methods that dictate the application's response to user interactions, such as clicks, and may include modern visual effects for hovering and clicking. The inherent complexity of modern GUI components makes the manual creation of each button from procedural code an impractical approach in software development.

Without the application of OOP principles, the construction of visual elements becomes procedurally complex, particularly given the large number of components in a typical application. For example, the message box in Figure 1.1 could include a *Cancel* button in addition to the *Ok* button. In a non-OOP context, creating the second button would require a complete replication of the implementation steps, from rendering the shape to defining its interactive behavior. The core OOP principle of reusability addresses this inefficiency. It allows for the creation of a single *Button* object for the *Ok* button, which can inherit behaviors and attributes from pre-existing classes. The button's shape, for instance, could be inherited from a *Shape* class that already contains logic for coordinate-based placement. Consequently, adding a *Cancel* button becomes a matter of instantiating another *Button* object, promoting code reuse within the same application and across future projects. Therefore, OOP is integral to modern GUI development. A comprehen-



Figure 1.1: A Simple GUI with a Button

sive understanding of its concepts is a prerequisite for the topics covered in the remainder of this coursebook.

1.1.1 JavaFX

Historically, GUI development in Java was accomplished using the `java.awt` (Oracle, 2026b) and `javax.swing` (Oracle, 2026c) packages, commonly known as AWT and Swing. A subsequent shift has occurred toward JavaFX (OpenJFX, 2026a), a modern framework that is now the predominant approach for developing user interfaces in Java. Positioned as the successor to AWT and Swing, JavaFX is described on its official website as a collaborative project among various contributors to produce a modern, efficient, and full-featured toolkit for developing rich client applications. Accordingly, this coursebook utilizes JavaFX as the primary framework for examining GUI development and the event-driven programming paradigm. Installation and configuration details for the required software are provided in the Appendix.

Several key features contribute to the adoption of JavaFX for client application development:

- Full compatibility with all Java APIs, which allows developers to access an extensive ecosystem of existing libraries.
- Support for cross-platform development, enabling applications to run on various operating systems without modification.
- Accessibility for new developers through visual design tools that support drag-and-drop component building.
- Versatility in application deployment, as it can be used to implement solutions for desktop, mobile, web, and embedded systems. Figure 1.2 provides an example of a JavaFX application in an embedded systems context.

A GUI in JavaFX is constructed using a scene graph, which is a hierarchical tree structure of visual elements referred to as nodes. In a manner analogous to the organization of HTML elements within a Document Object Model (DOM) tree, JavaFX nodes are arranged in parent-child relationships. These nodes encompass a diverse range of types, such as buttons, labels, text fields, shapes, and media elements for audio and video. Collectively, these nodes are equipped to handle various user input events, including mouse movements, hovers, and drag-and-drop operations. This framework provides a comprehensive set of visual elements for the development of client applications.

A JavaFX application's user interface is defined within a hierarchy of containers, primarily the Stage and the Scene. The Stage represents the main window or top-level container, while the Scene acts as the container for all visual content. In the provided example, Script 1.1 instantiates a Stage and a Scene with dimensions of 300x275 pixels. The content of the Scene is structured as a hierarchical node graph, with a StackPane functioning as the root node. The resulting GUI is displayed in Figure 1.3.

It is important to note the convention used for line numbers in Script 1.1 and subsequent code listings. The plus (+) and minus (–) symbols indicate line-level differences between file versions. A '+' denotes an added line, and a '-' denotes a deleted line. As this is the initial presentation of `HelloApplication.java`, all lines are marked with '+'. This convention is maintained throughout the coursebook.

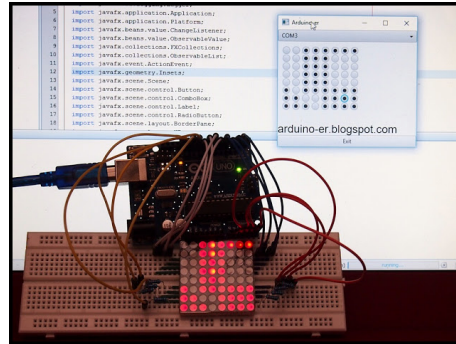
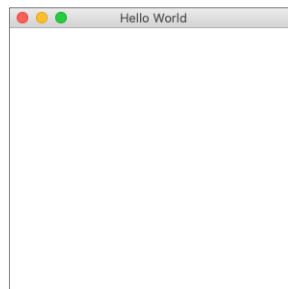


Figure 1.2: JavaFX Embedded System Development Example (Arduino-er, 2015)

Script 1.1: Hello World Example in JavaFX**HelloApplication.java**

```
+ 1 import javafx.application.Application;
+ 2 import javafx.scene.Scene;
+ 3 import javafx.scene.layout.StackPane;
+ 4 import javafx.stage.Stage;
+ 5
+ 6 public class HelloApplication extends Application {
+ 7     @Override
+ 8     public void start(Stage primaryStage) {
+ 9         StackPane root = new StackPane();
+10         primaryStage.setTitle("Hello World");
+11         primaryStage.setScene(new Scene(root, 300, 275));
+12         primaryStage.show();
+13     }
+14     public static void main(String[] args) { launch(args); }
+15 }
```

**Figure 1.3: The Rendered GUI of HelloApplication.java**

Script 1.2 demonstrates the modification of `HelloApplication.java` to include a button element. The implementation follows a general procedure that is applicable to the incorporation of any JavaFX visual element. The first step requires the visual element, in this case, a button, to be instantiated as a JavaFX node. Subsequently, this newly created node is added to an existing parent node within the application's scene graph, a process analogous to appending an HTML node to a

parent in the DOM tree.

As shown in the script, the button is instantiated in Line 5, and its text label is assigned in Line 6. Finally, in Line 8, the button is added to a StackPane element, ensuring its presence in the Application Scene graph and its subsequent display upon application launch. After compilation and execution, the button will be rendered centrally within the application window, as depicted in Figure 1.4.

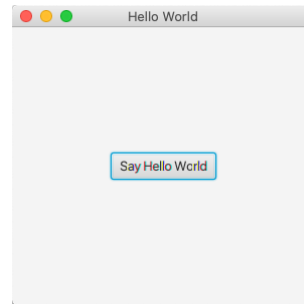


Figure 1.4: Result of Adding a Button to HelloApplication.java

Script 1.2: Adding a Button to HelloApplication.java

```
1 //Imports are omitted
2 public class HelloApplication extends Application {
3     @Override
4     public void start(Stage primaryStage) {
+ 5         Button btn = new Button();
+ 6         btn.setText("Say Hello World");
7         StackPane root = new StackPane();
+ 8         root.getChildren().add(btn);
9         primaryStage.setTitle("Hello World");
10        primaryStage.setScene(new Scene(root, 300, 275));
11        primaryStage.show();
12    }
13    public static void main(String[] args) { launch(args); }
14 }
```

Appending a JavaFX node to a parent node is accomplished using the `getChildren().add()` method. This structure implies that the `getChildren()` method returns a list-like object that facilitates the addition of new elements. To confirm this, the JavaDoc documentation for the `getChildren()` method is consulted (Oracle, 2026a). The left-hand side of Figure 1.5 shows that the return type of this method is `ObservableList <Node>`, which corroborates the initial assumption.

A further examination of the `ObservableList <Node>` type, depicted on the right-hand side of Figure 1.5, shows that it is an interface that extends other widely recognized interfaces from the Java Collections Framework, including `List<E>`, `Collection<E>`, and `Iterable<E>`. It also extends the `Observable` interface. This extension from standard collection interfaces allows common Java methods for list operations, such as `add`, `contains`, `forEach`, and `toArray`, to be employed directly on JavaFX element collections.

In contrast, the `Observable` interface may be less familiar. Its methods, `addListener` and `removeListener`, are integral to the event-handling mechanism in JavaFX. They provide the functionality for an object to register listeners that can respond to user-initiated signals. However, before examining the specifics of listeners, it is beneficial to first establish a foundational understanding of events (Oracle, 2026a).

getChildren <pre>protected ObservableList<Node> getChildren() Description copied from class: Parent Gets the list of children of this Parent. See the class documentation for Node for scene graph structure restrictions on setting a Parent's children list. If these restrictions are violated by a change to the list of children, the change is ignored and the previous value of the children list is restored. An IllegalArgumentException is thrown in this case. If this Parent node is attached to a Scene attached to a Window that is showing (Window.isShowing()), then its list of children must only be modified on the JavaFX Application Thread. An IllegalStateException is thrown if this restriction is violated. Note to subclasses: if you override this method, you must return from your implementation the result of calling this super method. The actual list instance returned from any getChildren() implementation must be the list owned and managed by this Parent. The only typical purpose for overriding this method is to promote the method to be public. Overrides: getChildren in class Parent Returns: the list of children of this Parent.</pre>	<pre>Module javafx.base Package javafx.collections Interface ObservableList<E> Type Parameters: E - the list element type All Superinterfaces: Collection<E>, Iterable<E>, List<E>, Observable All Known Subinterfaces: ObservableListValue<E>, WritableListValue<E> All Known Implementing Classes: FilteredList, ListBinding, ListExpression, ListProperty, ListPropertyBase, ModifiableObservableListBase, ObservableListBase, ReadOnlyListProperty, ReadOnlyListPropertyBase, ReadOnlyListWrapper, SimpleListProperty, SortedList, TransformationList</pre>
---	--

Figure 1.5: JavaDoc Specifications for the `getChildren()` Method and the `ObservableList` Interface (Oracle, 2026a)

1.2 Events

Events are broadly categorized into two types: foreground and background. This classification is primarily distinguished by the nature of user interaction. Foreground events are initiated by direct user actions, such as button clicks, mouse movements, or drag-and-drop operations. Conversely, background events are not triggered by users but by other software or hardware processes. Examples of background events include operating system interrupts, hardware or software failures, timer expirations, and the completion of operations.

In JavaFX, the event model comprises three fundamental components: an event source, an event listener, and an event handler. An event source refers to a GUI component that serves as the origin for user-initiated events. An event listener is an object that monitors for specific events from a source. The event handler is the method that contains the logic to process the event. In operation, an event is generated at the source, which is then detected by the registered listener. The listener, in turn, invokes the appropriate event handler to process the signal and execute the required application logic. Programmatically, enabling this interaction requires registering a listener with a source and implementing the logic within the corresponding handler method.

Script 1.3 demonstrates the addition of an event handler to the `HelloApplication.java` class. In this modified example, a click event on the button triggers the display of the text *Hello World!* in the system's terminal. On Line 7 of the script, an event listener is registered with the button instance. The use of the `@Override` annotation in this context illustrates a practical application of polymorphism, a core principle of OOP.

A noteworthy aspect of this implementation is the creation of the event handler directly within the `setOnAction` method, without a separate, formal instance declaration. This technique demonstrates the use of an anonymous class, which implements the `EventHandler` interface and overrides its `handle()` method to define the logic that outputs *Hello World!* to the terminal.

Script 1.3: Implementation of an Event Handler in HelloApplication.java

```
1 //Imports are omitted
2 public class HelloApplication extends Application {
3     @Override
4     public void start(Stage primaryStage) {
5         Button btn = new Button();
6         btn.setText("Say Hello World");
+ 7         btn.setOnAction(new EventHandler<ActionEvent>() {
+ 8             @Override
+ 9                 public void handle(ActionEvent event) {
+ 10                     System.out.println("Hello World!");
+ 11                 }
+ 12             });
13         StackPane root = new StackPane();
14         root.getChildren().add(btn);
15         primaryStage.setTitle("Hello World");
16         primaryStage.setScene(new Scene(root, 300, 275));
17         primaryStage.show();
18     }
19     public static void main(String[] args) { launch(args); }
20 }
```

In practical applications, multiple events often occur concurrently, requiring the application to provide a distinct response for each event. The example presented in Script 1.3, by contrast, is limited to a single event source, listener, and handler in order to illustrate the fundamental principles of event handling in JavaFX. However, even in this simplified context, a single event can trigger multiple actions. For instance, a click on a form-submission button might not only transmit data to a server but also display a confirmation message to the user indicating a successful submission.

1.3 Event Driven Programming

Event-driven programming can be defined as a paradigm for developing programs composed of multiple subprograms, where each subprogram is designed to execute in response to a specific event or signal. In the context of a video game, for example,

the action of moving a character forward constitutes one such subprogram. This subprogram remains inactive until the user provides a corresponding signal, such as a specific key press. The activation of a different signal, such as another key press, would invoke a different subprogram with a distinct outcome, such as initiating an attack.

The event-driven programming paradigm represents a foundational approach in numerous modern technologies, including web applications, the Internet of Things (IoT), and various AI-powered automated systems. Within these technological domains, numerous tasks can be conceptualized as subprograms initiated by specific signals. For instance, smart irrigation systems employed in modern agriculture utilize sensors for the continuous monitoring of environmental conditions. The data generated by these sensors function as event triggers, invoking subprograms that, based on their underlying algorithms, determine and execute appropriate actions such as the adjustment of watering schedules. An additional illustrative example is the adaptive cruise control system in an automobile. This system operates in response to signals from the surrounding environment, which in turn invoke subprograms responsible for modulating vehicle speed and maintaining safe following distances. In both of these examples, the event-driven paradigm provides the framework for the system to make informed decisions and execute appropriate actions in response to received signals. This capability facilitates efficient and responsive interactions between the system and its environment.

In summary, event-driven programming is a paradigm that allows for the design of programs structured around signals or events. These signals, which can originate from users or other system components, trigger the execution of specific subprograms, often referred to as event handlers. This approach is particularly effective for applications where distinct user actions or external events require distinct programmatic responses. By structuring programs in this manner, developers can create more flexible, interactive, and responsive software.

1.3.1 Events and GUI

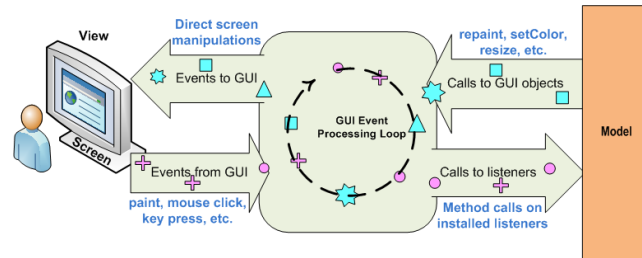


Figure 1.6: Conceptual Model of a GUI System in Java (Wong, 2017)

Figure 1.6 illustrates the conceptual model of a GUI system in Java (Wong, 2017). The core mechanism of this system is the GUI event processing loop, as depicted in the center of the figure. This loop operates as an active background process, functioning similarly to an infinite loop dedicated to dispatching events. User actions, such as a mouse click on a button, are encapsulated as event objects and added to an event queue. A listener object, which has been previously registered with the event's source component, is then notified. This listener dequeues the event and invokes the associated handler method to perform the required processing. The execution of the handler may, in turn, modify the application's state, resulting in updates to the GUI.

In the context of Script 1.3, the event is initiated by a user's mouse click on the button. This action is encapsulated as an `ActionEvent` object and placed in the event queue. The event loop subsequently dequeues the event and dispatches it to the appropriate listener. The system identifies the correct handler for the button click based on this pre-registered listener, which in this example is an instance that implements the `javafx.event.EventHandler<ActionEvent>` interface. It should be noted that multiple listeners can be registered for a single event source or type of action. Figure 1.7 provides a graphical representation of this event-handling process.

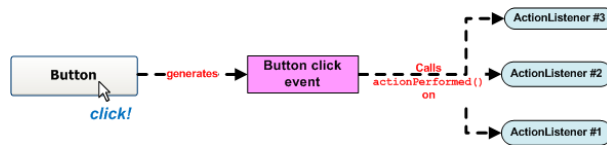


Figure 1.7: Event Handling Process for a Button Click in Java (Wong, 2017)

More examples

Script 1.4 demonstrates the addition of two further event listeners to the `HelloApplication.java` file. These listeners monitor the application's Scene and are triggered in response to changes in the window's dimensions. To facilitate the registration of these listeners, the Scene object is first instantiated and assigned to a local variable (Lines 17-18). Consequently, this scene object serves as the event source for the two listeners added in this example.

The first listener, defined in Lines 18-23, monitors the `widthProperty` of the application scene. Its overridden method functions as the event handler and is invoked when the scene's width value changes. Upon being triggered, this handler prints the new width of the scene to the terminal. The second listener, located in Lines 24-29, is functionally analogous, but monitors the scene's `heightProperty`.

This example highlights a recurring pattern for registering JavaFX event listeners. As illustrated by the code, this pattern can be generalized as: `[source element].[property to be observed].addListener(new listener interface [overridden handler method] );`. This structure cohesively defines the event source, the specific property being monitored, the listener implementation, and the handler logic within the required overridden method.

Script 1.5 provides a concrete example of OOP design, where event listeners are registered with an event source as part of a reusable component's implementation. The objective of this script is to create a `Circle` object accompanied by a `Label`. This label displays the mouse coordinates when the pointer is within the circle's bounds; otherwise, it indicates that the pointer is outside the circle.

Script 1.4: Registration of Window-Resizing Event Listeners in HelloWorld.java

```
1 //Imports are omitted
2 public class Launcher extends Application {
3     @Override
4     public void start(Stage primaryStage) throws Exception{
5         Button btn = new Button();
6         btn.setText("Say Hello World");
7         btn.setOnAction(new EventHandler<ActionEvent>() {
8             @Override
9             public void handle(ActionEvent event) {
10                 System.out.println("Hello World!");
11             }
12         });
13         StackPane root = new StackPane();
14         root.getChildren().add(btn);
15         primaryStage.setTitle("Hello World");
+ 16         primaryStage.setScene(new Scene(root, 300, 275));
+ 17         Scene scene = new Scene(root, 300, 275);
+ 18         scene.widthProperty().addListener(new ChangeListener<Number>() {
+ 19             @Override
+ 20             public void changed(ObservableValue<? extends Number> observableValue,
+                               Number oldSceneWidth, Number newSceneWidth) {
+ 21                 System.out.println("Width: " + newSceneWidth);
+ 22             }
+ 23         });
+ 24         scene.heightProperty().addListener(new ChangeListener<Number>() {
+ 25             @Override
+ 26             public void changed(ObservableValue<? extends Number> observableValue,
+                               Number oldSceneHeight, Number newSceneHeight) {
+ 27                 System.out.println("Height: " + newSceneHeight);
+ 28             }
+ 29         });
+ 30         primaryStage.setScene(scene);
31         primaryStage.show();
32     }
33     public static void main(String[] args) {
34         launch(args);
35     }
36 }
```

A non-event-driven approach would necessitate the continuous monitoring of the mouse pointer's position relative to the circle. The event-driven paradigm, however, simplifies this logic by executing code only in response to a relevant state change. This change-oriented design prompts a conceptual shift toward managing program state. State, in this context, refers to the data or information that is modified during program execution. For this example, the states correspond to the mouse pointer entering and exiting the circle's bounds. Event handlers are consequently implemented to manage these state transitions, while no handling is required as long as the pointer remains outside the circle.

An examination of Script 1.5 shows that the component is created by the `createMonitoredCircle()` method (Lines 15-35). The listeners for the circle's mouse events are registered using the `setOnMouseMoved()` and `setOnMouseExited()` methods. Specifically, the handler registered with `setOnMouseMoved()` is invoked repeatedly while the mouse pointer is within the circle's boundaries, whereas the handler for `setOnMouseExited()` is invoked at the moment the pointer leaves these boundaries.

Given this design, the reuse of the component created by `createMonitoredCircle()`, as illustrated in Script 1.6, implies that its associated label should be included. Therefore, encapsulating these related elements within a single, new object represents a practical design approach. In this implementation, the visual elements are organized within a `VBox` layout container, which is subsequently placed inside a `StackPane` container. Consequently, the return type of the `createMonitoredCircle()` method is a `StackPane` node, which acts as the root of this composite component. To facilitate the reuse of this composite component, it can be added as a child to any parent node at a higher level within the application's scene graph. In this particular example, the component is added to a JavaFX `FlowPane` layout container, as shown in Line 7 of the script. Upon compilation and execution, the application will display the result illustrated in Figure 1.8.

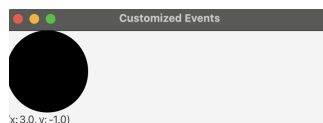


Figure 1.8: GUI Output of `customizedEvent.java`

Script 1.5: Implementation of a Reusable Component with Custom Event Listeners CustomizedEvent.java

```

+ 1 //Imports are omitted
+ 2 public class CustomizedEvent extends Application {
+ 3     @Override
+ 4     public void start(Stage stage) {
+ 5         FlowPane pane = new FlowPane();
+ 6         StackPane circle1 = createMonitoredCircle();
+ 7         pane.getChildren().addAll(circle1);
+ 8         Scene scene = new Scene(pane);
+ 9         stage.setTitle("Customized Events");
+10         stage.setScene(scene);
+11         stage.show();
+12     }
+13     private StackPane createMonitoredCircle() {
+14         StackPane pane = new StackPane();
+15         VBox layout = new VBox();
+16         Label reporter = new Label("Outside Circle");
+17         Circle circle = new Circle(50);
+18         circle.setOnMouseMoved(new EventHandler<MouseEvent>() {
+19             @Override
+20             public void handle(MouseEvent event) {
+21                 reporter.setText("(x: " + event.getX() + ", y: " + event.getY() + ")");
+22             }
+23         });
+24         circle.setOnMouseExited(new EventHandler<MouseEvent>() {
+25             @Override
+26             public void handle(MouseEvent event) {
+27                 reporter.setText("Outside Circle");
+28             }
+29         });
+30         layout.getChildren().setAll(circle, reporter);
+31         pane.getChildren().add(layout);
+32         return pane;
+33     }
+34     public static void main(String[] args) { launch(args); }
+35 }

```

**Script 1.6: Demonstration of Reusing the Composite Component
CustomizedEvent.java**

```
1 //Imports are omitted
2 public class CustomizedEvent extends Application {
3     @Override
4     public void start(Stage stage) {
5         FlowPane pane = new FlowPane();
6         StackPane circle1 = createMonitoredCircle();
+ 7         StackPane circle2 = createMonitoredCircle();
- 8         pane.getChildren().addAll(circle1);
+ 9         pane.getChildren().addAll(circle1, circle2);
10         Scene scene = new Scene(pane);
11         stage.setTitle("Customized Events");
12         stage.setScene(scene);
13         stage.show();
14     }
15     private StackPane createMonitoredCircle() {
16         StackPane pane = new StackPane();
17         VBox layout = new VBox();
18         Label reporter = new Label("Outside Circle");
19         Circle circle = new Circle(30);
20         circle.setOnMouseMoved(new EventHandler<MouseEvent>() {
21             @Override
22             public void handle(MouseEvent event) {
23                 reporter.setText("(x: " + event.getX() + ", y: " + event.getY() + ")");
24             }
25         });
26         circle.setOnMouseExited(new EventHandler<MouseEvent>() {
27             @Override
28             public void handle(MouseEvent event) {
29                 reporter.setText("Outside Circle");
30             }
31         });
32         layout.getChildren().setAll(circle, reporter);
33         pane.getChildren().add(layout);
34         return pane;
35     }
36     public static void main(String[] args) { launch(args); }
37 }
```

1.3.2 Mouse events

Script 1.6 demonstrates the reuse of the composite component described previously (Lines 7-9). This example utilizes listeners for mouse movement and exit events; however, the JavaFX framework provides a more extensive set of mouse event types. According to the official JavaDoc (OpenJFX, 2026b), the mouse events available in the current version of JavaFX include the following:

- `setOnMouseEntered`
- `setOnMouseExited`
- `setOnMouseMoved`
- `setOnMousePressed`
- `setOnMouseReleased`
- `setOnMouseDragged`
- `setOnMouseDragEntered`
- `setOnMouseDragExited`
- `setOnMouseDragOver`
- `setOnMouseDragReleased`

Drag and drop

An effective method for examining multiple mouse events concurrently is through the implementation of drag-and-drop functionality. The design of such functionality requires several key considerations:

Initiation: The user actions that trigger the start of a drag operation.

Draggable Objects: The types of GUI components that are designated as draggable.

Target Detection: The mechanisms for detecting when the mouse pointer is positioned over a valid drop target.

Visual Feedback: The visual changes to a component during the drag-and-drop process.

Component Movement: The logic governing the movement of the dragged component, including any constraints.

Termination: The user actions that signal the end of a drag operation (i.e., the drop).

Coordinate Management: The role of mouse and component coordinates in the implementation.

A thorough analysis of these considerations provides a comprehensive understanding of how mouse events are applied in complex user interactions. This section details the implementation of a specific drag-and-drop task, illustrated in Figure 1.9, which is defined by the following requirements:

1. Two circles are present: a blue source circle and a red target circle.
2. A mouse click on the source circle brings it to the front of the z-order and initiates its movement in synchronization with the mouse pointer.
3. If the mouse button is released while the source circle is over the target circle, the source circle snaps to the position of the target circle.
4. If the mouse button is released while the source circle is not over the target circle, it returns to its original coordinates.

The implementation of these requirements, while seemingly straightforward, involves several design considerations. Given that both the source and target circles share common characteristics (e.g., associated labels), an advisable design approach is to create a dedicated class for their instantiation. This promotes reusability by the main application class or other components. Script 1.7 demonstrates this approach through the implementation of the `FigureGroup` class.

Script 1.7: FigureGroup Class for Drag-and-Drop

FigureGroup.java

```
+ 1 //Imports are omitted
+ 2 public class FigureGroup {
+ 3     private StackPane figure;
+ 4     public FigureGroup(int x, int y, int radius, Color color, String str) {
+ 5         Circle circle = new Circle(radius, color);
+ 6         Text text = new Text(str);
+ 7         this.figure = new StackPane( circle, text );
+ 8         this.figure.setLayoutX(x);
+ 9         this.figure.setLayoutY(y);
+10     }
+11     public StackPane getFigure() { return figure; }
+12 }
```

Script 1.8: Launcher for a Drag-and-Drop Task

DragAndDrop.java

```
+ 1 //Imports are omitted
+ 2 public class DragAndDrop extends Application {
+ 3     @Override
+ 4     public void start(Stage stage) {
+ 5         stage.setTitle("Hello Drag and Drop");
+ 6         Group root = new Group();
+ 7         Scene scene = new Scene(root, 500, 200);
+ 8         scene.setFill(Color.WHITE);
+ 9         FigureGroup source = new FigureGroup(80,50,40,Color.LIGHTBLUE,"SOURCE");
+10         FigureGroup target = new FigureGroup(320,50,50,Color.RED,"DROP TARGET");
+11         root.getChildren().addAll(source.getFigure(),target.getFigure());
+12         stage.setScene(scene);
+13         stage.show();
+14     }
+15     public static void main(String[] args) {
+16         Application.launch(args);
+17     }
+18 }
```

Script 1.8 demonstrates the utilization of the `FigureGroup` object within the main application class. Lines 9 and 10 show the instantiation of two `FigureGroup` objects, which produces two circles, each with an associated label. Executing the script at this stage of the implementation displays the initial GUI depicted in Figure 1.9.

The subsequent implementation step is to define the drag-and-drop behavior. To facilitate the movement of the circle in synchronization with the mouse pointer, the component's coordinates are required for positional calculations. Furthermore, to satisfy requirement #4, which requires that releasing the mouse button outside the target circle must return the source circle to its original position, it is necessary to record the component's starting coordinates. This necessitates storing both the initial mouse coordinates and the component's origin coordinates prior to any movement. Accordingly, the implementation for the source circle requires the registration of listeners for the following events: `setOnMousePressed`, `setOnDragDetected`, `setOnMouseDragged`, and `setOnMouseReleased`.

The necessary modifications to the `FigureGroup` and `DragAndDrop` classes are presented in Scripts 1.9 and 1.10, respectively. Within the `FigureGroup` class, the modifications are primarily concerned with implementing the functionality to return the component to its origin. This involves a variable declaration on Line 4 and the implementation of methods for setting and resetting the component's location, as shown in Lines 12-19.

Furthermore, four event listeners are registered with the source circle in the updated `DragAndDrop` class (Script 1.10). The listener registered via the `setOnMousePressed()` method (Lines 13-19) is invoked when a mouse press occurs on the event source. Its handler captures and stores the initial coordinates in the `mousex` and `mousey` variables. The `setOnDragDetected()` listener is triggered when a drag gesture is initiated. Its handler brings the source component to the front of the z-order, ensuring that subsequent drag events are directed to this component until the gesture concludes. The `setOnMouseDragged()` listener's handler continuously updates the source component's position to match the mouse pointer's coordinates during the drag operation. Finally, the handler for the `setOnMouseReleased()` listener implements the logic to return the source circle to its original coordinates upon the release of the mouse button. Consequently, this implementation initially causes the source circle to revert to its origin regardless of the drop location.

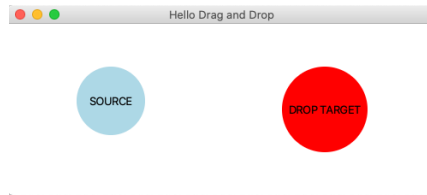


Figure 1.9: GUI Output of the Drag-and-Drop Application

Script 1.9: Modification of FigureGroup.java to Manage Origin Coordinates
FigureGroup.java

```

1  //Imports are omitted
2  public class FigureGroup {
3      private StackPane figure;
+ 4      private double origin_x, origin_y;
5      public FigureGroup(int x, int y, int radius, Color color, String str) {
6          Circle circle = new Circle(radius, color);
7          Text text = new Text(str);
8          this.figure = new StackPane( circle, text );
9          this.figure.setLayoutX(x);
10         this.figure.setLayoutY(y);
+ 11         this.origin_x = x;
+ 12         this.origin_y = y;
13     }
+ 14     public void setBackOrigin() {
+ 15         this.figure.setLayoutX(origin_x);
+ 16         this.figure.setLayoutY(origin_y);
+ 17     }
+ 18     public void setPosition(double x, double y) {
+ 19         this.figure.setLayoutX(this.figure.getLayoutX()+x);
+ 20         this.figure.setLayoutY(this.figure.getLayoutY()+y);
+ 21     }
22     public StackPane getFigure() { return figure; }
23 }

```

Script 1.10: Implementation of Event Listeners for the Source Component DragAndDrop.java

```

1  //Imports are omitted
2  public class DragAndDrop extends Application {
+ 3      double mouse_x, mouse_y;
      ...
11     FigureGroup source = new FigureGroup(80,50,40,Color.LIGHTBLUE,"SOURCE");
12     FigureGroup target = new FigureGroup(320,50,50,Color.RED,"DROP TARGET");
+ 13     source.getFigure().setOnMousePressed(new EventHandler<MouseEvent>() {
+ 14         @Override
+ 15         public void handle(MouseEvent event) {
+ 16             mouse_x = event.getScreenX();
+ 17             mouse_y = event.getScreenY();
+ 18         }
+ 19     });
+ 20     source.getFigure().setOnDragDetected(new EventHandler<MouseEvent>() {
+ 21         @Override
+ 22         public void handle(MouseEvent event) {
+ 23             source.getFigure().setMouseTransparent(true);
+ 24             source.getFigure().toFront();
+ 25             source.getFigure().startFullDrag();
+ 26         }
+ 27     });
+ 28     source.getFigure().setOnMouseDragged(new EventHandler<MouseEvent>() {
+ 29         @Override
+ 30         public void handle(MouseEvent event) {
+ 31             double deltaX = event.getScreenX() - mouse_x;
+ 32             double deltaY = event.getScreenY() - mouse_y;
+ 33             source.setPosition(deltaX, deltaY);
+ 34             mouse_x = event.getScreenX();
+ 35             mouse_y = event.getScreenY();
+ 36         }
+ 37     });
+ 38     source.getFigure().setOnMouseReleased(new EventHandler<MouseEvent>() {
+ 39         @Override
+ 40         public void handle(MouseEvent event) {
+ 41             source.setBackOrigin();
+ 42             source.getFigure().setMouseTransparent(false);
+ 43         }
+ 44     });
45     root.getChildren().addAll(source.getFigure(),target.getFigure());
46     stage.setScene(scene);
47     stage.show();
48 }
49 public static void main(String[] args) {
50     Application.launch(args);
51 }
52 }
```

The final implementation step is to satisfy requirement #3. Because the required response to a mouse release action is conditional upon the pointer's location, a flag variable is a suitable implementation tool. A flag, typically implemented as a boolean variable, is a logical construct analogous to an electronic switch that controls a binary state (e.g., on/off). In programming contexts, a flag variable is used to track the state of the system, indicating whether a specific condition is met or a particular event has occurred.

The use of flag variables is a common technique in introductory programming for solving problems such as primality testing. For instance, to determine if an integer n is prime, an algorithm can test for divisibility by numbers from 2 up to the square root of n . A boolean flag, initialized to false, can be used to track whether a divisor has been found. If any integer in the range divides n evenly, the flag is set to true. Upon the loop's completion, the final state of the flag indicates whether n is a prime number.

This same concept is frequently employed in GUI programming for tasks such as managing the state of toggle buttons, menus, or conditionally rendered components. In the context of the current drag-and-drop example, a flag variable is likewise necessary to implement the conditional logic of requirement #3. While a mouse release event signifies the termination of the drag operation, the programmatic response is dependent on the location of the release. A release outside the target object must cause the source object to return to its origin, whereas a release over the target must cause the source to snap to the target's position.

Script 1.11 further modifies the implementation by introducing a flag variable and two additional event listeners. Initially, a boolean flag variable named `isEntered` is declared (Line 4) and initialized to false (Line 13). The two new listeners, defined in Lines 54-65, are responsible for modifying the flag's state. Handlers registered via the `setOnMouseDragEntered()` and `setOnMouseDragExited()` methods, which are bound to the target circle, toggle the `isEntered` variable accordingly. The value of this flag is then utilized within the modified `setOnMouseReleased()` handler of the source circle. As shown in Lines 46-51, when the mouse is released, the handler logic examines the state of this flag to determine if the release occurred over the target circle, and executes the appropriate action.

Script 1.11: Implementation of Target Listeners and State Flag for Conditional Drop

DragAndDrop.java

```

1 //Imports are omitted
2 public class DragAndDrop extends Application {
3     double mouse_x, mouse_y;
+ 4     boolean isEntered;
5     @Override
6     public void start(Stage stage) {
7         stage.setTitle("Hello Drag and Drop");
8         Group root = new Group();
9         Scene scene = new Scene(root, 500, 200);
10        scene.setFill(Color.WHITE);
11        FigureGroup source = new FigureGroup(80,50, 40, Color.LIGHTBLUE, "SOURCE");
12        FigureGroup target = new FigureGroup(320,50, 50, Color.RED, "DROP TARGET");
+ 13        isEntered = false;
14        ...
41        source.getFigure().setOnMouseReleased(new EventHandler<MouseEvent>() {
42            @Override
43            public void handle(MouseEvent event) {
- 44                source.setBackOrigin();
- 45                source.getFigure().setMouseTransparent(false);
+ 46                if(!isEntered) {
+ 47                    source.setBackOrigin();
+ 48                    source.getFigure().setMouseTransparent(false);
+ 49                } else {
+ 50                    target.getFigure().getChildren().add(source.getFigure());
+ 51                }
52            }
53        });
+ 54        target.getFigure().setOnMouseDragEntered(new EventHandler<MouseEvent>() {
+ 55            @Override
+ 56            public void handle(MouseEvent event) {
+ 57                isEntered = true;
+ 58            }
+ 59        });
+ 60        target.getFigure().setOnMouseDragExited(new EventHandler<MouseEvent>() {
+ 61            @Override
+ 62            public void handle(MouseEvent event) {
+ 63                isEntered = false;
+ 64            }
+ 65        });
        ...

```

Dragging and dropping items in a ListView

Figure 1.10 illustrates a common application of drag-and-drop functionality: moving items between lists. Unlike the previous example, this functionality requires data transfer in conjunction with the drag-and-drop gestures. For instance, dragging the item Java from the source list to the target list transfers not only the visual representation of the item but also its underlying data. Therefore, this task involves several considerations beyond those required for moving simple shapes:

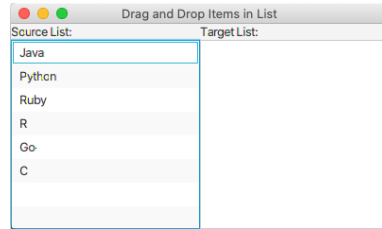


Figure 1.10: Drag-and-Drop Item Interaction between Lists

Data Transfer: The mechanisms for transferring the data associated with an item between the source and target lists.

Target Acceptance: The logic that determines when and how a target list accepts data from a source list.

Post-Drop Operations: The actions required on both the source and target lists subsequent to a successful drop operation.

Addressing these considerations is crucial for implementing a functional drag-and-drop feature that transfers items and their associated data between lists. The implementation of this functionality requires at least four primary event types: `dragDetected`, `dragOver`, `dragDropped`, and `dragDone`. Because either list can serve as both a source and a target, an efficient design approach is to create a single class for the list component, allowing it to be reused for both sides of the application window. Script 1.12 presents the class for this list component. A `ListView` control is created in Line 5, and the listeners for the four required event types are registered with this `ListView` between Lines 7 and 40.

Script 1.12: Implementation of a Reusable ListView Component with Drag-and-Drop Functionality

DragListModel.java

```
+ 1 //Imports are omitted
+ 2 public class DragListModel {
+ 3     ListView<String> lv;
+ 4     public DragListModel(){
+ 5         lv = new ListView<String>();
+ 6         lv.setPrefSize(200, 200);
+ 7         lv.setOnDragDetected(new EventHandler<MouseEvent>() {
+ 8             public void handle(MouseEvent event) {
+ 9                 Dragboard dragboard = lv.startDragAndDrop(TransferMode.MOVE);
+10                 String selectedItems = lv.getSelectionModel().getSelectedItem();
+11                 ClipboardContent content = new ClipboardContent();
+12                 content.putString(selectedItems);
+13                 dragboard.setContent(content);
+14             }
+15         });
+16         lv.setOnDragOver(new EventHandler <DragEvent>() {
+17             public void handle(DragEvent event) {
+18                 Dragboard dragboard = event.getDragboard();
+19                 if (event.getGestureSource() != lv && dragboard.hasString()) {
+20                     event.acceptTransferModes(TransferMode.MOVE);
+21                 }
+22             }
+23         });
+24         lv.setOnDragDropped(new EventHandler <DragEvent>() {
+25             public void handle(DragEvent event) {
+26                 boolean dragCompleted = false;
+27                 Dragboard dragboard = event.getDragboard();
+28                 if(dragboard.hasString()) {
+29                     String list = dragboard.getString();
+30                     lv.getItems().addAll(list);
+31                     dragCompleted = true;
+32                 }
+33                 event.setDropCompleted(dragCompleted);
+34             }
+35         });
+36         lv.setOnDragDone(new EventHandler <DragEvent>() {
+37             public void handle(DragEvent event) {
+38                 lv.getItems().remove(lv.getSelectionModel().getSelectedItem());
+39             }
+40         });
+41     }
+42 }
```


The handler for the `setOnDragDetected` event is responsible for creating a `DragBoard`, which is a standard JavaFX container for managing data transfer between a drag source and a drop target. Although this specific example transfers only a string value (e.g., Java), the `DragBoard` is capable of managing the transfer of entire serialized objects in more complex applications.

Next, the handler for the `setOnDragOver` event, which is invoked when an object is dragged over the `ListView`, contains the logic to determine if the target list can accept the data. This determination is made by verifying that the `DragBoard` contains data of a compatible type, in this case, a string.

Subsequently, the handler for the `setOnDragDropped` event employs a boolean flag to track the completion status of the drop operation. This flag, declared in Line 27, is set to true upon the successful transfer of the string data to the target list's data model.

Script 1.13: Main Application Class for the `ListView` Drag-and-Drop Example
`DragListExample.java`

```
+ 1 //Imports are omitted
+ 2 public class DragListExample extends Application {
+ 3     @Override
+ 4     public void start(Stage stage) {
+ 5         Label sourceListLbl = new Label("Source List: ");
+ 6         Label targetListLbl = new Label("Target List: ");
+ 7         ObservableList<String> list = FXCollections.observableArrayList();
+ 8         list.addAll("Java", "Python", "Ruby", "R", "Go", "C");
+ 9         DragListModel sourceView = new DragListModel();
+10         DragListModel targetView = new DragListModel();
+11         sourceView.lv.getItems().addAll(list);
+12         GridPane pane = new GridPane();
+13         pane.addRow(1, sourceListLbl, targetListLbl);
+14         pane.addRow(2, sourceView.lv, targetView.lv);
+15         VBox root = new VBox();
+16         root.getChildren().add(pane);
+17         stage.setScene(new Scene(root));
+18         stage.setTitle("Drag and Drop Items in List");
+19         stage.show();
+20     }
    ...
```

Finally, the handler for the `dragDone` event is responsible for removing the item from the source `ListView`'s data model, conditioned on the successful transfer to the target. The main application class for this example, shown in Script 1.13, first creates the underlying data models (Lines 7-8). It then instantiates two `ListView` components (Lines 9-10) and populates the source `ListView` with the initial data. Executing this script produces the GUI shown in Figure 1.10.

1.3.3 Keyboard events

The handling of mouse events and keyboard events differs in several fundamental aspects. In the case of mouse events, the user's intent is typically unambiguous, as it is spatially directed toward a specific visual element by the mouse pointer. This explicitness of the event target simplifies the implementation of event-handling logic. Keyboard events, by contrast, are often not associated with a specific GUI component, and the outcome of a key press may not be intuitively apparent to the user without prior instruction. Therefore, the behavior associated with keyboard events must be explicitly predefined, and the event source is typically registered at a global level, such as the entire application scene. For example, in a video game, specific keys are designated for character movement, and these key bindings apply globally regardless of the mouse pointer's position.

Script 1.14 demonstrates an application in which a component's movement is controlled by keyboard arrow keys. The visual component in this example is an instance of the `FigureGroup` class from Script 1.9. The code segment in Lines 12-21 implements the movement logic. This is achieved by creating a listener for key press events on the main scene. The handler for this listener processes the received key code and updates the component's position relative to its previous coordinates.

Script 1.14: Implementation of Keyboard-Controlled Component Movement
KeyboardEventExample.java

```
+ 1 //Imports are omitted
+ 2 public class KeyboardEventExample extends Application {
+ 3     @Override
+ 4     public void start(Stage stage) {
+ 5         stage.setTitle("Keyboard Event Example");
+ 6         Group root = new Group();
+ 7         Scene scene = new Scene(root, 500, 200);
+ 8         scene.setFill(Color.WHITE);
+ 9         FigureGroup obj = new FigureGroup(200,50,50,Color.CORAL,"FIGURE");
+10         scene.setOnKeyPressed(new EventHandler<KeyEvent>() {
+11             @Override
+12             public void handle(KeyEvent event) {
+13                 if (event.getCode() == KeyCode.UP) {
+14                     obj.setPosition(0,-5);
+15                 } else if (event.getCode() == KeyCode.DOWN) {
+16                     obj.setPosition(0,5);
+17                 } else if (event.getCode() == KeyCode.LEFT) {
+18                     obj.setPosition(-5,0);
+19                 } else if (event.getCode() == KeyCode.RIGHT) {
+20                     obj.setPosition(5,0);
+21                 }
+22             }
+23         });
+24         root.getChildren().add(obj.getFigure());
+25         stage.setScene(scene);
+26         stage.show();
+27     }
+28     public static void main(String[] args) {
+29         Application.launch(args);
+30     }
+31 }
```

1.4 Case Study

AI Advisory

Event-driven programming with JavaFX is an area where relying on AI tools too early can lead to code that works but is difficult to understand. This is because event-driven applications do not run in a simple, linear sequence. Instead, program execution is controlled by user interactions, such as mouse clicks, which act as signals to trigger specific event handlers. Because this flow of control is managed behind the scenes by the JavaFX framework, letting an AI write these interactions prevents a developer from learning exactly when and how the code is executed.

Recommended Approach

In this chapter, the links between the Model, View, and Controller components are held together entirely by the structure of the code. An AI tool without heavy configuration may not reliably determine which controller method is meant to update a specific visual component. Furthermore, the tool is unlikely to fully comprehend the lifecycle of an application without access to the entire codebase. The most reliable way to learn here is to manually trace the execution path from the event source to the listener, and finally to the handler that updates the data. Once a clear understanding of this flow is established, AI tools can then be used to inspect the code or generate specific segments. However, the generated code must always be verified to ensure it matches the intended design and does not introduce unintended side effects.

Appropriate Uses

- ✓ Run the application after adding small sections of code to see exactly what changes.

- ✓ Draw a diagram of the project architecture before writing any event handlers.
- ✓ Read the official documentation to understand a method before using it.
- ✓ Use an AI to explain general concepts (for example, asking about the difference between two methods) only after trying to figure the answer out manually.

Practices to Avoid

- ✗ Do not give an AI the entire project and ask for the event connections to be written.
- ✗ Do not ask an AI to link the controller to the view. External tools can easily make mistakes with specific variable names or layout details. At this stage of learning, it remains difficult to verify the correctness of generated code without a clear understanding of the underlying logic.
- ✗ Do not accept AI-generated code for complex tasks like drag-and-drop without reading and understanding every step of the process.
- ✗ Do not ask an AI to fix a `NullPointerException` before manually checking if variable names and annotations match perfectly.

Practicing this manual process will prepare the developer for Chapter 3. When background threads are introduced later, they introduce significantly more complex bugs that AI tools often struggle to accurately detect or resolve unless fully understand the entire context.

This chapter's case study involves the implementation of a GUI for a game inventory system, a feature commonly found in role-playing games (RPGs). The final application is depicted in Figure 1.11. The implementation will be structured around two primary entities: Character and Item. The Character entity will have two initial subtypes, Physical and Magical, and two corresponding item categories, Weapon and Armor, will also be implemented.

The right side of the GUI will display the character's avatar and a list of associated attributes. Positioned below the attributes, a Generate Character button will provide functionality to randomly generate new character statistics. The left side of the GUI will contain two equipment slots. These slots will support drag-and-drop functionality, allowing items from a list at the bottom of the window to be equipped, thereby enhancing the character's attributes (e.g., a weapon increasing attack power). The system will also support unequipping items, which will revert the character's attributes to their base values. An example of the project's parameter configuration is illustrated in Figure 1.12.

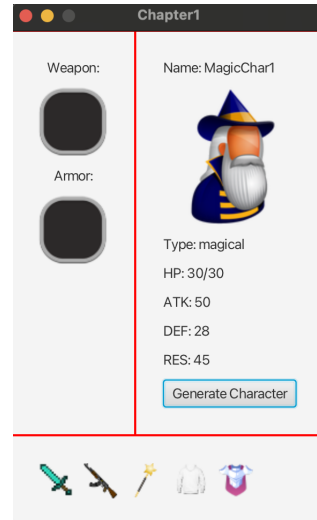
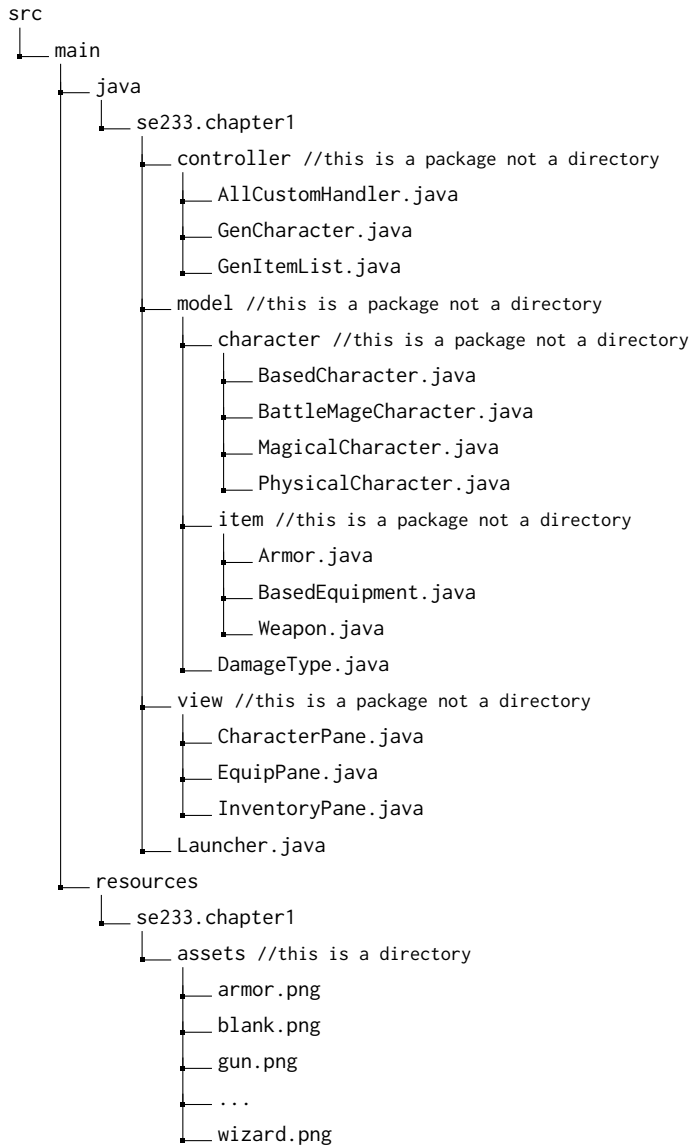


Figure 1.11: GUI Output of the Case Study Application

The directory and file structures of the project are as given below:



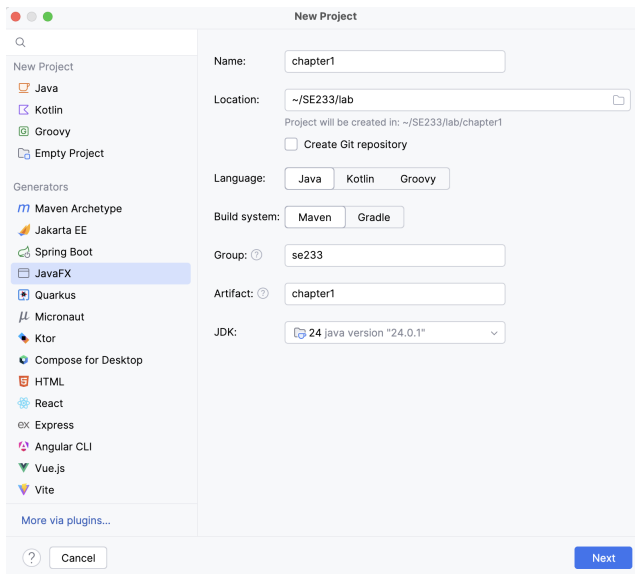


Figure 1.12: Project Parameter Configuration Example

The assets folder is the designated location for storing all program assets, which are available at <https://cmu.to/SE233CH1>. The project's structure is organized according to the MVC design pattern. The Launcher class functions as the main application entry point. The view component encompasses all elements related to presentation, such as displaying the attribute values stored within a Character instance. The model component is responsible for data modeling, data structures, and business logic, with entities implemented as distinct objects. For instance, the BasicEquipment class serves as the parent class for the Armor and Weapon classes, defining their common attributes. Finally, the controller component contains the core application logic and manages the interactions between the view and the model.

It is worth noting that the communication between these three components is largely indirect. The Controller accesses the Model through direct method calls, retrieving or updating data as needed. The Controller then updates the View either by invoking methods on view components or by modifying observable properties

to which the View is bound. The View, however, has no direct reference to the Model; all data flows through the Controller.

In this chapter, the View is constructed entirely in Java code rather than through FXML layout files. FXML-based view construction, where the Controller class declares fields annotated with `@FXML` and links them to visual components via matching `fx:id` attributes, will be introduced in Chapter 3. When that approach is adopted, a mismatch between the field name in the Controller and the `fx:id` in the FXML file results in a `null` reference at runtime, a common source of errors that can only be identified by careful inspection of both files. The Java-code approach used here makes the binding between components explicit and is therefore a useful starting point for understanding MVC communication before the additional layer of FXML indirection is introduced.

1.4.1 Model components

The examination of the project implementation begins with the model components. Scripts 1.15, 1.16, 1.17, and 1.18 illustrate the classes associated with the game's Character entity. The two character types, `Physical` and `Magical`, are implemented as subclasses derived from the `BasedCharacter` class. A significant advantage of this inheritance structure is that polymorphism allows instances of either `PhysicalCharacter` or `MagicalCharacter` to be treated as the `BasedCharacter` type, simplifying their management in the GUI. In this example, `Physical` characters possess a higher Full Health Point (`FullHp`) attribute compared to `Magical` characters, which is balanced by a lower base power attribute.

An examination of the `PhysicalCharacter` and `MagicalCharacter` classes (Scripts 1.17 and 1.18, respectively) reveals an absence of local variable declarations. This design choice is based on the commonality of their attributes; it is more efficient to declare these shared variables in the `BasedCharacter` superclass. The shared variables include `name`, `fullHP`, and `basedPow`.

Script 1.15: model/BasedCharacter.java

```
+ 1 //Imports are omitted
+ 2 public class BasedCharacter {
+ 3     protected String name, imgpath;
+ 4     protected DamageType type;
+ 5     protected Integer fullHp, basedPow, basedDef, basedRes;
+ 6     protected Integer hp, power, defense, resistance;
+ 7     protected Weapon weapon;
+ 8     protected Armor armor;
+ 9     public String getName () { return name; }
+10     public String getImagepath() { return imgpath; }
+11     public Integer getHp() { return hp; }
+12     public Integer getFullHp() { return fullHp; }
+13     public Integer getPower() { return power; }
+14     public Integer getDefense() { return defense; }
+15     public Integer getResistance() { return resistance; }
+16     @Override
+17     public String toString() { return name; }
+18     public DamageType getType() {
+19         return type;
+20     }
+21 }
```

Script 1.16: model/character/DamageType.java

```
+ 1 //Imports are omitted
+ 2 public enum DamageType {
+ 3     physical,
+ 4     magical
+ 5 }
```

Script 1.17: model/character/PhysicalCharacter.java

```
+ 1 //Imports are omitted
+ 2 public class PhysicalCharacter extends BasedCharacter {
+ 3     public PhysicalCharacter(String name,String imgpath,int basedDef,int basedRes) {
+ 4         this.name = name;
+ 5         this.type = DamageType.physical;
+ 6         this.imgpath = imgpath;
+ 7         this.fullHp = 50;
+ 8         this.basedPow = 30;
+ 9         this.basedDef = basedDef;
+10         this.basedRes = basedRes;
+11         this.hp = this.fullHp;
+12         this.power = this.basedPow;
+13         this.defense = basedDef;
+14         this.resistance = basedRes;
+15     }
+16 }
```

Script 1.18: model/character/MagicalCharacter.java

```
+ 1 //Imports are omitted
+ 2 public class MagicalCharacter extends BasedCharacter {
+ 3     public MagicalCharacter(String name,String imgpath,int basedDef,int basedRes) {
+ 4         this.name = name;
+ 5         this.type = DamageType.magical;
+ 6         this.imgpath = imgpath;
+ 7         this.fullHp = 30;
+ 8         this.basedPow = 50;
+ 9         this.basedDef = basedDef;
+10         this.basedRes = basedRes;
+11         this.hp = this.fullHp;
+12         this.power = this.basedPow;
+13         this.defense = basedDef;
+14         this.resistance = basedRes;
+15     }
+16 }
```

Next, the classes associated with the `Item` entity are implemented. Scripts 1.19, 1.20, and 1.21 present the implementation of these three classes. Following a design pattern similar to that of the `Character` entity, the two item types, `Weapon` and `Armor`, are implemented as subclasses that extend the `BasedEquipment` superclass. This use of inheritance allows the inventory list and equipment slots to be typed as `BasedEquipment`, enabling them to hold instances of either the `Weapon` or `Armor` subclasses through polymorphism.

Script 1.19: model/item/BasedEquipment.java

```
+ 1 //Imports are omitted
+ 2 public class BasedEquipment {
+ 3     protected String name;
+ 4     protected String imgpath;
+ 5     public String getName() { return name; }
+ 6     public String getImagepath() { return imgpath; }
+ 7     public void setImagepath(String imgpath) { this.imgpath = imgpath; }
+ 8 }
```

As shown in the implementations of `Weapon` (Script 1.20) and `Armor` (Script 1.21), some variables are declared locally within the subclasses rather than being inherited from `BasedEquipment`. This design is necessary because certain attributes, such as defense and resistance, are specific to one subclass and not shared by the other.

Script 1.20: model/item/Weapon.java

```
+ 1 //Imports are omitted
+ 2 public class Weapon extends BasedEquipment{
+ 3     private int power;
+ 4     private DamageType damageType;
+ 5     public Weapon(String name, int power, DamageType damageType, String imgpath) {
+ 6         this.name = name;
+ 7         this.imgpath = imgpath;
+ 8         this.power = power;
+ 9         this.damageType = damageType;
+10     }
+11     public int getPower() { return power; }
+12     public void setPower(int power) { this.power = power; }
+13     public void setDamageType(DamageType weaponType) {
+14         this.damageType = weaponType;
+15     }
+16     public DamageType getDamageType() { return damageType; }
+17     @Override
+18     public String toString() { return name; }
+19 }
```

Script 1.21: model/item/Armor.java

```
+ 1 //Imports are omitted
+ 2 public class Armor extends BasedEquipment {
+ 3     private int defense, resistance;
+ 4     public Armor(String name, int defense, int resistance, String imgpath) {
+ 5         this.name = name;
+ 6         this.imgpath = imgpath;
+ 7         this.defense = defense;
+ 8         this.resistance = resistance;
+ 9     }
+10     public int getDefense() { return defense; }
+11     public void setDefense(int defense) { this.defense = defense; }
+12     public int getResistance() { return resistance; }
+13     public void setResistant(int resistance) { this.resistance = resistance; }
+14     @Override
+15     public String toString() { return name; }
+16 }
```

1.4.2 View components

Subsequent to the implementation of the model, the development process typically proceeds to the implementation of the view components. The view is responsible for the visual representation of the model's state. As depicted in Figure 1.11, the application's main window is composed of three distinct sub-panes. Each of these sub-panes is implemented as a separate class, named descriptively according to its function: EquipPane for the left-hand side, CharacterPane for the right-hand side, and InventoryPane for the bottom sub-pane, respectively. The implementations for these view component classes are presented in Scripts 1.22, 1.23, and 1.24.

Script 1.22: view/InventoryPane.java

```
+ 1 //Imports are omitted
+ 2 public class InventoryPane extends ScrollPane {
+ 3     private BasedEquipment[] equipmentArray;
+ 4     public InventoryPane() { }
+ 5     private Pane getDetailsPane() {
+ 6         Pane inventoryInfoPane = new HBox(10);
+ 7         inventoryInfoPane.setBorder(null);
+ 8         inventoryInfoPane.setPadding(new Insets(25, 25, 25, 25));
+ 9         if (equipmentArray!=null) {
+10             ImageView[] imageViewList = new ImageView[equipmentArray.length];
+11             for(int i=0 ; i< equipmentArray.length ; i++) {
+12                 imageViewList[i] = new ImageView();
+13                 imageViewList[i].setImage(new Image(Launcher.class.getResource(
+14                     equipmentArray[i].getImagepath()).toString()));
+15             }
+16             inventoryInfoPane.getChildren().addAll(imageViewList);
+17         }
+18         return inventoryInfoPane;
+19     }
+20     public void drawPane(BasedEquipment[] equipmentArray) {
+21         this.equipmentArray = equipmentArray;
+22         Pane inventoryInfo = getDetailsPane();
+23         this.setStyle("-fx-background-color:Red;");
+24         this.setContent(inventoryInfo);
+25     }
+26 }
```

Script 1.23: view/EquipPane.java

```

+ 1 //Imports are omitted
+ 2 public class EquipPane extends ScrollPane {
+ 3     private Weapon equippedWeapon;
+ 4     private Armor equippedArmor;
+ 5     public EquipPane() { }
+ 6     private Pane getDetailsPane() {
+ 7         Pane equipmentInfoPane = new VBox(10);
+ 8         equipmentInfoPane.setBorder(null);
+ 9         ((VBox) equipmentInfoPane).setAlignment(Pos.CENTER);
+10         equipmentInfoPane.setPadding(new Insets(25, 25, 25, 25));
+11         Label weaponLbl, armorLbl;
+12         ImageView weaponImg = new ImageView();
+13         ImageView armorImg = new ImageView();
+14         if (equippedWeapon != null) {
+15             weaponLbl = new Label("Weapon:\n"+equippedWeapon.getName());
+16             weaponImg.setImage(new Image(Launcher.class.getResource(equippedWeapon.
+17                 getImagepath()).toString()));
+18         } else {
+19             weaponLbl = new Label("Weapon:");
+20             weaponImg.setImage(new Image(Launcher.class.getResource("assets/blank.png"
+21                 ).toString()));
+22         }
+23         if (equippedArmor != null) {
+24             armorLbl = new Label("Armor: \n"+equippedArmor.getName());
+25             armorImg.setImage(new Image(Launcher.class.getResource(equippedArmor.
+26                 getImagepath()).toString()));
+27         } else {
+28             armorLbl = new Label("Armor:");
+29             armorImg.setImage(new Image(Launcher.class.getResource("assets/blank.png"
+30                 ).toString()));
+31         }
+32         equipmentInfoPane.getChildren().addAll(weaponLbl, weaponImg, armorLbl,
+33             armorImg);
+34         return equipmentInfoPane;
+35     }
+36     public void drawPane(Weapon equippedWeapon, Armor equippedArmor) {
+37         this.equippedWeapon = equippedWeapon;
+38         this.equippedArmor = equippedArmor;
+39         Pane equipmentInfo = getDetailsPane();
+40         this.setStyle("-fx-background-color:Red;");
+41         this.setContent(equipmentInfo);
+42     }
+43 }

```

Script 1.24: view/CharacterPane.java

```

+ 1 //Imports are omitted
+ 2 public class CharacterPane extends ScrollPane {
+ 3     private BasedCharacter character;
+ 4     public CharacterPane() { }
+ 5     private Pane getDetailsPane() {
+ 6         Pane characterInfoPane = new VBox(10);
+ 7         characterInfoPane.setBorder(null);
+ 8         characterInfoPane.setPadding(new Insets(25, 25, 25, 25));
+ 9         Label name,type,hp,atk,def,res;
+10         ImageView mainImage = new ImageView();
+11         if (this.character != null) {
+12             name = new Label("Name: "+character.getName());
+13             mainImage.setImage(new Image(Launcher.class.getResource(character.
+14                 getImagepath()).toString()));
+15             hp = new Label("HP: "+character.getHp().toString()+"/"+character.
+16                 getFullHp().toString());
+17             type = new Label("Type: "+character.getType().toString());
+18             atk = new Label("ATK: "+character.getPower());
+19             def = new Label("DEF: "+character.getDefense());
+20             res = new Label("RES: "+character.getResistance());
+21         } else {
+22             name = new Label("Name: ");
+23             mainImage.setImage(new Image(Launcher.class.getResource("assets/unknown.
+24                 png").toString()));
+25             hp = new Label("HP: ");
+26             type = new Label("Type: ");
+27             atk = new Label("ATK: ");
+28             def = new Label("DEF: ");
+29             res = new Label("RES: ");
+30         }
+31         Button genCharacter = new Button();
+32         genCharacter.setText("Generate Character");
+33         characterInfoPane.getChildren().addAll(name,mainImage,type,hp,atk,def,res,
+34             genCharacter);
+35         return characterInfoPane;
+36     }
+37     public void drawPane(BasedCharacter character) {
+38         this.character = character;
+39         Pane characterInfo = getDetailsPane();
+40         this.setStyle("-fx-background-color:Red;");
+41         this.setContent(characterInfo);
+42     }
+43 }

```


In Script 1.22, an `ImageView` array is created (Line 10) and initialized within a `for-loop` (Lines 11-14). This array, sized according to the total number of equipment items, is responsible solely for the visual presentation of the item images. The underlying `BasedEquipment` data objects are stored separately in the `equipmentArray` variable. The `getDetailPane()` method returns a `Pane` object, which is a node that can be attached to the scene graph. The `drawPane()` method subsequently adds this returned `Pane` as a child node. Because the `InventoryPane` class itself extends the JavaFX `ScrollPane`, it functions as a composite node that can be directly added to the main application's scene graph.

Next, in Script 1.23, the core functionality involves generating a different visual representation based on whether an item is equipped. For example, the conditional statement `equippedWeapon != null` (Lines 14-20) checks for the presence of an equipped weapon. This technique, known as conditional rendering, is common in GUI development. If a weapon is equipped, its corresponding image is displayed; otherwise, a default placeholder image (`blank.png`) is displayed. A similar technique is employed for displaying the character's image in the `CharacterPane` class, as shown in Line 21 of Script 1.24.

1.4.3 Controller components and programming logic

Following the implementation of the model and view components, the final step is to integrate these elements within the `Launcher` class, which functions as the controller. The initial implementation of this class is presented in Script 1.25. The core logic for assembling the GUI is located in Lines 20-30, where all three sub-panes are combined into a single JavaFX visual element of the type `BorderPane`. Additionally, the `refreshPane()` method, implemented in Lines 31-35, demonstrates a common GUI programming technique: the use of a single method to refresh the state of all application sub-components.

The implementation then proceeds to the core program logic. The initial logic required is the setup of the `Character` and `Item` lists, which occurs upon application startup and produces the GUI shown in Figure 1.11. Script 1.26 presents the `setUpCharacter()` method for this purpose. This method initially declares a `character` variable of the type `BasedCharacter` (Line 4). This variable is subsequently instantiated as an object of a `BasedCharacter` subclass, either `MagicalCharacter`

Script 1.25: Launcher.java

```
+ 1 //Imports are omitted
+ 2 public class Launcher extends Application {
+ 3     private static Scene mainScene;
+ 4     private static BasedCharacter mainCharacter = null;
+ 5     private static BasedEquipment[] allEquipments = null;
+ 6     private static Weapon equippedWeapon = null;
+ 7     private static Armor equippedArmor = null;
+ 8     private static CharacterPane characterPane = null;
+ 9     private static EquipPane equipPane = null;
+10     private static InventoryPane inventoryPane = null;
+11     @Override
+12     public void start(Stage primaryStage) throws Exception{
+13         primaryStage.setTitle("Chapter1");
+14         primaryStage.setResizable(false);
+15         primaryStage.show();
+16         Pane mainPane = getMainPane();
+17         mainScene = new Scene(mainPane);
+18         primaryStage.setScene(mainScene);
+19     }
+20     public Pane getMainPane() {
+21         BorderPane mainPane = new BorderPane();
+22         characterPane = new CharacterPane();
+23         equipPane = new EquipPane();
+24         inventoryPane = new InventoryPane();
+25         refreshPane();
+26         mainPane.setCenter(characterPane);
+27         mainPane.setLeft(equipPane);
+28         mainPane.setBottom(inventoryPane);
+29         return mainPane;
+30     }
+31     public static void refreshPane() {
+32         characterPane.drawPane(mainCharacter);
+33         equipPane.drawPane(equippedWeapon,equippedArmor);
+34         inventoryPane.drawPane(allEquipments);
+35     }
+36     public static BasedCharacter getMainCharacter() { return mainCharacter; }
+37     public static void setMainCharacter(BasedCharacter mainCharacter) {
+38         Launcher.mainCharacter = mainCharacter;
+39     }
+40     public static void main(String[] args) {
+41         launch(args);
+42     }
+43 }
```

or `PhysicalCharacter`, as shown in Lines 10 and 12, respectively. This use of a superclass reference to hold a subclass instance is a practical illustration of how polymorphism can simplify program design.

Following this, Script 1.27 demonstrates the initialization of an array of `BasedEquipment` objects. Finally, the logic from these setup scripts is integrated into the main application through modifications to the `Launcher` class, as shown in Lines 33-34 of Script 1.28.

The next phase of the implementation involves the second set of program logic. Script 1.29 introduces a listener for click events on the Generate Character button. To manage the numerous event handlers required by the application, the implementation consolidates this logic within a dedicated `AllCustomHandler` class, as illustrated in Script 1.30. The sole responsibility of the handler method for this button is to invoke the `setUpCharacter()` method, which regenerates the character with new randomized attributes and subsequently triggers a refresh of the GUI.

Script 1.26: controller/GenCharacter.java

```
+ 1 //Imports are omitted
+ 2 public class GenCharacter {
+ 3     public static BasedCharacter setUpCharacter() {
+ 4         BasedCharacter character;
+ 5         Random rand = new Random();
+ 6         int type = rand.nextInt(2)+1;
+ 7         int basedDef = rand.nextInt(50)+1;
+ 8         int basedRes = rand.nextInt(50)+1;
+ 9         if (type == 1) {
+ 10             character = new MagicalCharacter("MagicChar1", "assets/wizard.png",
+ 11                 basedDef,basedRes);
+ 12         } else {
+ 13             character = new PhysicalCharacter("PhysicalChar1", "assets/knight.png",
+ 14                 basedRes,basedRes);
+ 15         }
+ 16     }
+ 17 }
```

Script 1.27: controller/GenItemList.java

```
+ 1 //Imports are omitted
+ 2 public class GenItemList {
+ 3     public static BasedEquipment[] setUpItemList() {
+ 4         BasedEquipment[] itemLists = new BasedEquipment[5];
+ 5         itemLists[0] = new Weapon("Sword", 10, DamageType.physical, "assets/sword.png");
+ 6         itemLists[1] = new Weapon("Gun", 20, DamageType.physical, "assets/gun.png");
+ 7         itemLists[2] = new Weapon("Staff", 30, DamageType.magical, "assets/staff.png");
+ 8         itemLists[3] = new Armor("Shirt", 0, 50, "assets/shirt.png");
+ 9         itemLists[4] = new Armor("Armor", 50, 0, "assets/armor.png");
+10         return itemLists;
+11     }
+12 }
```

Script 1.28: Launcher.java

```
1 //Imports are omitted
2 public class Launcher extends Application {
    ...
30     primaryStage.setTitle("Chapter1");
31     primaryStage.setResizable(false);
32     primaryStage.show();
+33     mainCharacter = GenCharacter.setUpCharacter();
+34     allEquipments = GenItemList.setUpItemList();
35     Pane mainPane = getMainPane();
    ...
}
```

Script 1.29: /src/view/CharacterPane.java

```
1 //Imports are omitted
2 public class CharacterPane extends ScrollPane {
    ...
42     genCharacter.setText("Generate Character");
+43     genCharacter.setOnAction(new AllCustomHandler.GenCharacterHandler());
44     characterInfoPane.getChildren().addAll(name, mainImage, type, hp, atk, def, res,
        genCharacter);
45     return characterInfoPane;
    ...
}
```

Script 1.30: controller/AllCustomHandler.java

```

+ 1 //Imports are omitted
+ 2 public class AllCustomHandler {
+ 3     public static class GenCharacterHandler implements EventHandler<ActionEvent> {
+ 4         @Override
+ 5         public void handle(ActionEvent event) {
+ 6             Launcher.setMainCharacter(GenCharacter.setUpCharacter());
+ 7             Launcher.refreshPane();
+ 8         }
+ 9     }
+ 10 }

```

The completion of this implementation step allows the application to be executed with its initial program logic integrated, producing the GUI depicted in Figure 1.13. Furthermore, since the Generate Character button is already implemented, this enables the logic that may produce a Physical character, like that in Figure 1.11, or a Magical character, like in Figure 1.13.

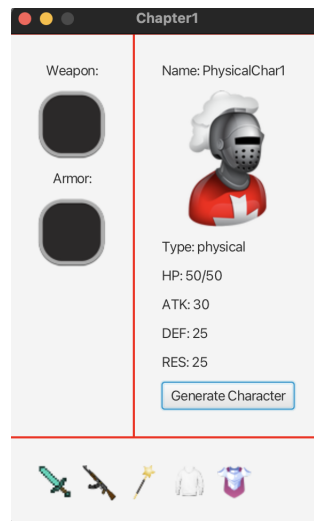


Figure 1.13: An Alternative GUI Output of the Case Study Application

1.4.4 Flexible-size container

Standard arrays are data structures designed to store a fixed number of elements, a size that is predetermined and immutable during program execution. In the context of this application, however, the number of items in the inventory is dynamic, as items may be added or removed during gameplay. Consequently, an `ArrayList`, which is a dynamic data structure capable of resizing itself, represents a more suitable choice for implementing the inventory. Script 1.31 demonstrates the refactoring of the item container from a standard array to an `ArrayList`.

Script 1.31: controller/GenItemList.java

```
1 //Imports are omitted
  ...
8 public class GenItemList {
- 9     public static BasedEquipment[] setUpItemList() {
- 10         BasedEquipment[] itemLists = new BasedEquipment[5];
- 11         itemLists[0] = new Weapon("Sword", 10, DamageType.physical, "assets/sword.png");
- 12         itemLists[1] = new Weapon("Gun", 20, DamageType.physical, "assets/gun.png");
- 13         itemLists[2] = new Weapon("Staff", 30, DamageType.magical, "assets/staff.png");
- 14         itemLists[3] = new Armor("Shirt", 0, 50, "assets/shirt.png");
- 15         itemLists[4] = new Armor("Armor", 50, 0, "assets/armor.png");
+ 16     public static ArrayList<BasedEquipment> setUpItemList() {
+ 17         ArrayList<BasedEquipment> itemLists = new ArrayList<BasedEquipment>(5);
+ 18         itemLists.add(new Weapon("Sword", 10, DamageType.physical, "assets/sword.png"));
+ 19         itemLists.add(new Weapon("Gun", 20, DamageType.physical, "assets/gun.png"));
+ 20         itemLists.add(new Weapon("Staff", 30, DamageType.magical, "assets/staff.png"));
+ 21         itemLists.add(new Armor("Shirt", 0, 50, "assets/shirt.png"));
+ 22         itemLists.add(new Armor("Armor", 50, 0, "assets/armor.png"));
23         return itemLists;
24     }
25 }
```

The modifications from the updated `GenItemList` class (Script 1.31) are integrated into the application through corresponding adjustments in the `Inventory-Pane` and `Launcher` classes, as shown in Scripts 1.32 and 1.33, respectively. These adjustments primarily involve replacing array-specific syntax with the equivalent

methods provided by the `ArrayList` class. For instance, the `length` property for determining an array's size is replaced with the `size()` method, and element retrieval is performed using the `get()` method instead of direct array indexing. These changes are evident in Lines 22, 24, and 27 of Script 1.32. Within the `Launcher` class, the only required modification is changing the declared type of the `allEquipments` variable, as shown in Lines 22 of Script 1.33.

Script 1.32: `view/InventoryPane.java`

```

1  //Imports are omitted
   ...
12 public class InventoryPane extends ScrollPane {
- 13     private BasedEquipment[] equipmentArray;
+ 14     private ArrayList<BasedEquipment> equipmentArray;
15     public InventoryPane() { }
16     private Pane getDetailsPane() {
17         Pane inventoryInfoPane = new HBox(10);
18         inventoryInfoPane.setBorder(null);
19         inventoryInfoPane.setPadding(new Insets(25, 25, 25, 25));
20         if (equipmentArray!=null) {
- 21             ImageView[] imageViewList = new ImageView[equipmentArray.length];
+ 22             ImageView[] imageViewList = new ImageView[equipmentArray.size()];
- 23             for(int i=0 ; i< equipmentArray.length ; i++) {
+ 24             for(int i=0 ; i< equipmentArray.size() ; i++) {
25                 imageViewList[i] = new ImageView();
- 26                 imageViewList[i].setImage(new Image(Launcher.class.getResource(
                    equipmentArray[i].getImagepath()).toString()));
+ 27                 imageViewList[i].setImage(new Image(Launcher.class.getResource(
                    equipmentArray.get(i).getImagepath()).toString()));
28             }
29             inventoryInfoPane.getChildren().addAll(imageViewList);
30         }
31         return inventoryInfoPane;
32     }
- 33     public void drawPane(BasedEquipment[] equipmentArray) {
+ 34     public void drawPane(ArrayList<BasedEquipment> equipmentArray) {
        ...

```

Script 1.33: Launcher.java

```
1 //Imports are omitted
  ...
18 public class Launcher extends Application {
19     private static Scene mainScene;
20     private static BasedCharacter mainCharacter = null;
- 21     private static BasedEquipment[] allEquipments = null;
+ 22     private static ArrayList<BasedEquipment> allEquipments = null;
    ...
```

Executing the application at this point in the development will not produce any observable visual differences when compared to the previous version that utilized a standard array. This architectural modification, however, provides the necessary flexibility to dynamically add or remove items from the inventory, a feature that was not possible with a fixed-size array.

1.4.5 Drag-and-drop functionality

With the fundamental components established, the subsequent focus is the implementation of the drag-and-drop functionality. The drag-and-drop feature in this case study is primarily concerned with data transfer. Specifically, this involves dragging an item's visual representation and dropping it onto a character's equipment slot, which results in the modification of the character's attributes.

The initiation of the drag-and-drop process requires the declaration of a data format and a Dragboard, as demonstrated in Line 4 of Script 1.34. The item image within the InventoryPane acts as the event source. This image can be dragged and dropped onto the event targets, which are the two equipment slots within the EquipPane. To manage this functionality, several handler methods are implemented in Script 1.35. The handler registered via the `setOnDragDetected()` method defines a Dragboard variable (`db`) to temporarily store data during the transfer. This is achieved using the `imgView.startDragAndDrop(TransferMode.ANY)` method, which functions as a clipboard for transferring data between the drag source and drop target.

Script 1.34: model/Item/BasedEquipment.java

```
1 //Imports are omitted
  ...
- 3 public class BasedEquipment {
+ 4 public class BasedEquipment implements Serializable {
+ 5     public static final DataFormat DATA_FORMAT = new DataFormat( "src.main.java.
      se233.chapter1.model.item.BasedEquipment");
6     protected String name;
7     protected String imgpath;
8     public String getName() { return name; }
9     public String getImagepath() { return imgpath; }
10    public void setImagepath(String imgpath) { this.imgpath = imgpath; }
11 }
```

Script 1.35: controller/AllCustomHandler.java

```
1 //Imports are omitted
2 public class AllCustomHandler {
  ...
+ 15 public static void onDragDetected(MouseEvent event, BasedEquipment equipment,
      ImageView imgView) {
+ 16     Dragboard db = imgView.startDragAndDrop(TransferMode.ANY);
+ 17     db.setDragView(imgView.getImage());
+ 18     ClipboardContent content = new ClipboardContent();
+ 19     content.put(equipment.DATA_FORMAT, equipment);
+ 20     db.setContent(content);
+ 21     event.consume();
+ 22 }
23 }
```

The updated InventoryPane implementation (Script 1.36) binds the setOnDragDetected event listener to the event sources. Because all item images in the inventory function as event sources, a for-loop is employed to register the listener with each image, as shown in Lines 26-31.

Script 1.36: view/InventoryPane.java

```

1  //Imports are omitted
2  public class InventoryPane extends ScrollPane {
3      private ArrayList<BasedEquipment> equipmentArray;
4      ...
23     for(int i=0 ; i< equipmentArray.size() ; i++) {
24         imageViewList[i] = new ImageView();
25         imageViewList[i].setImage(new Image(Launcher.class.getResource(
+ 26             equipmentArray.get(i).getImagepath()).toString());
+ 27         int finalI = i;
+ 28         imageViewList[i].setOnDragDetected(new EventHandler<MouseEvent>() {
+ 29             public void handle(MouseEvent event) {
+ 30                 onDragDetected(event, equipmentArray.get(finalI), imageViewList[
+ 31                     finalI]);
+ 32             }
+ 33         });
34     }
35     inventoryInfoPane.getChildren().addAll(imageViewList);
36     ...

```

With the event sources configured, the implementation focus shifts to the event targets. Script 1.37 introduces handlers for the `onDragOver` and `onDragDropped` events to the `AllCustomHandler` class. Line 33 demonstrates a technique for verifying the class name of the dragged data. This allows the handler to ensure that only an appropriate equipment type (Weapon or Armor) is accepted by a given equipment slot.

Line 47 of the script demonstrates a technique for managing the equipment slots. The logic checks if the equipment slot container (a `Pane`) already contains a child node. The presence of a child node signifies that an item is currently equipped. If the slot is occupied, the currently equipped item is first unequipped before the new item is equipped.

Subsequently, these event handlers are bound to the target view components. As shown in Lines 51-66 of Script 1.38, listeners that invoke the `onDragOver()` and `onDragDropped()` handler methods are registered with the `weaponImgGroup` and `armorImgGroup` components of the `EquipPane` class. These components display a

Script 1.37: controller/AllCustomHandler.java

```

1 //Imports are omitted
  ...
+ 30 public static void onDragOver(DragEvent event, String type) {
+ 31     Dragboard dragboard = event.getDragboard();
+ 32     BasedEquipment retrievedEquipment = (BasedEquipment)dragboard.getContent(
        BasedEquipment.DATA_FORMAT);
+ 33     if (dragboard.hasContent(BasedEquipment.DATA_FORMAT) && retrievedEquipment.
        getClass().getSimpleName().equals(type))
+ 34         event.acceptTransferModes(TransferMode.MOVE);
+ 35 }
+ 36 public static void onDragDropped(DragEvent event, Label lbl, StackPane imgGroup) {
+ 37     boolean dragCompleted = false;
+ 38     Dragboard dragboard = event.getDragboard();
+ 39     if(dragboard.hasContent(BasedEquipment.DATA_FORMAT)) {
+ 40         BasedEquipment retrievedEquipment = (BasedEquipment)dragboard.getContent(
            BasedEquipment.DATA_FORMAT);
+ 41         if(retrievedEquipment.getClass().getSimpleName().equals("Weapon")) {
+ 42             Launcher.setEquippedWeapon((Weapon) retrievedEquipment);
+ 43         } else {
+ 44             Launcher.setEquippedArmor((Armor) retrievedEquipment);
+ 45         }
+ 46         ImageView imgView = new ImageView();
+ 47         if (imgGroup.getChildren().size()!=1) {
+ 48             imgGroup.getChildren().remove(1);
+ 49             Launcher.refreshPane();
+ 50         }
+ 51         lbl.setText(retrievedEquipment.getClass().getSimpleName() + ":\n" +
            retrievedEquipment.getName());
+ 52         imgView.setImage(new Image(Launcher.class.getResource(retrievedEquipment.
            getImagepath()).toString()));
+ 53         imgGroup.getChildren().add(imgView);
+ 54         dragCompleted = true;
+ 55     }
+ 56     event.setDropCompleted(dragCompleted);
+ 57 }
58 }

```

Script 1.38: view/EquipPane.java

```

1  //Imports are omitted {
    ...
24  Label weaponLbl,armorLbl;
+ 25  StackPane weaponImgGroup = new StackPane();
+ 26  StackPane armorImgGroup = new StackPane();
+ 27  ImageView bg1 = new ImageView();
+ 28  ImageView bg2 = new ImageView();
29  ImageView weaponImg = new ImageView();
30  ImageView armorImg = new ImageView();
+ 31  bg1.setImage(new Image(Launcher.class.getResource("assets/blank.png").toString()));
+ 32  bg2.setImage(new Image(Launcher.class.getResource("assets/blank.png").toString()));
+ 33  weaponImgGroup.getChildren().add(bg1);
+ 34  armorImgGroup.getChildren().add(bg2);
35  if (equippedWeapon != null) {
36      weaponLbl = new Label("Weapon:\n"+equippedWeapon.getName());
37      weaponImg.setImage(new Image(Launcher.class.getResource(equippedWeapon.
        getImagepath()).toString()));
+ 38      weaponImgGroup.getChildren().add(weaponImg);
    ...
43  if (equippedArmor != null) {
44      armorLbl = new Label("Armor: \n"+equippedArmor.getName());
45      armorImg.setImage(new Image(Launcher.class.getResource(equippedArmor.getImagepath
        ()).toString()));
+ 46      armorImgGroup.getChildren().add(armorImg);
    ...
+ 51  weaponImgGroup.setOnDragOver(new EventHandler<DragEvent>() {
+ 52      @Override
+ 53      public void handle(DragEvent e) { onDragOver(e, "Weapon"); }
+ 54  });
+ 55  armorImgGroup.setOnDragOver(new EventHandler<DragEvent>() {
+ 56      @Override
+ 57      public void handle(DragEvent e) { onDragOver(e, "Armor"); }
+ 58  });
+ 59  weaponImgGroup.setOnDragDropped(new EventHandler<DragEvent>() {
+ 60      @Override
+ 61      public void handle(DragEvent e) { onDragDropped(e, weaponLbl, weaponImgGroup); }
+ 62  });
+ 63  armorImgGroup.setOnDragDropped(new EventHandler<DragEvent>() {
+ 64      @Override
+ 65      public void handle(DragEvent e) { onDragDropped(e, armorLbl, armorImgGroup); }
+ 66  });
- 67  equipmentInfoPane.getChildren().addAll(weaponLbl,weaponImg,armorLbl,armorImg);
+ 68  equipmentInfoPane.getChildren().addAll(weaponLbl,weaponImgGroup,armorLbl,
        armorImgGroup);
    ...

```

default placeholder image (blank.png), which is replaced by the equipped item's image when an item is successfully dropped into the slot.

This initial implementation, however, deviates from the common practice where an equipped item is displayed layered on top of a slot's background image. Therefore, further modifications are introduced to the EquipPane class (Lines 25-28, 31-34, 38, 46, and 68 of Script 1.38) to support the layering of the equipped item's image over the placeholder image, which provides a more visually intuitive representation.

At this stage of development, the compiler may indicate the need for accessor (getter) and mutator (setter) methods in several classes. Figure 1.14 illustrates the process of adding these methods. It is crucial that all required accessor and mutator methods are implemented, particularly within the Launcher class, before the application is re-executed.

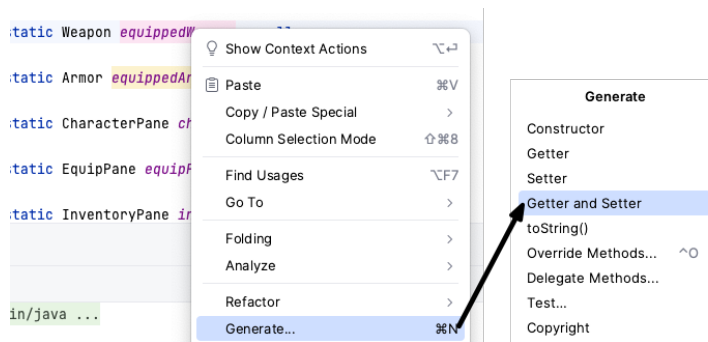


Figure 1.14: Generation of Accessor and Mutator Methods in IntelliJ

Upon the completion of this implementation step, the application will enable drag-and-drop functionality for weapons and armor, as depicted in Figure 1.15.

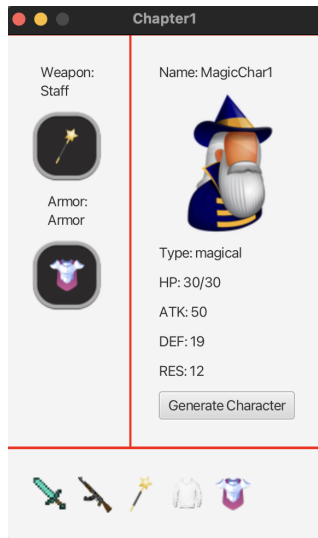


Figure 1.15: GUI Output of the Case Study Application with Weapon and Armor Equipped

1.4.6 Attributes propagation

The primary objective of this drag-and-drop implementation is to modify character status attributes based on the equipped items. This is achieved by transferring the equipment status stored within the dragged object to adjust the character's status. Notably, the status modification logic must propagate the value changes to both the model and view components. Upon mouse release and acceptance of the dropped object by the target event node, the `BasedCharacter` object is retrieved from the `Launcher` class, and its status values are modified. The attribute value adjustment logic is detailed between Lines 22 and 30 of Script 1.39. Following the modification, the object state is updated using its setter methods, as shown in the added lines of Script 1.40.

Script 1.39: model/character/BasedCharacter.java

```

1  //Imports are omitted
2  public class BasedCharacter {
    ...
+ 22  public void equipWeapon( Weapon weapon) {
+ 23      this.weapon = weapon;
+ 24      this.power = this.basedPow + weapon.getPower();
+ 25  }
+ 26  public void equipArmor( Armor armor) {
+ 27      this.armor = armor;
+ 28      this.defense = this.basedDef + armor.getDefense();
+ 29      this.resistance= this.basedRes + armor.getResistance();
+ 30  }
31  @Override
32  public String toString() { return name; }
33  }

```

Script 1.40: controller/AllCustomHandler.java

```

1  //Imports are omitted
    ...
37  public static void onDragDropped(DragEvent event, Label lbl, StackPane imgGroup){
    ...
40      if(dragboard.hasContent(BasedEquipment.DATA_FORMAT)) {
41          BasedEquipment retrievedEquipment = (BasedEquipment)dragboard.getContent(
              BasedEquipment.DATA_FORMAT);
+ 42          BasedCharacter character = Launcher.getMainCharacter();
43          if(retrievedEquipment.getClass().getSimpleName().equals("Weapon")) {
44              Launcher.setEquippedWeapon((Weapon) retrievedEquipment);
+ 45              character.equipWeapon((Weapon) retrievedEquipment);
46          } else {
47              Launcher.setEquippedArmor((Armor) retrievedEquipment);
+ 48              character.equipArmor((Armor) retrievedEquipment);
49          }
+ 50          Launcher.setMainCharacter(character);
+ 51          Launcher.refreshPane();
52          ImageView imgView = new ImageView();
    ...

```

The final functionality implemented in this case study is the management of the inventory list, which involves removing an item when it is equipped and returning a previously equipped item to the list. This implementation is presented in Script 1.41. The code in Lines 86-93 handles the removal of the equipped item from the `allEquipments` `ArrayList`. The implementation employs a technique analogous to a flag variable to locate the item to be removed. A position variable, `pos`, is initialized to -1. A for-loop (Line 86) iterates through the list to find a match for the equipped item. If a match is found, `pos` is updated with the item's index. After the loop, a non-negative value for `pos` indicates that the item was found and should be removed from the `ArrayList`, as it is now equipped.

Finally, Script 1.42 shows the registration of the `onEquipDone` event listener with the appropriate event sources (Lines 22-27). It should also be noted that corresponding accessor and mutator methods for the `allEquipments` collection are required in the `Launcher` class to facilitate this interaction.

The completion of this final implementation step allows the application to render the GUI as depicted in Figures 1.11, 1.13, and 1.15. At this stage, the weapon and armor equipping logic is fully operational, successfully propagating attribute values to dynamically modify the character's statistics. It should be noted, however, that several minor aspects of the equipment logic remain unimplemented. For instance, equipped items currently persist in their respective slots when a new character is generated, and the explicit functionality to unequip items has not yet been provided. The implementation of these remaining features, along with other system enhancements, is deferred to the exercise section at the conclusion of this chapter.

Script 1.41: controller/AllCustomHandler.java

```

1 //Imports are omitted
...
38 public static void onDragDropped(DragEvent event, Label lbl, StackPane imgGroup) {
39     boolean dragCompleted = false;
40     Dragboard dragboard = event.getDragboard();
+ 41     ArrayList<BasedEquipment> allEquipments = Launcher.getAllEquipments();
42     if(dragboard.hasContent(BasedEquipment.DATA_FORMAT)) {
43         BasedEquipment retrievedEquipment = (BasedEquipment)dragboard.getContent(
            BasedEquipment.DATA_FORMAT);
44         BasedCharacter character = Launcher.getMainCharacter();
45         if(retrievedEquipment.getClass().getSimpleName().equals("Weapon")) {
+ 46             if (Launcher.getEquippedWeapon() != null)
+ 47                 allEquipments.add(Launcher.getEquippedWeapon());
48                 Launcher.setEquippedWeapon((Weapon) retrievedEquipment);
49                 character.equipWeapon((Weapon) retrievedEquipment);
50         } else {
+ 51             if (Launcher.getEquippedArmor() != null)
+ 52                 allEquipments.add(Launcher.getEquippedArmor());
53                 Launcher.setEquippedArmor((Armor) retrievedEquipment);
54                 character.equipArmor((Armor) retrievedEquipment);
55         }
56         Launcher.setMainCharacter(character);
+ 57         Launcher.setAllEquipments(allEquipments);
58         Launcher.refreshPane();
...
+ 73 public static void onEquipDone(DragEvent event) {
+ 74     Dragboard dragboard = event.getDragboard();
+ 75     ArrayList<BasedEquipment> allEquipments = Launcher.getAllEquipments();
+ 76     BasedEquipment retrievedEquipment = (BasedEquipment)dragboard.getContent(
        BasedEquipment.DATA_FORMAT);
+ 77     int pos = -1;
+ 78     for(int i=0; i<allEquipments.size() ; i++) {
+ 79         if (allEquipments.get(i).getName().equals(retrievedEquipment.getName())) {
+ 80             pos = i;
+ 81         }
+ 82     }
+ 83     if (pos != -1) {
+ 84         allEquipments.remove(pos);
+ 85     }
+ 86     Launcher.setAllEquipments(allEquipments);
+ 87     Launcher.refreshPane();
+ 88 }
...

```

Script 1.42: view/InventoryPane.java

```

1  //Imports are omitted
2  public class InventoryPane extends ScrollPane {
    ...
21  private Pane getDetailsPane() {
22      Pane inventoryInfoPane = new HBox(10);
23      inventoryInfoPane.setBorder(null);
24      inventoryInfoPane.setPadding(new Insets(25, 25, 25, 25));
25      if (equipmentArray!=null) {
26          ImageView[] imageViewList = new ImageView[equipmentArray.size()];
27          for(int i=0 ; i< equipmentArray.size() ; i++) {
28              imageViewList[i] = new ImageView();
29              imageViewList[i].setImage(new Image(Launcher.class.getResource(
30                  equipmentArray.get(i).getImagepath()).toString()));
31              int finalI = i;
32              imageViewList[i].setOnDragDetected(new EventHandler<MouseEvent>() {
33                  public void handle(MouseEvent event) {
34                      onDragDetected(event, equipmentArray.get(finalI), imageViewList[
35                          finalI]);
36                  }
37              });
38              imageViewList[i].setOnDragDone(new EventHandler<DragEvent>() {
39                  @Override
40                  public void handle(DragEvent event) {
41                      onEquipDone(event);
42                  }
43              });
44          inventoryInfoPane.getChildren().addAll(imageViewList);
45      }
    ...

```

1.5 Exercise

1. Implement an additional item and add it to the item list. The image for this new item should be consistent in size with the existing item images.

2. Implement a new character subclass named `BattleMage`, with its `fullHp` and `basedPow` attributes initialized to 40. Associate a suitable image with this new character type, ensuring its dimensions are consistent with the other character images.
3. Modify the drag-and-drop logic so that if an item is dropped outside of a valid equipment slot, it is returned to the inventory list.
4. Incorporate a new button into the `EquipPane` that, when clicked, unequips all items from the character.
5. Implement functionality to automatically unequip all items whenever a new character is generated via the `Generate Character` button.
6. Extend the equipment logic based on the `DamageType` enum associated with each weapon. The implementation must enforce a rule where a character can only equip weapons that match their own `DamageType` (e.g., a `Physical` character can only equip `Physical` weapons). An exception to this rule is the `BattleMage`, which can equip weapons of any `DamageType` but is prohibited from equipping any `Armor`.

1.6 Discussion and Summary

This chapter has established the foundational principles of event-driven programming utilizing the JavaFX framework. The discussion progressed from the application of OOP concepts in GUI construction to the architectural design of the scene graph. Furthermore, it detailed the event-handling mechanism, comprising event sources, listeners, and handlers, and examined the primary categories of user input, specifically action, mouse, and keyboard events. Finally, the case study synthesized these theoretical concepts into a functional, MVC-structured application, demonstrating practical implementations of drag-and-drop functionality and attribute propagation.

Knowledge Outcomes

Upon the completion of this chapter, a student should demonstrate the ability to:

- Explain the fundamental role of OOP principles, particularly inheritance and polymorphism, in the architecture of modern GUI frameworks.
- Describe the JavaFX scene graph as a hierarchical node tree, explicitly identifying the functions of the Stage, Scene, and root nodes.
- Differentiate between foreground events, such as user-initiated clicks or keystrokes, and background events, including property-change listeners monitoring window dimensions.
- Explain the interaction within the event-source, listener, and handler triad, detailing how this mechanism governs the propagation of an `ActionEvent` from a GUI component to the underlying application logic.
- Outline the sequence of mouse events (specifically `onDragDetected`, `onDragOver`, `onDragDropped`, and `onDragDone`) required to implement drag-and-drop functionality within JavaFX.
- Evaluate the utilization of state variables and flags in coordinating application behavior and maintaining state consistency across multiple event handlers.

Points for Discussion

1. Analyze how the architecture of the case study application would require modification if a secondary view, such as a battle interface, needed to concurrently observe and reflect the state of the shared `Character` model.
2. Identify scenarios in which the use of anonymous inner classes for event handlers introduces maintenance challenges, and evaluate the alternative approaches provided by the Java language.

Suggested Extensions and Project Ideas

- Extend the case study inventory system with a trading interface that allows two characters to exchange equipment via drag-and-drop between separate panes.
- Implement a party management screen where multiple characters are displayed simultaneously, each with independent equipment slots, requiring coordinated state management across shared item pools.
- Build a tile-based map editor using a `GridPane`, where tiles are placed and rearranged through drag-and-drop, and keyboard shortcuts control tile rotation and deletion.
- Create a drawing application with shape tools (rectangle, circle, line) that respond to mouse press, drag, and release events, with an undo stack managed through state flags.

Building upon the event-handling paradigms established in this chapter, the subsequent chapter introduces exception handling and lambda expressions. These concepts provide a more concise syntax for the implementation of event handlers and are presented alongside an exploration of stream-based data processing techniques.

Progress Checklist

- ☐ Explain why OOP is essential for GUI development.
- ☐ Describe the JavaFX scene graph hierarchy and the roles of Stage, Scene, and Node.
- ☐ Distinguish between foreground events and background events in a GUI application.
- ☐ Explain the event-source, listener, and handler relationship in JavaFX event handling.
- ☐ Implement a `setOnAction` handler for a `Button` node.
- ☐ Implement mouse event handlers for drag-and-drop interactions between two nodes.
- ☐ Implement a keyboard event handler that responds to specific key inputs.
- ☐ Structure a JavaFX application using the MVC pattern with separate packages for each component.

References

- Arduino-er. (2015). Control Arduino + 8x8 LED matrix. Retrieved May, 2026, from <http://arduino-er.blogspot.com/2015/09/javajavafxjssc-control-arduino-8x8-led.html>
- OpenJFX. (2026a). JavaFX. Retrieved May, 2026, from <https://openjfx.io>
- OpenJFX. (2026b). Overview (JavaFX 26). Retrieved May, 2026, from <https://openjfx.io/javadoc/26/index.html>
- Oracle. (2026a). FXCollections (JavaFX 26). Retrieved May, 2026, from <https://docs.oracle.com/en/java/java-components/javafx/26/docs/javafx.base/javafx/collections/ \ FXCollections.html>
- Oracle. (2026b). Package java.awt. Retrieved May, 2026, from <https://docs.oracle.com/en/java/javase/26/docs/api/java.desktop/java/awt/package-summary.html>
- Oracle. (2026c). Package javax.swing. Retrieved May, 2026, from <https://docs.oracle.com/en/java/javase/26/docs/api/java.desktop/javax/swing/package-summary.html>
- Wong, S. (2017). Java GUI programming primer. Retrieved May, 2026, from <https://www.clear.rice.edu/comp310/JavaResources/GUI/>

CHAPTER 2

Exception Handling

Prerequisites

- Completion of Chapter 1, including familiarity with JavaFX application structure, event handling, and the Model-View-Controller (MVC) pattern.
- Understanding of the Java class hierarchy, inheritance, and method overriding.
- Experience writing and compiling multi-class Java projects within an Integrated Development Environment (IDE).
- Basic familiarity with control flow statements (if-else, loops) and method signatures.

Expected Outcomes

- Distinguish between checked and unchecked exceptions and explain when each category applies.
- Implement try-catch-finally blocks to handle exceptions and maintain application stability.
- Define and throw custom exception classes tailored to application-specific error conditions.
- Write lambda expressions as concise replacements for anonymous inner classes in event handling and collection processing.
- Construct stream pipelines using intermediate operations (map, filter) and terminal operations (reduce, collect) to process data declaratively.

A well-known heuristic principle, commonly referred to as Murphy's law (Bloch, 2003), states that *Anything that can go wrong will go wrong*. In the context of software applications, this concept manifests as a wide range of potential runtime problems, such as network disruptions, file access errors, or invalid computational inputs. Such scenarios can lead to critical system failures, exemplified by the system-halt errors in early versions of the Windows operating system. To address this class of problems, modern programming languages, including Java, incorporate the concept of exception handling, which provides a structured mechanism for managing and mitigating such failures.

It is important to note that exception handling is not a universal solution capable of preventing all forms of program termination. Rather, it provides a structured mechanism that enables developers to anticipate, manage, and potentially recover from anomalous conditions that may arise during program execution. The exception-handling mechanism in Java allows developers to define alternative execution paths within a method, which are invoked when an anomalous condition, known as an exception, occurs. Such exceptions can originate from various sources, including invalid arithmetic operations (e.g., division by zero) or resource

access issues. This capability facilitates the graceful management of exceptional circumstances and can prevent abrupt program termination.

2.1 Categorization of Exceptions

Exceptions in Java are classified into two distinct types: checked and unchecked. The fundamental distinction between these two types is the point at which they are verified: checked exceptions are verified by the compiler at compile time, whereas unchecked exceptions are verified only at runtime.

2.1.1 Checked exceptions

As their name implies, checked exceptions are subject to rigorous examination by the Java compiler. If a method contains code that could potentially throw a checked exception, the Java compiler enforces a requirement that the method must either handle the exception internally or declare that it may throw the exception. This compile-time enforcement ensures that such exceptions are explicitly addressed by the developer.

Common examples of checked exceptions include `IOException` (related to input and output operations) and `SQLException` (related to database interactions). These exceptions typically represent recoverable error conditions, and the requirement for their explicit declaration encourages the implementation of robust error-handling mechanisms designed to restore the application to a stable state.

2.1.2 Unchecked exceptions

Conversely, unchecked exceptions are not subject to verification by the compiler. Their occurrence is contingent upon specific runtime conditions or data values, and thus cannot be reliably predicted during compilation. Examples include `ArrayIndexOutOfBoundsException` and `ArithmeticException` (e.g., division by zero), which are determined by the specific values and operations encountered at runtime.

Unchecked exceptions typically represent logical programming errors or other conditions from which recovery may not be feasible. The `RuntimeException` class serves as the base class for all unchecked exceptions, encompassing a wide range of

issues such as `ArrayIndexOutOfBoundsException`, `NullPointerException`, and `IllegalArgumentException`. Although these exceptions are not enforced at compile time and may seem improbable during development, the principle expressed by Murphy's Law remains relevant: *Anything that can go wrong will go wrong*. Therefore, despite the absence of compiler enforcement, diligent developers must anticipate and handle such exceptions to ensure application robustness and reliability.

2.2 Exception Handling

2.2.1 Catching the exceptions

The fundamental logic of exception handling is analogous to a specialized form of an if statement, a control flow mechanism that facilitates the conditional logic illustrated in the comments of Script 2.1. This script further demonstrates the formal implementation of this mechanism, beginning on Line 8.

This control flow statement, formally known as a try-catch block, provides a structured approach for managing exceptional conditions. Without such a mechanism, the occurrence of an unhandled exception would lead to an application crash, as the runtime environment would lack instructions for error recovery. For instance, the system-halt errors common in older operating systems arise from an inability to resolve a critical, unhandled problem. Although an application cannot predict the occurrence of all such issues, the developer is responsible for implementing logic that defines the application's response to specific categories of exceptions.

Try-catch block

Prior to examining the design of exception-handling methods, an understanding of the structure of an `Exception` is beneficial. The Java language provides a comprehensive hierarchy of predefined exception types to represent various potential issues, such as those related to input/output (I/O), memory, and arithmetic operations. Additionally, Java allows developers to create custom exceptions, although

Script 2.1: Conceptual Overview of Exception Handling**ExceptionOverview.java**

```
+ 1  /*
+ 2      if(is no problem occurs) {
+ 3          normal flow
+ 4      } else {
+ 5          exception flow
+ 6      }
+ 7  */
+ 8  public class Launcher {
+ 9      public static void main(String[] args) {
+10          try {
+11              normalFlow();
+12          } catch (Exception e) {
+13              exceptionFlow();
+14          }
+15      }
+16  }
```

this practice is less common, given the extensive range of standard exceptions available.

The fundamental principle of exception handling is the detection and appropriate management of anomalous conditions upon their occurrence. Users typically expect applications to function seamlessly, even when encountering unexpected operational issues. For instance, in video games, frequent application crashes that force players to resume from a distant save point can significantly degrade the user experience. Therefore, the successful detection and handling of exceptions are crucial for enabling graceful application recovery, thereby preventing data loss and maintaining a positive user experience.

Scripts 2.2, 2.3, and 2.4 present three classes that constitute a concrete example of a method call stack. The program reads content from a text file named `dummy.txt`, located at a relative path within the project's resources folder. The execution of this program follows these steps:

1. The `Launcher.main` method is invoked, which subsequently invokes the constructor of the `Document` class (Line 3 of Script 2.2).

2. Within the constructor of the Document class, the specified file path is passed to the FileReader class (Line 4 of Script 2.3).
3. The retrieve() method in the FileReader class reads the content from the specified file path. It utilizes a Scanner to read the file line-by-line and subsequently returns the full content to the calling method (Line 4 of Script 2.3).
4. If the retrieval is successful, the content returned by the FileReader.retrieve() method is displayed on the standard output by the code in Line 4 of Script 2.2.

Script 2.2: Call Stack Example: The Launcher Class

Launcher.java

```
+ 1 public class Launcher {  
+ 2     public static void main(String[] args) {  
+ 3         Document d = new Document();  
+ 4         System.out.println(d.getContent());  
+ 5     }  
+ 6 }
```

Script 2.3: Call Stack Example: The Document Class

Document.java

```
+ 1 public class Document {  
+ 2     String content = "";  
+ 3     public Document() {  
+ 4         content = FileReader.retrieve("dummy.txt");  
+ 5     }  
+ 6     public String getContent() {  
+ 7         return content;  
+ 8     }  
+ 9 }
```

Script 2.4: Call Stack Example: The FileReader Class

FileReader.java

```
+ 1 public class FileReader {
+ 2     public static String retrieve(String filepath) {
+ 3         String data = "";
+ 4         InputStream inputStream = getClass().getResourceAsStream(filepath);
+ 5         Scanner myReader = new Scanner(inputStream);
+ 6         while (myReader.hasNextLine()) {
+ 7             data += myReader.nextLine();
+ 8         }
+ 9         myReader.close();
+10         return data;
+11     }
+12 }
```

However, if the file `dummy.txt` is not found, an exception of type `FileNotFoundException` (a subclass of `IOException`) will be thrown. This event triggers a search for a suitable handler, which involves traversing the call stack in reverse order, starting from the point of failure and moving back through the chain of method invocations. If no appropriate handler is found within the current method, the search continues up the call stack. If no handler is located anywhere in the call stack, the Java Virtual Machine will terminate the program and report an unhandled exception. The provided scripts do not contain any explicit exception-handling mechanisms. Consequently, the absence of the specified file will cause the program to terminate due to this unhandled `FileNotFoundException`.

The process of handling an exception involves two primary components: the `try` block and the `catch` block.

- **try block:** This statement block, as shown in Script 2.1, encloses code that may potentially throw an exception.
- **catch block:** This statement block, which must immediately follow a `try` block, contains the handler logic for a specific type of exception.

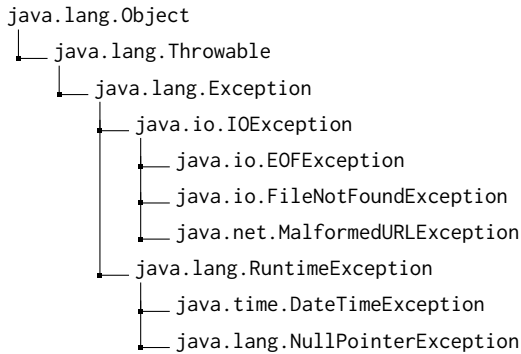
Script 2.5 provides an example in which an exception handler is added to the `FileReader` class from the previous example. The code enclosed within the try-catch structure (Lines 4-13) prevents abrupt program termination in the event of an exception. Instead, it allows for a controlled response, in this case, by notifying the user of the error and executing the logic defined within the catch block (Lines 11-13).

Script 2.5: Adding a try-catch Block**FileReader.java**

```
1 public class FileReader {
2     public String retrieve(String filepath) {
3         String data = "";
+ 4         try {
5             InputStream inputStream = getClass().getResourceAsStream(filepath);
6             Scanner myReader = new Scanner(inputStream);
7             while (myReader.hasNextLine()) {
8                 data += myReader.nextLine();
9             }
10            myReader.close();
+ 11        } catch (Exception e){
+ 12            System.err.println(e);
+ 13        }
14        return data;
15    }
16 }
```

However, in practice, it is a recommended practice to handle exceptions using the most specific exception type applicable, rather than relying on a generic `Exception` handler. For instance, in file operations, the exceptions that are likely to be thrown are related to I/O. The most relevant exception class for this domain is `IOException`.

To determine the appropriate exception type, the JavaDoc for the `Exception` class (Oracle, 2026d) serves as the definitive reference. It presents the hierarchy of exception classes, illustrating the relationships between different types, such as:



Script 2.6 demonstrates an example of replacing the generic Exception catch block with a more specific IOException handler. In practical scenarios, it is advisable to handle different types of exceptions separately, tailoring recovery actions to each specific issue.

Script 2.6: Handling a Specific Exception Type (IOException)**FileReader.java**

```
1 public class FileReader {
2     public String retrieve(String filepath) {
3         String data = "";
4         try {
5             ...
- 11     } catch (Exception e){
- 12         System.err.println(e);
+ 13     } catch (IOException e){
+ 14         System.err.println("Error due to I/O: " + e.getMessage());
15     }
16     return data;
17 }
18 }
```


This approach offers several benefits:

- **Targeted recovery:** Allows for the implementation of distinct recovery actions tailored to each specific exception type.
- **Code clarity:** Improves code readability and maintainability by grouping related error-handling logic.
- **Simplified debugging:** Facilitates easier troubleshooting and debugging by isolating error-handling logic for different failure modes.
- **Architectural flexibility:** Provides the flexibility to handle different exception types at different levels of the application architecture.

For example, in the context of file operations, I/O-related issues can be decomposed into more specific categories, such as:

- A failure to locate the specified file, represented by `FileNotFoundException`.
- An unexpected end-of-file condition, represented by an `EOFException`.

Both of these exception types are subclasses of `IOException`. Script 2.7 demonstrates how these specific exceptions can be handled in separate catch blocks (Lines 13-16), thereby allowing for tailored responses based on the specific I/O error encountered.

The finally block

A `finally` block serves as a guaranteed execution block that follows a `try` block and any associated catch blocks. Its inclusion is optional, providing developers with flexibility in their implementation. When a `finally` block is present, the code within it is guaranteed to execute regardless of whether an exception is thrown or caught within the `try` block. A `try` statement can be associated with at most one `finally` block.

Although its utility may not be immediately apparent, the `finally` block serves a crucial purpose in robust program design. It provides a designated location for

Script 2.7: Handling Multiple, Specific Exception Subtypes

FileReader.java

```
1 public class FileReader {
2     public String retrieve(String filepath) {
3         String data = "";
4         try {
5             ...
10         myReader.close();
- 11     } catch (IOException e){
- 12         System.err.println("Error due to I/O: " + e.getMessage());
+ 13     } catch (FileNotFoundException e){
+ 14         System.err.println("File not found: " + e.getMessage());
+ 15     } catch (EOFException e){
+ 16         System.err.println("The end of the file is reached unexpectedly: " + e.
            getMessage());
17     }
18     return data;
19 }
20 }
```

resource-management code, such as cleanup operations, that must execute under all possible exit conditions. This design guarantees that essential operations, such as closing files or releasing system resources, are not bypassed by control-flow statements like `continue`, `break`, or an unhandled exception. It is therefore a recommended practice to utilize a `finally` block to ensure the deterministic execution of such cleanup code. When used in conjunction with `catch` blocks, the `finally` block is placed after all `catch` blocks. Script 2.8 demonstrates a modification of Script 2.5 that incorporates a `finally` block.

2.2.2 Throwing the exceptions

An alternative to handling an exception within the method where it occurs is to postpone its handling by propagating the exception up the call stack to the calling method. This action, referred to as throwing an exception, delegates the responsibility of handling the exception to the caller. For checked exceptions, this propaga-

Script 2.8: Implementation of a try-catch-finally Block**FileReader.java**

```

1 public class FileReader {
2     public String retrieve(String filepath) {
3         String data = "";
4         try {
5             Scanner myReader = new Scanner(getClass().getResourceAsStream(filepath));
6             while (myReader.hasNextLine()) {
7                 data += myReader.nextLine();
8             }
9             myReader.close();
10        } catch (FileNotFoundException e){
11            System.err.println("File not found: " + e.getMessage());
12        } catch (EOFException e){
13            System.err.println("The end of the file is reached unexpectedly: " + e.
14                getMessage());
15        } finally {
16            System.out.println("Read file done.");
17            myReader.close();
18        }
19        return data;
20    }
21 }

```

tion must be explicitly declared using the `throws` keyword in the method signature. For unchecked exceptions, a `throws` declaration is not required, as they are implicitly propagated by the runtime system upon encountering an error condition.

In the context of the previous example (Script 2.8), the `try-catch` block could be omitted from the `retrieve()` method of the `FileReader` class. This would allow the `retrieve()` method to propagate any thrown exceptions to the `init()` method of the `Document` class, which is its caller. This requires the following modifications:

- **For checked exceptions**, such as `FileNotFoundException` and `EOFException`, the following modifications are necessary:
 1. The signature of the `retrieve()` method in the `FileReader` class must be modified to include a `throws` clause specifying `FileNotFoundException`-

ception and EOFException.

2. The try-catch block is relocated from the `FileReader` class to the calling method, which is the `init()` method in the `Document` class.
- **For unchecked exceptions**, such as a potential `NullPointerException` the following steps are taken:
 1. The try-catch block is relocated to the calling method, the `init()` method of the `Document` class. An explicit throws clause is not required in the method signature for unchecked exceptions.
 2. Additional catch blocks for `RuntimeException` or any of its anticipated subclasses can be added within the `init()` method.

These modifications are presented in Scripts 2.9 and 2.10. By propagating an exception, the responsibility for handling it shifts to the calling method, which affords greater flexibility in the overall exception management strategy. In Script 2.9, the try-catch block is removed from the `FileReader` class and relocated to the `Document` class in Script 2.10. The following steps outline the revised exception-handling process:

1. Upon encountering an exception within the `FileReader.retrieve()` method, it propagates to the calling method, in this case, the `init()` method of the `Document` class, at the point of invocation (`fr.retrieve("dummy.txt")`).
2. For a checked exception, the `throws` keyword is mandatory in the `retrieve()` method's signature to declare this propagation. For an unchecked exception, this declaration is not required.
3. Because the point of invocation in the calling method is now enclosed within a try-catch block, the exception can be handled at this higher level.
4. Alternatively, the `Document` method could also declare a `throws` clause, re-throwing the exception to propagate it further up the call stack.
5. If an exception propagates to the main method (the top-level entry point in this example) and is not handled, the application will terminate.

Script 2.9: Propagating an Exception from the FileReader Class `FileReader.java`

```
1 public class FileReader {
- 2     public String retrieve(String filepath) {
+ 3     public String retrieve(String filepath) throws FileNotFoundException,
        EOFException {
4         String data = "";
- 5         try {
6             Scanner myReader = new Scanner(getClass().getResourceAsStream(filepath));
7             while (myReader.hasNextLine()) {
8                 data += myReader.nextLine();
9             }
+ 10        myReader.close();
- 11    } catch (FileNotFoundException e){
- 12        System.err.println("File not found: " + e.getMessage());
- 13    } catch (EOFException e){
- 14        System.err.println("The end of the file is reached unexpectedly: " + e.
            getMessage());
- 15    } finally {
- 16        System.out.println("Read file done.");
- 17        myReader.close();
- 18    }
19    return data;
20 }
21 }
```

The handling of exceptions at an appropriate level in the call stack is therefore a crucial aspect of robust application design. In the absence of a suitable handler in the main method, any unhandled exception will result in the termination of the application.

Script 2.10: Handling a Propagated Exception in the Document Class `Document.java`

```
1 public class Document {
2     String content = "Default content";
3     public Document() {
+ 4         try {
5             content = FileReader.retrieve("dummy.txt");
+ 6         } catch (FileNotFoundException e){
+ 7             System.err.println("File not found: " + e.getMessage());
+ 8         } catch (EOFException e){
+ 9             System.err.println("Unexpected end of file: " + e.getMessage());
+ 10        } catch (RuntimeException e){
+ 11            System.err.println("A runtime error occurred: " + e.getMessage());
+ 12        } catch (IOException e) {
+ 13            System.err.println("I/O error: " + e.getMessage());
+ 14        } catch (Exception e) {
+ 15            System.err.println("An unexpected error occurred: " + e.getMessage());
+ 16        }
17    }
18    public String getContent() {
19        return content;
20    }
21 }
```

2.2.3 Customized exception

In scenarios where a specific error condition is not represented by any of the pre-defined exception types in the Java Application Programming Interface (API) (Oracle, 2026d), developers have the capability to create custom exception classes. For instance, if an application must not process empty files and this case should be handled as an exception, a design decision must be made as to whether the custom exception should be checked or unchecked. This decision involves choosing a suitable superclass (e.g., `IOException` or `RuntimeException`) and is guided by the following considerations:

- **Nature of the Exception:**

- **Checked (IOException):** If an empty file is considered an exceptional but potentially recoverable condition, defining it as a checked exception ensures that callers are forced by the compiler to either handle it or declare it.
- **Unchecked (RuntimeException):** If an empty file indicates a logical programming error or a condition that does not require mandatory handling by the caller, defining it as an unchecked exception simplifies the calling code.

- **Caller Awareness and Handling:**

- **Checked:** This approach forces caller awareness, which is beneficial when the calling context can meaningfully recover from the error, such as by providing a default file, prompting the user for action, or logging a specific message.
- **Unchecked:** This approach is suitable when the responsibility for handling the error should not be imposed on the caller, often because the error indicates a bug that should be fixed rather than handled at runtime.

For the scenario described, a custom exception named `EmptyFileException` is created. Because an empty file represents a recoverable, I/O-related error condition, extending `IOException` is the preferable design choice. This approach ensures that the calling method is made aware of the potential issue at compile time and can implement an appropriate response, such as providing a default file, prompting the user for a different file, or logging the error.

Script 2.11 illustrates the implementation of the `EmptyFileException` class, which demonstrates its derivation from `IOException` in the class signature. Script 2.12 modifies the `FileReader` class to incorporate the logic for throwing an `EmptyFileException` under the specified condition (Line 9). The method signature for `retrieve()` is updated to declare all three potential exception types in its throws clause.

The handling of this custom exception is implemented within a catch block in the Document class, as shown in Lines 6-7 of Script 2.13. If the dummy.txt file is empty, execution of Launch.java produces a message on standard error: *File is empty: This application does not accept empty files. Set the content as 'Default content'*. Concurrently, the message *Default content* is displayed on standard output.

Script 2.11: A Custom Exception Class**EmptyFileException.java**

```
+ 1 public class EmptyFileException extends IOException {
+ 2     public EmptyFileException(String message) {
+ 3         super(message);
+ 4     }
+ 5 }
```

Script 2.12: Throwing a Custom Exception in FileReader.java**FileReader.java**

```
1 public class FileReader {
- 2     public String retrieve(String filepath) {
+ 3     public String retrieve(String filepath) throws FileNotFoundException,
        EOFException, EmptyFileException {
4         String data = "";
5         Scanner myReader = new Scanner(getClass().getResourceAsStream(filepath));
6         while (myReader.hasNextLine()) {
7             data += myReader.nextLine();
8         }
+ 9         if (data.isEmpty()) {
+ 10             throw new EmptyFileException("This application does not accept empty
                files. Set the content as 'Default content'.");
+ 11         }
12         myReader.close();
13         return data;
14     }
15 }
```


Script 2.13: Handling a Custom Exception in Document.java**Document.java**

```
1 public class Document {
2     String content = "Default content";
3     public Document() {
4         try {
5             content = FileReader.retrieve("dummy.txt");
+ 6         } catch (EmptyFileException e) {
+ 7             System.err.println("File is empty: " + e.getMessage());
8         } catch (FileNotFoundException e){
9             System.err.println("File not found: " + e.getMessage());
10        } catch (EOFException e){
11            System.err.println("Unexpected end of file: " + e.getMessage());
12        } catch (RuntimeException e){
13            System.err.println("A runtime error occurred: " + e.getMessage());
14        } catch (IOException e) {
15            System.err.println("I/O error: " + e.getMessage());
16        } catch (Exception e) {
17            System.err.println("An unexpected error occurred: " + e.getMessage());
            ...
        }
```

2.3 Extra Topics: Lambda Expressions and Streams in Java

Introduced in Java 8, lambda expressions provide a concise syntax for representing anonymous functions. This feature is particularly beneficial when implementing functional interfaces, which are interfaces that contain a single abstract method. Lambda expressions facilitate a more functional programming style by emphasizing a function's behavior rather than its formal declaration.

2.3.1 What is a Lambda expression?

Lambda calculus, which serves as a theoretical foundation for functional programming, is a system where all computations are expressed as functions. Java's lambda expressions adopt this principle, allowing for the definition of functions without requiring an explicit name. For instance, the function $\text{square_sum}(x, y) = x^2 + y^2$

can be expressed concisely as $(x, y) \rightarrow x^2 + y^2$ in contexts where an explicit function name is unnecessary.

Prior to the introduction of lambda expressions in Java 8, anonymous classes were the primary mechanism for the inline implementation of interfaces, a technique frequently used in event handling. Script 2.14 provides an example from the case study in Chapter 1, where an anonymous class is used to define the event handler for an `ImageView` object. The `setOnDragDone()` method requires an object that implements the `EventHandler` interface; this requirement is satisfied by creating an anonymous class that overrides the `handle()` method.

Although functional, the syntax for anonymous classes can be verbose, particularly for simple, single-method implementations where a formal class structure is unnecessarily cumbersome. In response to this, Java 8 introduced lambda expressions as a more concise and expressive alternative. Script 2.15 demonstrates the refactoring of the code from Script 2.14 using a lambda expression. As demonstrated, the lambda expression achieves the same functionality as the anonymous class but with a significantly more compact syntax. The compiler infers that the provided lambda expression is an implementation of the `handle()` method, allowing the event parameter to be passed directly to the `onEquipDone()` method.

Script 2.14: Event Handler Implemented Using an Anonymous Class

```
+ 1 imageViewList[i].setOnDragDone(new EventHandler<DragEvent>() {  
+ 2     @Override  
+ 3     public void handle(DragEvent event) {  
+ 4         onEquipDone(event);  
+ 5     }  
+ 6 });
```

Script 2.15: Event Handler Implemented Using a Lambda Expression

```
+ 1 imageViewList[i].setOnDragDone(event -> onEquipDone(event));
```

2.3.2 Illustrative example: String sorting

The evolution from anonymous classes to lambda expressions can be further illustrated through a string-sorting example. Scripts 2.16, 2.17, and 2.18 each achieve the same functional outcome, which is the alphabetical sorting of strings, but they do so using progressively more modern implementation techniques.

Script 2.16 presents a solution common prior to Java 1.1, which requires the implementation of an explicit, custom `Comparator` class. An instance of this class is then passed to the `Arrays.sort()` method. Script 2.17 demonstrates the approach common between Java 1.1 and Java 8, which utilizes an anonymous class through a Class Instance Creation Expression (CICE). This technique, analogous to the `EventHandler` implementation in Script 2.14, involves defining the comparison logic within the body of an anonymous class. Finally, Script 2.18 presents the modern, concise alternative, which refactors the anonymous class into a lambda expression. The lambda expression provides a more streamlined and readable syntax by directly representing the comparison logic as an anonymous function, thereby expressing the core functionality without the syntactic overhead of a full class definition.

Script 2.16: String Sorting Using a Custom Comparator Class

```
+ 1 public class StringLengthComparator implements Comparator {
+ 2     public static final StringLengthComparator INSTANCE = new
        StringLengthComparator();
+ 3     public int compare(Object o1, Object o2) {
+ 4         String s1 = (String) o1, s2 = (String) o2;
+ 5         return s1.length() - s2.length();
+ 6     }
+ 7
+ 8     public static void main(String[] args) {
+ 9         String[] arr = {"ABC", "AB", "A", "ABCD"};
+10         Arrays.sort(arr, StringLengthComparator.INSTANCE);
+11         System.out.println(Arrays.toString(arr));
+12     }
+13 }
```

Script 2.17: String Sorting Using an Anonymous Class (CICE)

```
+ 1 public static void main(String[] args) {  
+ 2     String[] arr = {"ABC", "AB", "A", "ABCD"};  
+ 3     Arrays.sort(arr, new Comparator() {  
+ 4         @Override  
+ 5         public int compare(Object o1, Object o2) {  
+ 6             String s1 = (String) o1, s2 = (String) o2;  
+ 7             return s1.length() - s2.length();  
+ 8         }  
+ 9     });  
+ 10     System.out.println(Arrays.toString(arr));  
+ 11 }
```

Script 2.18: String Sorting Using a Lambda Expression

```
+ 1 public static void main(String[] args) {  
+ 2     String[] arr = {"ABC", "AB", "A", "ABCD"};  
+ 3     Arrays.sort(arr, (s1, s2) -> s1.length() - s2.length());  
+ 4     System.out.println(Arrays.toString(arr));  
+ 5 }
```

Lambda expressions, therefore, represent a powerful tool for writing concise and expressive code, particularly in scenarios that require the implementation of single-method interfaces. Their introduction in Java 8 significantly enhanced the capabilities of the Java language and encouraged the adoption of a more functional programming style.

2.3.3 Lambda syntax

A lambda expression conforms to a specific syntax composed of three distinct elements:

1. **Parameters:** This component defines the input parameters for the anonymous function. For multiple parameters, the list is enclosed in parentheses

with parameters separated by commas. Parentheses are optional for a single parameter. For a function with no parameters, empty parentheses () are mandatory. For example:

- `x -> x + 1` (single parameter without parentheses)
 - `(x) -> x + 1` (single parameter with parentheses)
 - `() -> System.out.println("Hello")` (no parameters)
2. **Arrow (->):** The arrow token (->) serves as a delimiter that separates the parameter list from the expression's body. It signifies a mapping from the input parameters on the left to the implementation on the right.
 3. **Body:** This component contains the implementation logic of the anonymous function. It can be one of two forms:

Expression Body: A single expression whose result is implicitly returned by the function. For example: `x -> x + 1`.

Block Body: A block of statements enclosed in curly braces ({}). If the function returns a value, an explicit return statement is required within the block.

Table 2.1 provides further examples, illustrating the syntactic flexibility of lambda expressions.

2.3.4 Utilizing Lambda expressions

Lambda expressions are frequently employed in event handling, where they often replace the traditional anonymous class syntax for implementing the `handle()` method. Additionally, they are widely utilized to provide implementations for functional interfaces from standard libraries, such as `Comparator` and `Runnable`. This application results in more concise code, which enhances readability by reducing syntactic complexity and boilerplate.

Modern IDEs, such as IntelliJ IDEA, often provide automated suggestions to refactor a Class Instance Creation Expression (CICE) into a lambda expression

Table 2.1: Syntactic Forms of Java Lambda Expressions with Examples

Syntax	Example
parameter -> expression	<code>x -> x + 1</code> <i>// A single-parameter function that returns the result of an expression.</i>
parameter -> block	<code>s -> System.out.println(s);</code> <i>// A single-parameter function with a block body that performs an action.</i>
parameters -> expression	<code>(x, y) -> x + y</code> <i>// A multi-parameter function that returns the result of an expression.</i>
parameters -> block	<code>(s1, s2) -> System.out.println(s1 + `, ' + s2);</code> <i>// A multi-parameter function with a block body.</i>
parameter(s) with types -> expression	<code>(double x, double y) -> Math.sqrt(x * x + y * y)</code> <i>// A multi-parameter function with explicit type declarations.</i>
parameter(s) with type -> block	<code>(List<?> list) -> Arrays.sort(list.toArray());</code> <i>// A function with an explicit generic type parameter and a void block body.</i>

where applicable. As depicted in Figure 2.1, these suggestions are typically indicated by an icon (e.g., a lightbulb). Accepting this suggestion automatically performs the code transformation.

**Figure 2.1: IDE Suggestion to Convert an Anonymous Class to a Lambda Expression**

2.3.5 Introducing streams

The Stream API, introduced in Java 8, provides a mechanism for processing sequences of objects. A stream is derived from a source, such as a collection, an array, or an I/O channel, and supports aggregate operations for efficient bulk data processing. A standard stream pipeline is composed of three distinct stages:

1. **Source:** A data source (e.g., a Collection) from which the stream is generated.
2. **Intermediate Operations:** A chain of zero or more operations that transform or filter the elements of the stream (e.g., `filter()`, `map()`).
3. **Terminal Operation:** A single operation that produces a final result or a side effect and consumes the stream (e.g., `forEach()`, `collect()`).

Stream sources

Script 2.19 presents three examples that demonstrate the creation of a Stream from different data sources. In Line 3, a stream is constructed from a List of strings. In Line 4, a stream is generated from an array of Integer objects. Finally, in Line 5, a stream is created from a list of characters. In each of these examples, the `forEach()` terminal operation is used to print every element of the resulting stream to the console.

Script 2.19: Demonstration of Various Stream Sources

```
+ 1 public static void main(String[] args) {  
+ 2     List<String> stringList = Arrays.asList("Hello", "World!", "How", "Are", "You");  
+ 3     stringList.stream().forEach(a -> System.out.println(a));  
+ 4     IntStream.range(0, 10).forEach(a->System.out.println(a));  
+ 5     "Hello world!".chars().forEach(x -> System.out.print((char) x));  
+ 6 }
```

Intermediate stream operations

Intermediate operations provide a versatile toolkit for transforming and filtering a stream's elements before a terminal operation is applied to produce a final result. Common intermediate operations include the following:

- **Mapping:** Applies a function to each element of the stream, transforming it into a new element.

- **Filtering:** Selects only the elements from the stream that satisfy a given predicate.
- **Distinct:** Returns a stream consisting of the unique elements of the original stream.
- **Sorting:** Arranges the stream's elements according to their natural order or a specified Comparator.

The `map()` method applies a provided function to each element, transforming it into a new element. The method's argument is typically a lambda expression that defines this transformation. Script 2.20 demonstrates this operation, where the first character of each string in a list is extracted to create a new stream of characters.

The `filter()` method selects elements from a stream that satisfy a specific condition, or predicate. Its argument is a lambda expression that evaluates to a boolean value. Script 2.21 provides an example where only strings with a length greater than three characters are retained in the resulting stream.

The `distinct()` operation returns a stream containing only the unique elements of the source stream, based on the `equals()` method for comparison. Script 2.22 provides an example in which duplicate characters are eliminated from a list.

The `sorted()` operation returns a stream whose elements are arranged according to their natural order or a provided Comparator. Script 2.23 demonstrates this by arranging a stream of characters into alphabetical order.

Script 2.20: Using the `map()` Operation to Transform Stream Elements

```
+ 1 public static void main(String[] args) {  
+ 2     List<String> stringList = Arrays.asList("Hello", "World!", "How", "Are", "You");  
+ 3     List<String> mappedList = stringList.stream()  
+ 4         .map(s -> s.substring(0, 1))  
+ 5         .collect(Collectors.toList());  
+ 6     System.out.println(mappedList);  
+ 7 }
```


Script 2.21: Using the filter() Operation to Transform Stream Elements

```
+ 1 public static void main(String[] args) {  
+ 2     List<String> stringList = Arrays.asList("Hello", "World!", "How", "Are", "You");  
+ 3     List<String> filteredList = stringList.stream()  
+ 4         .filter(s -> s.length() > 3)  
+ 5         .collect(Collectors.toList());  
+ 6     System.out.println(filteredList);  
+ 7 }
```

Script 2.22: Using the distinct() Operation to Transform Stream Elements

```
+ 1 public static void main(String[] args) {  
+ 2     List<String> stringList = Arrays.asList("Hello", "World!", "How", "Are", "You");  
+ 3     List<String> sortedList = stringList.stream()  
+ 4         .map(s -> s.substring(0, 1))  
+ 5         .distinct()  
+ 6         .collect(Collectors.toList());  
+ 7     System.out.println(sortedList);  
+ 8 }
```

Script 2.23: Using the sorted() Operation to Transform Stream Elements

```
+ 1 public static void main(String[] args) {  
+ 2     List<String> stringList = Arrays.asList("Hello", "World!", "How", "Are", "You");  
+ 3     List<String> sortedList = stringList.stream()  
+ 4         .map(s -> s.substring(0, 1))  
+ 5         .sorted()  
+ 6         .collect(Collectors.toList());  
+ 7     System.out.println(sortedList);  
+ 8 }
```

Stream parallelism

Parallelism is a technique applicable to stream operations that can significantly enhance performance. The overall performance of a stream operation is influenced by several factors, including the nature of the operations, the size of the data set, and whether parallelism is employed. Script 2.24 presents three functionally equivalent operations that calculate the sum of base-10 logarithms for numbers from 1 to 10^8 .

Script 2.24: Performance Comparison of Sequential and Parallel Stream Operations

```
+ 1 public static void main(String[] args) {
+ 2     long tStart = System.currentTimeMillis();
+ 3     double sum = 0;
+ 4     for(long j=1 ; j<1e8 ; j++)
+ 5         sum += Math.log10(j);
+ 6     long tEnd = System.currentTimeMillis();
+ 7     long elapsedSeconds = tEnd - tStart;
+ 8     System.out.println("sum: " + sum + "\ntime: " + elapsedSeconds + " ms");
+ 9
+10     tStart = System.currentTimeMillis();
+11     sum = LongStream.range(1, (long) 1e8)
+12         .mapToDouble(x -> Math.log10(x))
+13         .sum();
+14     tEnd = System.currentTimeMillis();
+15     elapsedSeconds = tEnd - tStart;
+16     System.out.println("sum: " + sum + "\ntime: " + elapsedSeconds + " ms");
+17
+18     tStart = System.currentTimeMillis();
+19     sum = LongStream.range(1, (long) 1e8)
+20         .parallel()
+21         .mapToDouble(x -> Math.log10(x))
+22         .sum();
+23     tEnd = System.currentTimeMillis();
+24     elapsedSeconds = tEnd - tStart;
+25     System.out.println("sum: " + sum + "\ntime: " + elapsedSeconds + " ms");
+26 }
```

Lines 2-8 of the script compute the sum using a traditional for-loop that iterates the variable j from 1 to 10^8 , where the variable `sum` accumulates the total. In contrast, the code in Lines 10-15 performs the same calculation using a sequential stream, which is created from a range of long values in Line 11. A `mapToDouble()` operation is then applied (Line 12), which uses a lambda expression (`Math::log10`) to transform each long element into a double representing its base-10 logarithm. Subsequently, the `sum()` terminal operation (Line 13) aggregates these double values to produce the final result.

A significant advantage of the Stream API is its built-in support for parallelization. By incorporating the `parallel()` intermediate operation, the stream processing can be partitioned into subtasks and distributed across multiple CPU cores, potentially leading to substantial performance gains. This is achieved by invoking the `.parallel()` method on the stream. In the provided example (Script 2.24), it is added at Line 20, while the remaining operations are identical to the sequential stream implementation.

In this specific example, execution on a 2020 MacBook Air with an M1 CPU yielded an average execution time of 3.31 seconds over three runs for the for-loop implementation, while the sequential stream implementation averaged 3.78 seconds. This performance difference is attributable to the overhead associated with creating and managing the stream pipeline for the sequential operation. However, when parallelized, the stream's average execution time decreased to 1.72 seconds, which demonstrates a substantial performance improvement over both the traditional for-loop and the sequential stream. This improvement is a direct result of utilizing multiple processing cores, enabling the computation to be divided into smaller, independent tasks and executed concurrently. Although there is an inherent overhead in managing parallelism, the ability to leverage multiple computational units often outweighs this cost, resulting in faster overall execution.

Parallel streams are most effective when the operations performed on the stream elements are independent and stateless, meaning the outcome for one element does not affect another. Additionally, parallelism typically yields greater benefits for large datasets, as the overhead of parallelization becomes less significant relative to the computational gains. Furthermore, the underlying data source must be efficiently splittable into sub-problems for parallel execution. It is crucial to benchmark performance both with and without parallelization to verify that a net per-

formance gain is achieved. In some cases, particularly for small datasets or operations with inherent sequential dependencies, the overhead of parallelization can outweigh its benefits.

Terminal stream operations

A terminal operation consumes a Java Stream to produce a final result or a side effect. In contrast to intermediate operations, which return a new stream and enable pipeline chaining, a terminal operation concludes the pipeline. Some common terminal operations include:

- `anyMatch(Predicate<T>)`, `allMatch(Predicate<T>)`, and `noneMatch(Predicate<T>)`: These operations test whether any, all, or no elements of the stream match a given predicate.
- `collect(Collector<T, A, R>)`: Performs a mutable reduction operation on the elements of the stream, accumulating them into a container such as a `List` or `Map`. This versatile operation utilizes predefined `Collector` implementations for common tasks.
- `count()`: Returns a count of the elements in the stream.
- `findFirst()` and `findAny()`: These operations return an `Optional` describing the first element of the stream (`findFirst`), or an arbitrary element of the stream (`findAny`).
- `forEach(Consumer<T>)`: Performs a specified action for each element of the stream. This operation is typically used for its side effects, such as printing elements to a console.
- `min(Comparator<T>)` and `max(Comparator<T>)`: Return the minimum or maximum element of the stream according to a provided `Comparator`.
- `reduce(T, BinaryOperator<T>)`: Performs a reduction on the elements of the stream, using an associative accumulation function, and returns an `Optional` describing the reduced value.

- `sum()`: Returns the sum of elements in a primitive numeric stream (e.g., `IntStream`, `LongStream`, `DoubleStream`).
- `toArray()`: Collects the elements of the stream into a new array.

A comprehensive list and detailed specifications for all intermediate and terminal stream operations are available in the official JavaDoc documentation (Oracle, 2026e).

Example: Prime number generation

Script 2.25 presents an implementation of a prime number generator that utilizes streams and lambda expressions. The `primeNumbers(int n)` function employs a stream pipeline for this purpose. Initially, `IntStream.rangeClosed(2, n)` creates a stream of primitive integers from 2 to `n`. An intermediate `filter()` operation is then applied, which retains only the elements for which the `isPrime()` predicate method returns `true`. The subsequent `boxed()` method converts the `IntStream` into a `Stream<Integer>`, which allows the use of the `collect(Collectors.toList())` terminal operation to accumulate the resulting prime numbers into a `List<Integer>`.

The helper method, `isPrime(int number)`, determines the primality of a given integer. It accomplishes this by generating an `IntStream` for the range from 2 to the square root of the input number and utilizing the `noneMatch()` terminal operation to test for divisibility. If `noneMatch()` returns `true`, indicating that no divisors were found, the number is determined to be prime.

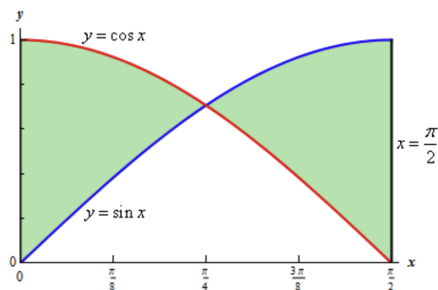
Notably, the `IntStream.rangeClosed()` method is functionally equivalent to a traditional `for` loop for generating a sequence of integers. As a potential enhancement, the `.parallel()` operation could be incorporated into this stream pipeline to leverage multi-threaded computation, which may improve performance for large input ranges.

Script 2.25: Prime Number Generation Using a Stream Pipeline

```

+ 1 public class PrimeStream {
+ 2     public static List<Integer> primeNumbers(int n) {
+ 3         return IntStream.rangeClosed(2, n)
+ 4             .filter(x -> isPrime(x)).boxed()
+ 5             .collect(Collectors.toList());
+ 6     }
+ 7
+ 8     private static boolean isPrime(int number) {
+ 9         return IntStream.rangeClosed(2, (int) (Math.sqrt(number)))
+10             .noneMatch(x -> number % x == 0);
+11     }
+12
+13     public static void main(String[] args) {
+14         long tStart = System.currentTimeMillis();
+15         System.out.println(primeNumbers((int) 1e6));
+16         long tEnd = System.currentTimeMillis();
+17         long elapsedSeconds = tEnd - tStart;
+18         System.out.println("time: " + elapsedSeconds + " ms");
+19     }
+20 }

```

Example: Estimating area between curves**Figure 2.2: Area Estimation Between Two Curves**

This example demonstrates the estimation of the area enclosed between two curves, as highlighted in Figure 2.2. A straightforward approach to this problem is the partitioning of the integration range into two segments: $[0, \pi/4]$ and $[\pi/4, \pi/2]$. The area for the first segment is determined by the integral of $\cos(x) - \sin(x)$, while the area for the second is determined by the integral of $\sin(x) - \cos(x)$. This approach approximates the total area by subdividing these segments into numerous smaller intervals, calculating the area for each sub-interval, and summing the results.

Script 2.26 presents a solution to this problem that utilizes Java Streams and lambda expressions. Although the mathematical problem may appear complex, the conciseness of the implementation highlights the expressive power of Java Streams and lambda expressions. A general-purpose method, `AreaUnderCurve()`, is provided to perform this numerical integration for any two given curves. This implementation first defines a functional interface, `DoubleFunction`, to represent mathematical functions that accept a double input and return a double output. The `AreaUnderCurve()` method operates by partitioning a given interval `[start, end]` into a large number of sub-intervals. It then uses a parallel `LongStream` to calculate the differential area between the upper and lower function bounds for each sub-interval. These differential areas are then summed to produce the final approximation of the total area. The `main()` method demonstrates this by calculating the area between the cosine and sine curves for the two separate intervals and summing the results.

2.3.6 Map and Reduce paradigm

The map and reduce paradigm is a fundamental concept in parallel and distributed computing and is widely employed in technologies designed for large-scale data processing, such as Hadoop and MongoDB.

- **Map**(f, x): This operation applies a function (f) to each element within an input collection (x), producing a new collection of the same size where each element corresponds to the result of applying (f) to an element in (x).
- **Reduce**(f, x): This operation iteratively combines the elements of a collection (x) using an associative function (f), progressively reducing the collection to a single output value.

Script 2.26: Numerical Integration Using Streams and Lambda Expressions

```
+ 1 public class AUCStream {
+ 2     public interface DoubleFunction {
+ 3         public double f(double x);
+ 4     }
+ 5     public static double AreaUnderCurve( DoubleFunction upper, DoubleFunction lower,
+ 6         double start, double end, int chunk) {
+ 7         double piece = (end - start)/chunk;
+ 8         return LongStream.range(0, chunk)
+ 9             .parallel()
+ 10            .mapToDouble(x -> upper.f(start+x*piece)*piece - lower.f(start+x*piece)*
+ 11                piece)
+ 12            .sum();
+ 13     }
+ 14     public static void main(String[] args) {
+ 15         double area1 = AreaUnderCurve(x->Math.cos(x), x->Math.sin(x),0,Math.PI/4,(int)1
+ 16             e6);
+ 17         double area2 = AreaUnderCurve(x->Math.sin(x), x->Math.cos(x),Math.PI/4,Math.PI
+ 18             /2,(int)1e6);
+ 19         System.out.println(area1+area2);
+ 20     }
+ 21 }
```

In the context of Java Streams, the conceptual reduce operation is often implemented using terminal operations that perform grouping or aggregation, such as `collect()`.

Example: Word count using Map and Reduce

Script 2.27 demonstrates a word-counting implementation using the map and reduce paradigm within a Java Stream. The implementation involves the following steps:

- A string variable, `text`, is defined, and a `Pattern` object is created to split the text into words based on whitespace.

- The `splitAsStream()` method generates an initial stream of words. A `map()` operation is then applied to clean each word by removing non-alphabetic characters and converting it to lowercase.
- A `filter()` operation removes any empty strings that may have resulted from the cleaning process.
- A subsequent `map()` operation transforms each word into a `SimpleEntry` (a key-value pair), with the word as the key and the integer 1 as its value, representing a single occurrence.
- The `collect()` terminal operation, which serves as the reduce step, aggregates the stream into a `Map<String, Integer>`. The `toMap()` collector is used with a merge function (`v1, v2`) $\rightarrow$ `v1 + v2` to sum the counts for duplicate keys (words).
- Finally, the `forEach()` method is used to iterate over the resulting map and print each word and its final count.

Script 2.27: Word Count Implementation Using the Map-Reduce Paradigm with Java Streams

```
+ 1 public static void main(String[] args) {  
+ 2     String text = "Java is a programming language. Python is also a programming  
+ 3         language";  
+ 4     Pattern pattern = Pattern.compile(" ");  
+ 5     Map<String, Integer> wordCount = pattern.splitAsStream(text)  
+ 6         .map(word -> word.replaceAll("[^a-zA-Z]", "").toLowerCase().trim())  
+ 7         .filter(word -> word.length() > 0)  
+ 8         .map(word -> new AbstractMap.SimpleEntry<>(word, 1))  
+ 9         .collect(toMap(e -> e.getKey(), e -> e.getValue(), (v1, v2) -> v1 + v2));  
+ 10    wordCount.forEach((k, v) -> System.out.println(String.format("%s: %d", k, v)));  
+ 11 }
```

2.4 Case Study

AI Advisory

Exception handling is a topic where AI tools produce syntactically correct code almost every time, and where that code can still be dangerously wrong. A broad catch (`Exception e`) { `e.printStackTrace();` } block will compile and silence every error, but it teaches nothing about the actual failure mode and leaves the application in an undefined state. At this stage of the course, the objective is to recognize *why* an exception occurs, not merely how to suppress it.

Recommended Approach

Before accepting any AI-generated catch block, the developer must first identify what has actually gone wrong and whether the handler is recovering from the problem or merely hiding it. A network timeout, a malformed JSON response, and an unsupported currency code are three fundamentally different problems; collapsing them into a single generic handler removes the diagnostic information that makes debugging possible. Similarly, when working with lambda expressions and stream pipelines, each intermediate operation (`filter`, `map`, `sorted`) should be describable in plain language before it is included. An AI can generate a syntactically valid pipeline in seconds, but a pipeline that cannot be explained step by step will become a maintenance liability as the application grows.

Appropriate Uses

- ✓ Use an AI to explain the exception hierarchy (`Throwable` → `Exception` → `RuntimeException`) if the class diagram is unclear.
- ✓ Use an AI to verify whether a specific exception is checked or unchecked, then confirm the answer against the official JavaDoc.

- ✓ Use an AI to show the anonymous inner class equivalent of a lambda expression, to verify that the transformation is understood.

Practices to Avoid

- ✗ Do not ask an AI to fix a compile error caused by an unhandled checked exception without first understanding why the compiler requires it.
- ✗ Do not allow an AI to write the catch block for a network call without specifying which failure modes the application must distinguish.
- ✗ Do not use an AI-generated stream pipeline without being able to describe the purpose of each intermediate and terminal operation.

The pattern-recognition developed through lambda expressions and streams in this chapter makes it substantially safer to evaluate AI-generated code in subsequent chapters, where these constructs reappear in increasingly complex contexts such as concurrent task management and test fixture construction.

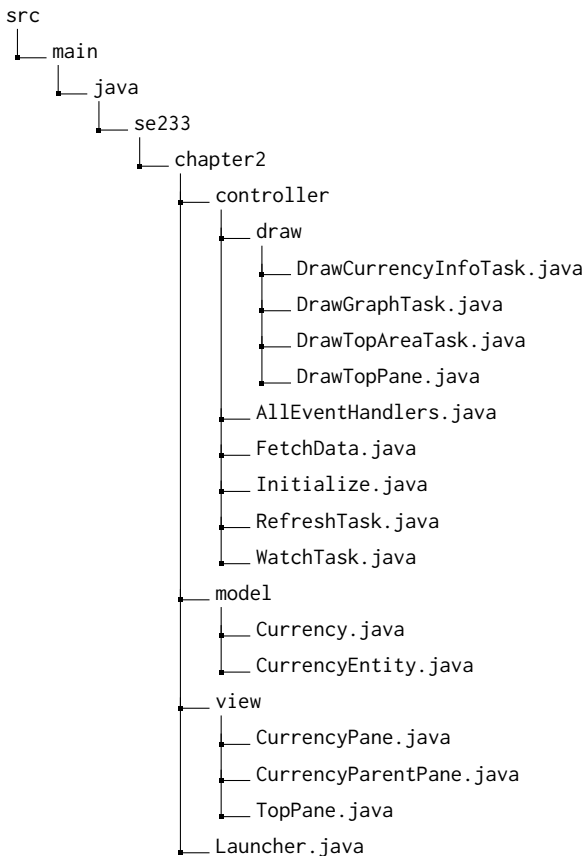
The case study for this chapter involves the development of an application designed to visualize historical currency exchange rate data. The graphical user interface (GUI) for the final application is depicted in Figure 2.3. The initial requirements for the application are as follows:

1. The application must display current exchange rate data on its main screen.
2. The application must present historical exchange rate data for the preceding seven-day period in a line chart.
3. The user must have the capability to select a target currency, for which the exchange rate against the Thai Baht will be displayed.

4. The application must provide a notification to the user if the exchange rate falls below a user-specified threshold.

As illustrated in Figure 2.3, the application, designated Currency Watcher, incorporates three primary control buttons: *Refresh*, *Change*, and *Watch*. The *Refresh* button reinitializes the application's state by fetching the latest data and refreshing the visual components. The *Change* button allows the user to modify the target currency, while the *Watch* button allows the user to set a target exchange rate for monitoring purposes.

The following project structure is to be established:



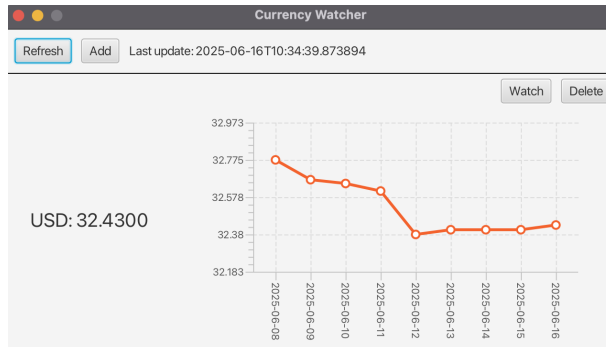


Figure 2.3: GUI of the Currency Watcher Case Study Application

A custom API service has been developed specifically for this case study. The service is implemented using a technology stack orchestrated by Amazon Web Services (AWS, 2026). This API is composed of several AWS components, including AWS Lambda, Amazon S3, Amazon EventBridge, and the AWS API Gateway. Detailed information regarding the API's implementation is provided in Appendix B of this coursebook. The following is a guide to using this Currency Exchange API.

2.4.1 Currency exchange API

The API endpoint provides historical exchange rate data between two specified currencies across a designated time frame. It can be accessed utilizing the Hypertext Transfer Protocol (HTTP) GET method at <https://cmu.to/SE233currencyapi>

Parameters

The following parameters are accepted via the HTTP GET method upon querying the API endpoint:

- **base** (str): The base currency for the exchange rate (for instance, THB). This parameter is mandatory and must be chosen from the supported currencies list.

- **symbol** (str): The target currency for the exchange rate (for instance, EUR). This parameter is mandatory and must be chosen from the supported currencies list.
- **start_date** (str): The starting date for the exchange rate history (for instance, 2026-05-01). This parameter is mandatory and should be in YYYY-MM-DD format.
- **end_date** (str): The ending date for the exchange rate history (for instance, 2026-05-07). This parameter is mandatory and should be in the YYYY-MM-DD format.

Constraints

- The maximum date range for the query is 30 days.
- The earliest allowable start date is 90 days prior to the query date.
- Both the start_date and end_date cannot exceed the current date.

Supported currencies

Table 2.2: Supported Currency Codes and Names

Code	Currency Name	Code	Currency Name
AUD	Australian Dollar	INR	Indian Rupee
CAD	Canadian Dollar	JPY	Japanese Yen
CNY	Chinese Yuan	MYR	Malaysian Ringgit
EUR	Euro	PHP	Philippine Peso
GBP	British Pound Sterling	SGD	Singapore Dollar
HKD	Hong Kong Dollar	THB	Thai Baht
IDR	Indonesian Rupiah	USD	US Dollar

Return value

The API endpoint responds with a JavaScript Object Notation (JSON) object comprising the following fields:

- **message** (str): Indicates the status of the request. It will be *Success* if the request was successful, otherwise, it will contain an error message.
- **base** (str): The base currency (identical to the input parameter).
- **symbol** (str): The target currency (identical to the input parameter).
- **start_date** (str): The starting date (identical to the input parameter).
- **end_date** (str): The ending date (identical to the input parameter).
- **rates** (dict): A dictionary wherein the keys are dates (YYYY-MM-DD) and the values are the corresponding exchange rates for the specified base and symbol currencies.

Example query and response

For example, a query for the exchange rate between THB and EUR for the period of 2026-05-01 to 2026-05-07 would be constructed using the following Uniform Resource Locator (URL): https://cmu.to/SE233currencyapi?base=THB&symbol=EUR&start_date=2026-05-01&end_date=2026-05-07

A successful request to this URL returns a JSON object with the following structure and data:

```
{
  "message": "Success",
  "base": "THB",
  "symbol": "EUR",
  "start_date": "2026-05-01",
  "end_date": "2026-05-07",
  "rates": {
    "2026-05-01": 38.274600982666016,
    "2026-05-02": 38.274600982666016,
    "2026-05-03": 38.274600982666016,
    "2026-05-04": 38.274600982666016,
    "2026-05-05": 38.28959655761719,
    "2026-05-06": 38.01139831542969,
    "2026-05-07": 37.92490005493164
  }
}
```

2.4.2 Extracting information from the API

The initial implementation step involves establishing a package structure within the `src` folder, consisting of packages named `controller`, `model`, and `view`. Within this structure, the first two classes to be implemented are responsible for connecting to the API and retrieving the exchange rate data. These classes are `CurrencyEntity` and `FetchData`, whose implementations are presented in Scripts 2.28 and 2.29, respectively.

Script 2.28: `controller/FetchData.java`

```
+ 1 //Imports are omitted
+ 2 public class FetchData {
+ 3     private static final DateTimeFormatter formatter = DateTimeFormatter.ofPattern(
+ 4         "yyyy-MM-dd");
+ 5     public static List<CurrencyEntity> fetchRange(String symbol, int N) {
+ 6         String dateEnd = LocalDate.now().format(formatter);
+ 7         String dateStart = LocalDate.now().minusDays(N).format(formatter);
+ 8         String urlStr = String.format("https://cmu.to/SE233currencyapi?base=THB&
+ 9             symbol=%s&start_date=%s&end_date=%s", symbol, dateStart, dateEnd);
+10         List<CurrencyEntity> histList = new ArrayList<>();
+11         return histList;
+12     }
+13 }
```

The `CurrencyEntity` class serves as a data model to represent a single currency data point, associating an exchange rate with a specific date. The `FetchData` class contains a single method, `fetchRange()`, which accepts two arguments: a `String` `src` representing the target currency, and an `int` `N` representing the number of days for the historical data range. The `fetchRange()` method first calculates the required date range for the API query, using the current date as the end date (Lines 5-6). Subsequently, the complete API request URL is assembled (Line 7). An `ArrayList` is then prepared to be a container for the query results, which is ultimately returned to the caller (Lines 8-9).

The JSON-formatted data retrieved from the API must be parsed prior to its utilization. Two Java libraries, *Apache Commons IO* and *JSON in Java*, provide the

Script 2.29: model/CurrencyEntity.java

```
+ 1 //Imports are omitted
+ 2 public class CurrencyEntity {
+ 3     private Double rate;
+ 4     private String date;
+ 5     public CurrencyEntity(Double rate, String date) {
+ 6         this.rate = rate;
+ 7         this.date = date;
+ 8     }
+ 9     public Double getRate() {
+10         return rate;
+11     }
+12     public String getTimestamp() {
+13         return date;
+14     }
+15     @Override
+16     public String toString() {
+17         return String.format("%s %.4f",date,rate);
+18     }
+19 }
```

necessary methodologies for this parsing task. The integration of these libraries is accomplished by adding their dependency information to the project's `pom.xml` file. Script 2.30 demonstrates this update, where the required dependency declarations are added within the `<dependencies>` section (Lines 39-48). A detailed explanation of Maven dependency management is provided in Chapter 6. Upon modifying the `pom.xml` file, the project's Maven configuration must be reloaded. This is typically accomplished within an IDE by selecting the Reload project option from the Maven context menu, as illustrated in Figure 2.4.

Furthermore, recent versions of the IntelliJ IDEA IDE manage JavaFX projects as modular projects by default. This modular approach necessitates the declaration of required dependencies in the `src/main/java/module-info.java` file. Script 2.31 demonstrates the required module configuration for the *Apache Commons IO* and *JSON in Java* libraries.

With the integration of these two libraries, the application is now equipped

Script 2.30: pom.xml

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.
5      apache.org/xsd/maven-4.0.0.xsd">
6      <modelVersion>4.0.0</modelVersion>
7      ...
36     <version>${junit.version}</version>
37     <scope>test</scope>
38     </dependency>
+ 39 <dependency>
+ 40     <groupId>commons-io</groupId>
+ 41     <artifactId>commons-io</artifactId>
+ 42     <version>2.22.0</version>
+ 43 </dependency>
+ 44 <dependency>
+ 45     <groupId>org.json</groupId>
+ 46     <artifactId>json</artifactId>
+ 47     <version>20251224</version>
+ 48 </dependency>
49 </dependencies>
...

```

to fetch the currency exchange rate data from the API. Script 2.32 presents the updated FetchData class. The code in Line 11 establishes a connection to the API endpoint URL. The IOUtils.toString() method is then used to read the entire response from the connection's input stream into a raw string. A while-loop (Lines 20-24) iteratively parses the JSON data, extracting the date (key) and exchange rate (value) pairs.

After all key-value pairs have been extracted and added to the histList, the code in Lines 25-30 sorts the histList collection in ascending order by date. This sorting step is necessary because JSON objects do not guarantee key order. Finally, the sorted list of currency data is returned to the caller (Line 31).

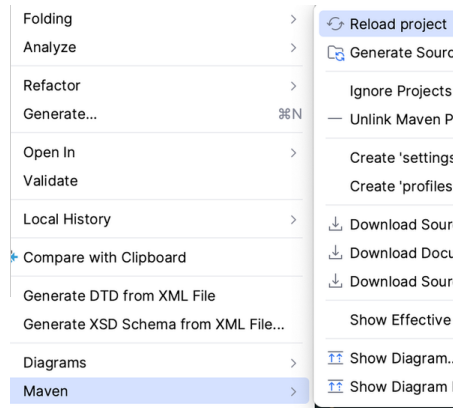


Figure 2.4: Reloading the Maven Configuration

Script 2.31: module-info.java

```

1 module se233.chapter2 {
2     requires javafx.controls;
3     requires javafx.fxml;
+ 4     requires org.apache.commons.io;
+ 5     requires org.json;
6
7     opens se233.chapter2 to javafx.fxml;
8     exports se233.chapter2;
9 }

```

2.4.3 Adding exception handlers

An examination of Script 2.32 within an IDE reveals a compilation error at Line 9. This error is due to an unhandled checked exception, for which the compiler typically suggests resolution strategies. In this scenario, the potential issues that can disrupt the normal program flow are an invalid URL format and an invalid format for the retrieved data. Script 2.33 presents a try-catch implementation that addresses these potential exceptions, thereby satisfying the compiler. Notably, there

Script 2.32: controller/FetchData.java

```

1  //Imports are omitted
2  public class FetchData {
    ...
16      List<CurrencyEntity> histList = new ArrayList<>();
+ 17      String retrievedJson = IOUtils.toString(new URL(urlStr), Charset.
           defaultCharset());
+ 18      JSONObject jsonObj = new JSONObject(retrievedJson).getJSONObject("rates");
+ 19      Iterator keysToCopyIterator = jsonObj.keys();
+ 20      while(keysToCopyIterator.hasNext()) {
+ 21          String key = (String) keysToCopyIterator.next();
+ 22          Double rate = Double.parseDouble(jsonObj.get(key).toString());
+ 23          histList.add(new CurrencyEntity(rate, key));
+ 24      }
+ 25      histList.sort(new Comparator<CurrencyEntity>() {
+ 26          @Override
+ 27          public int compare(CurrencyEntity o1, CurrencyEntity o2) {
+ 28              return o1.getTimestamp().compareTo(o2.getTimestamp());
+ 29          }
+ 30      });
31      return histList;
32  }
33  }

```

are two catch blocks in this example, each designed to handle a specific failure mode: one for URL formatting errors and another for data parsing errors.

Script 2.34 provides the initial implementation of the Launcher class for the application. When the application is executed at this stage, the retrieved data should be displayed in the system console in an array format without any errors.

Upon the completion of this implementation step, executing the application will render a blank window with “Currency Watcher” displayed in the title bar. Concurrently, an examination of the output terminal within the IDE will reveal an array of date and exchange rate pairs spanning the preceding seven days.

Script 2.33: controller/FetchData.java

```

1  //Imports are omitted
2  public class FetchData {
    ...
22     List<CurrencyEntity> histList = new ArrayList<>();
+ 23     try {
24         String retrievedJson = IOUtils.toString(new URL(urlStr), Charset.
            defaultCharset());
25         JSONObject jsonObj = new JSONObject(retrievedJson).getJSONObject("rates");
26         Iterator keysToCopyIterator = jsonObj.keys();
            ...
35         return o1.getTimestamp().compareTo(o2.getTimestamp());
36     }
37     });
+ 38 } catch (MalformedURLException e) {
+ 39     System.err.println("Encounter a Malformed Url exception");
+ 40 } catch (IOException e) {
+ 41     System.err.println("Encounter an IO exception");
+ 42 }
43     return histList;
44 }
45 }

```

Script 2.34: Launcher.java

```

+ 1  //Imports are omitted
+ 2  public class Launcher extends Application {
+ 3      private static Stage primaryStage;
+ 4      @Override
+ 5      public void start(Stage stage) {
+ 6          primaryStage = stage;
+ 7          primaryStage.setTitle("Currency Watcher");
+ 8          primaryStage.setResizable(false);
+ 9          System.out.println(FetchData.fetchRange("USD",6));
+ 10         primaryStage.show();
+ 11     }
+ 12 }

```

2.4.4 The view components

Upon initial execution, the application is required to establish a connection with the data provider to retrieve the exchange rate data. The implementation of the application's GUI components proceeds from this point, beginning with the model. Script 2.35 creates a class named `Currency` within the model package. This `Currency` class encapsulates the data for a specific currency, including its short code, a state indicating if it is being monitored, and the specified watch rate. In Line 5 of the script, the `historical` variable is declared with a type that matches the return type of the `fetchRange()` method, as it is intended to store the data retrieved by that method. Accessor and mutator methods have been omitted from this course-book for brevity, but they can be generated using the same approach demonstrated in the previous chapter's case study (Figure 2.5).

The subsequent implementation step is the creation of the view components. As depicted in Figure 2.3, the application's layout is composed of two primary sections: the top pane and the body pane, designated as `TopPane` and `CurrencyPane`, respectively. Script 2.36 presents the implementation of the `TopPane` class. The pane contains two buttons, whose event-handling logic will be implemented subsequently. Of note in this class is the `refreshPane()` method (Lines 16-18). This method is designed to be invoked when the *Refresh* button is clicked, at which point it will update the timestamp information displayed on the pane.

The `CurrencyPane` class extends the `BorderPane` class, a layout container that organizes child nodes into five regions: top, bottom, left, right, and center. In this implementation, these regions will contain the currency information area, the historical data graph, and the watch list area. These areas are implemented as three distinct custom panes: `InfoPane`, `GraphPane`, and `WatchPane`, respectively. Script 2.37 presents the implementation of this `CurrencyPane` container class.

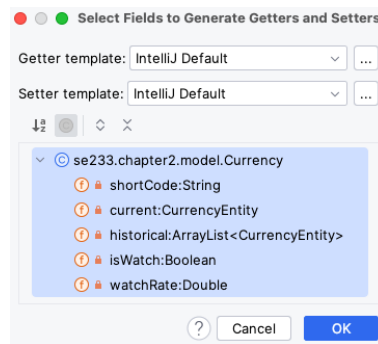


Figure 2.5: Generation of Accessor and Mutator Methods for Currency Class

Script 2.35: model/Currency.java

```
+ 1 //Imports are omitted
+ 2 public class Currency {
+ 3     private String shortCode;
+ 4     private CurrencyEntity currency;
+ 5     private List<CurrencyEntity> historical;
+ 6     private Boolean isWatch;
+ 7     private Double watchRate;
+ 8     public Currency(String shortCode) {
+ 9         this.shortCode = shortCode;
+10         this.isWatch = false;
+11         this.watchRate = 0.0;
+12     }
+13 }
```

Script 2.36: view/TopPane.java

```
+ 1 //Imports are omitted
+ 2 public class TopPane extends FlowPane {
+ 3     private Button refresh;
+ 4     private Button change;
+ 5     private Label update;
+ 6     public TopPane() {
+ 7         this.setPadding(new Insets(10));
+ 8         this.setHgap(10);
+ 9         this.setPrefSize(640,20);
+10         change = new Button("Change");
+11         refresh = new Button("Refresh");
+12         update = new Label();
+13         refreshPane();
+14         this.getChildren().addAll(refresh,change,update);
+15     }
+16     public void refreshPane() {
+17         update.setText(String.format("Last update: %s",LocalDateTime.now().toString()));
+18     }
+19 }
```

Script 2.37: view/CurrencyPane.java

```

+ 1 //Imports are omitted
+ 2 public class CurrencyPane extends BorderPane {
+ 3     private Currency currency;
+ 4     private Button watch;
+ 5     public CurrencyPane(Currency currency) {
+ 6         this.watch = new Button("Watch");
+ 7         this.setPadding(new Insets(0));
+ 8         this.setPrefSize(640,300);
+ 9         this.setStyle("-fx-border-color: black");
+10         this.refreshPane(currency);
+11     }
+12     public void refreshPane(Currency currency) {
+13         this.currency = currency;
+14         Pane currencyInfo = genInfoPane();
+15         Pane topArea = genTopArea();
+16         this.setTop(topArea);
+17         this.setLeft(currencyInfo);
+18     }
+19     private Pane genInfoPane() {
+20         VBox currencyInfoPane = new VBox(10);
+21         currencyInfoPane.setPadding(new Insets(5, 25, 5, 25));
+22         currencyInfoPane.setAlignment(Pos.CENTER);
+23         Label exchangeString = new Label("");
+24         Label watchString = new Label("");
+25         exchangeString.setStyle("-fx-font-size: 20;");
+26         watchString.setStyle("-fx-font-size: 14;");
+27         if (this.currency != null) {
+28             exchangeString.setText(String.format("%s: %.4f",this.currency.getShortCode
+29                 (),this.currency.getCurrency().getRate()));
+30             if(this.currency.getWatch() == true) {
+31                 watchString.setText(String.format("(Watch @%.4f)",this.currency.
+32                     getWatchRate()));
+33             }
+34         }
+35         currencyInfoPane.getChildren().addAll(exchangeString,watchString);
+36         return currencyInfoPane;
+37     }
+38     private HBox genTopArea() {
+39         HBox topArea = new HBox(10);
+40         topArea.setPadding(new Insets(5));
+41         topArea.getChildren().addAll(watch);
+42         ((HBox) topArea).setAlignment(Pos.CENTER_RIGHT);
+43         return topArea;
+44     }
+45 }

```


2.4.5 The controller components

The Launcher class is subsequently modified to assemble and display all previously implemented view components. The Initialize class, presented in Script 2.38, is responsible for creating and initializing the primary Currency object used throughout the application. Script 2.39 presents the updated Launcher class. As shown in Line 16, the `initMainPane()` method is invoked to initialize the `TopPane` and `CurrencyPane` instances. At this juncture, the only remaining view component to be implemented is the `GraphPane`.

Script 2.40 illustrates a class named `DrawGraphTask` responsible for generating the visual elements of the graph,. This class implements the Java `Callable` interface, a standard interface that allows an object's logic to be executed as a background task concurrently with the main application thread. The fundamentals of background processes and threads will be explained in the subsequent chapter. The `call()` method serves as the entry point for this background task. The method iterates through the currency data points stored in the historical variable of the `Currency` instance (Lines 20-24). Subsequently, it plots these data points onto the graph according to their calculated (x, y) coordinates (Line 29). The resulting graph is then added to the `GraphPane` (Line 31), which is subsequently returned to the caller (Line 32). The invocation of the `DrawGraphTask`'s `call()` method is managed by the `refreshPane()` method, as shown in Script 2.41.

Script 2.38: controller/Initialize.java

```
+ 1 //Imports are omitted
+ 2 public class Initialize {
+ 3     public static Currency initializeApp() {
+ 4         Currency c = new Currency("USD");
+ 5         List<CurrencyEntity> cList = FetchData.fetchRange(c.getShortCode(), 8);
+ 6         c.setHistorical(cList);
+ 7         c.setCurrent(cList.get(cList.size() -1 ));
+ 8         return c;
+ 9     }
+ 10 }
```

Script 2.39: Launcher.java

```
1 //Imports are omitted
  ...
7 public class Launcher extends Application {
8     private static Stage primaryStage;
+ 9     private static FlowPane mainPane;
+10     private static TopPane topPane;
+11     private static CurrencyPane currencyPane;
+12     private static Currency currency;
13
14     @Override
15     public void start(Stage stage) {
16         primaryStage = stage;
17         primaryStage.setTitle("Currency Watcher");
18         primaryStage.setResizable(false);
- 19         System.out.println(FetchData.fetchRange("USD",6));
+20         currency = Initialize.initializeApp();
+21         initMainPane();
+22         Scene mainScene = new Scene(mainPane);
+23         primaryStage.setScene(mainScene);
24         primaryStage.show();
25     }
+26     public void initMainPane() {
+27         mainPane = new FlowPane();
+28         topPane = new TopPane();
+29         currencyPane = new CurrencyPane(currency);
+30         mainPane.getChildren().add(topPane);
+31         mainPane.getChildren().add(currencyPane);
+32     }
33 }
```

Script 2.40: controller/DrawGraphTask.java

```

+ 1 //Imports are omitted
+ 2 public class DrawGraphTask implements Callable<VBox> {
+ 3     Currency currency;
+ 4     public DrawGraphTask(Currency currency) {
+ 5         this.currency = currency;
+ 6     }
+ 7     @Override
+ 8     public VBox call() throws Exception {
+ 9         VBox graphPane = new VBox(10);
+10         graphPane.setPadding(new Insets(0, 25, 5, 25));
+11         CategoryAxis xAxis = new CategoryAxis();
+12         NumberAxis yAxis = new NumberAxis();
+13         yAxis.setAutoRanging(true);
+14         LineChart lineChart = new LineChart(xAxis,yAxis);
+15         lineChart.setLegendVisible(false);
+16         if (this.currency != null) {
+17             XYChart.Series series = new XYChart.Series();
+18             double minY = Double.MAX_VALUE;
+19             double maxY = Double.MIN_VALUE;
+20             for (CurrencyEntity c : currency.getHistorical()) {
+21                 series.getData().add(new XYChart.Data(c.getTimestamp(),c.getRate()));
+22                 if(c.getRate() > maxY) maxY = c.getRate();
+23                 if(c.getRate() < minY) minY = c.getRate();
+24             }
+25             yAxis.setAutoRanging(false);
+26             yAxis.setLowerBound(minY-(maxY-minY)/2);
+27             yAxis.setUpperBound(maxY+(maxY-minY)/2);
+28             yAxis.setTickUnit((maxY-minY)/2);
+29             lineChart.getData().add(series);
+30         }
+31         graphPane.getChildren().add(lineChart);
+32         return graphPane;
+33     }
+34 }

```

Script 2.41 demonstrates the propagation of an exception. The `refreshPane()` method, shown in Line 30, is designed to propagate any exceptions that occur during its execution, rather than handling them internally. This is accomplished by declaring the relevant exception types in its `throws` clause, thereby delegating the handling responsibility to the calling method.

Script 2.41: view/CurrencyPane.java

```

1  //Imports are omitted
2  public class CurrencyPane extends BorderPane {
    ...
16  public CurrencyPane(Currency currency) {
17      this.watch = new Button("Watch");
18      this.setPadding(new Insets(0));
19      this.setPrefSize(640,300);
20      this.setStyle("-fx-border-color: black");
+21  try {
22      this.refreshPane(currency);
+23  } catch (ExecutionException e) {
24      System.out.println("Encountered an execution exception.");
+25  } catch (InterruptedException e) {
26      System.out.println("Encountered an interrupted exception.");
+27  }
28  }
-29  public void refreshPane(Currency currency) {
+30  public void refreshPane(Currency currency) throws ExecutionException,
    InterruptedException {
31      this.currency = currency;
32      Pane currencyInfo = genInfoPane();
+33      FutureTask futureTask = new FutureTask<VBox>(new DrawGraphTask(currency));
+34      ExecutorService executor = Executors.newSingleThreadExecutor();
+35      executor.execute(futureTask);
+36      VBox currencyGraph = (VBox) futureTask.get();
37      Pane topArea = genTopArea();
38      this.setTop(topArea);
39      this.setLeft(currencyInfo);
+40      this.setCenter(currencyGraph);
41  }
    ...

```

2.4.6 Event handlers

In its current state, the application's GUI is displayed as depicted in Figure 2.3, although the historical data is limited to a seven-day period. The subsequent implementation phase involves defining the functionality for the control buttons, beginning with the Refresh button. Scripts 2.42, 2.43, and 2.44 present minor modifications to the Launcher and TopPane classes and introduce a new AllEventHandlers class, which is designed to consolidate all event-handling logic for the application.

Script 2.42: Launcher.java

```
1 //Imports are omitted
2 public class Launcher extends Application {
    ...
36     mainPane.getChildren().add(currencyPane);
37 }
+ 38 public static void refreshPane() throws InterruptedException, ExecutionException {
+ 39     topPane.refreshPane();
+ 40     currencyPane.refreshPane(currency);
+ 41 }
42 }
```

Script 2.43: controller/AllEventHandlers.java

```
+ 1 //Imports are omitted
+ 2 public class AllEventHandlers {
+ 3     public static void onRefresh() {
+ 4         try {
+ 5             Launcher.refreshPane();
+ 6         } catch (Exception e) {
+ 7             e.printStackTrace();
+ 8         }
+ 9     }
+ 10 }
```

Script 2.44: view/TopPane.java

```
1  //Imports are omitted
2  public class TopPane extends FlowPane {
    ...
19     refresh = new Button("Refresh");
+ 20     refresh.setOnAction(new EventHandler<ActionEvent>() {
+ 21         @Override
+ 22         public void handle(ActionEvent event) {
+ 23             AllEventHandlers.onRefresh();
+ 24         }
+ 25     });
26     update = new Label();
27     refreshPane();
    ...
}
```

Upon the completion of this step, the application will render the GUI depicted in Figure 2.6, with only the Refresh button functionality implemented. However, for the purposes of this case study, a hypothetical scenario is introduced wherein a client requests several functional enhancements. Specifically, whereas the current version permits monitoring only a single currency, a more practical requirement is the concurrent monitoring of multiple currencies. To accommodate this, the Change button is to be replaced with dedicated Add and Delete buttons. Furthermore, the implementation must satisfy all remaining unfulfilled requirements from the initial specification. Upon the integration of these new requirements, the final application will feature an updated GUI that reflects this enhanced functionality.

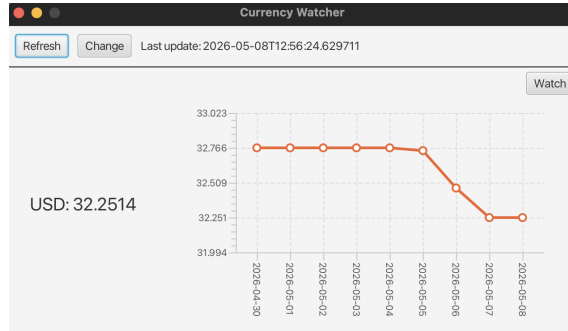


Figure 2.6: GUI of the Currency Watcher Case Study Application After Completing the Refresh Button Functionality

2.4.7 Requirement change handling

In response to the new requirement to handle multiple currency objects concurrently, a modification of the underlying data structure is necessary. The transition from representing the model as a single object to a collection is required to support the dynamic addition and removal of currencies. For this purpose, an `ArrayList` is an appropriate data structure.

First, the change from a single currency object to a `List` (the superinterface of `ArrayList`) is propagated to the `Initialize` class to ensure data model consistency, as shown in Script 2.45.

The management of these multiple currency objects introduces the requirement to dynamically update the view. This involves assembling the various view sub-components for each currency into a single, cohesive presentation. Because the `List` can store multiple currencies, the application must be capable of displaying multiple `CurrencyPane` instances concurrently. A dedicated controller component is therefore required to manage this collection of `CurrencyPane` objects. Script 2.46 introduces such a controller class.

Finally, the `Launcher` class is refactored to use the `List` data model and incorporate the new controller. As illustrated in Script 2.47, the `Launcher` now orchestrates the creation and management of the collection of currency panes, completing the transition to a multi-currency architecture.

Script 2.45: controller/Initialize.java

```
1 //Imports are omitted
...
8 public class Initialize {
- 9     public static Currency initializeApp() {
+ 10     public static List<Currency> initializeApp() {
        ...
        c.setCurrency(cList.get(cList.size() -1 ));
- 14     return c;
+ 15     List<Currency> currencyList = new ArrayList<>();
+ 16     currencyList.add(c);
+ 17     return currencyList;
18 }
19 }
```

Script 2.46: view/CurrencyParentPane.java

```
+ 1 //Imports are omitted
+ 2 public class CurrencyParentPane extends FlowPane {
+ 3     public CurrencyParentPane(List<Currency> currencyList) throws ExecutionException,
        InterruptedException {
+ 4         this.setPadding(new Insets(0));
+ 5         refreshPane(currencyList);
+ 6     }
+ 7     public void refreshPane(List<Currency> currencyList) throws ExecutionException,
        InterruptedException {
+ 8         this.getChildren().clear();
+ 9         for(int i=0 ; i<currencyList.size() ; i++) {
+ 10             CurrencyPane cp = new CurrencyPane(currencyList.get(i));
+ 11             this.getChildren().add(cp);
+ 12         }
+ 13     }
+ 14 }
```

It is important to note that accessor and mutator methods are necessary for the `CurrencyList` collection in the `Launcher` class. These methods should be implemented following the approach demonstrated previously (Figure 2.5).

Script 2.47: Launcher.java

```

1  //Imports are omitted
2  public class Launcher extends Application {
    ...
- 19  private static CurrencyPane currencyPane;
+ 20  private static CurrencyParentPane currencyParentPane;
- 21  private static Currency currency;
+ 22  private static List<Currency> currencyList;
- 23  public void start(Stage stage) {
+ 24  public void start(Stage stage) throws ExecutionException, InterruptedException {
    25      primaryStage = stage;
    26      primaryStage.setTitle("Currency Watcher");
    27      primaryStage.setResizable(false);
- 28      currency = Initialize.initializeApp();
+ 29      currencyList = Initialize.initializeApp();
    ...
- 33  public void initMainPane() {
+ 34  public void initMainPane() throws ExecutionException, InterruptedException {
    35      mainPane = new FlowPane();
    36      topPane = new TopPane();
- 37      currencyPane = new CurrencyPane(currency);
+ 38      currencyParentPane = new CurrencyParentPane(currencyList);
    39      mainPane.getChildren().add(topPane);
- 40      mainPane.getChildren().add(currencyPane);
+ 41      mainPane.getChildren().add(currencyParentPane);
    42  }
    43  public void refreshPane() throws ExecutionException, InterruptedException {
    44      topPane.refreshPane();
- 45      currencyPane.refreshPane(currency);
+ 46      currencyParentPane.refreshPane(currencyList);
+ 47      primaryStage.sizeToScene();
    48  }
    49  }

```

The implementation of the *Add* and *Delete* buttons follows. The functionality for the *Add* button is straightforward, as a single button can be used to create a new Currency object and append it to the central List. The functionality for deletion is more complex, as a single, global *Delete* button would be ambiguous. Consequently, a separate *Delete* button must be associated with each individual CurrencyPane. The required modifications to the AllEventHandlers and TopPane classes are illustrated in Scripts 2.48 and 2.49, respectively.

Script 2.48: controller/AllEventHandlers.java

```
1 //Imports are omitted
2 public class AllEventHandlers {
3     ...
+ 13     public static void onAdd() {
+ 14         try {
+ 15             TextInputDialog dialog = new TextInputDialog();
+ 16             dialog.setTitle("Add Currency");
+ 17             dialog.setContentText("Currency code:");
+ 18             dialog.setHeaderText(null);
+ 19             dialog.setGraphic(null);
+ 20             Optional<String> code = dialog.showAndWait();
+ 21             if (code.isPresent()){
+ 22                 List<Currency> currencyList = Launcher.getCurrencyList();
+ 23                 Currency c = new Currency(code.get());
+ 24                 List<CurrencyEntity> cList = FetchData.fetchRange(c.getShortCode(),8);
+ 25                 c.setHistorical(cList);
+ 26                 c.setCurrency(cList.get(cList.size() -1 ));
+ 27                 currencyList.add(c);
+ 28                 Launcher.setCurrencyList(currencyList);
+ 29                 Launcher.refreshPane();
+ 30             }
+ 31         } catch (InterruptedException e) {
+ 32             e.printStackTrace();
+ 33         } catch (ExecutionException e) {
+ 34             e.printStackTrace();
+ 35         }
+ 36     }
37 }
```

Script 2.49: view/TopPane.java

```
1 //Imports are omitted
  ...
13 public class TopPane extends FlowPane {
14     private Button refresh;
- 15     private Button change;
+ 16     private Button add;
    ...
- 21     change = new Button("Change");
+ 22     add = new Button("Add");
23     refresh = new Button("Refresh");
24     refresh.setOnAction(new EventHandler<ActionEvent>() {
25         @Override
26         public void handle(ActionEvent event) {
27             AllEventHandlers.onRefresh();
28         }
29     });
+ 30     add.setOnAction(new EventHandler<ActionEvent>() {
+ 31         @Override
+ 32         public void handle(ActionEvent event) {
+ 33             AllEventHandlers.onAdd();
+ 34         }
+ 35     });
36     update = new Label();
37     refreshPane()
- 38     this.getChildren().addAll(refresh, change, update);
+ 39     this.getChildren().addAll(refresh, add, update);
40 }
41 public void refreshPane() {
42     update.setText(String.format("Last update: %s", LocalDateTime.now().toString()));
43 }
```

The subsequent functionality to be implemented is the deletion process. The programming logic for this is outlined in Script 2.50 between Lines 44 and 64. The input for this method is the short currency code, and the method will search the `ArrayList` for the `Currency` object whose short code matches the search query. The technique employs an array index as a flag variable, mirroring the approach applied in the earlier chapter's case study. The event handling for this delete functionality is established between Lines 26 and 31 of Script 2.51.

Script 2.50: controller/AllEventHandlers.java

```
1 //Imports are omitted
2 public class AllEventHandlers {
3     ...
+ 44     public static void onDelete(String code) {
+ 45         try {
+ 46             List<Currency> currencyList = Launcher.getCurrencyList();
+ 47             int index = -1;
+ 48             for(int i=0 ; i<currencyList.size() ; i++) {
+ 49                 if (currencyList.get(i).getShortCode().equals(code) ) {
+ 50                     index = i;
+ 51                     break;
+ 52                 }
+ 53             }
+ 54             if (index != -1) {
+ 55                 currencyList.remove(index);
+ 56                 Launcher.setCurrencyList(currencyList);
+ 57                 Launcher.refreshPane();
+ 58             }
+ 59         } catch (InterruptedException e) {
+ 60             e.printStackTrace();
+ 61         } catch (ExecutionException e) {
+ 62             e.printStackTrace();
+ 63         }
+ 64     }
65 }
```

Script 2.51: view/CurrencyPane.java

```

1  //Imports are omitted
   ...
19 public class CurrencyPane extends BorderPane {
20     private Currency currency;
21     private Button watch;
+ 22     private Button delete;
23     public CurrencyPane(Currency currency) {
24         this.watch = new Button("Watch");
+ 25         this.delete = new Button("Delete");
+ 26         this.delete.setOnAction(new EventHandler<ActionEvent>() {
+ 27             @Override
+ 28             public void handle(ActionEvent event) {
+ 29                 AllEventHandlers.onDelete(currency.getShortCode());
+ 30             }
+ 31         });
32         this.setPadding(new Insets(0));
   ...
72     private HBox genTopArea() {
73         HBox topArea = new HBox(10);
74         topArea.setPadding(new Insets(5));
- 75         topArea.getChildren().addAll(watch);
+ 76         topArea.getChildren().addAll(watch, delete);
77         ((HBox) topArea).setAlignment(Pos.CENTER_RIGHT);
78         return topArea;
79     }
80 }

```

The final button functionality to be implemented is for the Watch button. Scripts 2.53 and 2.52 present the necessary modifications to the CurrencyPane and AllEventHandlers classes to implement this feature. A requirement for this feature is that the specified watch rate, if defined by the user, must be displayed beneath the current exchange rate on the GUI. The implementation of this logic utilizes two state variables, `isWatch` and `watchRate`, to track the currency's monitoring status and the target rate, respectively.

Script 2.52: controller/AllEventHandlers.java

```
1  //Imports are omitted
2  public class AllEventHandlers {
    ...
+ 65  public static void onWatch(String code) {
+ 66      try {
+ 67          List<Currency> currencyList = Launcher.getCurrencyList();
+ 68          int index = -1;
+ 69          for(int i = 0 ; i < currencyList.size() ; i++) {
+ 70              if (currencyList.get(i).getShortCode().equals(code) ) {
+ 71                  index = i;
+ 72                  break;
+ 73              }
+ 74          }
+ 75          if (index != -1) {
+ 76              TextInputDialog dialog = new TextInputDialog();
+ 77              dialog.setTitle("Add Watch");
+ 78              dialog.setContentText("Rate:");
+ 79              dialog.setHeaderText(null);
+ 80              dialog.setGraphic(null);
+ 81              Optional<String> retrievedRate = dialog.showAndWait();
+ 82              if (retrievedRate.isPresent()){
+ 83                  double rate = Double.parseDouble(retrievedRate.get());
+ 84                  currencyList.get(index).setWatch(true);
+ 85                  currencyList.get(index).setWatchRate(rate);
+ 86                  Launcher.setCurrencyList(currencyList);
+ 87                  Launcher.refreshPane();
+ 88              }
+ 89              Launcher.setCurrencyList(currencyList);
+ 90              Launcher.refreshPane();
+ 91          }
+ 92      } catch (InterruptedException e) {
+ 93          e.printStackTrace();
+ 94      } catch (ExecutionException e) {
+ 95          e.printStackTrace();
+ 96      }
+ 97  }
+ 98 }
```

Script 2.53: view/CurrencyPane.java

```
1 //Imports are omitted
...
22 public class CurrencyPane extends BorderPane {
23     private Currency currency;
24     private Button watch;
25     private Button delete;
26     public CurrencyPane(Currency currency) {
27         this.watch = new Button("Watch");
28         this.delete = new Button("Delete");
+ 29         this.watch.setOnAction(new EventHandler<ActionEvent>() {
+ 30             @Override
+ 31             public void handle(ActionEvent event) {
+ 32                 AllEventHandlers.onWatch(currency.getShortCode());
+ 33             }
+ 34         });
...

```

2.4.8 Background process

In addition to the event handler for the Watch button, the watch functionality requires a background process to periodically refresh the application's data. For example, to monitor the watch rate, the application must be updated at a regular interval, e.g., every five seconds. This requires the implementation of a recurring task on a background thread that executes the following sequence during each cycle:

1. Pauses execution for a specified interval (e.g., five seconds).
2. Invokes a refresh of all application components to fetch the latest data.
3. Compares the current exchange rates against the user-defined watch rates and issues a notification if the condition is met.

The RefreshTask class, shown in Script 2.54, contains the logic for this recurring background process. It implements a loop that manages the five-second interval and subsequently invokes the refreshPane() method in the Launcher class to trigger a full application refresh.

Script 2.54: controller/RefreshTask.java

```

+ 1 //Imports are omitted
+ 2 public class RefreshTask extends Task<Void> {
+ 3     @Override
+ 4     protected Void call() throws InterruptedException {
+ 5         for (;;) {
+ 6             try {
+ 7                 Thread.sleep((long) (60 * 1e3));
+ 8             } catch (InterruptedException e) {
+ 9                 System.out.println("Encountered an interrupted exception");
+10             }
+11             Platform.runLater(new Runnable() {
+12                 @Override
+13                 public void run() {
+14                     try {
+15                         Launcher.refreshPane();
+16                         List<Currency> allCurrency = Launcher.getCurrencyList();
+17                         String found = "";
+18                         for(int i = 0 ; i < allCurrency.size() ; i++) {
+19                             if (allCurrency.get(i).getWatchRate() != 0 && allCurrency.get(i).
+20                                 getWatchRate() > allCurrency.get(i).getCurrency().getRate()) {
+21                                 if(found.equals(""))
+22                                     found = allCurrency.get(i).getShortCode();
+23                                 else
+24                                     found = found + " and " + allCurrency.get(i).getShortCode();
+25                             }
+26                             if (!found.equals("")) {
+27                                 Alert alert = new Alert(Alert.AlertType.WARNING);
+28                                 alert.setTitle(null);
+29                                 alert.setHeaderText(null);
+30                                 if(found.length()>3) {
+31                                     alert.setContentText(String.format("%s have become lower than
+32                                         the watch rate!", found));
+33                                 } else {
+34                                     alert.setContentText(String.format("%s has become lower than the
+35                                         watch rates!", found));
+36                                     alert.showAndWait();
+37                                 }
+38                             }
+39                         } catch (Exception e) {
+40                             e.printStackTrace();
+41                         }
+42                     }
+43                 }
+44             }

```


Subsequently, the Launcher class is updated as shown in Script 2.55. This modification involves initializing and starting a new thread that executes an instance of the RefreshTask class. The Platform.runLater() method is employed to ensure that any GUI updates originating from the background thread are safely executed on the main JavaFX Application Thread, thereby preventing potential concurrency issues.

Script 2.55: Launcher.java

```
1 //Imports are omitted
2 public class Launcher extends Application {
3     private static Stage primaryStage;
4     private static Scene mainScene;
5     ...
23 @Override
24 public void start(Stage stage) {
25     ...
33     primaryStage.show();
+ 34     RefreshTask r = new RefreshTask();
+ 35     Thread th = new Thread(r);
+ 36     th.setDaemon(true);
+ 37     th.start();
    ...
}
```

A remaining user experience issue is that the background thread continues to run after a watch notification is triggered, resulting in redundant alerts. To resolve this, the background task is modified to pause until the notification is dismissed by the user. This is achieved by transferring the notification logic from the RefreshTask class to a new WatchTask class that implements the javafx.concurrent.Task interface, as shown in Scripts 2.56 and 2.57.

The completion of these modifications brings the application close to fulfilling the initial requirements. Remaining functionalities are intentionally left unaddressed and are proposed as end-of-chapter exercises.

Script 2.56: controller/WatchTask.java

```
+ 1 //Imports are omitted
+ 2 class WatchTask implements Callable<Void> {
+ 3     @Override
+ 4     public Void call() {
+ 5         List<Currency> allCurrency = Launcher.getCurrencyList();
+ 6         String found = "";
+ 7         for(int i=0 ; i < allCurrency.size() ; i++) {
+ 8             if (allCurrency.get(i).getWatchRate() != 0 && allCurrency.get(i).
+ 9                 getWatchRate() > allCurrency.get(i).getCurrency().getRate()) {
+10                 if(found.equals("")) {
+11                     found = allCurrency.get(i).getShortCode();
+12                 } else {
+13                     found = found + " and " + allCurrency.get(i).getShortCode();
+14                 }
+15             }
+16             if (!found.equals("")) {
+17                 Alert alert = new Alert(Alert.AlertType.WARNING);
+18                 alert.setTitle(null);
+19                 alert.setHeaderText(null);
+20                 if(found.length()>3) {
+21                     alert.setContentText(String.format("%s have become lower than the watch
+22                         rate!",found));
+23                 } else {
+24                     alert.setContentText(String.format("%s has become lower than the watch
+25                         rates!",found));
+26                 }
+27                 alert.showAndWait();
+28             }
+29             return null;
+30         }
+31     }
+32 }
```

Script 2.57: controller/RefreshTask.java

```

1  //Imports are omitted
2  public class RefreshTask extends Task<Void> {
    ...
22      public void run() {
23          try {
24              Launcher.refreshPane();
- 25              List<Currency> allCurrency = Launcher.getCurrencyList();
- 26              String found = "";
- 27              for(int i=0 ; i<allCurrency.size() ; i++) {
- 28                  if (allCurrency.get(i).getWatchRate() != 0 && allCurrency.get(i).
                      getWatchRate() > allCurrency.get(i).getCurrent().getRate()) {
- 29                      if(found.equals(""))
- 30                          found = allCurrency.get(i).getShortCode();
- 31                      else
- 32                          found = found + " and " + allCurrency.get(i).getShortCode();
- 33                  }
- 34              }
- 35              if (!found.equals("")) {
- 36                  Alert alert = new Alert(Alert.AlertType.WARNING);
- 37                  alert.setTitle(null);
- 38                  alert.setHeaderText(null);
- 39                  if(found.length() > 3) {
- 40                      alert.setContentText(String.format("%s have become lower than
                          the watch rate!",found));
- 41                  } else {
- 42                      alert.setContentText(String.format("%s has become lower than the
                          watch rates!",found));
- 43                      alert.showAndWait();
- 44                  }
- 45              }
46              } catch (Exception e) {
47                  e.printStackTrace();
48              }
49          }
50      });
+ 51      FutureTask futureTask = new FutureTask(new WatchTask());
+ 52      Platform.runLater(futureTask);
+ 53      try {
+ 54          futureTask.get();
+ 55      } catch (InterruptedException e) {
+ 56          System.out.println("Encountered an interrupted exception");
+ 57      } catch (ExecutionException e) {
+ 58          System.out.println("Encountered an execution exception");
+ 59      }
    ...

```

Upon the completion of this implementation step, the application is capable of displaying multiple currencies concurrently. Furthermore, it facilitates the addition and deletion of currencies within the list, and incorporates a background process designed to monitor exchange rates until a predefined threshold is reached. An example of the resulting GUI output is depicted in Figure 2.7.

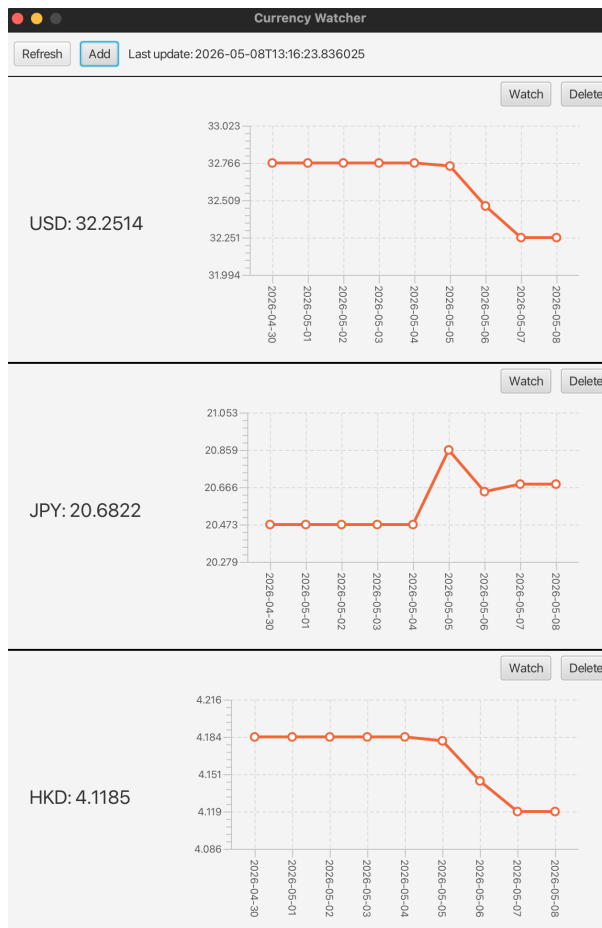


Figure 2.7: GUI Output of the Completed Currency Watcher Application

The `RefreshTask` and `WatchTask` classes introduced in this section represent a preliminary approach to background processing within a JavaFX application. The use of raw `Thread` objects and `Platform.runLater()` is sufficient for simple periodic tasks; however, more complex concurrent scenarios, such as coordinating multiple simultaneous downloads or managing shared mutable state across threads, demand a more structured concurrency framework. Chapter 3 provides a formal exploration of the JavaFX concurrent utilities, including `Task`, `Service`, and `ScheduledService`, which address these requirements with built-in lifecycle management and thread-safe GUI update mechanisms.

2.5 Exercise

1. Modify the application to enable the display of historical exchange rate data for a period of up to 30 days for each currency.
2. Incorporate an *Unwatch* button adjacent to the *Watch* button. The handler for this button should remove the watch rate from the display and deactivate all associated monitoring logic.
3. Enhance the currency input functionality to be case-insensitive, accepting both lowercase and uppercase letters for currency codes.
4. Implement error-handling logic to notify the user when an invalid currency code is entered and to prompt for valid input. **Hint:** A try-catch-finally block may be suitable for parsing and validating the API response.
5. Design and implement a user interface (UI) component that allows the user to configure the application's base currency.
6. Refactor the two custom sub-pane classes within the `CurrencyPane` to implement the `Callable` interface, following the design pattern used for the `DrawGraphTask` class. Subsequently, use an `ExecutorService` to manage the concurrent execution of these `Callable` tasks.

2.6 Discussion and Summary

This chapter has addressed two complementary topics. The first is exception handling, which provides a structured mechanism for managing anomalous runtime conditions. The discussion began with the categorization of exceptions into checked and unchecked types, proceeded through the mechanics of try-catch-finally blocks and exception propagation via throw and throws, and concluded with the design of custom exception classes. During the application design phase, all potential `Exception` types must be anticipated, and a strategy must be formulated to determine where and how each exception should be handled. This process constitutes a form of risk management, guided by the principle that user-triggered exceptions should prompt corrective input, sensible defaults should be assumed where possible, and informative error messages should guide subsequent actions. The consistent practice of exception handling encourages the separation of normal and exceptional program flows, which is a critical skill for both academic coursework and professional development.

The second topic is lambda expressions and streams, introduced as a more concise and expressive syntax for patterns frequently encountered in event-driven GUI programming. Lambda expressions reduce the verbosity of anonymous inner classes when implementing functional interfaces, while the stream API enables declarative data processing through pipelines of intermediate and terminal operations. The map-reduce paradigm, applied to aggregate and transform collections, provides a foundation for parallel data processing that extends naturally into the concurrency concepts presented in Chapter 3.

The case study synthesized both topics into a functional Currency Watcher application. Exception handling was applied to manage network failures and invalid API responses, while lambda expressions simplified the event handler implementations. The introduction of a background refresh task within the case study further demonstrated how these concepts interact in a realistic, multi-component application.

Knowledge Outcomes

Upon the completion of this chapter, a student should demonstrate the ability to:

- Classify a given Java exception as checked or unchecked, and explain the implications of each category for method signatures and caller responsibilities.
- Trace the execution flow through a try-catch-finally block, including the ordering of multiple catch clauses from most specific to most general.
- Explain the role of the throws keyword in method declarations and describe how exception propagation transfers responsibility to calling methods.
- Justify the design decision to create a custom exception class rather than reusing a standard Java exception type.
- Define a lambda expression in terms of its relationship to a functional interface and explain how the compiler resolves the target type.
- Describe the distinction between intermediate operations (`filter`, `map`, `sorted`) and terminal operations (`reduce`, `collect`, `forEach`) in a stream pipeline.

Points for Discussion

1. Evaluate the trade-offs between catching exceptions at the point of occurrence versus propagating them to a centralized handler, using the Currency Watcher API integration as a concrete example.
2. Analyze the consequences of using a single catch (Exception e) block to handle all exception types, and discuss alternative approaches that preserve diagnostic specificity.
3. Compare the readability and maintainability of the anonymous inner class approach from Chapter 1 with the equivalent lambda expression refactoring presented in this chapter.

Suggested Extensions and Project Ideas

- Refactor the event handlers in the Chapter 1 case study (e.g., drag-and-drop handlers, button click handlers) from anonymous inner classes to lambda expressions, and compare the resulting code length and readability.

- Build a weather dashboard application that queries a public weather API, parses the JSON response using streams to extract temperature and humidity data, and throws a custom `ApiQuotaExceededException` when the rate limit is reached.
- Implement a chain-of-responsibility pattern for exception handling in which different handler objects process different exception types (e.g., network errors, parsing errors, validation errors), and compare this design with nested try-catch blocks.
- Create a stream-based grade calculator that reads a list of student records, uses `filter` to exclude incomplete entries, `map` to compute weighted scores, and `reduce` to determine class statistics such as average and standard deviation.

The concurrency primitives introduced at the conclusion of the case study, including raw thread creation and `Platform.runLater()`, represent a preliminary approach to background processing. The subsequent chapter provides a formal treatment of the JavaFX concurrency framework, extending these concepts into structured task management, thread-safe GUI updates, and the coordination of parallel workloads.

Progress Checklist

- ☐ Distinguish between checked and unchecked exceptions and identify which category a given exception belongs to.
- ☐ Implement a try-catch-finally block that handles multiple exception types with appropriate ordering.
- ☐ Propagate an exception from a method using the throws keyword and handle it at the call site.
- ☐ Define a custom exception class that extends Exception and throw it under a specific error condition.
- ☐ Rewrite an anonymous inner class as a lambda expression for a functional interface.
- ☐ Construct a stream pipeline that applies filter, map, and a terminal operation to a collection.
- ☐ Apply the map-reduce paradigm to aggregate values from a data set using stream operations.
- ☐ Integrate a Representational State Transfer (REST) API response into a JavaFX application with appropriate exception handling for network failures.

References

- AWS. (2026). Cloud Computing Services – Amazon Web Services. Retrieved May, 2026, from <https://aws.amazon.com/>
- Bloch, A. (2003). *Murphy's law*. Penguin.
- Oracle. (2026d). Exception (Java SE 26). Retrieved May, 2026, from <https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/lang/Exception.html>
- Oracle. (2026e). java.util.stream (Java SE 26). Retrieved May, 2026, from <https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/util/stream/package-summary.html>

CHAPTER 3

Concurrency

Prerequisites

- Completion of Chapters 1 and 2, including familiarity with JavaFX application structure, event handling, and stream pipelines.
- Confidence with lambda expressions and functional interfaces, as concurrent application programming interfaces (APIs) rely heavily on passing behaviour as arguments.
- Understanding of the try-catch mechanism for exception handling and the distinction between checked and unchecked exceptions.
- Experience with the Model-View-Controller pattern and background task execution using raw threads and `Platform.runLater()`, as introduced in the Chapter 2 case study.
- Basic understanding of processes, threads, and the concept of shared memory.

Expected Outcomes

- Explain why graphical user interface applications require background threads and identify the constraints imposed by the JavaFX Application Thread.
- Implement a Task subclass with progress reporting and bind its observable properties to user interface components.
- Distinguish between Task, Service, and ScheduledService and select the appropriate component for a given concurrency requirement.
- Submit multiple tasks to an ExecutorService and collect results using an ExecutorCompletionService.
- Identify data-race conditions in shared mutable state and apply synchronization mechanisms to prevent them.
- Decompose a computation into independent subtasks and execute them in parallel to reduce overall processing time.

MODERN commodity computers are commonly equipped with multicore central processing units (CPUs), facilitating the concurrent execution of multiple tasks. This capability is evident in common computing scenarios, where activities such as application development and media playback occur simultaneously on a single machine. Moreover, multicore CPUs can be leveraged to execute computationally demanding tasks in parallel, such as graphic rendering or simulations. However, the utilization of this parallel processing power is not an automatic feature of software; it requires the deliberate design of concurrent programs to effectively manage hardware resources and achieve performance improvements.

Conventional programming instruction often focuses on a sequential programming model, in which instructions are executed in a single, linear sequence. While effective for many applications, this sequential approach fails to utilize the parallel

processing capabilities of modern hardware, typically leaving most CPU cores idle. Concurrent programming offers an alternative paradigm, enabling applications to harness the power of multiple cores for improved performance and responsiveness.

An analogy can be used to illustrate the benefits and challenges of concurrency. Consider a group academic project. Ideally, the division of labor allows for faster completion than if one person performed all the work, potentially by a factor equal to the number of group members. However, this ideal speedup may not always be realized. Excessive communication overhead, such as frequent meetings, can reduce overall project efficiency. Conversely, insufficient communication can lead to integration errors that are discovered late in the process, causing further delays.

Similarly, achieving optimal speedup with concurrency in software development is challenging due to the inherent overhead of inter-thread communication or synchronization. If a computational problem can be partitioned into independent subtasks that require no inter-communication, a near-linear performance speedup may be attainable. Therefore, achieving an optimal balance between task decomposition and communication overhead is a crucial aspect of concurrent program design.

The fundamental units of execution in concurrent programming are processes and threads. A process is an instance of a running program that operates within an isolated memory space with dedicated resources; the concurrent execution of multiple processes is known as multiprocessing. A thread, by contrast, is a more lightweight unit of execution that exists within a process and shares its resources. This model, where a single process executes multiple instruction streams concurrently, is known as multithreading. The primary distinction between these two mechanisms lies in memory allocation. Because threads share the same memory space, inter-thread communication is typically less resource-intensive, and thread creation has lower overhead compared to process creation. This shared memory model generally affords multithreading a significant performance advantage for many applications.

3.1 Purpose

3.1.1 Concurrency and GUI programming

Multithreading is a fundamental technique for developing responsive and efficient graphical user interfaces (GUIs). By allowing multiple threads to execute concurrently, multithreading enables an application to perform background tasks while maintaining an interactive user interface (UI). This capability is particularly important for computationally intensive operations, during which the UI must remain responsive to user input.

A key advantage of multithreading in UI design is the ability to facilitate inter-thread communication and interruption. For instance, consider a video editing application where a rendering process is initiated on a separate thread. This design allows the user to continue interacting with the UI without interruption. If the user chooses to cancel the operation, a separate thread can signal the rendering thread to terminate, demonstrating the flexibility afforded by this concurrent approach.

In contrast, a single-threaded application would become unresponsive during such time-consuming operations. Figure 3.1 illustrates this problem: if a computationally intensive task is triggered by a button click and executed on the same thread that handles GUI events, the application will appear to freeze until the task completes. This lack of responsiveness significantly degrades the user experience, particularly in applications that require continuous interaction.

To mitigate the responsiveness issues inherent in single-threaded GUI applications, a standard design pattern employs at least two types of threads: the Event Dispatch Thread (EDT) and one or more worker threads. The EDT is dedicated exclusively to processing all GUI-related events, ensuring the prompt handling of user input. Worker threads, by contrast, are responsible for executing the long-running tasks triggered by these events.

To maintain UI responsiveness, all operations that modify the GUI must be

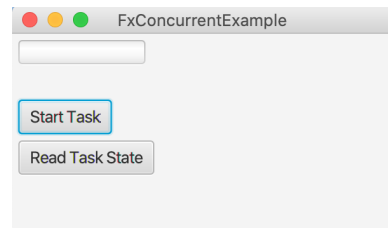


Figure 3.1: A JavaFX Application Demonstrating Concurrency Principles

executed on the EDT. Therefore, a fundamental principle of concurrent GUI programming is to delegate any time-consuming operations, such as file input/output (I/O), network requests, or complex calculations, to separate worker threads. By following this principle, developers can prevent the GUI from becoming unresponsive during these operations and ensure a smooth, interactive user experience.

3.1.2 Concurrency and accelerated computing

Multiprocessing is frequently employed for computationally intensive tasks where increased processing speed is the primary objective. In such scenarios, the operating system can allocate dedicated CPU cores and isolated memory spaces to each process, which can simplify development by reducing the need for explicit synchronization mechanisms. Multithreading, by contrast, operates on a shared memory model. This requires developers to explicitly manage resource access to prevent concurrency hazards such as data races, introducing additional implementation complexity. Despite this added complexity, the benefits of multithreading often extend beyond raw performance improvements.

A common consideration in concurrent programming is the magnitude of performance improvement that can be achieved. The degree of performance gain depends on several factors, most notably the nature of the algorithm and the available hardware resources. For computationally intensive tasks, such as those common in data science, the impact of concurrency can be substantial.

For example, a study by Phannachitta and Matsumoto (2019) involved conducting thousands of experiments to compare the performance of various machine learning algorithms on an empirical software engineering problem. Executing these experiments on a standard dual-core computer required nearly a month of computation time. However, by migrating the workload to a render farm equipped with dual 8-core Intel XEON CPUs, the total execution time was reduced to less than one week.

A subsequent, larger-scale experiment was conducted as part of research by Phannachitta (2020). By leveraging high-performance computing resources available through the Amazon Web Services (AWS) Cloud Credits for Research Program, this more complex experiment was completed in several days. Further tests utilizing a high-performance AWS instance (c5.24xlarge) demonstrated that a large-

scale data processing project, which would have required months of computation on a standard machine, could be completed in approximately one day. However, the significant operational cost of such high-performance instances raises concerns regarding their cost-effectiveness for applications where the return on investment is uncertain. For example, at the time of the study, the on-demand rental cost for a c5.24xlarge instance could exceed one million baht per year.

Synchronization

The fundamental principle of synchronization in concurrent programming is to coordinate the execution of multiple threads to prevent interference, particularly when accessing shared resources. This coordination is crucial for maintaining data integrity and consistency. Without effective synchronization, race conditions can occur, leading to unpredictable and erroneous program behavior.

Script 3.1: Illustrating a Race Condition with Unsynchronized Bank Transactions

```
+ 1  int withdraw (int account, int amount) {  
+ 2      double balance = readBalance(account);  
+ 3      double balance = balance - amount;  
+ 4      updateBalance(account, balance);  
+ 5      return balance;  
+ 6  }  
+ 7  int deposit (int account, int amount) {  
+ 8      double balance = readBalance(account);  
+ 9      double balance = balance + amount;  
+10      updateBalance(account, balance);  
+11      return balance;  
+12  }
```

Consider a scenario involving two threads operating on a shared bank account: one depositing funds and the other withdrawing, a scenario illustrated by the pseudocode in Script 3.1. Without synchronization, both threads could read the initial balance concurrently. If the deposit thread adds funds and writes the new balance, its update could be immediately overwritten by the withdrawal thread, which was

operating on the stale, initial balance. This would result in a final balance that incorrectly reflects only the withdrawal, compromising the integrity of the data.

Script 3.2 illustrates this problem through a minimal, executable example. In this script, two threads concurrently increment a shared counter variable 100,000 times. Because the `counter++` statement is not an atomic operation, it necessitates three distinct steps: reading the current value, incrementing that value by one, and writing the result back to memory. Consequently, the execution of these steps by the two threads can interleave. A thread may read the variable's value before the competing thread has completed its write operation, resulting in the silent loss of increments. Executing this program reliably produces a final count significantly below the expected 200,000, with the exact numerical result varying between executions.

Script 3.2: Data Race Two Threads Incrementing a Shared Counter Without Synchronization

```
+ 1 static int counter = 0;
+ 2
+ 3 public static void main(String[] args) throws Exception {
+ 4     Runnable task = () -> {
+ 5         for (int i = 0; i < 100_000; i++) {
+ 6             counter++;
+ 7         }
+ 8     };
+ 9     Thread t1 = new Thread(task);
+10     Thread t2 = new Thread(task);
+11     t1.start(); t2.start();
+12     t1.join(); t2.join();
+13     System.out.println("Counter = " + counter);
+14     // Expected: 200000 | Actual: varies, e.g. 137422
+15 }
```

Synchronized code blocks provide a solution by ensuring the consistent state of shared data objects. When a thread enters a synchronized block or method associated with an object, it acquires a lock on that object. This lock grants the thread exclusive access, preventing other threads from executing synchronized code on the same object. Upon exiting the block, the lock is released, and any modifications to the shared data are made visible to other threads, thereby ensuring data consistency.

Essentially, synchronization and concurrency represent opposing design considerations. Synchronization prioritizes data integrity by enforcing controlled, sequential access to shared resources, while concurrency aims to maximize performance by enabling the parallel execution of threads. The primary challenge is achieving an optimal balance between these two goals. Excessive synchronization can degrade performance by creating unnecessary contention and, in severe cases, deadlocks. Conversely, insufficient synchronization leads to data corruption due to uncoordinated access to shared resources.

This trade-off can be illustrated with an analogy. Consider a school hallway during a change of classes. Without any rules of passage, students might collide or create bottlenecks, delaying everyone. In a multithreaded program, synchronization mechanisms function like these rules. They direct the flow of threads and manage access to shared resources to prevent conflicts. Just as rules might dictate that students keep to one side of the hallway, synchronization primitives, such as locks, allow only one thread to modify a shared resource at a time. This management of access prevents data corruption and helps maintain the integrity of shared data structures.

In the context of GUI programming, multithreading is an essential technique for maintaining application responsiveness. By utilizing multiple threads, a GUI application can execute long-running operations in the background without interrupting the user's ability to interact with the interface. Without this concurrent approach, the application would become unresponsive during time-consuming tasks, leading to a poor user experience. Consequently, GUI frameworks typically employ a dedicated thread for all UI updates, often called the Event Dispatch Thread (EDT). All long-running operations, such as network I/O or intensive computations, should be delegated to separate worker threads to prevent blocking the EDT and ensure a fluid, interactive application.

3.2 The JavaFX Concurrent Framework

The `DrawGraphTask` class examined in Chapter 2 provided an initial examination of background task execution. By implementing the `Callable` interface and utilizing a `FutureTask`, the application maintained GUI responsiveness while rendering currency graphs. The JavaFX concurrent framework formalizes this pattern through the provision of higher-level abstractions, specifically `Task`, `Service`, and `ScheduledService`. These abstractions integrate directly with observable properties and the JavaFX Application Thread.

The framework is primarily composed of four core classes: `Task`, `Service`, `ScheduledService`, and `WorkerStateEvent`. The `Task`, `Service`, and `ScheduledService` classes all implement the `Worker` interface, enabling their instances to be executed on background threads. The `Worker.State` enum represents the distinct states of a `Worker` object during its lifecycle, all of which can be safely observed from the main application thread.

The primary distinction between `Task`, `Service`, and `ScheduledService` lies in their reusability and scheduling capabilities. An instance of the `Task` class is designed for a single execution and is not reusable. In contrast, instances of `Service` and `ScheduledService` represent reusable background tasks. The `ScheduledService` class, which extends the `Service` class, is specifically designed for tasks that must be executed repeatedly at specified intervals. Finally, an instance of the `WorkerStateEvent` class represents an event that is triggered in response to a state change in a `Worker` object. Event handlers can be registered to respond to these state-change events.

3.2.1 Components

The `Worker` interface

The `Worker<V>` interface defines the contract for an object that performs work on a background thread. The generic parameter `V` specifies the data type of the task's return value. A key feature of the `Worker` interface is that its state is observable, and any state changes are published on the main JavaFX Application Thread. This

design facilitates communication with the JavaFX Scene Graph and allows for the safe modification of UI components from background threads.

A `Worker` object transitions through a set of distinct states during its lifecycle, from creation to completion. These states are represented by the `Worker.State` enum, which includes the following predefined constants: `READY`, `SCHEDULED`, `RUNNING`, `SUCCEEDED`, `FAILED`, and `CANCELLED`.

Upon its creation, a `Worker` is in the `READY` state. It transitions to the `SCHEDULED` state just prior to execution and enters the `RUNNING` state once execution begins. If the task completes without errors, its final state becomes `SUCCEEDED`. Conversely, if an unhandled exception occurs during execution, the state transitions to `FAILED`. The `CANCELLED` state indicates that the `Worker`'s execution was intentionally terminated before completion.

The Task class

The `Task<V>` class is a concrete, one-time-use implementation of the `Worker<V>` interface. It is designed to encapsulate the logic for a background operation. To implement a `Task`, developers must override the abstract `call()` method, which contains the code to be executed on the background thread.

Like any `Worker`, the state of a `Task` instance can be monitored throughout its lifecycle. In addition to the standard `Worker` states, the `Task` class provides several observable properties that offer more detailed feedback, such as `progress`, `message`, and `value`. These properties are particularly useful for updating the UI to reflect the ongoing status of a background operation, such as displaying a progress bar.

The Service and ScheduledService class

The `Service<V>` class is designed for reusable background operations. Unlike a `Task`, which can only be executed once, a `Service` can be restarted multiple times. Internally, a `Service` creates and manages `Task` instances to perform its work. The `ScheduledService<V>` class extends `Service<V>` to provide functionality for tasks that must be executed repeatedly according to a defined schedule. It can be con-

figured to restart automatically after a successful completion or even after a failure, based on a specified delay.

3.2.2 Example: JavaFX with concurrent threads

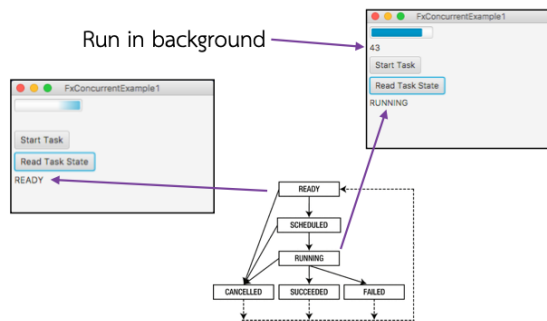


Figure 3.2: Graphical Representation of the FxConcurrentExample Application

Script 3.3 illustrates a multithreaded JavaFX application. Line 7 of the script presents the `call()` method, which contains the logic to be executed on a background thread (Lines 8-14). The program's functionality is to increment a counter from 1 to 50, with a one-second delay between increments, which is controlled by the `Thread.sleep(1000)` command in Line 12. This command requires exception handling, as discussed in the previous chapter. For brevity, this example uses a `throws Exception` declaration; however, a `try-catch-finally` block is the recommended practice for production code. Line 10 updates the progress bar to inform the user of the task's current execution status. For example, the progress bar will indicate 50% completion after 25 seconds. Line 11 updates a label on the GUI with the current value of the counter.

Outside the background task logic, the application's control elements are declared in Lines 17-22. These consist of a progress bar, two labels, and two buttons. The code from Line 23 onward defines the GUI layout. Figure 3.1 shows the displayed GUI of this example application.

Script 3.3: A Multithreaded JavaFX Application Example `FxConcurrentExample.java`

```

+ 1 //Imports are omitted
+ 2 public class FxConcurrentExample extends Application {
+ 3     @Override
+ 4     public void start(Stage primaryStage) {
+ 5         final Task task = new Task<Void>() {
+ 6             @Override
+ 7             protected Void call() throws Exception {
+ 8                 int max = 50;
+ 9                 for (int i = 1; i <= max; i++) {
+10                     updateProgress(i, max);
+11                     updateMessage(String.valueOf(i));
+12                     Thread.sleep(1000);
+13                 }
+14                 return null;
+15             }
+16         };
+17         ProgressBar pBar = new ProgressBar();
+18         pBar.setProgress(0);
+19         Label lblCount = new Label();
+20         final Label lblState = new Label();
+21         Button btnStart = new Button("Start Task");
+22         Button btnReadTaskState = new Button("Read Task State");
+23         VBox vBox = new VBox();
+24         vBox.setPadding(new Insets(5, 5, 5, 5));
+25         vBox.setSpacing(5);
+26         vBox.getChildren().addAll(pBar, lblCount, btnStart, btnReadTaskState,
+27                                 lblState);
+28         StackPane root = new StackPane();
+29         root.getChildren().add(vBox);
+30         Scene scene = new Scene(root, 300, 250);
+31         primaryStage.setTitle("FxConcurrentExample");
+32         primaryStage.setScene(scene);
+33         primaryStage.show();
+34     }
+35     public static void main(String[] args) { launch(args); }

```

Script 3.4 introduces four event handlers to the application, as shown in Lines 21, 23, 26, and 28. The first two handlers are bound to the task's observable properties and automatically synchronize the progress bar and message label with the task's current status. The remaining two handlers are associated with the buttons: execution of the background task is initiated by a click on the btnStart button, while a click on the btnReadTaskState button retrieves and displays the worker's current state in the lblState component. Figure 3.2 provides a graphical representation of the complete application logic.

Script 3.4: Multithreaded Example with Event Handler Integration
FxConcurrentExample.java

```
1 //Imports are omitted
2 public class FxConcurrentExample extends Application {
    ...
20     pBar.setProgress(0);
+ 21     pBar.progressProperty().bind(task.progressProperty());
22     Label lblCount = new Label();
+ 23     lblCount.textProperty().bind(task.messageProperty());
24     final Label lblState = new Label();
25     Button btnStart = new Button("Start Task");
+ 26     btnStart.setOnAction(t -> new Thread(task).start());
27     Button btnReadTaskState = new Button("Read Task State");
+ 28     btnReadTaskState.setOnAction(t -> lblState.setText(task.getState().toString()));
    ...
}
```

3.3 Example project: Download manager

The preceding analysis has illustrated two distinct applications of multithreading: enhancing GUI responsiveness and accelerating computational tasks. The following example integrates both principles within a single application. The project materials are available at https://github.com/mhrimaz/JavaFXWorkshop_06, and the application's GUI is depicted in Figure 3.3a. This project demonstrates the process of enhancing an existing application with new functionality.

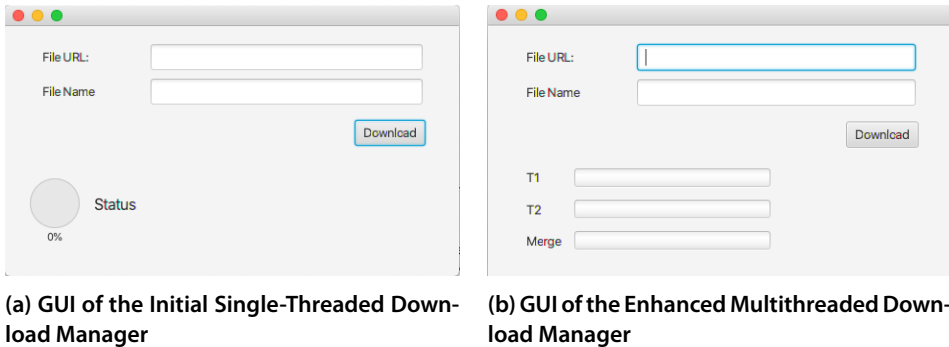


Figure 3.3: Comparison of Download Manager GUIs

The original version of the application is a basic download manager. Its core design involves dispatching a download task to a background thread when the *Download* button is clicked. The main application thread then monitors the download's progress and provides real-time updates to the GUI. The initial application is composed of three Java files: `Downloader.java`, `FXMLDocumentController.java`, and `JavaFXWorkshop_06.java`. The `Downloader` class is responsible for establishing the Uniform Resource Locator (URL) connection, reading the file's content into a buffer using a `ByteArrayOutputStream`, and writing the buffered data to local storage. The `FXMLDocumentController` class instantiates the various UI control elements and registers their corresponding event handlers. Finally, the `JavaFXWorkshop_06` class serves as the main application launcher.

This example demonstrates a practical utilization of threads in GUI programming, enhancing application performance by introducing multithreaded file download functionality. Because the target file is partitioned into distinct segments, or chunks, the download qualifies as an embarrassingly parallel task wherein each assigned thread executes independently. Specifically, this modification employs two concurrent downloader tasks to retrieve these segments simultaneously, culminating in the final state illustrated in Figure 3.3b. To minimize synchronization overhead during this concurrent execution, the application adheres to the following sequence of operations:

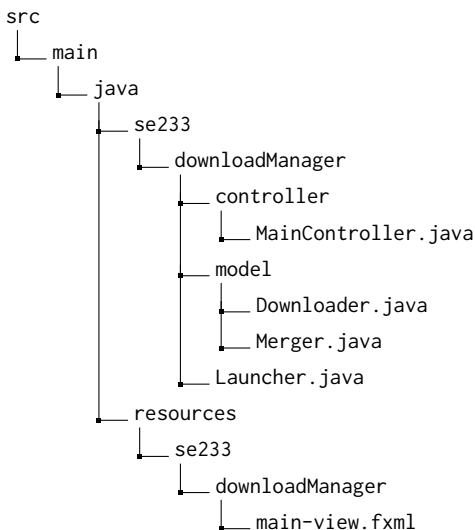
1. The total size of the target file is obtained.

2. The file size is divided by the number of download threads to determine the size of each data segment.
3. A fixed-size thread pool is created, and all download tasks are submitted for execution.
4. The main thread waits for the completion of all download tasks.
5. The downloaded data segments are merged into a single file.

The required modification involves refactoring the `Downloader` class to support multipart downloading and introducing a new `Merger` class responsible for combining the downloaded file segments.

From the original project, the `Downloader.java` and `FXMLDocument.fxml` files are retained, with the latter being renamed to `main-view.fxml`. The `JavaFXWorkshop_06.java` and `FXMLDocumentController.java` files are no longer required and can be removed. Their functionality is replaced by two new files: `MainController.java` and `Launcher.java`.

The structure of the modified project is as follows:



Initially, the module declarations within the `module-info.java` file must be modified as illustrated in Script 3.5.

Script 3.5: module-info.java

```
1 module se233.fxconcurrent {
2     requires javafx.controls;
+ 3     requires javafx.fxml;
+ 4     requires java.logging;
5     opens se233.downloadManager to javafx.fxml;
+ 6     opens se233.downloadManager.controller to javafx.fxml;
7     exports se233.downloadManager;
8     exports se233.downloadManager.controller;
9 }
```

The modified Downloader class is presented in Script 3.6. The variable declarations up to Line 13 introduce three new instance variables: `startByte`, `endByte`, and `doneSignal`. The `startByte` and `endByte` variables define the byte range that a specific downloader instance is responsible for retrieving. For example, in a two-downloader implementation, the `startByte` for the first downloader would be zero. The `startByte` for the second downloader would be set to the midpoint of the file, ensuring that each downloader operates on a distinct segment.

The `doneSignal` variable, of type `CountDownLatch`, provides a simple yet effective synchronization mechanism. This latch is used to signal the `Merger` class upon the completion of all download tasks. A controller class initializes the latch with a count equal to the number of downloader tasks (two, in this example). Upon completing its individual download task, each downloader thread decrements the latch's count by one. When the latch's count reaches zero, it releases any waiting threads, signaling to the `Merger` instance that all file segments have been downloaded and are ready for merging.

The implementation of the `Merger` class is presented in Script 3.7. The primary function of the `Merger` class is to wait for a start signal, which is implemented using the `await()` method of the `CountDownLatch` in Line 18. Upon receiving the signal, it iterates through the downloaded file segments. The segments are processed in sequential order, which is determined by the numeric suffix of each part's filename (Line 22). After each segment is read and appended to the final output file, the temporary segment file is deleted. This process is repeated until all segments have been merged.

Script 3.6: model/Downloader.java

```

1  //Imports are omitted
2  public class Downloader extends Task<Void> {
    ...
+ 5  private long startByte, endByte;
+ 6  private CountDownLatch doneSignal;
- 7  public Downloader(URL url, String fileName) {
+ 8  public Downloader(URL url, String fileName, long startByte, long endByte,
    CountDownLatch doneSignal) {
    ...
+ 11  this.startByte = startByte;
+ 12  this.endByte = endByte;
+ 13  this.doneSignal = doneSignal;
14  }
15  @Override
16  protected Void call() throws Exception {
17  try {
18      URLConnection openConnection = url.openConnection();
+ 19      openConnection.setRequestProperty("Range", "bytes="+ startByte+"-"+endByte);
20      int fileSize = openConnection.getContentLength();
- 21      ByteArrayOutputStream out;
- 22      try (InputStream in = new BufferedInputStream(url.openStream())) {
- 23          out = new ByteArrayOutputStream();
+ 24          OutputStream out = new FileOutputStream(this.fileName);
+ 25          InputStream in = openConnection.getInputStream();
26          byte[] buf = new byte[5120];
- 27          int n = 0;
28          long downloaded = 0;
- 29          while (-1 != (n = in.read(buf))) {
+ 30          while (downloaded < fileSize) {
+ 31              int n = in.read(buf, 0, buf.length);
+ 32              if (n != -1) {
33                  downloaded += n;
34                  out.write(buf, 0, n);
35                  updateProgress(downloaded, fileSize);
- 36                  updateMessage("Downloaded : " + (downloaded/(1024F*1024F*8F))+ " MB");
37              }
38          }
39          out.close();
+ 40          in.close();
- 41          byte[] response = out.toByteArray();
- 42          try (FileOutputStream fos = new FileOutputStream(fileName))
- 43              fos.write(response);
44      } catch (Exception e) { }
+ 45      doneSignal.countDown();
46      return null;
47  }
48  }

```

Script 3.7: model/Merger.java

```
+ 1 //Imports are omitted
+ 2 public class Merger extends Task<Void> {
+ 3     private String fileName;
+ 4     private int totalParts;
+ 5     private CountDownLatch startSignal;
+ 6     public Merger(String fileName, int totalParts, CountDownLatch startSignal) {
+ 7         this.fileName = fileName;
+ 8         this.totalParts = totalParts;
+ 9         this.startSignal = startSignal;
+10     }
+11     @Override
+12     protected Void call() {
+13         byte[] buf = new byte[4096];
+14         OutputStream out = null;
+15         InputStream in = null;
+16         File f = null;
+17         try {
+18             startSignal.await();
+19             out = new FileOutputStream(this.fileName);
+20             for (int i = 0; i < this.totalParts; i++) {
+21                 int toWrite;
+22                 f = new File(this.fileName + "-part" + (i+1));
+23                 in = new FileInputStream(f);
+24                 while ((toWrite = in.read(buf)) != -1) {
+25                     out.write(buf, 0, toWrite);
+26                 }
+27                 f.delete();
+28                 updateProgress((i + 1), this.totalParts);
+29                 in.close();
+30             }
+31             out.close();
+32         } catch (Exception e) {
+33             e.printStackTrace();
+34         }
+35         return null;
+36     }
+37 }
```

Script 3.8 presents the application's Launcher class, which is responsible for loading the user interface from an FXML file and attaching it to the main application scene. The application's view component is defined in an FXML file (Script 3.9). FXML is an XML-based markup language used for defining the structure of a JavaFX user interface.

A dedicated visual layout tool named SceneBuilder is available for designing FXML files. While some versions of IDEs like IntelliJ IDEA offered integrated support, it is now common practice to install SceneBuilder as a separate application. The main-view.fxml file can then be opened in the standalone SceneBuilder application (available for download at <https://gluonhq.com/products/scene-builder>) to visually edit the GUI layout.

The application's main controller, illustrated in Script 3.10, first binds its instance variables to the UI components defined in the FXML file. This binding is achieved through the use of the `fx:id` attribute within the FXML markup (e.g., Lines 41, 42, and 44 of Script 3.9). In Line 15 of the controller, the `CountDownLatch` is initialized with a count of two. Subsequently, a fixed-size thread pool is created with a corresponding size of two (Line 17). This thread pool is used to execute the two downloader tasks, which are instantiated in Lines 26-27 and submitted for concurrent execution in Lines 34-36. After all download tasks have completed, the thread pool is terminated (Line 37). This sequence concludes the entire multi-threaded download and merge operation.

Script 3.8: Launcher.java

```
+ 1 //Imports are omitted
+ 2 public class Launcher extends Application {
+ 3     @Override
+ 4     public void start(Stage stage) throws Exception {
+ 5         FXMLLoader fxmlLoader = new FXMLLoader(Launcher.class.getResource("main-view.
+             fxml"));
+ 6         Scene scene = new Scene(fxmlLoader.load());
+ 7         stage.setScene(scene);
+ 8         stage.show();
+ 9     }
+10     static void main(String[] args) { launch(args); }
+11 }
```

Script 3.9: /src/resources/se233/chapter3//main-view.fxml

```

10 //Imports are omitted
- 11 <AnchorPane id="AnchorPane" prefHeight="261.0" prefWidth="474.0" xmlns:fx="http://
    javafx.com/fxml/1" xmlns="http://javafx.com/javafx/8.0.65" fx:controller="
    javafxworkshop_06.FXMLDocumentController">
+ 12 <AnchorPane id="AnchorPane" prefHeight="261.0" prefWidth="474.0" xmlns:fx="http://
    javafx.com/fxml" fx:controller="se233.downloadManager.controller.MainController">
    ...
19 </GridPane>
20 <Button layoutX="365.0" layoutY="98.0" mnemonicParsing="false" onAction="#"
    handleDownloadAction" text="Download" />
- 21 <ProgressIndicator fx:id="progress" layoutX="25.0" layoutY="159.0" prefHeight="
    72.0" prefWidth="53.0" progress="0.0" stylesheets="@style.css" />
- 22 <Label fx:id="status" layoutX="93.0" layoutY="176.0" prefHeight="19.0" prefWidth
    ="345.0" text="Status" wrapText="true">
- 23 <font>
- 24 <Font name="Segoe UI Bold" size="16.0" />
- 25 </font>
- 26 </Label>
+ 27 <GridPane layoutX="39.0" layoutY="139.0" prefHeight="97.0" prefWidth="397.0">
+ 28 <columnConstraints>
+ 29 <ColumnConstraints hgrow="SOMETIMES" maxWidth="351.0" minWidth="10.0"
    prefWidth="49.0" />
+ 30 <ColumnConstraints hgrow="SOMETIMES" maxWidth="351.0" minWidth="10.0"
    prefWidth="206.0" />
+ 31 <ColumnConstraints hgrow="SOMETIMES" maxWidth="205.0" minWidth="10.0"
    prefWidth="142.0" />
+ 32 </columnConstraints>
+ 33 <rowConstraints>
+ 34 <RowConstraints minHeight="10.0" prefHeight="30.0" vgrow="SOMETIMES" />
+ 35 <RowConstraints minHeight="10.0" prefHeight="30.0" vgrow="SOMETIMES" />
+ 36 <RowConstraints minHeight="10.0" prefHeight="30.0" vgrow="SOMETIMES" />
+ 37 </rowConstraints>
+ 38 <children>
+ 39 <Label text="T1" GridPane.rowIndex="0" />
+ 40 <Label text="T2" GridPane.rowIndex="1" />
+ 41 <ProgressBar fx:id="thread1" prefWidth="200.0" progress="0.0" GridPane.
    columnIndex="1" />
+ 42 <ProgressBar fx:id="thread2" prefWidth="200.0" progress="0.0" GridPane.
    columnIndex="1" GridPane.rowIndex="1" />
+ 43 <Label text="Merge" GridPane.rowIndex="2" />
+ 44 <ProgressBar fx:id="merge_bar" prefWidth="200.0" progress="0.0" GridPane.
    columnIndex="1" GridPane.rowIndex="2" />
+ 45 </children>
+ 46 </GridPane>
47 </children>
48 </AnchorPane>

```

Script 3.10: controller/MainController.java

```
+ 1 //Imports are omitted
+ 2 public class MainController {
+ 3     @FXML
+ 4     private TextField urlField;
+ 5     @FXML
+ 6     private TextField fileField;
+ 7     @FXML
+ 8     private ProgressBar thread1,thread2,merge_bar;
+ 9
+10     long totalSizeOfFile;
+11     private static ExecutorService executor;
+12
+13     @FXML
+14     private void handleDownloadAction() {
+15         CountDownLatch countDownLatch = new CountDownLatch(2);
+16         try {
+17             executor = Executors.newFixedThreadPool(2);
+18
+19             URL url = new URL(urlField.getText());
+20             String filename = fileField.getText();
+21             URLConnection openConnection = url.openConnection();
+22
+23             totalSizeOfFile = openConnection.getContentLength();
+24             long baseLength = totalSizeOfFile/2;
+25
+26             Downloader downloader1 = new Downloader(url, filename+"-part1", 0,
+27                 baseLength, countDownLatch);
+28             Downloader downloader2 = new Downloader(url, filename+"-part2", (baseLength
+29                 +1),totalSizeOfFile, countDownLatch);
+30             Merger merger = new Merger(filename, 2, countDownLatch);
+31
+32             thread1.progressProperty().bind(downloader1.progressProperty());
+33             thread2.progressProperty().bind(downloader2.progressProperty());
+34             merge_bar.progressProperty().bind(merger.progressProperty());
+35
+36             executor.submit(downloader1);
+37             executor.submit(downloader2);
+38             executor.submit(merger);
+39             executor.shutdown();
+40         } catch (Exception e) {
+41             e.printStackTrace();
+42         }
+43     }
+44 }
```


3.4 Case Study

AI Advisory

Concurrency defects are among the most difficult software issues to reproduce and diagnose, and AI tools are consistently unreliable sources of threading advice. Thread-safety errors, such as updating a JavaFX node from a background thread or accessing a shared data structure without synchronization, do not always produce immediate, reproducible crashes. Instead, these errors frequently produce intermittent failures that may not surface until hours or days later. The prevention of such defects requires a deep comprehension of the threading model and the ability to reason about execution flow prior to writing any concurrent code.

Recommended Approach

When working through the concepts in this chapter, the generation of a Task implementation via an AI tool must be avoided. Instead, the development process should begin by identifying which data is shared between threads and which specific operations must be confined to the JavaFX Application Thread. The Task subclass must be constructed manually, ensuring that `Platform.runLater()` is utilized for every scene-graph update originating from a worker thread. Furthermore, potential race conditions must be tested by executing the application repeatedly under varying computational loads. Only after the implementation is functioning correctly, should an AI tool be consulted for code review or optimization suggestions.

Appropriate Uses

- ✓ Utilize an AI tool to clarify the distinctions between Task, Service, and ScheduledService after consulting the framework documentation provided in this chapter.

- ✓ Employ an AI tool to explain the specific function of `Platform.runLater()` and the circumstances under which it is required, after observing its usage within the codebase.
- ✓ Utilize an AI tool to verify that thread-unsafe operations have been annotated or guarded correctly, but only after the implementation is complete and functioning.

Practices to Avoid

- ✗ Do not request an AI tool to automatically refactor code to execute on a background thread. Such refactoring involves non-trivial architectural decisions regarding shared data and synchronization requirements.
- ✗ Do not accept an AI suggestion to add a synchronized keyword without first independently identifying the critical section and the specific shared state requiring protection.
- ✗ Do not employ an AI tool to debug a `ConcurrentModificationException` without first manually tracing which thread is modifying the collection and identifying the originating call site.
- ✗ Do not delegate the implementation of an `ExecutorCompletionService` to an AI tool, as the order in which concurrent results arrive holds significant semantic importance within this chapter's case study.

This chapter details the implementation of an inverted index creation application. When a set of Portable Document Format (PDF) files is placed into the application's designated drop zone, the application extracts all words, calculates their frequencies, and generates an inverted index. The terms within the index are then sorted in descending order based on their frequency. The resulting inverted index facilitates the rapid location of documents that contain specific search terms. The inverted index is a fundamental component of modern search engines, enabling the efficient retrieval of relevant documents or webpages in response to user

queries. Additionally, the sorted frequency count can serve as an indicator of a word's relative importance within a document corpus.

This case study follows a systematic, step-by-step implementation process, beginning with the GUI design. Subsequently, the implementation of the file-drop functionality and the core indexing logic are detailed. Finally, the application is enhanced through the integration of concurrency to improve performance. Figure 3.4 provides a visual representation of the final application GUI.

The project's file structure is outlined as follows:

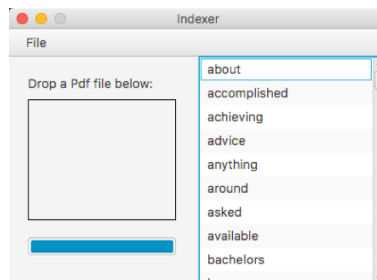
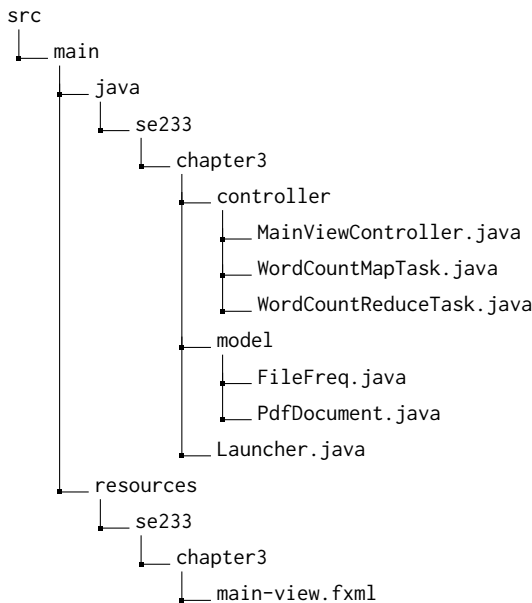


Figure 3.4: Inverted Index Creation Application Interface

The implementation begins with the modification of the `src/main/java/module-info.java` file, as shown in Script 3.11. The required `pdfbox` library can be added to the project as a Maven dependency, following the procedure detailed in the preceding chapter. Specifically, the `pom.xml` file is modified as illustrated in Script 3.12.

Script 3.11: module-info.java

```
1 module se233.chapter3 {
2     requires javafx.controls;
3     requires javafx.fxml;
+ 4     requires org.apache.pdfbox;
+ 5     requires org.apache.pdfbox.io;
6
7     opens se233.chapter3 to javafx.fxml;
8     opens se233.chapter3.controller to javafx.fxml;
+ 9     exports se233.chapter3;
10    exports se233.chapter3.controller;
11 }
```

Script 3.12: pom.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.
5         apache.org/xsd/maven-4.0.0.xsd">
6     <modelVersion>4.0.0</modelVersion>
7     ...
36     <version>${junit.version}</version>
37     <scope>test</scope>
38 </dependency>
+ 39 <dependency>
+ 40     <groupId>org.apache.pdfbox</groupId>
+ 41     <artifactId>pdfbox</artifactId>
+ 42     <version>3.0.7</version>
+ 43 </dependency>
44 </dependencies>
45 ...
```

3.4.1 Constructing the GUI with fxml

Within an integrated development environment (IDE) such as IntelliJ IDEA, the application’s GUI can be designed using the SceneBuilder toolkit. The GUI component definitions for this application are located in the `main-view.fxml` file. This file can be edited directly as XML within the IDE or visually manipulated using the standalone SceneBuilder application, which can be obtained from <https://gluonhq.com/products/scene-builder>, if not already installed.

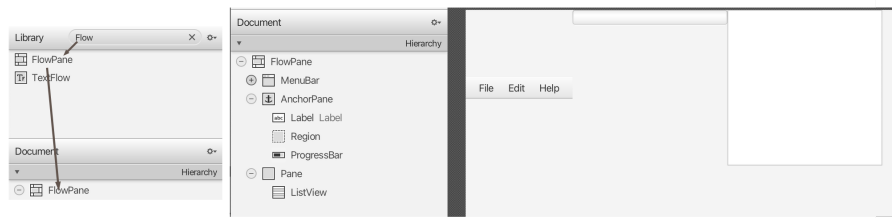


Figure 3.5: GUI Structure Tree Generated by SceneBuilder

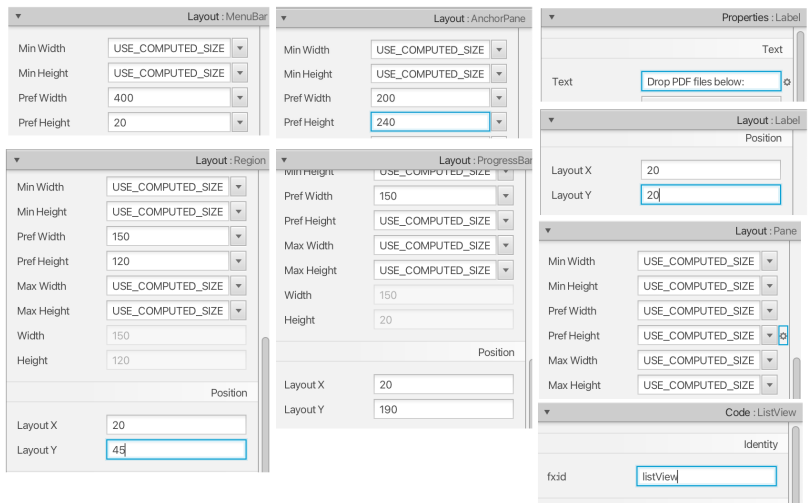


Figure 3.6: Summary of GUI Parameters for Configured Components

The GUI for this application is constructed by arranging JavaFX components in the Hierarchy pane of SceneBuilder. Components can be added from the Library pane, and their arrangement can be modified via drag-and-drop operations within the Hierarchy pane. After the components are assembled to match the structure shown in Figure 3.5, their properties must be manually configured according to the following specifications:

- For the `FlowPane` component, all size-related properties (Min Width through Max Height) under the Layout section are set to `USE_COMPUTED_SIZE`.
- For the `MenuBar` component, the `Pref Width` and `Pref Height` properties are set to 400 and 20, respectively.
- For the `AnchorPane` component, the `Pref Width` and `Pref Height` properties are set to 200 and 240, respectively.
- For the `Label` component, the `text` property is set to *Drop PDF files below:*, and its position is set with `Layout X = 20` and `Layout Y = 20`.
- For the `Region` component, its dimensions are set with `Pref Width = 150` and `Pref Height = 120`, and its position is set with `Layout X = 20` and `Layout Y = 45`. In the `Code` section, its `fx:id` is set to `dropRegion`. In the `Properties` section, its `Style` is set to `-fx-border-color: black;`.
- For the `ProgressBar` component, its `Pref Width` is set to 150, and its position is set with `Layout X = 20` and `Layout Y = 190`. In the `Code` section, its `fx:id` is set to `progressBar`.
- For the `Pane` component, all size-related properties are set to `USE_COMPUTED_SIZE`.
- For the `ListView` component, its `Pref Width` and `Pref Height` are set to 200 and 240, respectively. In the `Code` section, its `fx:id` is set to `listView`.

Figure 3.6 provides a summary of these configuration steps. Upon completion, the GUI layout will be rearranged as depicted in Figure 3.7. Saving the FXML file and returning to the IDE will show that the underlying XML markup has been modified to reflect these configured parameters.

The final step is to associate a controller class with the FXML layout. This is accomplished in SceneBuilder by navigating to the Controller section and setting the Controller class field to `se233.chapter3.controller.MainViewController`, as illustrated in Figure 3.8.

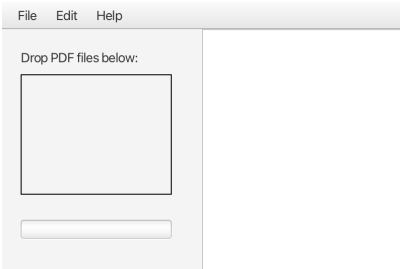


Figure 3.7: GUI Layout After Parameter Configuration

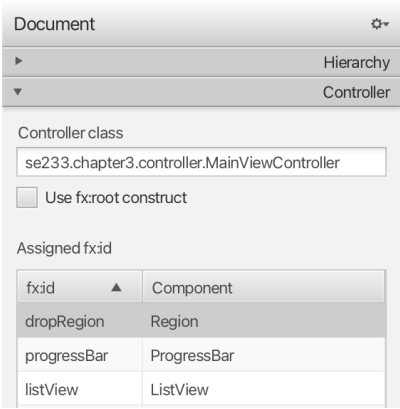


Figure 3.8: Declaration of the Main Application Controller

3.4.2 The drop-file functionality

The drag-and-drop functionality implemented in the previous chapter can be adapted to support the file-drop feature required for this application. The application launcher,

presented in Script 3.13, is structured similarly to the previous example. Script 3.14 introduces a controller for the view elements. This controller enables the designated drop region of the application window to accept PDF files for subsequent processing. In Line 13, a check is performed to verify that the dropped items are files and that they are in the PDF format. If this condition is met, the Dragboard accepts the transfer. The handler for the `setOnDragDropped` event (Lines 27-31) then iterates through each accepted PDF file and outputs its filename to the console. At this stage of development, the implementation can be tested by executing the application and confirming that dropped PDF files are correctly processed.

Script 3.13: Launcher.java

```
+ 1 //Imports are omitted
+ 2 public class Launcher extends Application {
+ 3     @Override
+ 4     public void start(Stage stage) throws Exception {
+ 5         FXMLLoader fxmlLoader = new FXMLLoader(Launcher.class.getResource("main-
+             view.fxml"));
+ 6         Scene scene = new Scene(fxmlLoader.load());
+ 7         stage.setTitle("Indexer");
+ 8         stage.setScene(scene);
+ 9         stage.show();
+10     }
+11     static void main(String[] args) { launch(args); }
+12 }
```


Script 3.14: controller/MainViewController.java

```
+ 1 //Imports are omitted
+ 2 public class MainViewController {
+ 3     @FXML
+ 4     private Region dropRegion;
+ 5     @FXML
+ 6     private ProgressBar progressBar;
+ 7     @FXML
+ 8     private ListView listView;
+ 9     @FXML
+10     public void initialize() {
+11         dropRegion.setOnDragOver(event -> {
+12             Dragboard db = event.getDragboard();
+13             final boolean isAccepted = db.getFiles().get(0).getName().toLowerCase().
+                endsWith(".pdf");
+14             if (db.hasFiles() && isAccepted) {
+15                 event.acceptTransferModes(TransferMode.COPY);
+16             } else {
+17                 event.consume();
+18             }
+19         });
+20         dropRegion.setOnDragDropped(event -> {
+21             Dragboard db = event.getDragboard();
+22             boolean success = false;
+23             if (db.hasFiles()) {
+24                 success = true;
+25                 String filePath;
+26                 int total_files = db.getFiles().size();
+27                 for (int i = 0; i < total_files; i++) {
+28                     File file = db.getFiles().get(i);
+29                     filePath = file.getAbsolutePath();
+30                     System.out.println(filePath);
+31                 }
+32                 progressBar.setProgress(100);
+33             }
+34             event.setDropCompleted(success);
+35             event.consume();
+36         });
+37     }
+38 }
```

3.4.3 Pdf file extraction

The generation of an inverted index first requires the extraction of all words from the provided PDF files. The data structures for storing the document information are defined in the `FileFreq` and `PdfDocument` classes (Scripts 3.15 and 3.16). Accessor and mutator methods should be implemented for both classes, following the approaches demonstrated in previous chapters. With the model and view structures established, the subsequent step is to implement the application's core logic. This logic primarily involves the word-processing procedure, which encompasses two main steps: (1) extracting all words from each PDF file and counting their intra-document frequencies, and (2) merging these frequency counts across all documents to create the final index. The `WordCountMapTask` class is responsible for the initial extraction and frequency counting, while the `WordCountReduceTask` class is responsible for merging these counts. These two classes are presented in Scripts 3.17 and 3.18, respectively.

Script 3.15: `model/FileFreq.java`

```
+ 1 //Imports are omitted
+ 2 public class FileFreq {
+ 3     private String name;
+ 4     private String path;
+ 5     private Integer freq;
+ 6     public FileFreq(String name, String path, Integer freq) {
+ 7         this.name = name;
+ 8         this.path = path;
+ 9         this.freq = freq;
+10     }
+11     @Override
+12     public String toString() {
+13         return String.format("%s:%d", name, freq);
+14     }
+15 }
```

Script 3.16: model/PdfDocument.java

```

+ 1 //Imports are omitted
+ 2 public class PdfDocument {
+ 3     private String name;
+ 4     private String filePath;
+ 5     private PDDocument document;
+ 6     private LinkedHashMap<String, List<FileFreq>> uniqueSets;
+ 7     public PdfDocument(String filePath) throws IOException {
+ 8         this.name = Paths.get(filePath).getFileName().toString();
+ 9         this.filePath = filePath;
+10         File pdfFile = new File(filePath);
+11         this.document = Loader.loadPDF(new RandomAccessReadBufferedFile(pdfFile));
+12     }
+13 }

```

Script 3.17: controller/WordCountMapTask.java

```

+ 1 //Imports are omitted
+ 2 public class WordCountMapTask{
+ 3     private Map<String, FileFreq> wordCount;
+ 4     public WordCountMapTask(PdfDocument doc) throws IOException {
+ 5         PDFTextStripper reader = new PDFTextStripper();
+ 6         Pattern pattern = Pattern.compile(" ");
+ 7         String s = reader.getText(doc.getDocument());
+ 8         this.wordCount = pattern.splitAsStream(s)
+ 9             .map(word -> word.replaceAll("[^a-zA-Z]", "").toLowerCase().trim())
+10             .filter(word -> word.length() > 3)
+11             .map(word -> new AbstractMap.SimpleEntry<>(word, 1))
+12             .collect(toMap(e -> e.getKey(), e -> e.getValue(), (v1, v2) -> v1 + v2))
+13             .entrySet()
+14             .stream()
+15             .filter(e -> e.getValue() > 1)
+16             .collect(Collectors.toMap(e -> e.getKey(), e -> new FileFreq(doc.getName
+17                                     (), doc.getFilePath(), e.getValue())));
+18 }

```

Script 3.18: controller/WordCountReduceTask.java

```

+ 1 //Imports are omitted
+ 2 public class WordCountReduceTask {
+ 3     private LinkedHashMap<String, List<FileFreq>> uniqueSets;
+ 4     public WordCountReduceTask(Map<String, FileFreq>[] wordMap) {
+ 5         List<Map<String, FileFreq>> wordMapList = new ArrayList<>(Arrays.asList(
+ 6             wordMap));
+ 7         this.uniqueSets = wordMapList.stream()
+ 8             .flatMap(m -> m.entrySet().stream())
+ 9             .collect(Collectors.groupingBy(
+10                 e->e.getKey(),
+11                 Collector.of(
+12                     () -> new ArrayList<FileFreq>(),
+13                     (list, item) -> list.add(item.getValue()),
+14                     (current_list, new_items) -> {
+15                         current_list.addAll(new_items);
+16                         return current_list; })
+17             ));
+18         .entrySet()
+19         .stream()
+20         .sorted(Map.Entry.comparingByKey())
+21         .collect(Collectors.toMap(e->e.getKey(), e->e.getValue(),
+22             (v1,v2)->v1, () -> new LinkedHashMap<>()));
+23     }
+24 }

```

These two classes utilize the Stream API and lambda expressions (Oracle, 2026e), which significantly reduces the amount of required implementation code. The stream pipeline within the WordCountMapTask class, beginning on Line 8, performs the following sequence of operations:

1. The input text is split into a stream of words using whitespace as the delimiter.
2. Each word is normalized by removing all non-alphabetic characters and converting it to lowercase.
3. Each normalized word is then mapped to an integer value of 1, representing a single occurrence.

4. A reduce operation is performed to collect and sum the occurrences of each unique word within the document.
5. A new stream is created from the resulting map of words and their frequencies.
6. The stream is filtered to retain only words with a frequency count greater than one.
7. A final map is generated, associating each word with a tuple containing the filename, file path, and frequency count.

In the `WordCountReduceTask` class, the code in Lines 8-16 creates a stream from the word-frequency maps of all processed documents. A reduce operation is then applied to aggregate the frequency counts for each word across all documents. The code in Lines 17-21 then sorts the aggregated results to produce the application's final output. At this stage of the implementation, the output is sorted alphabetically by word; a subsequent modification will change the sorting order to be based on frequency.

With the individual model, view, and controller classes prepared, the final step is their integration. Script 3.19 presents the modified `MainViewController`, which replaces the previous file-name printing logic with the new word-processing tasks. In Lines 37-38, an array of `WordCountMapTask` instances and a `Map` data structure are created. These are utilized within the `for-loop` (Lines 39-53), where each PDF file is processed individually to extract its word-frequency data. The output of this mapping phase is stored in the `wordMap` variable. This collection of maps is then passed to a `WordCountReduceTask` instance for the final reduction and merging step (Line 42). The final, merged result is then passed to the GUI for display.

Script 3.19: controller/MainViewController.java

```

1  //Imports are omitted
2  public class MainViewController {
3      ...
30     dropRegion.setOnDragDropped(event -> {
31         Dragboard db = event.getDragboard();
32         boolean success = false;
33         if (db.hasFiles()) {
34             success = true;
35             String filePath;
36             int total_files = db.GetFiles().size();
+ 37             WordCountMapTask[] wordCountMapTaskArray = new WordCountMapTask[total_files];
+ 38             Map<String, FileFreq>[] wordMap = new Map[total_files];
39             for (int i = 0; i < total_files; i++) {
- 40                 File file = db.GetFiles().get(i);
- 41                 filePath = file.getAbsolutePath();
- 42                 System.out.println(filePath);
+ 43                 try {
+ 44                     File file = db.GetFiles().get(i);
+ 45                     filePath = file.getAbsolutePath();
+ 46                     PdfDocument p = new PdfDocument(filePath);
+ 47                     wordCountMapTaskArray[i] = new WordCountMapTask(p);
+ 48                     wordMap[i] = wordCountMapTaskArray[i].getWordCount();
+ 49                     progressBar.setProgress(i / wordCountMapTaskArray.length * 100);
+ 50                 } catch (IOException e) {
+ 51                     e.printStackTrace();
+ 52                 }
53             }
+ 54             WordCountReduceTask merger = new WordCountReduceTask(wordMap);
+ 55             LinkedHashMap<String, List<FileFreq>> uniqueSets = merger.getUniqueSets();
+ 56             listView.getItems().addAll(uniqueSets.entrySet());
57             progressBar.setProgress(100);
58         }
59         event.setDropCompleted(success);
60         event.consume();
61     });
62 }
63 }

```

At this stage, the application's functionality satisfies the initial requirements. However, several usability enhancements could be implemented. These potential improvements include the following:

- The addition of a *Start Indexing* button, which would allow a user to initiate the indexing process manually after dropping files.
- An enhancement to the readability of the result list displayed in the right-hand pane.
- Functionality to allow a user to click on an entry in the result list to open the corresponding PDF file.
- A mechanism, such as a loading screen or progress indicator, to inform the user that word extraction is being processed in the background.

To incorporate a *Start Indexing* button, several visual elements and their associated logic require modification. For instance, the static Region serving as the drop zone can be replaced with a `ListView` to display the list of files queued for indexing. Additionally, the existing `ProgressBar` could be considered optional, and its allocated space repurposed for the new button.

These GUI modifications can be accomplished using `SceneBuilder` or by directly editing the `FXML` markup, as shown in Script 3.20. Scripts 3.21 and 3.22 present the corresponding updates to the `MainViewController` class. In the updated controller, the `setOnDragOver` handler is now associated with the `inputListView` component (Line 22). Notably, the handler no longer initiates the indexing process immediately upon a file drop; instead, this task is now triggered by the new button. The event handler for this new button is detailed in Script 3.22. The `setOnAction` handler contains the logic that was previously executed immediately upon a file drop. This

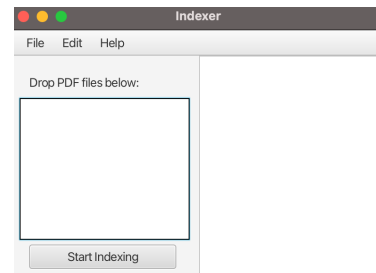


Figure 3.9: Application Interface with Added *Start Indexing* Button

logic (Lines 61-70) iterates through the list of PDF files in the `inputListView`, performs the word extraction and frequency count for each, merges the results into a single data structure, and displays the final index in the right-hand pane of the GUI.

Executing the application after the update outlined in Script 3.22 will yield the rendered results as depicted in Figure 3.9.

Script 3.20: `src/resources/se233/chapter3/main-view.fxml`

```

1 //Imports are omitted
2 <FlowPane xmlns="http://javafx.com/javafx/8.0.121" xmlns:fx="http://javafx.com/fxml/1">
3   <children>
4     ...
36   <AnchorPane prefHeight="240.0" prefWidth="200.0">
37     <children>
38       <Label layoutX="20.0" layoutY="20.0" text="Drop a PDF file below:" />
- 39       <Region fx:id="dropRegion" layoutX="20.0" layoutY="45.0" prefHeight="120.0
+ 40         " prefWidth="150.0" style="-fx-border-color: black;" />
- 41       <ListView fx:id="inputListView" layoutX="10" layoutY="45.0" prefHeight="150"
+ 42         prefWidth="180" style="-fx-border-color: black;" />
- 41       <ProgressBar layoutX="20.0" layoutY="190.0" prefWidth="150.0" progress="0.0" />
+ 42       <Button fx:id="startButton" layoutX="20.0" layoutY="200.0" prefHeight="20.0"
        prefWidth="156.0" text="Start Indexing"/>
43     </children>
44   </AnchorPane>
45   <Pane>
46     <children>
47       <ListView fx:id="listView" prefHeight="240.0" prefWidth="200.0" />
48     </children>
49   </Pane>
50 </children>
51 </FlowPane>

```


Script 3.21: controller/MainViewController.java

```

1 //Imports are omitted
...
18 public class MainViewController {
+ 19     LinkedHashMap<String, List<FileFreq>> uniqueSets;
20     @FXML
- 21     private Region dropRegion;
+ 22     private ListView<String> inputListView;
23     @FXML
- 24     private ProgressBar progressBar;
+ 25     private Button startButton;
26     @FXML
27     private ListView listView;
28     @FXML
29     public void initialize() {
- 30         dropRegion.setOnDragOver(event -> {
+ 31         inputListView.setOnDragOver(event -> {
...
36     });
- 37     dropRegion.setOnDragDropped(event -> {
+ 38     inputListView.setOnDragDropped(event -> {
39         Dragboard db = event.getDragboard();
40         boolean success = false;
41         if (db.hasFiles()) {
...
43             int total_files = db.GetFiles().size();
44             WordCountMapTask[] wordCountMapTaskArray = new WordCountMapTask[total_files];
45             Map<String, FileFreq>[] wordMap = new Map[total_files];
46             for (int i = 0; i < total_files; i++) {
- 47                 try {
48                     File file = db.GetFiles().get(i);
49                     filePath = file.getAbsolutePath();
+ 50                     inputListView.getItems().add(filePath);
- 51                     PdfDocument p = new PdfDocument(filePath);
- 52                     wordCountMapTaskArray[i] = new WordCountMapTask(p);
- 53                     wordMap[i] = wordCountMapTaskArray[i].getWordCount();
- 54                     progressBar.setProgress(i / wordCountMapTaskArray.length * 100);
- 55                 } catch (IOException e) {
- 56                     e.printStackTrace();
- 57                 }
58             }
- 59             WordCountReduceTask merger = new WordCountReduceTask(wordMap);
- 60             LinkedHashMap<String, List<FileFreq>> uniqueSets = merger.getUniqueSets();
- 61             listView.getItems().addAll(uniqueSets.entrySet());
- 62             progressBar.setProgress(100);
63         }
...

```

Script 3.22: controller/MainViewController.java

```
1  //Imports are omitted
2  public class MainViewController {
3      private LinkedHashMap<String,ArrayList<FileFreq>> uniqueSets;
4      ...
53     event.setDropCompleted(success);
54     event.consume();
55     });
+56     startButton.setOnAction(event -> {
+57         List<String> inputListViewItems = inputListView.getItems();
+58         int total_files = inputListViewItems.size();
+59         WordCountMapTask[] wordCountMapTaskArray = new WordCountMapTask[total_files];
+60         Map<String, FileFreq>[] wordMap = new Map[total_files];
+61         for (int i = 0; i < total_files; i++) {
+62             try {
+63                 String filePath = inputListViewItems.get(i);
+64                 PdfDocument p = new PdfDocument(filePath);
+65                 wordCountMapTaskArray[i] = new WordCountMapTask(p);
+66                 wordMap[i] = wordCountMapTaskArray[i].getWordCount();
+67             } catch (IOException e) {
+68                 e.printStackTrace();
+69             }
+70         }
+71         WordCountReduceTask merger = new WordCountReduceTask(wordMap);
+72         uniqueSets = merger.getUniqueSets();
+73         listView.getItems().addAll(uniqueSets.keySet());
+74     });
75 }
76 }
```

3.4.4 Map and ListView

The next functionality to be implemented is user interaction with the list of extracted words. This feature will allow a user to click an item in the list to open the corresponding PDF file from which the word was extracted. This process involves modifications to two classes: the Launcher and the MainViewController classes, as illustrated in Scripts 3.23 and 3.24, respectively. In Script 3.23, the `primaryStage` variable is declared as a static class variable to provide global access for controlling GUI elements. A new `HostServices` variable, `hs`, is also introduced. This object provides a mechanism to interact with the host operating system, which in this case is used to open PDF files. This `hs` variable is utilized in Line 80 of Script 3.24.

Script 3.23: Launcher.java

```
1 //Imports are omitted
  ...
8 public class Launcher extends Application {
+ 9     public static Stage primaryStage;
+ 10     public static HostServices hs;
11     @Override
12     public void start(Stage stage) throws Exception{
+ 13         primaryStage = stage;
+ 14         hs = getHostServices();
15         FXMLLoader fxmlLoader = new FXMLLoader(Launcher.class.getResource("main-view.
            fxml"));
16         Scene scene = new Scene(fxmlLoader.load());
- 17         stage.setTitle("Indexer");
- 18         stage.setScene(scene);
- 19         stage.show();
+ 20         primaryStage.setTitle("Indexer");
+ 21         primaryStage.setScene(scene);
+ 22         primaryStage.show();
23     }
24     public static void main(String[] args) { launch(args); }
25 }
```

In Script 3.24, a new `setOnMouseClicked` event handler is implemented for the `ListView`. The underlying data structure enabling this functionality is a `Map`. A `Map` is used to associate the displayed information (the key) with related, but non-displayed, data (the value). In this implementation, the displayed word-frequency string serves as the key, while the value is an object containing additional details, such as the file path. The `for`-loop in Lines 78-82 populates this `Map`. Consequently, when a user clicks an item in the list, the handler for the `setOnMouseClicked` event (Line 84) uses the selected item as a key to retrieve the associated file path from the `Map` and open the corresponding document.

Script 3.24: controller/MainViewController.java

```

1  //Imports are omitted
2  public class MainViewController {
    ...
73      listView.getItems().addAll(uniqueSets.keySet());
74  };
+ 75      listView.setOnMouseClicked(event -> {
+ 76          List<FileFreq> listOfLinks = uniqueSets.get(listView.getSelectionModel().
              getSelectedItem());
+ 77          ListView<FileFreq> popupListView = new ListView<>();
+ 78          LinkedHashMap<FileFreq,String> lookupTable = new LinkedHashMap<>();
+ 79          for (int i=0 ; i<listOfLinks.size() ; i++) {
+ 80              lookupTable.put(listOfLinks.get(i),listOfLinks.get(i).getPath());
+ 81              popupListView.getItems().add(listOfLinks.get(i));
+ 82          }
+ 83          popupListView.setPrefWidth(Region.USE_COMPUTED_SIZE);
+ 84          popupListView.setPrefHeight(popupListView.getItems().size() * 40);
+ 85          popupListView.setOnMouseClicked(innerEvent -> {
+ 86              Launcher.hs.showDocument("file:///"+lookupTable.get(popupListView.
                  getSelectedItem().getSelectedItem()));
+ 87              popupListView.getScene().getWindow().hide();
+ 88          });
+ 89          Popup popup = new Popup();
+ 90          popup.getContent().add(popupListView);
+ 91          popup.show(Launcher.primaryStage);
+ 92      });
93  }
94  }
```

3.4.5 Multithreaded indexing implementation

Indexing tasks are typically performed on large document corpora, an operation that often demands substantial computational power. This example leverages multithreading to create concurrent indexer tasks. To enable a class to be executed concurrently in JavaFX, a common approach is to have it implement the `Callable` interface. This requires the implementation of a `call()` method, which serves as the entry point for the background task. An instance of such a class can then be submitted for execution on a background thread. Scripts 3.25 and 3.26 modify the `WordCountMapTask` and `WordCountReduceTask` classes accordingly. The submission of these tasks is initiated by the event handler for the `startButton` within the `MainViewController` class.

The final version of the `MainViewController` class is presented in Script 3.27. In this implementation, two service instances are declared: an `ExecutorService` and an `ExecutorCompletionService` (Lines 59-60). The `ExecutorCompletionService` is introduced here for the first time. This class decorates an `ExecutorService` to assist in managing asynchronous tasks, providing a way to retrieve results as they become available. This service allows for the retrieval of results from completed tasks in the order they finish, facilitating further processing.

As shown in Line 71, the updated `WordCountMapTask` instance is submitted to the `ExecutorCompletionService` for each PDF file that requires processing. The results from these completed map tasks are then retrieved within a `for-loop` (Lines 76-83). Within this loop, the `take()` method of the completion service is used, which returns a `Future` object representing the result of the next completed task. A `Future` represents the result of an asynchronous computation; it provides methods to check if the computation is complete and to retrieve its result once it is. Similarly, the `WordCountReduceTask` is submitted to an `ExecutorService` (Line 86), and a `Future` is used to retrieve the final result of this reduction task. Overall, the modifications to these classes exemplify a common approach to managing concurrent data processing tasks in a JavaFX application.

Script 3.25: controller/WordCountMapTask.java

```

1  //Imports are omitted
   ...
-15 public class WordCountMapTask{
+16 public class WordCountMapTask implements Callable<Map<String,FileFreq>> {
-17     private Map<String, FileFreq> wordCount;
-18     public WordCountMapTask(PdfDocument doc) throws IOException {
+19     private PdfDocument doc;
+20     public WordCountMapTask(PdfDocument doc) throws IOException {
+21         this.doc = doc;
+22     }
+23     @Override
+24     public Map<String, FileFreq> call() throws Exception {
+25         Map<String, FileFreq> wordCount;
+26         PDFTextStripper reader = new PDFTextStripper();
+27         Pattern pattern = Pattern.compile(" ");
+28         String s = reader.getText(doc.getDocument());
-29         this.wordCount = pattern.splitAsStream(s)
+30         wordCount = pattern.splitAsStream(s)
+31             .map(word -> word.replaceAll("[^a-zA-Z]", "").toLowerCase().trim())
+32             .filter(word -> word.length() > 3)
+33             .map(word -> new AbstractMap.SimpleEntry<>(word, 1))
+34             .collect(toMap(e -> e.getKey(), e -> e.getValue(), (v1, v2) -> v1 + v2))
+35             .entrySet()
+36             .stream()
+37             .filter(e -> e.getValue() > 1)
+38             .collect(Collectors.toMap(e -> e.getKey(), e -> new FileFreq(doc.getName
+39                 (),doc.getFilePath(),e.getValue())));
+40         return wordCount;
+41     }
-41     public Map<String, FileFreq> getWordCount() {
-42         return wordCount;
-43     }
-44     public void setWordCount(Map<String, FileFreq> wordCount) {
-45         this.wordCount = wordCount;
-46     }
+47 }

```

Script 3.26: controller/WordCountReduceTask.java

```

1  //Imports are omitted
   ...
- 9  public class WordCountReduceTask {
+ 10 public class WordCountReduceTask implements Callable<LinkedHashMap<String, List<
      FileFreq>>> {
- 11     private LinkedHashMap<String, List<FileFreq>> uniqueSets;
- 12     public WordCountReduceTask(Map<String, FileFreq>[] wordMap) {
+ 13     private Map<String, FileFreq>[] wordMap;
+ 14     public WordCountReduceTask(Map<String, FileFreq>[] wordMap) {
+ 15         this.wordMap = wordMap;
+ 16     }
+ 17     @Override
+ 18     public LinkedHashMap<String, List<FileFreq>> call() throws Exception {
+ 19         LinkedHashMap<String, List<FileFreq>> uniqueSets;
+ 20         List<Map<String, FileFreq>> wordMapList = new ArrayList<>(Arrays.asList(
            wordMap));
- 21         this.uniqueSets = wordMapList.stream()
+ 22         uniqueSets = wordMapList.stream()
+ 23             .flatMap(m -> m.entrySet().stream())
+ 24             .collect(Collectors.groupingBy(
+ 25                 e->e.getKey(),
+ 26                 Collector.of(
+ 27                     () -> new ArrayList<FileFreq>(),
+ 28                     (list, item) -> list.add(item.getValue()),
+ 29                     (current_list, new_items) -> {
+ 30                         current_list.addAll(new_items);
+ 31                         return current_list; })
+ 32                 ))
+ 33             .entrySet()
+ 34             .stream()
+ 35             .sorted(Map.Entry.comparingByKey())
+ 36             .collect(Collectors.toMap(e->e.getKey(), e->e.getValue(),
+ 37                 (v1,v2)->v1, () -> new LinkedHashMap<>()));
+ 38         return uniqueSets;
+ 39     }
- 40     public LinkedHashMap<String, List<FileFreq>> getUniqueSets() {
- 41         return uniqueSets;
- 42     }
- 43     public void setUniqueSets(LinkedHashMap<String, List<FileFreq>> uniqueSets) {
- 44         this.uniqueSets = uniqueSets;
- 45     }
+ 46 }

```

Script 3.27: controller/MainViewController.java

```

1  //Imports are omitted
2  public class MainViewController {
    ...
58      startButton.setOnAction(event -> {
+ 59          ExecutorService executor = Executors.newFixedThreadPool(4);
+ 60          final ExecutorCompletionService<Map<String, FileFreq>> completionService = new
              ExecutorCompletionService<>(executor);
61          List<String> inputListViewItems = inputListView.getItems();
62          int total_files = inputListViewItems.size();
- 63          WordCountMapTask[] wordCountMapTaskArray = new WordCountMapTask[total_files];
64          Map<String, FileFreq>[] wordMap = new Map[total_files];
65          for (int i = 0; i < total_files; i++) {
66              try {
67                  String filePath = inputListViewItems.get(i);
68                  PdfDocument p = new PdfDocument(filePath);
- 69                  wordCountMapTaskArray[i] = new WordCountMapTask(p);
- 70                  wordMap[i] = wordCountMapTaskArray[i].getWordCount();
+ 71                  completionService.submit(new WordCountMapTask(p));
72              } catch (IOException e) {
73                  e.printStackTrace();
74              }
75          }
+ 76          for (int i = 0; i < total_files; i++) {
+ 77              try {
+ 78                  Future<Map<String, FileFreq>> future = completionService.take();
+ 79                  wordMap[i] = future.get();
+ 80              } catch (Exception e) {
+ 81                  e.printStackTrace();
+ 82              }
+ 83          }
+ 84          try {
85              WordCountReduceTask merger = new WordCountReduceTask(wordMap);
+ 86              Future<LinkedHashMap<String, List<FileFreq>>> future = executor.submit(
                  merger);
- 87              uniqueSets = merger.getUniqueSets();
+ 88              uniqueSets = future.get();
89              listView.getItems().addAll(uniqueSets.keySet());
+ 90          } catch (Exception e) {
+ 91              e.printStackTrace();
+ 92          } finally {
+ 93              executor.shutdown();
+ 94          }
95      });
    ...

```


3.4.6 Incorporating a Loading Indicator

Providing feedback to the user during computationally intensive operations, such as the inverted index creation in this application, is an essential aspect of responsive design. In this example, this feedback is provided by a loading indicator, specifically a spinner, which is displayed within the *Start Indexing* button. It is important to note that, by default, JavaFX event handlers execute on the main JavaFX application thread. Consequently, if a long-running task is executed directly within an event handler, the application thread will be blocked, preventing any GUI updates until the task completes. This makes the application appear unresponsive. Therefore, a specific technique is required to perform the long-running task on a background thread while updating the loading indicator on the main application thread.

A standard approach for this is to encapsulate the time-consuming logic within a Task object, which can be executed on a background thread. Script 3.28 presents the modified MainViewController class that incorporates this loading indicator. The code in Lines 51-58 defines the call() method for the Task, into which the existing event handler logic has been relocated. A ProgressIndicator instance is introduced to serve as the visual loading indicator and is associated with the button. The code in Lines 98-100 declares an event handler that binds the visibility of the loading indicator to the running state of the background task. Finally, the code in Lines 101-103 creates and starts a new background thread to execute the Task when the button is clicked.

At this final stage of implementation, the application allows a user to drop one or more PDF files onto its interface. The application then lists the paths of these files for confirmation. The user can then initiate the indexing process by clicking the *Start Indexing* button. Upon this action, the computationally intensive indexing tasks are executed on background threads. While these tasks are running, a loading indicator is displayed to inform the user that processing is underway. Upon completion, the final inverted index is displayed in the right-hand pane. Users can interact with this result list; clicking an entry opens the corresponding PDF file associated with the selected term.

Script 3.28: controller/MainViewController.java

```
1  //Imports are omitted
...
25 public class MainViewController {
    ...
62     startButton.setOnAction(event -> {
+ 63         Parent bgRoot = Launcher.primaryStage.getScene().getRoot();
+ 64         Task<Void> processTask = new Task<Void>() {
+ 65             @Override
+ 66             public Void call() throws IOException {
+ 67                 ProgressIndicator pi = new ProgressIndicator();
+ 68                 VBox box = new VBox(pi);
+ 69                 box.setAlignment(Pos.CENTER);
+ 70                 Launcher.primaryStage.getScene().setRoot(box);
71                 ExecutorService executor = Executors.newFixedThreadPool(4);
                ...
93                 try {
94                     WordCountReduceTask merger = new WordCountReduceTask(wordMap);
                        ...
98                 } catch (Exception e) {
99                     e.printStackTrace();
100                } finally {
101                    executor.shutdown();
102                }
+ 103                return null;
+ 104            }
+ 105        };
+ 106        processTask.setOnSucceeded( e -> {
+ 107            Launcher.primaryStage.getScene().setRoot(bgRoot);
+ 108        });
+ 109        Thread thread = new Thread(processTask);
+ 110        thread.setDaemon(true);
+ 111        thread.start();
112    });
    ...
}
```

3.5 Exercise

1. Modify the sorting logic for the result `ListView`. The entries, which are currently sorted alphabetically, should instead be ordered by their total frequency count in descending order. For words that appear in multiple documents, the total frequency is the sum of their frequencies across all documents.
2. Enhance the display format for words that appear in multiple PDF files. The frequency counts from each document should be displayed individually and in descending order. For example, if the word *about* appears in three separate documents with frequencies of 1, 4, and 2, the corresponding entry in the `ListView` should be formatted as: *about* (4, 2, 1).
3. Refactor the input `ListView` (the file drop zone) to display only the filename for each dropped file, rather than the full file path, which can be difficult to read.
4. Incorporate a `MenuBar` at the top of the application window. This menu bar must contain a *File* menu with, at a minimum, a *Close* option that terminates the application.
5. Implement functionality to close the document-view window that appears after clicking a term in the result list. This should be accomplished by registering an event handler that listens for the *Escape* (ESC) key press.

3.6 Discussion and Summary

This chapter has introduced the principles and practical mechanisms of concurrent programming within the JavaFX framework. The discussion began by addressing the necessity of delegating long-running computations to background threads to preserve GUI responsiveness. The JavaFX concurrent framework, specifically `Task`, `Service`, and `ScheduledService`, was examined as the standard abstraction for managing these operations. This was followed by an exploration of `ExecutorService` and `ExecutorCompletionService` for coordinating multiple asyn-

chronous tasks. Furthermore, the chapter analyzed the fundamental tension between parallelism and data integrity, illustrating how unsynchronized access to shared mutable state produces data races. Finally, the case study synthesized these concepts through an inverted-index builder that processes PDF files in parallel utilizing a map-reduce pattern.

Knowledge Outcomes

Upon the completion of this chapter, a student should demonstrate the ability to:

- Explain how the single-threaded rendering model of modern GUI frameworks necessitates the delegation of long-running computations to background threads to prevent interface freezing.
- Differentiate between the three levels of abstraction in the JavaFX concurrent framework: one-shot Task operations, reusable Service components, and periodic ScheduledService executions.
- Bind observable properties of Worker objects (such as progress and message) directly to user interface controls to maintain responsiveness without relying on manual callbacks.
- Manage thread pools utilizing an ExecutorService and process asynchronous results in their order of completion via an ExecutorCompletionService.
- Identify data race conditions caused by unsynchronized thread access to shared mutable variables, and explain the resulting non-deterministic behaviors.
- Apply synchronization primitives, including synchronized blocks and atomic variables, to enforce mutual exclusion while evaluating the architectural trade-offs between coarse-grained and fine-grained locking.

Points for Discussion

1. The case study processes each dropped PDF file utilizing a separate thread submitted to an executor. Under what specific circumstances might adding

more threads *decrease* overall throughput, and how does the ratio of CPU-bound work to I/O-bound work influence the optimal thread pool size?

2. Synchronization successfully eliminates data races but introduces the possibility of a deadlock condition, wherein two threads each hold a lock that the other requires. What specific strategies, such as strict lock ordering or timeout-based acquisition, can prevent deadlocks without sacrificing the safety guarantees provided by synchronization?
3. The inverted-index builder in the case study employs a map-reduce pattern, where each thread independently indexes one document (the map phase), and the results are subsequently merged into a single data structure (the reduce phase). How does this pattern generalize to computational problems outside of text processing, and what specific properties must a problem possess to benefit from this decomposition strategy?

Suggested Extensions and Project Ideas

1. Replace the raw synchronized counter in the data-race example with a `java.util.concurrent.atomic.AtomicInteger`, and compare the resulting code clarity and performance characteristics.
2. Incorporate a cancellation button into the case-study indexer that invokes `ExecutorService.shutdownNow()`, and observe how each running Task responds to an interrupt signal during the PDF parsing phase.
3. Convert the one-shot indexing operation into a `ScheduledService` that rescans a designated folder at a fixed interval, automatically detecting newly added PDF files and updating the result list.
4. Introduce a shared `ConcurrentHashMap` as the merge target during the reduce phase, and measure whether concurrent merging improves overall wall-clock execution time compared to the current sequential merge implementation.

Progress Checklist

- ☐ Explain why long-running operations must execute on a worker thread rather than the JavaFX Application Thread.
- ☐ Implement a Task subclass that reports progress and bind its observable properties to a progress bar and status label.
- ☐ Distinguish between Task, Service, and ScheduledService and select the appropriate class for a given scenario.
- ☐ Submit multiple Callable tasks to an ExecutorService and retrieve results through an ExecutorCompletionService.
- ☐ Identify a data race condition in a shared-counter example and explain why the observed result differs from the expected value.
- ☐ Apply synchronized blocks or methods to eliminate a race condition and verify correctness.
- ☐ Decompose a computation into independent subtasks, assign each subtask to a separate thread or executor, and combine the partial results into a final output.
- ☐ Trace the lifecycle states of a Worker object from READY through SUCCEEDED, FAILED, or CANCELLED.

References

- Oracle. (2026e). `java.util.stream` (Java SE 26). Retrieved May, 2026, from <https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/util/stream/package-summary.html>
- Phannachitta, P. (2020). On an optimal analogy-based software effort estimation. *Information and Software Technology*, 125, 106330.
- Phannachitta, P., & Matsumoto, K. (2019). Model-based software effort estimation—A robust comparison of 14 algorithms widely used in the data science community. *International Journal of Innovative Computing, Information and Control*, 15(2), 569–589.

Debugging

Prerequisites

- Completion of Chapters 1 through 3, encompassing the ability to construct and execute a JavaFX application utilizing FXML-based layouts and background threads.
- Comprehension of exception handling (try-catch-finally) and the distinction between checked and unchecked exceptions, as detailed in Chapter 2.
- Proficiency in the Model-View-Controller (MVC) pattern and event-driven control flow, as the debugging exercises within this chapter operate exclusively on MVC-structured applications.
- Access to an integrated development environment (IDE), such as IntelliJ IDEA, with a configured Java project, as breakpoint-based debugging necessitates dedicated IDE tooling.

Expected Outcomes

- Describe the limitations of print-statement probing and explain the rationale for utilizing structured logging as a more sustainable diagnostic strategy.
- Configure the Log4j2 logging framework utilizing a properties file, properly selecting logging levels, appenders, and layouts.
- Employ the IntelliJ IDEA debugger to establish breakpoints, step through execution flows, and inspect variable states within a running JavaFX application.
- Implement sprite animation techniques, encompassing rendering, movement, collision detection, and frame interpolation.
- Integrate structured logging at appropriate severity levels within a game loop to facilitate both development-time diagnostics and post-release evidence collection.

FAULTS and errors are an inherent part of the software development and production lifecycle. A method may produce an incorrect value, or an application may terminate unexpectedly due to exceptions. The potential causes of errors are numerous, including data input errors, incorrect parameter naming, improper variable type declarations, misuse of operators or operands, flawed computational implementation, and inappropriate algorithm selection. In essence, a fault is defined as a condition causing a software application to fail to perform its intended function, while an error refers to the difference between the actual output and the expected output. The occurrence of a fault or error indicates the presence of a defect within the source code that requires identification and correction. The process of identifying, localizing, and correcting these defects is known as debugging.

4.1 Debugging Methodologies

Faults and errors are common occurrences during software development. While some issues, such as syntax errors, are readily identified by the compiler, defects related to program logic or algorithmic flaws are often more challenging to diagnose. In most cases, the diagnosis of such defects requires an examination of the application's behavior with specific input data to identify patterns of incorrectness. Upon identifying a potential defect, the relevant software components must be isolated and examined individually. Debugging in practice involves a diverse range of tools and methodologies. This chapter will focus on three categories that are widely employed in the industry.

4.1.1 Probing

A fundamental debugging method is known as *probing*. This technique involves observing the runtime state of a variable or the execution flow of a method by inserting print statements into the source code. For example, to ascertain the exact value of a variable `x`, the code snippet `System.out.println(x)` is inserted into the program adjacent to the line of code under scrutiny. Additional print statements can subsequently be added to observe how the state of `x` changes during execution. This approach is frequently utilized due to its simplicity, particularly when the developer possesses a strong hypothesis regarding the location of the defect. However, this practice exhibits significant limitations when compared to more comprehensive debugging techniques.

The primary drawbacks of this technique include its inherent lack of flexibility and the inefficient process of relocating print statements to various investigation points. Furthermore, observing multiple variables simultaneously can become highly cumbersome. Crucially, these diagnostic code fragments must be manually removed after the defect is resolved, as their continued presence can introduce unintended side effects or performance degradation. In summary, probing serves as a straightforward initial approach to troubleshooting but rapidly becomes unmanageable as code complexity increases.

These limitations are significantly amplified in graphical user interface (GUI) applications. Because a JavaFX application executes event handlers and background

tasks on separate threads, concurrent `System.out.println` output from multiple threads can arrive in an unpredictable order, making it exceedingly difficult to reconstruct the actual sequence of events. The console output quickly becomes disorganized when print statements are scattered across controllers, model classes, and worker threads, particularly because they lack any indication of severity or origin. Furthermore, probing offers no mechanism to selectively enable or disable output for a specific component; every statement is either executed unconditionally or must be manually commented out. Perhaps most critically, developers frequently neglect to remove print statements prior to packaging an application for release, resulting in diagnostic messages persisting in production environments, where they become visible to end users or silently consume input/output (I/O) resources. These cumulative shortcomings necessitate the adoption of a structured logging framework. Such a framework provides level-based filtering, timestamped output, configurable destinations, and the capacity to remain in the codebase permanently without interfering with production output.

Script 4.1: Using `System.out.println()` to Verify Application Loading

```
1 public class HelloApplication extends Application {
2     @Override
3     public void start(Stage stage) throws IOException {
4         FXMLLoader fxmlLoader = new FXMLLoader(HelloApplication.class.getResource(
5             "hello-view.fxml"));
6         Scene scene = new Scene(fxmlLoader.load(), 320, 240);
7         stage.setTitle("Hello!");
8         stage.setScene(scene);
9         stage.show();
10        System.out.println("Successfully loaded.");
11    }
12
13    public static void main(String[] args) {
14        launch();
15    }
16 }
```

Script 4.2: Using `System.out.println()` to Verify a Button Click Event

```
1 public class HelloController {
2     @FXML
3     private Label welcomeText;
4
5     @FXML
6     protected void onHelloButtonClick() {
7         welcomeText.setText("Welcome to JavaFX Application!");
+ 8         System.out.println("Button clicked.");
9     }
10 }
```

Scripts 4.1 and 4.2 illustrate scenarios where this probing technique is used to confirm successful application loading and button click events, respectively. This approach can be beneficial when a fault occurs without an immediately obvious point of failure.

4.1.2 Logging

Logging is the recommended alternative to simple probing, offering significant improvements in both functionality and practicality. It is widely recognized as a best practice in modern software development, facilitating future maintenance and support. The practice of logging involves recording messages during program execution to a persistent or observable location, such as a file, a database, or the console. Each log entry typically comprises essential information, including a timestamp, severity level, and a formatted message containing contextual information.

This logging mechanism enables the reporting and persistence of informational, error, and warning messages, which can be later retrieved and analyzed. These captured log data are increasingly regarded as valuable assets in software development. In essence, logging allows for a comprehensive understanding of an application's behavior in production through the examination of log entries. More broadly, logging assets are utilized in mining software repositories, a process that analyzes data generated during software development to uncover meaningful and actionable information about software systems and projects. These assets play an important

role in maintaining high levels of collaboration and ensuring quality in numerous open-source software projects, including Mozilla Firefox, Eclipse, and LibreOffice.

In the Java ecosystem, logging is typically implemented using dedicated application programming interfaces (APIs), such as Log4j (2026) or SLF4J (2026). The essential components of a logging framework include:

- Logging frameworks;
- Level;
- Logger;
- Appender;
- Layout;
- Configuration;

Logging frameworks

The implementation of logging in a Java application requires the use of a logging framework. Java provides a built-in framework in the `java.util.logging` package, which contains the essential components for creating and persisting log messages. These components encompass objects, methods, and configuration options. However, this built-in framework is generally considered less capable and is less frequently used in production systems compared to third-party alternatives such as Log4j2 and SLF4J.

In modern Java development, the combination of SLF4J as a facade with a Log4j2 binding has emerged as a predominant logging architecture. This architecture provides extensive flexibility, allowing the underlying logging implementation to be changed without modifying the application code.

Scripts 4.3 and 4.4 demonstrate the integration of these logging techniques into the Hello application. The use of `System.out.println()` commands is replaced with logging calls. Line 3 of each script demonstrates the logger declaration. Subsequent calls to methods such as `logger.info()` and `logger.debug()` illustrate the replacement of `System.out.println()` statements with structured logging calls. A more comprehensive discussion of the configuration details for the Log4j2 framework is presented in the case study section of this chapter.

Script 4.3: Replacing Probing with Logging for Application Status

```
1 public class HelloApplication extends Application {
2
+ 3     private static final Logger logger = LogManager.getLogger(HelloApplication.class);
4
5     @Override
6     public void start(Stage stage) throws IOException {
7         FXMLLoader fxmlLoader = new FXMLLoader(HelloApplication.class.getResource("hello
            -view.fxml"));
8         Scene scene = new Scene(fxmlLoader.load(), 320, 240);
9         stage.setTitle("Hello!");
10        stage.setScene(scene);
11        stage.show();
- 12        System.out.println("Successfully loaded.");
+ 13        logger.info("Successfully loaded.");
14    }
15    public static void main(String[] args) {
16        launch();
17    }
18 }
```

Script 4.4: Replacing Probing with Logging for Event Status

```
1 public class HelloController {
2
+ 3     static final Logger logger = LogManager.getLogger(HelloController.class);
4
5     @FXML
6     private Label welcomeText;
7
8     @FXML
9     protected void onHelloButtonClick() {
10        welcomeText.setText("Welcome to JavaFX Application!");
- 11        System.out.println("Button clicked.");
+ 12        logger.debug("Button clicked.");
13    }
14 }
```

Logging levels

Logging levels are standardized categories assigned to log entries to indicate their severity or importance. In a production environment, messages with high severity often signify unexpected application behavior that could lead to negative consequences, such as data corruption or service unavailability. Clearly defined levels are crucial for enabling developers and operators to quickly assess the urgency of a given log entry. For instance, a log entry designated as a severe error requires immediate attention, whereas an entry containing routine informational data may only be relevant for certain types of analysis. A set of standard levels is widely adopted across numerous programming languages. The following list presents these common log levels, ordered from most to least severe:

- **FATAL:** Represents a critical error condition that will likely lead to the termination of the application. Due to their high severity, FATAL-level events require immediate intervention.
- **ERROR:** Represents a significant error that, while not necessarily causing application termination, prevents a primary business function from being fulfilled. The application may continue to operate but in an erroneous or degraded state. This level should be used judiciously to avoid diluting the significance of critical error reports.
- **WARN:** Indicates a potentially harmful situation or an unexpected event from which the application can recover. This level is appropriate for non-critical errors where a fix can be postponed or where the system is attempting a recovery action, such as reconnecting to a service.
- **INFO:** Represents messages that highlight the routine progress and state transitions of the application. This information is valuable for tracking normal application behavior in both development and production environments, such as user login events or major service initializations.
- **DEBUG:** Provides fine-grained diagnostic information useful for debugging application logic. DEBUG-level logs are primarily intended for use by developers during the development and troubleshooting phases.

- **TRACE:** Provides the most granular level of diagnostic information, offering more detail than DEBUG. In many practices, TRACE-level logs are used for highly detailed, step-by-step execution analysis.

Two additional pseudo-levels, OFF and ALL, are also available. OFF is used to disable all logging, while ALL enables the recording of all messages regardless of their level.

The operational principle of logging levels is based on a configurable threshold. During runtime, the application code generates logging requests, each with an associated severity level. The logging framework is configured with a specific log level that acts as this threshold. A logging request is processed only if its severity level is equal to or greater than the configured threshold level; otherwise, the request is discarded. The standard hierarchical order of severity is: TRACE < DEBUG < INFO < WARN < ERROR < FATAL. For instance, if the logging framework's threshold is set to ERROR, only logging requests with a level of ERROR or FATAL will be recorded.

Loggers

Loggers are named entities responsible for capturing log events. After applying a level-based filter, they forward the qualifying log events to the appropriate Appenders. They provide developers with fine-grained, runtime control over which log statements are processed and their destinations.

Appender

Appenders are components responsible for directing formatted log events to their final output destinations. These destinations can range from the console and local files to more complex systems like databases or remote servers. The following list outlines some widely used Appenders:

- **ConsoleAppender:** Appends log events to the console by passing formatted entries to `System.out` or `System.err`, with `System.out` as the default target. `System.err` can be used alternatively, often to segregate error messages. While frequently used during development due to its simplicity, it is generally less practical in production environments.

- **FileAppender:** Appends log events to a file at a predefined location. This is a common choice for production environments, as file-based logs can be archived, retrieved, and analyzed post-execution. This is crucial when addressing software defects, as timestamps can aid in identifying when a problem occurred.
- **RollingFileAppender:** A FileAppender variant that “rolls over” log files when certain conditions are met (e.g., the file reaches a specific size). This is widely used in production systems to manage log file growth and simplify archival.
- **DailyRollingFileAppender:** A time-based variation of RollingFileAppender that creates a new log file at a scheduled interval, typically daily.
- **SMTPAppender:** Formats log events as emails, typically used to provide immediate notification for ERROR or FATAL level events. This concept of real-time notification has been adapted for modern communication platforms, with custom appenders that send alerts to services like instant messaging applications.
- **JDBCAppender:** Appends log events by persisting them to a relational database. This structured storage facilitates complex querying and analysis of log data.

Layout

Layouts are components responsible for formatting a log event into a specific string representation before it is written by an Appender. Logging frameworks typically offer layouts for various formats, including plain text, HTML, and XML. In practice, a useful log entry includes not only the message but also contextual metadata (e.g., timestamp and thread name) that aids in diagnostics.

A common layout used in Java is `PatternLayout`, which allows for a highly customizable log format based on a conversion pattern string. An example format string is `%d{yyyy/MM/dd HH:mm:ss} [%-5p] %c1 - %m%n`, which produces log entries such as `2025/01/01 08:00:00 [DEBUG] HelloWorld - Message Logged`.

Understanding the conversion specifiers is essential for interpreting a `PatternLayout` out. The following list outlines some key specifiers:

- **%d...**: Formats the timestamp of the log event. The pattern inside the braces (e.g., `yyyy/MM/dd HH:mm:ss`) follows standard date formatting rules.
- **%p**: Displays the severity level of the log event (e.g., `DEBUG`, `INFO`, `ERROR`). Modifiers like `-5` can be used for padding.
- **%t**: Displays the name of the thread that generated the log event.
- **%c...**: Displays the name of the logger that generated the event. The number in braces specifies the precision (e.g., `1` shows only the simple class name).
- **%M**: Displays the name of the method where the logging request was issued.
- **%L**: Displays the line number of the source code where the logging request was issued.
- **%m**: Outputs the application-supplied log message.
- **%n**: Outputs the platform-specific line separator.

Configuration

The variety of available logging frameworks has led to the development of diverse configuration formats. This section focuses on the configuration format for the `Log4j2` framework, a widely adopted solution. Configuring `Log4j2` typically involves creating a `log4j2.properties` file within the project's resource folder. Script 4.5 provides an illustrative example of such a file. In this example, the root logger's level is set to `DEBUG`, and a single `ConsoleAppender` is configured as the output destination.

During program execution, when a logging request is made, the logging framework uses this configuration to determine the appropriate action. If the request's level meets the configured threshold, the framework formats the log message according to the specified `PatternLayout` and forwards it to all configured appenders.

Script 4.5: resources/se233/chapter4/log4j2.properties

```

+ 1 name = PropertiesConfig
+ 2 appender.console.type = Console
+ 3 appender.console.name = consoleLogger
+ 4 appender.console.layout.type = PatternLayout
+ 5 appender.console.layout.pattern = [%5p] %d{yyyy-MM-dd HH:mm:ss.SSS} [%t] %c{1} %
    msg%n
+ 6
+ 7 rootLogger.level = debug
+ 8 rootLogger.appenderRef.stderr.ref = consoleLogger

```

4.1.3 Debugging

Debugging is the systematic process of identifying and resolving defects within a running program. In contrast to the static analysis of stack traces, interactive debugging requires a dedicated debugger tool, which is essential when the source of an error is not immediately apparent. Such cases often involve subtle defects in logic or algorithms that may not manifest until a specific runtime condition is met, long after the faulty code has executed. Resolving these types of issues can be particularly challenging.

The central principle of interactive debugging is the ability to inspect the program's state at any point during its execution. This is accomplished by setting breakpoints, which pause the program's execution at specific lines of code, allowing for the analysis of variable values. A debugger thus enables a detailed examination of how variable values change as the program progresses from one statement to the next. This capability facilitates the identification of unexpected state changes that may indicate the root cause of a defect.

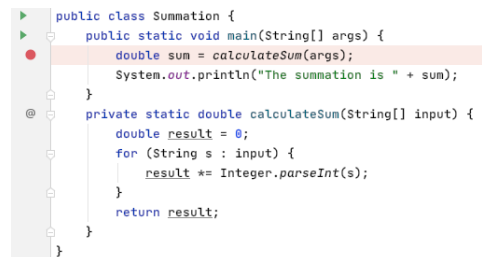


Figure 4.1: Establishing a Breakpoint in IntelliJ IDEA

To illustrate the debugging process, consider a simple scenario. Script 4.6 presents a program intended to calculate the sum of integer values provided as command-

line arguments. The program compiles and executes without errors; however, it produces an incorrect result. For an input of 1 2 3 4 5, the expected output is 15, but the program produces 0. In this situation, the first step is to form a hypothesis about the probable source of the error. Because this is a logical error rather than a compilation error, it must be investigated at runtime.

Script 4.6: Example Class for Demonstrating Debugging**Summation.java**

```
+ 1 public class Summation {
+ 2     public static void main(String[] args) {
+ 3         double sum = calculateSum(args);
+ 4         System.out.println("The summation is " + sum);
+ 5     }
+ 6     private static double calculateSum(String[] input) {
+ 7         double result = 0;
+ 8         for (String s : input) {
+ 9             result *= Integer.parseInt(s);
+10         }
+11         return result;
+12     }
+13 }
```

Using a debugger, a breakpoint can be set at any line of code. This breakpoint instructs the debugger to pause program execution at that specific line, allowing for an examination of the current values of all declared variables. Typically, a breakpoint is set just before the line of code where a defect is suspected. The following example demonstrates this process in the IntelliJ IDEA IDE. Initially, a breakpoint is set at Line 3 of Script 4.6. This is typically done by clicking in the area next to the line number, which places a red circle icon at that location, as shown in Figure 4.1. The program is then executed in debug mode.

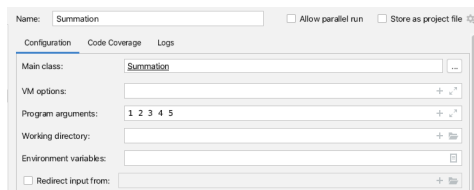


Figure 4.2: Configuring Program Arguments for Debugging in IntelliJ IDEA

To execute the program in debug mode within IntelliJ IDEA, the run configuration must first be edited by selecting `Run > Edit Configurations...` from the main menu. In the configuration window, the program arguments that produced the incorrect result (1 2 3 4 5) are entered. Figure 4.2 provides a visual illustration of this step.

The debug execution mode is then initiated via the *Debug* button, typically located next to the *Run* button. This action starts a debugging session, and program execution pauses at the predefined breakpoint. The Debug tool window, illustrating the current debugging process, is displayed in Figure 4.3. As shown in the tool window, because execution is paused before the invocation of the `calculateSum()` method, its local variables have not yet been initialized.

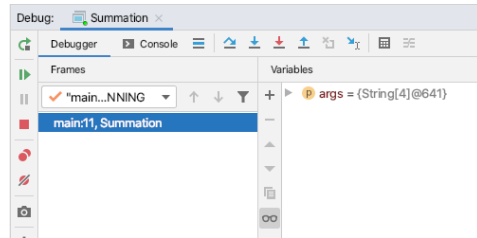


Figure 4.3: Debugger Tool Window in IntelliJ IDEA

From this paused state, the debugger can be used to execute the code statement by statement, observing the state of all variables after each step. This action is typically performed using the *Step Into* command. Stepping through each iteration of the `for`-loop reveals that the `result` variable consistently remains `0.0`. This observation indicates that the defect is related to either the initialization of the `result` variable (Line 7) or the operation within the loop (Line 9). A closer inspection reveals that the multiplication assignment operator (`*=`) in Line 9 is the root cause of the defect. Replacing this with the addition assignment operator (`+=`) corrects the logic, and re-executing the program yields the correct output of 15.

The *Step Into* command executes the current line and, if it contains a method call, moves the execution point to the first line inside that method. If the internal execution of a method is not of interest, the *Step Over* command can be used instead. This command executes the entire method call at once and pauses on the subsequent line in the current method.

Another powerful feature available in many modern debuggers, including IntelliJ IDEA's, is the ability to *Evaluate Expression*. This tool allows a developer to

execute arbitrary code expressions within the current execution context, which is useful for testing hypotheses without altering the source code and recompiling. This feature is typically accessed via a dedicated button or context menu option in the debugger tool window. Within this tool, arbitrary code expressions can be evaluated, and their results are displayed, reflecting the current state of the paused application.

4.2 Extra Topic: Sprite Animation

A simple game featuring sprite animation has been chosen as the case study for this chapter's discussion of logging and debugging. The numerous state changes inherent in game animation make it a suitable example for illustrating the challenges of maintaining consistency and correctness during program execution. Prior to examining the case study, a concise overview of game loops and sprite animation is provided.

Fundamentally, a game can be conceptualized as a combination of logic loops and rendering loops. Typically, at least two main loops are implemented: one for game logic computations and another for rendering visual effects. These loops often operate at different frequencies. The rendering loop should refresh at a high rate, typically 60 times per second (60 Hz) or more, to ensure fluid visual animation. In contrast, the logic loop may refresh less frequently, for example, 5 to 10 times per second, as update frequencies exceeding this rate offer no perceptual benefit for game logic.

The implementation of animation components introduces considerations beyond standard application programming, as the timing of drawing operations must be synchronized with human perception. Movements that are rendered too slowly or too quickly can result in an undesirable user experience. For example, consider the object movement implemented in Script 4.7. Although the code is free of errors, the resulting movement is imperceptible because its speed exceeds the threshold of human visual processing; it does not constitute animation. The solution is to control the refresh rate of the drawing loop. Script 4.8 presents an updated implementation, `redrawShape.java`, where a `Thread.sleep()` call is introduced into the loop. The value `1000/60` corresponds to a target refresh rate of 60 frames

per second, with each frame having a duration of approximately 16.7 milliseconds.

Script 4.7: Drawing Objects at High Speed

```
+ 1  for (int i = 0; i < 1000; i++) {  
+ 2      x++;  
+ 3      y++;  
+ 4      c.redraw();  
+ 5  }
```

Script 4.8: Implementing Movement with Thread.sleep for Smoother Animation

```
1  for (int i = 0; i < 1000; i++) {  
2      x++;  
3      y++;  
4      c.redraw();  
+ 5      try {  
+ 6          Thread.sleep(1000/60);  
+ 7      } catch (Exception e) {  
+ 8          e.printStackTrace();  
+ 9      }  
10 }
```

In programming terms, animation is synonymous with time-based motion, wherein an image is drawn, erased, and redrawn at a new coordinate within each iteration of the drawing loop. This process occurs at a refresh rate optimized for human perception. The refresh rate must strike a balance, as excessively slow rates result in a laggy experience, while overly fast rates create the illusion of teleportation rather than smooth movement. Broadly speaking, animation can be viewed as the time-based alteration of graphical objects through changes in their states, locations, sizes, and orientations (Haase & Guy, 2007). The essential components for implementing basic animation include drawing, clocks, movement, collision handling, and interpolation.

4.2.1 Drawing

In JavaFX, objects intended for animation are typically implemented as custom components. This can be achieved by extending a class such as `javafx.scene.layout.Pane`, which allows the custom component to be treated as a standard JavaFX node and added to the scene graph. A key consideration in 2D graphics programming is the coordinate system. In contrast to conventional Cartesian coordinate systems where the origin (0, 0) is at the bottom-left, the JavaFX coordinate system places the origin at the top-left corner of the drawing surface.

4.2.2 Clock

A clock, in the context of animation and game development, is a mechanism for synchronizing time-dependent operations. Similar to how a master clock synchronizes various devices in a complex system to prevent signal interference, a central timing loop in an application ensures that all animations and logic updates occur in a coordinated and consistent manner. In practice, a `for` or `while` loop, often referred to as the main game loop, is implemented to serve this function. The primary purpose of this clock or loop is to ensure that all visual components update their state based on a single, authoritative time reference. For the purposes of this course, the main programming loops will be synchronized with a master clock that “ticks” at a regular interval, such as every millisecond.

4.2.3 Movement

The implementation of object movement is primarily concerned with achieving a high degree of realism. A common methodology is to apply principles from basic physics. In this approach, a movable object is defined by its position, velocity, and acceleration at any given time. Instead of directly manipulating an object’s position, developers can manipulate its acceleration, allowing changes to propagate to its velocity and position according to physics-based formulas. The realism can be further enhanced by incorporating concepts such as force and momentum. For instance, keyboard events can be interpreted as forces applied to a game character in a specific direction. The character then moves with a calculated acceleration up to a predefined maximum velocity. When the key is released, the force is removed,

and the character decelerates until its velocity reaches zero. In this model, the position and velocity are not directly set but are emergent properties of the underlying physics simulation.

4.2.4 Collision handling

Collision handling, or collision detection, is the process of detecting when two or more graphical objects intersect. Because these objects are virtual, their reactions to such intersections must be explicitly defined in the program logic. This component of a game or simulation often involves significant mathematical calculations. Common responses to a detected collision include halting movement, modifying object attributes (e.g., reducing health points), triggering visual effects, or updating a score.

Script 4.9: Example Implementation of a Basic Game Loop

LogicLoop.java

```
+ 1 //Imports are omitted
+ 2 public class LogicLoop {
+ 3     public LogicLoop(Platform platform) {
+ 4         this.platform = platform;
+ 5         frameRate = 10;
+ 6         interval = 1000.0f / frameRate;
+ 7         running = true;
+ 8     }
+ 9     @Override
+ 10    public void run() {
+ 11        while (running) {
+ 12            update(platform.getCharacter());
+ 13            checkCollisions(platform.getCharacter());
+ 14            try {
+ 15                Thread.sleep((long) interval);
+ 16            } catch (InterruptedException e) {
+ 17                e.printStackTrace();
+ 18            }
+ 19        }
+ 20    }
+ 21 }
```

Before discussing interpolation, a summary of the game loop components is provided. Scripts 4.9 and 4.10 present the structures of the logic and rendering loops, respectively. Lines 5 and 6 of each script define their respective refresh rates; in this example, the logic loop operates at 10 Hz (10 times per second) and the rendering loop operates at 60 Hz. The logic loop (Script 4.9) updates the state of game objects and performs collision checks once per tick (every 100 milliseconds). The rendering loop (Script 4.10), by contrast, ticks more frequently (every 16.7 milliseconds). Each iteration is responsible for redrawing all visual components based on the current game state.

Script 4.10: Example Implementation of a Basic Drawing Loop DrawingLoop.java

```
+ 1 //Imports are omitted
+ 2 public class DrawingLoop {
+ 3     public DrawingLoop(Platform platform) {
+ 4         this.platform = platform;
+ 5         frameRate = 60;
+ 6         interval = 1000.0f / frameRate;
+ 7         running = true;
+ 8     }
+ 9     @Override
+10     public void run() {
+11         while (running) {
+12             checkDrawCollisions(platform.getCharacter());
+13             paint(platform.getCharacter());
+14             try {
+15                 Thread.sleep((long) interval);
+16             } catch (InterruptedException e) {
+17                 e.printStackTrace();
+18             }
+19         }
+20     }
+21 }
```



Figure 4.4: Sprite Sheet Example (Armstead, 2022)

4.2.5 Interpolation

The final component of animation discussed here is sprite sheet animation. This technique creates the illusion of movement by rapidly displaying a sequence of static images, or frames. These frames are often consolidated into a single image file known as a sprite sheet, which improves organization and memory management. Figure 4.4 illustrates an example of a sprite sheet.

To utilize a sprite sheet, a viewport is defined. The viewport acts as a sliding window that selects a single frame from the sheet to be displayed at any given time. With each tick of the rendering loop, the viewport's position is updated to select the next frame in the animation sequence. Initially, only the top-left Rockman image is displayed. After a frame's duration (e.g., 16.7 ms), the viewport moves to the next image in the same row. This process of sequentially displaying sub-images from a sprite sheet to create animated movement is a fundamental technique for 2D character animation. When the sequence reaches its end, it can loop back to the first frame. This technique enhances the perceived realism of moving objects.

Script 4.11 demonstrates a code fragment that uses a viewport for sprite sheet animation. The `tick()` method determines which frame to display during the current rendering cycle, based on the `curIndex` variable. The `columns` and `rows` variables represent the dimensions of the sprite grid on the sheet. The `interpolate()` method in this context sets the coordinates of the viewport on the sprite sheet, where `x` and `y` represent the top-left corner of the target frame. For the first frame (`curIndex = 0`), both `x` and `y` are zero. For the next frame, the value of `x` is incremented by the width of a single sub-image. A key requirement for this technique is that all frames within a sprite sheet must have uniform dimensions. Table 4.1

Script 4.11: Example Implementation of Sprite Sheet Animation Interpolation.java

```

+ 1 public void tick() {
+ 2     curColumnIndex = curIndex % columns;
+ 3     curRowIndex = curIndex / columns;
+ 4     curIndex = (curIndex+1) % (columns * rows);
+ 5     interpolate();
+ 6 }
+ 7 protected void interpolate() {
+ 8     final int x = curColumnIndex * width + offsetX;
+ 9     final int y = curRowIndex * height + offsetY;
+10     this.setViewport(new Rectangle2D(x, y, width, height));
+11 }

```

illustrates the relationship between `curIndex`, `curColumnIndex`, and `curRowIndex` for the sprite sheet in Figure 4.4, which is organized as a grid of five columns and two rows.

Table 4.1: Relationship between `curIndex`, `curColumnIndex`, and `curRowIndex` for the Sprite Sheet in Figure 4.4.

<code>curIndex</code>	<code>curColumnIndex</code>	<code>curRowIndex</code>
0	0	0
1	1	0
2	2	0
3	3	0
4	4	0
5	0	1
6	1	1
7	2	1
8	3	1
9	4	1

4.3 Case Study

AI Advisory

Debugging is a skill that AI tools can assist but cannot replace. Utilizing an AI tool as the primary resort when a defect emerges, such as pasting a stack trace and requesting an immediate solution, bypasses the diagnostic reasoning that this chapter is designed to cultivate. The ability to formulate a hypothesis, isolate the relevant code path, and verify the hypothesis through observation constitutes a core professional competency that improves exclusively through deliberate practice.

Recommended Approach

When a defect emerges, a structured methodology must be followed: first, formulate a hypothesis regarding the root cause; second, establish a breakpoint in the debugger to test this hypothesis; and third, consult a log file if the defect is timing-related or difficult to reproduce interactively. Only after a specific, testable hypothesis has been established should an AI tool be consulted to confirm or challenge the underlying reasoning. Employing an AI tool as a hypothesis generator, such as requesting potential causes for an observed symptom, is highly productive. Conversely, utilizing an AI tool as an automatic defect resolver bypasses the learning process entirely.

Appropriate Uses

- ✓ Utilize an AI tool to interpret an unfamiliar line within a stack trace, such as explaining the representation of `javafx.application.Application$1.run` in a thread dump.
- ✓ Employ an AI tool to suggest Log4j2 `PatternLayout` syntax options only after the basic pattern format is fully understood.

- ✓ Utilize an AI tool to generate a `log4j2.properties` skeleton after each field in the configuration file can be explained independently.

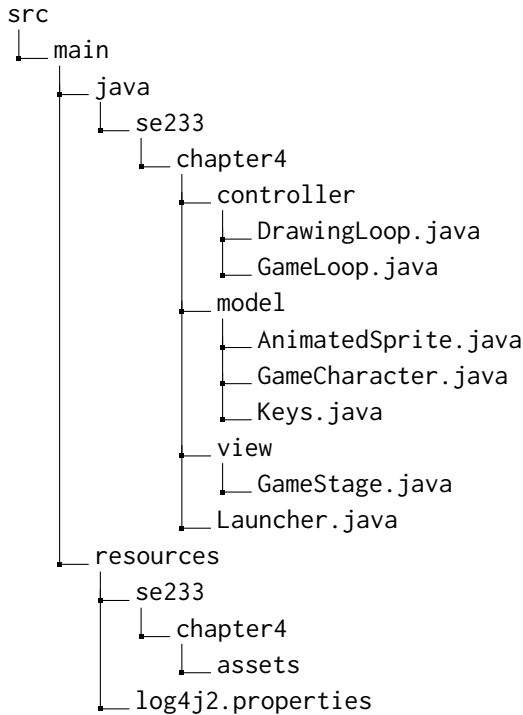
Practices to Avoid

- ✗ Do not paste a stack trace into an AI tool and request a resolution prior to establishing a breakpoint and confirming the failure location independently.
- ✗ Do not employ an AI tool to disable or silence excessive log output without first determining the underlying cause of the excessive logging.
- ✗ Do not delegate game-loop implementation to an AI tool. The temporal logic of frame interpolation and acceleration must be traced manually to be fully comprehended.

The case study for this chapter focuses on the implementation of a simple 2D platformer game in which character movement is controlled by the keyboard. The project, named `Platformer`, and its associated assets are available at <https://cmu.to/SE233CH4>.

The implementation for this case study is notable for its complexity, particularly due to the large number of state variables required. This complexity presents an ideal scenario for demonstrating the use of a debugger as a tool to assist in the development process. The use of a debugger for troubleshooting is highly recommended when encountering any development issues throughout this case study.

The project's file structure is organized as follows:



The implementation begins with the `GameCharacter` class, as shown in Script 4.12. This class is composed of three groups of variables. The first group defines the size of the character image using the `CHARACTER_WIDTH` and `CHARACTER_HEIGHT` variables. The second group represents the character image, its container, and the character's current position on the screen, defined by the `gameCharacterImg`, `imageView`, `x`, and `y` variables, respectively. The final group consists of the keyboard codes for moving the character, defined by the `leftKey`, `rightKey`, and `upKey` variables. The constructor of the class initializes the object instance's position, loads the visual representation of the character, and assigns the keyboard keys that will control the character's movement.

Following the `GameCharacter` class, the `GameStage` class is implemented to define the game's main scene. This class, illustrated in Script 4.13, also contains three groups of variables. The first group consists of constant variables: `WIDTH`, `HEIGHT`,

Script 4.12: model/GameCharacter.java

```
+ 1 //Imports are omitted
+ 2 public class GameCharacter extends Pane {
+ 3     public static final int CHARACTER_WIDTH = 64;
+ 4     public static final int CHARACTER_HEIGHT = 64;
+ 5     private Image gameCharacterImg;
+ 6     private ImageView imageView;
+ 7     private int x;
+ 8     private int y;
+ 9     private KeyCode leftKey;
+10     private KeyCode rightKey;
+11     private KeyCode upKey;
+12     public GameCharacter(int x, int y, KeyCode leftKey, KeyCode rightKey, KeyCode
        upKey) {
+13         this.x = x;
+14         this.y = y;
+15         this.setTranslateX(x);
+16         this.setTranslateY(y);
+17         this.gameCharacterImg = new Image(Launcher.class.getResourceAsStream("
            assets/StillMario.png"));
+18         this.imageView = new ImageView(this.gameCharacterImg);
+19         this.leftKey = leftKey;
+20         this.rightKey = rightKey;
+21         this.upKey = upKey;
+22         this.getChildren().addAll(this.imageView);
+23     }
+24 }
```

and GROUND. The WIDTH and HEIGHT variables define the size of the game screen in pixels, while the GROUND variable defines the y-coordinate of the virtual ground relative to the top of the background image. The value of GROUND is set to 300, indicating that the virtual ground is positioned 300 pixels from the top of the background image. The second group contains a single variable, gameStageImg, which holds the background image for the game stage. Finally, the third group consists of the gameCharacter variable, which is an instance of the GameCharacter class.

Script 4.13: view/GameStage.java

```
+ 1 //Imports are omitted
+ 2 public class GameStage extends Pane {
+ 3     public static final int WIDTH = 800;
+ 4     public static final int HEIGHT = 400;
+ 5     public final static int GROUND = 300;
+ 6     private Image gameStageImg;
+ 7     private GameCharacter gameCharacter;
+ 8     public GameStage() {
+ 9         gameStageImg = new Image(Launcher.class.getResourceAsStream("assets/
+ 10             Background.png"));
+ 11         ImageView backgroundImg = new ImageView(gameStageImg);
+ 12         backgroundImg.setFitHeight(HEIGHT);
+ 13         backgroundImg.setFitWidth(WIDTH);
+ 14         gameCharacter = new GameCharacter(30, 30, KeyCode.A, KeyCode.D, KeyCode.W);
+ 15         getChildren().addAll(backgroundImg, gameCharacter);
+ 16     }
+ 17     public GameCharacter getGameCharacter() {
+ 18         return gameCharacter;
+ 19     }
+ 20 }
```

Script 4.14: Launcher.java

```
+ 1 //Imports are omitted
+ 2 public class Launcher extends Application {
+ 3     public static void main(String[] args) { launch(args); }
+ 4     @Override
+ 5     public void start(Stage stage) {
+ 6         GameStage gameStage = new GameStage();
+ 7         Scene scene = new Scene(gameStage, gameStage.WIDTH, gameStage.HEIGHT);
+ 8         stage.setTitle("Mario");
+ 9         stage.setScene(scene);
+ 10        stage.show();
+ 11    }
+ 12 }
```

The application's Launcher class is a standard JavaFX launcher that instantiates the GameState class. Script 4.14 presents the initial implementation of this Launcher class. Executing the application after implementing these three classes will display a static image, as depicted in Figure 4.5.

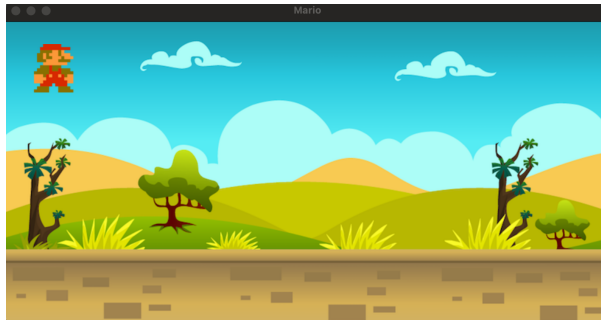


Figure 4.5: Initial Static GUI of the Case Study Application

4.3.1 Game loop

Before examining an implementation with separate logic and rendering loops, a version of the game that utilizes a single, unified game loop will be explored. Script 4.15 illustrates a class named `GameLoop` that implements the `Runnable` interface. Its primary component is an infinite loop that executes on a background thread and repeats at a frequency of 60 Hz (approximately every 16.7 milliseconds). Within each iteration, the `run()` method sequentially invokes the `update()`, `checkCollisions()`, and `paint()` methods.

To examine the operation of this `GameLoop`, the `GameCharacter`, `GameLoop`, and `Launcher` classes are modified as shown in Scripts 4.16, 4.17, and 4.18, respectively. The updated `GameCharacter` class in Script 4.16 implements the logic for the character to fall to the virtual ground. Two new state variables are introduced in Lines 19-20: `yVelocity`, which determines the rate of descent, and `isFalling`, a boolean flag indicating the character's falling state. Three new methods are also added: `moveY()` to update the character's y-coordinate, `checkReachFloor()` to detect when the character has reached the virtual ground, and `repaint()` to redraw the character on the platform.

Script 4.15: controller/GameLoop.java

```
+ 1 //Imports are omitted
+ 2 public class GameLoop implements Runnable {
+ 3     private GameState gameState;
+ 4     private int frameRate;
+ 5     private float interval;
+ 6     private boolean running;
+ 7     public GameLoop(GameState gameState) {
+ 8         this.gameState = gameState;
+ 9         frameRate = 60;
+10         interval = 1000.0f / frameRate;
+11         running = true;
+12     }
+13     private void update(GameCharacter gameCharacter) {
+14     }
+15     private void checkCollisions(GameCharacter gameCharacter) {
+16     }
+17     private void paint(GameCharacter gameCharacter) {
+18     }
+19     @Override
+20     public void run() {
+21         while (running) {
+22             float time = System.currentTimeMillis();
+23             update(gameState.getGameCharacter());
+24             checkCollisions(gameState.getGameCharacter());
+25             paint(gameState.getGameCharacter());
+26             time = System.currentTimeMillis() - time;
+27             if (time < interval) {
+28                 try {
+29                     Thread.sleep((long) (interval - time));
+30                 } catch (InterruptedException e) {
+31                     e.printStackTrace();
+32                 }
+33             } else {
+34                 try {
+35                     Thread.sleep((long) (interval - (interval % time)));
+36                 } catch (InterruptedException e) {
+37                     e.printStackTrace();
+38                 }
+39             }
+40         }
+41     }
+42 }
```

Script 4.16: model/GameCharacter.java

```
1  //Imports are omitted
2  public class GameCharacter extends Pane {
    ...
18  private KeyCode upKey;
+19  int yVelocity = 3;
+20  boolean isFalling = true;
21  public GameCharacter(int x, int y, KeyCode leftKey, KeyCode rightKey, KeyCode
    upKey) {
22      this.x = x;
    ...
32  }
+33  public void moveY() {
+34      if(isFalling) {
+35          y = y + yVelocity;
+36      }
+37  }
+38  public void checkReachFloor() {
+39      if(isFalling && y >= GameState.GROUND - CHARACTER_HEIGHT) {
+40          isFalling = false;
+41      }
+42  }
+43  public void repaint() {
+44      setTranslateX(x);
+45      setTranslateY(y);
+46  }
47  }
```

These new GameCharacter methods are controlled by the GameLoop. Within the loop, each method is invoked in sequence, as shown in Lines 18, 21, and 24 of Script 4.17. The integration is completed by instantiating the GameLoop and executing it on a background thread within the Launcher class (Lines 13 and 18 of Script 4.18). Upon compilation and execution, the character image will be animated to fall towards the ground, stopping when its bottom edge reaches the virtual ground, as depicted in Figure 4.6.

Script 4.17: controller/GameLoop.java

```
1 //Imports are omitted
2 public class GameLoop implements Runnable {
    ...
14     interval = 1000.0f / frameRate;
15     running = true;
16 }
17 private void update(GameCharacter gameCharacter) {
+ 18     gameCharacter.moveY();
19 }
20 private void checkCollisions(GameCharacter gameCharacter) {
+ 21     gameCharacter.checkReachFloor();
22 }
23 private void paint(GameCharacter gameCharacter) {
+ 24     gameCharacter.repaint();
25 }
    ...
```

Script 4.18: Launcher.java

```
1 //Imports are omitted
    ...
8 public class Launcher extends Application {
9     public static void main(String[] args) { launch(args); }
10     @Override
11     public void start(Stage stage) {
12         GameStage gameStage = new GameStage();
+ 13         GameLoop gameLoop = new GameLoop(gameStage);
14         Scene scene = new Scene(gameStage, gameStage.WIDTH, gameStage.HEIGHT);
15         primaryStage.setTitle("Mario");
16         primaryStage.setScene(scene);
17         primaryStage.show();
+ 18         (new Thread(gameLoop)).start();
19     }
20 }
```



Figure 4.6: Rendered GUI after Initiating the GameLoop Thread

4.3.2 Keys

Keyboard input is used to control the movement of the game character. For example, the *a*, *w*, and *d* keys could be mapped to leftward movement, jumping, and rightward movement, respectively. The binding of these keys requires the implementation of event listeners to handle the corresponding keyboard events. Script 4.19 presents a new `Keys` class that functions as a centralized input manager. This class tracks which keys are currently pressed during each game loop tick, enabling the application to respond to keyboard state. The `Keys` class is designed to be independent of the `GameCharacter` class to enhance modularity; for example, this separation would allow different key mappings for multiple players in a local cooperative mode. A `HashMap` is employed within the `Keys` class to efficiently track the state of multiple, simultaneous key presses, which is a common requirement for complex character control.

The integration of the `Keys` class requires modifications to the `GameStage`, `Launcher`, `GameLoop`, and `GameCharacter` classes, as shown in Scripts 4.20, 4.21, 4.22, and 4.23, respectively. The modifications in Script 4.20 integrate the `Keys` class with the main view component, `GameStage`.

In Script 4.21, two anonymous classes (Lines 16 and 17) are used to create event handlers. These handlers capture keyboard press and release events and forward them to the `Keys` object managed by the `GameStage`. This action updates the state of the `HashMap`, adding an entry when a key is pressed and removing it upon release.

Script 4.19: model/Keys.java

```
+ 1 //Imports are omitted
+ 2 public class Keys {
+ 3     private HashMap<KeyCode,Boolean> keys;
+ 4     public Keys() {
+ 5         keys = new HashMap<>();
+ 6     }
+ 7     public void add(KeyCode key) {
+ 8         keys.put(key, true);
+ 9     }
+10     public void remove(KeyCode key) {
+11         keys.put(key, false);
+12     }
+13     public boolean isPressed(KeyCode key) {
+14         return keys.containsKey(key);
+15     }
+16 }
```

Script 4.20: view/GameStage.java

```
1 //Imports are omitted
...
11 public class GameStage extends Pane {
...
15     private GameCharacter gameCharacter;
+16     private Keys keys;
17     public GameStage() {
+18         keys = new Keys();
19         gameStageImg = new Image(Launcher.class.getResourceAsStream("assets/
Background.png"));
20         ImageView backgroundImg = new ImageView(gameStageImg);
...
30         return gameCharacter;
31     }
+32     public Keys getKeys() {
+33         return keys;
+34     }
35 }
```

Script 4.21: Launcher.java

```
1 //Imports are omitted
  ...
9 public class Launcher extends Application {
10     public static void main(String[] args) { launch(args); }
11     @Override
12     public void start(Stage primaryStage) {
13         GameState gameState = new GameState();
14         GameLoop gameLoop = new GameLoop(gameState);
15         Scene scene = new Scene(gameState, gameState.WIDTH, gameState.HEIGHT);
+ 16         scene.setOnKeyPressed(event-> gameState.getKeys().add(event.getCode()));
+ 17         scene.setOnKeyReleased(event -> gameState.getKeys().remove(event.getCode()));
        ...
    }
```

Subsequently, the `GameCharacter` class is modified (Script 4.22) to include the logic for horizontal movement and stopping. Vertical movement (jumping) is not implemented in these methods, as it is managed separately by the physics logic (e.g., gravity). After implementing the necessary accessor methods for the key-binding variables in the `GameCharacter` class, the character can be moved using the *a*, *w*, and *d* keys.

Finally, the modified `GameLoop` class is presented in Script 4.23. Specifically, the changes are located in the `update()` method (Lines 17-28), where several conditional statements have been added. These statements poll the keyboard state from the `Keys` object to define the character's movement (left, right, or idle) based on the user's input. At this stage, the character can move horizontally but is not yet constrained by the game boundaries, as illustrated in Figure 4.7.

Script 4.22: model/GameCharacter.java

```
1  //Imports are omitted
   ...
10 public class GameCharacter extends Pane {
   ...
+20     int xVelocity = 0;
21     int yVelocity = 3;
22     boolean isFalling = true;
   ...
+49     public void moveLeft() {
+50         setScaleX(-1);
+51         go();
+52         x = x-xVelocity;
+53     }
+54     public void moveRight() {
+55         setScaleX(1);
+56         go();
+57         x = x+xVelocity;
+58     }
+59     public void go() {
+60         xVelocity = 5;
+61     }
+62     public void stop() {
+63         xVelocity = 0;
+64     }
   ...
}
```

Script 4.23: controller/GameLoop.java

```
1  //Imports are omitted
2  public class GameLoop implements Runnable {
    ...
17  private void update(GameCharacter gameCharacter) {
+ 18      boolean leftPressed = gameStage.getKeys().isPressed(gameCharacter.
        getLeftKey());
+ 19      boolean rightPressed = gameStage.getKeys().isPressed(gameCharacter.
        getRightKey());
+ 20
+ 21      if (leftPressed && rightPressed) {
+ 22          gameCharacter.stop();
+ 23      } else if (leftPressed) {
+ 24          gameCharacter.moveLeft();
+ 25      } else if (rightPressed) {
+ 26          gameCharacter.moveRight();
+ 27      } else {
+ 28          gameCharacter.stop();
+ 29      }
30
31      gameCharacter.moveY();
32  }
    ...
```

**Figure 4.7: Character Movement without Boundary Collision Detection**

4.3.3 Collision checking

The initial collision detection implemented in the application is designed to handle collisions between the character and the boundaries of the game world. This functionality requires modifications to the `GameLoop` and `GameCharacter` classes, as shown in Scripts 4.24 and 4.25, respectively. The modification to the `GameLoop` class involves the addition of a method call (Line 34 of Script 4.24) to check for collisions between the character and the game boundaries. The logic for this check is implemented within the `GameCharacter` class, as shown in Lines 40-46 of Script 4.25.

Script 4.24: controller/GameLoop.java

```

1  //Imports are omitted
2  public class GameLoop implements Runnable {
    ...
33  private void checkCollisions(GameCharacter gameCharacter) {
+ 34      gameCharacter.checkReachGameWall();
35      gameCharacter.checkReachFloor();
36  }
    ...

```

Script 4.25: model/GameCharacter.java

```

1  //Imports are omitted
2  public class GameCharacter extends Pane {
    ...
+ 40  public void checkReachGameWall() {
+ 41      if(x <= 0) {
+ 42          x = 0;
+ 43      } else if( x+getWidth() >= GameState.WIDTH) {
+ 44          x = GameState.WIDTH-(int)getWidth();
+ 45      }
+ 46  }
47  public void checkReachFloor() {
48      if(isFalling && y >= GameState.GROUND - CHARACTER_HEIGHT) {
49          isFalling = false;
50      }
51  }

```

The logic for this boundary check is straightforward. It verifies if the character's updated x-coordinate is either less than zero or greater than the width of the game stage. If a boundary is exceeded, the character's x-coordinate is adjusted to the boundary value itself. This correction occurs before the `GameLoop` invokes the `repaint()` method, ensuring the character is rendered in the correct position.

4.3.4 Jumping

When the designated key for upward movement (e.g., `w`) is pressed, a force is applied to the character to initiate a jump. Subsequently, the character should descend under the influence of gravity until it makes contact with the ground. The downward motion occurs at a non-constant speed due to gravitational acceleration. Consequently, the character's vertical velocity increases throughout its descent. However, a complete physics-based implementation of this jumping mechanism is overly complex for an initial example.

To simplify the initial implementation, a model is considered that represents the jump using only basic force, velocity, and position. Modifications begin with the `GameCharacter` class, as shown in Script 4.26. Three new variables are introduced: `canJump`, `isJumping`, and `highestJump`. The first two are boolean flags that track whether the character is permitted to jump and whether a jump is currently in progress, respectively. These flags are essential for preventing the user from initiating a new jump while the character is already airborne. While the `canJump` state is false, subsequent presses of the jump key will have no effect until the character has returned to the ground. The `highestJump` variable defines the maximum vertical displacement the character can achieve relative to the ground.

In this update, conditional logic for upward movement is added to the `moveY()` method (Lines 42-44). Additionally, two new methods, `jump()` and `checkReachHighest()`, are introduced (Lines 53-66). The `jump()` method is invoked when the jump key is pressed, but only if the `canJump` state is true. The `checkReachHighest()` method determines if the character has reached its maximum vertical height and, if so, initiates the descent by setting `isJumping` to false and `isFalling` to true. The `checkReachFloor()` method resets the character's state upon landing on the virtual ground. Specifically, it sets the `canJump` state back to true (Line 70), allowing a new jump to be initiated.

Script 4.26: model/GameCharacter.java

```

1  //Imports are omitted
...
10 public class GameCharacter extends Pane {
    ...
21     int yVelocity = 3;
22     boolean isFalling = true;
+ 23     boolean canJump = false;
+ 24     boolean isJumping = false;
+ 25     int highestJump = 100;
    ...
38     public void moveY() {
39         if(falling) {
40             y = y + yVelocity;
- 41         }
+ 42         } else if(isJumping) {
+ 43             y = y - yVelocity;
+ 44         }
45     }
46     public void checkReachGameWall() {
47         if(x <= 0) {
48             x = 0;
49         } else if( x+getWidth() >= GameState.WIDTH) {
50             x = GameState.WIDTH-(int)getWidth();
51         }
52     }
+ 53     public void jump() {
+ 54         if (canJump) {
+ 55             yVelocity = 5;
+ 56             canJump = false;
+ 57             isJumping = true;
+ 58             isFalling = false;
+ 59         }
+ 60     }
+ 61     public void checkReachHighest () {
+ 62         if(isJumping && y <= GameState.GROUND - CHARACTER_HEIGHT - highestJump) {
+ 63             isJumping = false;
+ 64             isFalling = true;
+ 65         }
+ 66     }
67     public void checkReachFloor() {
68         if(isFalling && y >= GameState.GROUND - CHARACTER_HEIGHT) {
69             isFalling = false;
+ 70             canJump = true;
71         }
72     }

```

The updated `GameCharacter` class is then integrated into the application via minor modifications to the `GameLoop` class, as illustrated in Script 4.27. To enable this functionality, the `GameLoop` polls the state of the jump key (Line 20) and invokes the character's `jump()` method when it is pressed (Lines 34-36). The `checkReachHighest()` method is invoked from within the `checkCollisions()` method (Line 42). This ensures the check is performed once per game loop tick, after the boundary collision checks have been completed.

Script 4.27: controller/GameLoop.java

```
1 //Imports are omitted
  ...
6 public class GameLoop implements Runnable {
  ...
17 private void update(GameCharacter gameCharacter) {
18     boolean leftPressed = gameStage.getKeys().isPressed(gameCharacter.getLeftKey
    ());
19     boolean rightPressed = gameStage.getKeys().isPressed(gameCharacter.
    getRightKey());
+ 20     boolean upPressed = gameStage.getKeys().isPressed(gameCharacter.getUpKey());
    ...
28 } else if (rightPressed) {
29     gameCharacter.moveRight();
30 } else {
31     gameCharacter.stop();
32 }
33
+ 34     if (upPressed) {
+ 35         gameCharacter.jump();
+ 36     }
37
38     gameCharacter.moveY();
39 }
40 private void checkCollisions(GameCharacter gameCharacter) {
41     gameCharacter.checkReachGameWall();
+ 42     gameCharacter.checkReachHighest();
43     gameCharacter.checkReachFloor();
    ...
}
```

The current implementation demonstrates the fundamental mechanics of a platform game. The character's movement is controlled via the keyboard, and the application responds to user input for actions such as horizontal movement and jumping. This state serves as a foundation for implementing the more advanced topics discussed previously, including multiple game loops, sprite interpolation, acceleration, and logging.

The first enhancement is the separation of rendering logic from the main game loop. This is achieved by creating a new `DrawingLoop` class. Script 4.28 presents this new class, which encapsulates methods related to collision detection and graphics rendering that were previously in the `GameLoop`. The original `GameLoop` is subsequently modified (Script 4.29) by reducing its refresh rate to 10 Hz (100 milliseconds per tick) and removing this rendering logic.

As discussed earlier, the drawing loop should only observe variable changes to render the consequences on the screen, without performing any programming logic computations. This necessitates adjustments to the `moveLeft` and `moveRight` methods in the `GameCharacter` class, detailed in Script 4.30. The primary change involves calculating movement on the x-axis similarly to the y-axis. Instead of directly modifying the location, external forces determine it, enhancing flexibility for future upgrades to the movement simulation engine. The calculations for the character's position on both x and y axes are now independent of character rendering.

In the updated `GameCharacter` class (Script 4.30), the character's x-coordinate is no longer modified directly within the input handling logic. Instead, the state of the character's horizontal movement is controlled by boolean flags, and a new `moveX()` method is responsible for applying this movement during the rendering phase. Consequently, the character's movement on both axes is now governed by four state flags: `isMoveLeft`, `isMoveRight`, `isFalling`, and `isJumping`. The increased complexity of this state-based logic highlights the utility of a debugger for verification and troubleshooting.

Script 4.28: model/DrawingLoop.java

```
+ 1 //Imports are omitted
+ 2 public class DrawingLoop implements Runnable {
+ 3     private GameState gameState;
+ 4     private int frameRate;
+ 5     private float interval;
+ 6     private boolean running;
+ 7     public DrawingLoop(GameState gameState) {
+ 8         this.gameState = gameState;
+ 9         frameRate = 60;
+10         interval = 1000.0f / frameRate; // 1000 ms = 1 second
+11         running = true;
+12     }
+13     private void checkDrawCollisions(GameCharacter gameCharacter) {
+14         gameCharacter.checkReachGameWall();
+15         gameCharacter.checkReachHighest();
+16         gameCharacter.checkReachFloor();
+17     }
+18     private void paint(GameCharacter gameCharacter) {
+19         gameCharacter.repaint();
+20     }
+21     @Override
+22     public void run() {
+23         while (running) {
+24             float time = System.currentTimeMillis();
+25             checkDrawCollisions(gameState.getGameCharacter());
+26             paint(gameState.getGameCharacter());
+27             time = System.currentTimeMillis() - time;
+28             if (time < interval) {
+29                 try {
+30                     Thread.sleep((long) (interval - time));
+31                 } catch (InterruptedException e) {
+32                 }
+33             } else {
+34                 try {
+35                     Thread.sleep((long) (interval - (interval % time)));
+36                 } catch (InterruptedException e) {
+37                 }
+38             }
+39         }
+40     }
+41 }
```


Script 4.29: controller/GameLoop.java

```

1  //Imports are omitted
   ...
6  public class GameLoop implements Runnable {
   ...
11  public GameLoop(GameStage gameStage) {
12      this.gameStage = gameStage;
- 13      frameRate = 60;
+ 14      frameRate = 10;
15      interval = 1000.0f / frameRate;
16      running = true;
17  }
18  private void update(GameCharacter gameCharacter) {
   ...
32      if (upPressed) {
33          gameCharacter.jump();
34      }
- 35
- 36      gameCharacter.moveY();
37  }
- 38  private void checkCollisions(GameCharacter gameCharacter) {
- 39      gameCharacter.checkReachGameWall();
- 40      gameCharacter.checkReachHighest();
- 41      gameCharacter.checkReachFloor();
- 42  }
- 43  private void paint(GameCharacter gameCharacter) {
- 44      gameCharacter.repaint();
- 45  }
46  @Override
47  public void run() {
48      while (running) {
49          float time = System.currentTimeMillis();
50          update(gameStage.getGameCharacter());
- 51          checkCollisions(gameStage.getGameCharacter());
- 52          paint(gameStage.getGameCharacter());
53          time = System.currentTimeMillis() - time;
   ...

```

Script 4.30: model/GameCharacter.java

```
1 //Imports are omitted
...
10 public class GameCharacter extends Pane {
    ...
- 20     int xVelocity = 0;
+ 21     int xVelocity = 5;
+ 22     boolean isMoveLeft = false;
+ 23     boolean isMoveRight = false;
    ...
+ 40     public void moveX() {
+ 41         setTranslateX(x);
+ 42         if(isMoveLeft) { x = x - xVelocity; }
+ 43         if(isMoveRight) { x = x + xVelocity; }
+ 44     }
    45     public void moveY() {
+ 46         setTranslateY(y);
    ...
    80     public void repaint() {
- 81         setTranslateX(x);
- 82         setTranslateY(y);
+ 83         moveX();
+ 84         moveY();
    85     }
    86     public void moveLeft() {
    87         setScaleX(-1);
- 88         go();
- 89         x = x-xVelocity;
+ 90         isMoveLeft = true;
+ 91         isMoveRight = false;
    92     }
    93     public void moveRight() {
    94         setScaleX(1);
- 95         go();
- 96         x = x+xVelocity;
+ 97         isMoveLeft = false;
+ 98         isMoveRight = true;
    99     }
- 100     public void go() {
- 101         xVelocity = 5;
- 102     }
    103     public void stop() {
- 104         xVelocity = 0;
+ 105         isMoveLeft = false;
+ 106         isMoveRight = false;
    107     }
    ...
}
```

To integrate these changes, the Launcher class is modified as shown in Script 4.31. This update registers the new DrawingLoop with the application and initiates its execution in a separate thread.

Script 4.31: Launcher.java

```
1  //Imports are omitted
   ...
9  public class Launcher extends Application {
10     public static void main(String[] args) { launch(args); }
11     @Override
12     public void start(Stage stage) {
13         GameState gameStage = new GameState();
14         GameLoop gameLoop = new GameLoop(gameStage);
+ 15         DrawingLoop drawingLoop = new DrawingLoop(gameStage);
       ...
22         (new Thread(gameLoop)).start();
+ 23         (new Thread(drawingLoop)).start();
       ...
   }
```

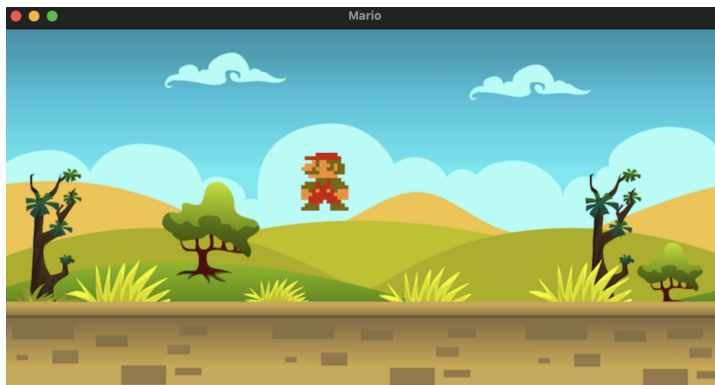


Figure 4.8: Character Movement with Jumping Mechanism Implementation

Upon the completion of this implementation step, the character is capable of jumping, as depicted in Figure 4.8; however, the resulting movement still appears

visually unrealistic. To address this, the subsequent sections will introduce interpolation and refine the movement mechanics through the integration of acceleration and gravity.

4.3.5 Interpolation

The final enhancement to be discussed is sprite animation, which is used to create more realistic character movement. An examination of the sprite sheet (e.g., `MarioSheet.png`) reveals a sequence of character poses. The rapid, sequential display of these individual images creates the illusion of fluid movement, which is the foundation of 2D sprite animation.

Script 4.32: `model/AnimatedSprite.java`

```
+ 1 //Imports are omitted
+ 2 public class AnimatedSprite extends ImageView {
+ 3     int count, columns, rows, offsetX, offsetY, width, height, curIndex,
        curColumnIndex = 0, curRowIndex = 0;
+ 4     public AnimatedSprite(Image image, int count, int columns, int rows, int
        offsetX, int offsetY, int width, int height) {
+ 5         this.setImage(image);
+ 6         this.count = count;
+ 7         this.columns = columns;
+ 8         this.rows = rows;
+ 9         this.offsetX = offsetX;
+10         this.offsetY = offsetY;
+11         this.width = width;
+12         this.height = height;
+13         this.setViewport(new Rectangle2D(offsetX, offsetY, width, height));
+14     }
+15     public void tick() {
+16         curColumnIndex = curIndex % columns;
+17         curRowIndex = curIndex / columns;
+18         curIndex = (curIndex+1) % (columns * rows);
+19         interpolate();
+20     }
+21     protected void interpolate() {
+22         final int x = curColumnIndex * width + offsetX;
+23         final int y = curRowIndex * height + offsetY;
+24         this.setViewport(new Rectangle2D(x, y, width, height));
+25     }
+26 }
```

This functionality is encapsulated within a new `AnimatedSprite` class, as shown in Script 4.32. Its core methods, `tick()` and `interpolate()`, use a viewport to sequentially select and display individual frames from the sprite sheet, with the frame changing at each tick of the rendering loop. The `curIndex` variable tracks the current frame in the animation sequence, while the `columns` and `rows` variables define the dimensions of the sprite grid.

Script 4.33: `model/GameCharacter.java`

```

1  //Imports are omitted
   ...
10  public class GameCharacter extends Pane {
- 11    public static final int CHARACTER_WIDTH = 64;
+ 12    public static final int CHARACTER_WIDTH = 32;
13    public static final int CHARACTER_HEIGHT = 64;
14    private Image gameCharacterImg;
- 15    private ImageView imageView;
+ 16    private AnimatedSprite imageView;
   ...
- 28    public GameCharacter(int x, int y, KeyCode leftKey, KeyCode rightKey, KeyCode
      upKey) {
+ 29    public GameCharacter(int x, int y, int offsetX, int offsetY, KeyCode leftKey,
      KeyCode rightKey, KeyCode upKey) {
30      this.x = x;
31      this.y = y;
32      this.setTranslateX(x);
33      this.setTranslateY(y);
- 34      this.gameCharacterImg = new Image(Launcher.class.getResourceAsStream("
        assets/StillMario.png"));
+ 35      this.gameCharacterImg = new Image(Launcher.class.getResourceAsStream("
        assets/MarioSheet.png"));
- 36      this.imageView = new ImageView(gameCharacterImg);
+ 37      this.imageView = new AnimatedSprite(gameCharacterImg, 4, 4, 1, offsetX,
        offsetY, 16, 32);
+ 38      this.imageView.setFitWidth(CHARACTER_WIDTH);
+ 39      this.imageView.setFitHeight(CHARACTER_HEIGHT);
40      this.leftKey = leftKey;
41      this.rightKey = rightKey;
42      this.upKey = upKey;
43      this.getChildren().addAll(this.imageView);
44    }
   ...

```

Script 4.34: view/GameStage.java

```
1  //Imports are omitted
   ...
11 public class GameStage extends Pane {
   ...
18 public GameStage()
   ...
23 backgroundImg.setFitWidth(WIDTH);
- 24 gameCharacter = new GameCharacter(30, 30, KeyCode.A, KeyCode.D, KeyCode.W);
+ 25 gameCharacter = new GameCharacter(30, 30, 0, 0, KeyCode.A, KeyCode.D, KeyCode.W);
26 getChildren().addAll(backgroundImg, character);
27 }
   ...
```

Given the direct relationship between sprite animation and the character's visual representation, the `GameCharacter` and `GameStage` classes require updates to integrate this new functionality (Scripts 4.33 and 4.34 detail these modifications).

With the model and view components updated, the focus shifts to the controller component. Due to the application's modular design, the only required controller modification is an update to the `update()` method of the `GameLoop` class (Lines 25 and 28). Script 4.35 presents this final revision of the `GameLoop` class. As a result, the character now displays a smooth animation cycle during horizontal movement, as depicted in Figure 4.9.

Script 4.35: controller/GameLoop.java

```
1  //Imports are omitted
   ...
6  public class GameLoop implements Runnable {
   ...
17 private void update(GameCharacter gameCharacter) {
   ...
22 if (leftPressed && rightPressed) {
23     gameCharacter.stop();
24 } else if (leftPressed) {
+ 25     gameCharacter.getImageView().tick();
26     gameCharacter.moveLeft();
27 } else if (rightPressed) {
+ 28     gameCharacter.getImageView().tick();
29     gameCharacter.moveRight();
30 }
   ...
```



Figure 4.9: Rendered GUI After Interpolation Implementation

4.3.6 Acceleration

The integration of sprite animation results in a more fluid running motion for the game character. Overall, the application has undergone substantial visual and interactive enhancements since its initial version. However, the character's movement can be made more realistic. The current implementation, which uses constant velocity, is a primary source of this unnatural movement.

The next enhancement introduces acceleration to the movement model. Movement along the x-axis will be determined by an applied input force, while movement along the y-axis will be influenced by both input force (for jumping) and gravity. A simplified acceleration model is implemented to demonstrate the improvement in realism compared to the previous constant-velocity model.

Due to the modular design, the implementation of acceleration only requires modifications to the `GameCharacter` class. For clarity, the updates to this class are presented across two separate scripts. Script 4.36 presents changes to variable declaration and assignment, along with the updated `moveX()` and `moveY()` methods. Script 4.37 details updates to the `jump()`, `checkReachHighest()`, and `checkReachFloor()` methods. Collectively, these changes implement a basic model of projectile motion by incorporating acceleration along both the x and y-axes.

The motion characteristics differ significantly between the two axes. On the x-axis, horizontal movement initiates from zero velocity and accelerates to a maximum defined velocity. Conversely, vertical movement (a jump) begins with an initial upward velocity that continuously decreases due to gravity until it reaches zero at the apex of the jump. Subsequently, the character accelerates downward until it makes contact with the virtual ground.

Script 4.36: model/GameCharacter.java

```
1  //Imports are omitted
   ...
10 public class GameCharacter extends Pane {
   ...
-20  int xVelocity = 5;
+21  int xVelocity = 0;
   22  boolean isMoveLeft = false;
   23  boolean isMoveRight = false;
-24  int yVelocity = 3;
+25  int yVelocity = 0;
   ...
-27  int highestJump = 100;
+28  int xAcceleration = 1;
+29  int yAcceleration = 1;
+30  int xMaxVelocity = 7;
+31  int yMaxVelocity = 17;
   ...
   45  public void moveX() {
   46      setTranslateX(x);
   47      if(isMoveLeft) {
+48          xVelocity = xVelocity>=xMaxVelocity? xMaxVelocity : xVelocity+xAcceleration;
   49          x = x - xVelocity;
   50      }
   51      if(isMoveRight) {
+52          xVelocity = xVelocity>=xMaxVelocity? xMaxVelocity : xVelocity+xAcceleration;
   53          x = x + xVelocity;
   54      }
   55  }
   56  public void moveY() {
   57      setTranslateY(y);
   58      if(falling) {
+59          yVelocity = yVelocity >= yMaxVelocity? yMaxVelocity : yVelocity+yAcceleration;
   60          y = y + yVelocity;
   61      }
   62      else if(jumping) {
+63          yVelocity = yVelocity <= 0 ? 0 : yVelocity-yAcceleration;
   64          y = y - yVelocity;
   65      }
   66  }
   ...
}
```


Script 4.37: model/GameCharacter.java

```

1  //Imports are omitted
  ...
10 public class GameCharacter extends Pane {
    ...
73     public void jump() {
74         if (canJump) {
-75             yVelocity = 5;
+76             yVelocity = yMaxVelocity;
77             canJump = false;
78             isJumping = true;
79             isFalling = false;
80         }
81     }
82     public void checkReachHighest() {
-83         if(isJumping && y <= GameStage.GROUND - CHARACTER_HEIGHT - highestJump) {
+84         if(isJumping && yVelocity <= 0) {
85             isJumping = false;
86             isFalling = true;
+87             yVelocity = 0;
88         }
89     }
90     public void checkReachFloor() {
91         if(falling && y >= GameStage.GROUND - CHARACTER_HEIGHT) {
92             isFalling = false;
93             canJump = true;
+94             yVelocity = 0;
95         }
96     }
    ...

```

4.3.7 Logging

While interactive debugging has been instrumental in the development process thus far, it is also appropriate to explore alternative diagnostic techniques such as probing and logging. Scripts 4.38 and 4.39 demonstrate the probing technique applied to the GameCharacter and GameLoop classes, respectively.

A `trace()` method is introduced to the `GameCharacter` class, which outputs the character's current coordinates and velocities to the console. Within the `GameLoop`, this `trace()` method is invoked to allow for the observation of these values during the character's horizontal movement.

However, as probing is often criticized as an overly simplistic technique, it is recommended to replace it with a structured logging implementation. Logging provides a more robust and configurable mechanism for tracing program state and troubleshooting issues.

Script 4.38: model/GameCharacter.java

```
1 //Imports are omitted
...
10 public class GameCharacter extends Pane {
    ...
+ 113     public void trace() {
+ 114         System.out.println(String.format("x:%d y:%d vx:%d vy:%d",x,y,xVelocity,
+                                     yVelocity));
+ 115     }
116 }
```

Script 4.39: controller/GameLoop.java

```
1 //Imports are omitted
...
6 public class GameLoop implements Runnable {
    ...
17     private void update(Character character) {
        ...
24     } else if (leftPressed) {
25         gameCharacter.getImageView().tick();
26         gameCharacter.moveLeft();
+ 27         gameStage.getGameCharacter().trace();
28     } else if (rightPressed) {
29         gameCharacter.getImageView().tick();
30         gameCharacter.moveRight();
+ 31         gameStage.getGameCharacter().trace();
32     }
    ...
}
```

Integrating a logging framework into the application begins by adding the necessary dependencies to the `pom.xml` file (Script 4.40). Typically, this involves dependencies for *Apache Log4j Core*, *Apache Log4j API*, and the *SLF4J Binding*. Subsequently, the `module-info.java` file is adjusted as illustrated in Script 4.41. After these dependencies are integrated and the project is refreshed, the probing statements can be replaced with logging calls. As shown in Script 4.42, the only required change in the `GameCharacter` class is the initialization of a logger instance (Line 5), which is then used to replace the previous probing implementation (Lines 121-122).

Script 4.40: `pom.xml`

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.
    apache.org/xsd/maven-4.0.0.xsd">
5      <modelVersion>4.0.0</modelVersion>
        ...
36     <version>${junit.version}</version>
37     <scope>test</scope>
38     </dependency>
+ 39     <dependency>
+ 40         <groupId>org.apache.logging.log4j</groupId>
+ 41         <artifactId>log4j-core</artifactId>
+ 42         <version>2.25.4</version>
+ 43     </dependency>
+ 44     <dependency>
+ 45         <groupId>org.apache.logging.log4j</groupId>
+ 46         <artifactId>log4j-api</artifactId>
+ 47         <version>2.25.4</version>
+ 48     </dependency>
+ 49     <dependency>
+ 50         <groupId>org.apache.logging.log4j</groupId>
+ 51         <artifactId>log4j-slf4j-impl</artifactId>
+ 52         <version>2.25.4</version>
+ 53     </dependency>
54 </dependencies>
    ...

```

Script 4.41: module-info.java

```
1 module se233.chapter4 {
2     requires javafx.controls;
3     requires javafx.fxml;
+ 4     requires org.apache.logging.log4j;
5     opens se233.chapter4 to javafx.fxml;
6     exports se233.chapter4;
7 }
```

Script 4.42: model/GameCharacter.java

```
1 package se233.chapter4.model;
2
+ 3 import org.apache.logging.log4j.LogManager;
+ 4 import org.apache.logging.log4j.Logger;
5 //Other imports are omitted
6 ...
12 public class GameCharacter extends Pane {
+ 13     private static final Logger logger = LogManager.getLogger(GameCharacter.class);
14     ...
116     public void trace() {
- 117         System.out.println(String.format("x:%d y:%d vx:%d vy:%d",x,y,xVelocity,
118                                         yVelocity));
+ 118         logger.info("x:{} y:{} vx:{} vy:{}",x,y,xVelocity,yVelocity);
119     }
120 }
```

With the dependencies configured, the next step requires the creation of a `log4j2.properties` configuration file. The contents of this file are presented in Script 4.43. This example configuration designates the console as the sole appender and sets the root logging level to `INFO`. The output pattern is defined to include the severity level, a detailed timestamp, the thread name, the class name, the log message, and a line separator. Upon compiling and executing the application, the console output will adhere to the newly configured log pattern.

Script 4.43: resources/se233/chapter4/log4j2.properties

```
+ 1 name = PropertiesConfig
+ 2 appender.console.type = Console
+ 3 appender.console.name = consoleLogger
+ 4 appender.console.layout.type = PatternLayout
+ 5 appender.console.layout.pattern = [%5p] %d{yyyy-MM-dd HH:mm:ss.SSS} [%t] %c{1} %
    msg%n
6
+ 7 rootLogger.level = info
+ 8 rootLogger.appenderRef.stderr.ref = consoleLogger
```

Finally it is important to note that Logging is evidence, not just debug output. Log files serve a purpose that extends well beyond development-time debugging. In a deployed application, structured log output constitutes the primary source of evidence when diagnosing failures that cannot be reproduced interactively. A well-configured logging framework records timestamps, thread identifiers, severity levels, and class origins automatically, information that is impossible to reconstruct from memory or from the unstructured output of `System.out.println` calls. Logging statements should therefore be treated as permanent, first-class components of the codebase, designed to remain in place across releases and configured through severity levels rather than removed when debugging is complete.

4.4 Exercise

1. Implement a second playable character within the application. The movement for this new character should be controlled by the *left*, *up*, and *right* arrow keys.
2. For the newly implemented character, define distinct maximum velocity values for movement along both the x-axis and y-axis.
3. Implement sprite animation for the new character using the sprite sheet resource available at <https://cmu.to/SE233rockman>.

4. Implement a DEBUG-level log message that is generated whenever a character collides with a game boundary. The logging configuration should be set appropriately to ensure this message is visible in the output.
5. Modify the logging configuration to use a `FileAppender` concurrently with the existing `ConsoleAppender`, such that all log entries are written to both the console and a designated log file.
6. Modify the `PatternLayout` in the logging configuration. The new pattern should include the source code line number and method name where the log event occurred. Additionally, adjust the timestamp format to include the day of the week.

4.5 Discussion and Summary

This chapter has examined three progressively capable strategies for identifying and correcting software defects. The discussion began with probing, a technique characterized by the insertion of `System.out.println` statements to observe runtime states. This approach exposed its inherent limitations in multi-threaded GUI applications, where concurrent, unstructured output rapidly becomes unmanageable. Structured logging was subsequently introduced as a sustainable alternative. A logging framework provides severity-based filtering, timestamped records, configurable output destinations, and the capacity to remain permanently in the codebase without interfering with production behavior. The chapter then progressed to breakpoint-based debugging within the IntelliJ IDEA environment. This section demonstrated how stepping through execution flows and inspecting variable states yields significantly more precise diagnostic information than either probing or logging alone. Finally, the case study synthesized all three techniques by applying them to a 2D platformer application, progressing from initial probing to logging integration, and concluding with the interactive debugging of collision, interpolation, and acceleration logic.

Knowledge Outcomes

Upon the completion of this chapter, a student should demonstrate the ability to:

- Describe the limitations of print-statement probing, including the inability to filter by severity, the absence of timestamps and thread identification, and the risk of diagnostic code leaking into production releases.
- Explain the Log4j2 architecture in terms of its core components, specifically the Logger, Appender, and Layout, and describe the role each component plays in the processing pipeline from log event to output destination.
- Distinguish between the standard logging levels (TRACE, DEBUG, INFO, WARN, ERROR, FATAL) and justify which level is appropriate for a given category of diagnostic message.
- Explain how a debugger breakpoint differs from a print statement in terms of execution interference, reproducibility, and the ability to inspect complex object states without modifying source code.
- Describe the game-loop pattern, encompassing the update, render, and sleep phases, and explain how frame interpolation smooths visual movement between discrete ticks.
- Explain why logging statements should be treated as permanent, first-class code rather than temporary debugging aids, emphasizing the role of log files as critical post-release diagnostic evidence.

Points for Discussion

1. The probing technique and structured logging both generate textual output; however, probing is considered a temporary diagnostic measure, whereas logging represents a permanent architectural component. What specific properties of a logging framework justify retaining log statements in production code, and under what conditions might the computational overhead of logging become a performance concern?

2. A breakpoint suspends the entire application at a specific line of code, which can alter the timing behavior of concurrent or animation-related operations. How might this observer effect influence the diagnosis of timing-dependent defects, and what alternative debugging strategies can be employed to mitigate this issue?
3. The case study integrates collision detection, interpolation, and acceleration within a single game loop. If a rendering anomaly occurs during gameplay, what systematic approach can be utilized to isolate the defect to one of these three subsystems prior to implementing any code modifications?

Suggested Extensions and Project Ideas

1. Add a `RollingFileAppender` to the `log4j2` configuration that rotates log files daily and retains the seven most recent files, then verify the rotation behaviour by adjusting the system clock or using a scheduled test.
2. Implement a conditional breakpoint in IntelliJ IDEA that suspends execution only when a character's velocity exceeds a defined threshold, and use this technique to diagnose an intermittent overshooting defect.
3. Extend the game loop by integrating a frame-rate counter displayed within the application window. Cross-reference any observed frame drops with WARN-level log entries to identify which subsystem consumes the greatest execution time per tick.
4. Introduce a replay mechanism that reads a log file of timestamped input events and re-executes them against the game loop, enabling deterministic reproduction of defects that were originally observed only in live play.

Progress Checklist

- ☐ Describe the limitations of print-statement probing and explain why structured logging provides a more sustainable alternative for diagnosing defects.
- ☐ Configure a Log4j2 properties file specifying a root logging level, a console appender, and a pattern layout that includes timestamps, thread names, and severity levels.
- ☐ Distinguish between logging levels (TRACE, DEBUG, INFO, WARN, ERROR, FATAL) and select the appropriate level for a given diagnostic scenario.
- ☐ Set a breakpoint in IntelliJ IDEA, step through a running JavaFX application, and inspect variable states at each suspension point.
- ☐ Implement a sprite animation pipeline comprising rendering, clock-driven updates, movement, and collision detection.
- ☐ Apply frame interpolation to smooth character movement between discrete game-loop ticks.
- ☐ Integrate structured log statements into a game loop at appropriate severity levels to capture runtime behavior without disrupting gameplay.
- ☐ Read a log output file and trace the sequence of events that led to an observed defect.

References

- Armstead, L. (2022). Exemplo De Spritesheet – Sprite Sheet Megaman Png, Transparent Png. Retrieved June, 2026, from https://www.kindpng.com/imgv/mwToi_exemplo-de-spritesheet-sprite-sheet-megaman-png-transparent/
- Haase, C., & Guy, R. (2007). *Filthy rich clients: Developing animated and graphical effects for desktop Java applications*. Pearson Education.
- Log4j. (2026). Log4j – Apache Log4j 2. Retrieved May, 2026, from <https://logging.apache.org/log4j/>
- SLF4J. (2026). SLF4J. Retrieved May, 2026, from <http://www.slf4j.org/>

CHAPTER 5

Unit Testing

Prerequisites

- Completion of Chapters 1 through 4, including event-driven graphical user interface (GUI) development, exception handling, concurrency, and debugging techniques.
- Familiarity with object-oriented programming (OOP) in Java, particularly class design, inheritance, and polymorphism, as unit tests operate directly on class-level behaviour.
- Experience with the Model-View-Controller (MVC) pattern and JavaFX scene-graph architecture, as the case studies apply testing to game components built in earlier chapters. In particular, the 2D platformer game developed in Chapter 4 serves as the direct foundation for the first case study.
- Access to an integrated development environment (IDE), such as IntelliJ IDEA, with Maven configured, as the testing frameworks covered in this chapter are managed through project dependencies.

Expected Outcomes

- Explain the V-Model of software testing and distinguish unit testing from integration testing and system testing within that framework.
- Write JUnit 5 test classes utilising `@Test`, `@BeforeEach`, and `@AfterEach` annotations with appropriate assertion methods.
- Apply Mockito to create mock objects that isolate the class under test from its dependencies.
- Utilize Java reflection to access and test private methods and fields that are inaccessible through the public application programming interface (API).
- Describe the test-driven development (TDD) red-green-refactor cycle and apply it to implement a simple feature from a failing test.
- Design a JUnit test suite that achieves meaningful branch coverage for a multi-class JavaFX application.

SOFTWARE testing is a fundamental process in software engineering that serves to ensure the quality of a software product. Its purpose is to evaluate and verify that a software application functions according to its specified requirements. The benefits of a rigorous testing process include the prevention of defects, reduced long-term maintenance costs, and enhanced overall software performance.

The aim of software testing is to determine whether an application can operate correctly when subjected to conditions designed to induce failure. These conditions are introduced through systematically designed test cases, which enable the observation of the software's behavior under specific circumstances. Test cases are typically designed to cover a broad spectrum of potential issues that may arise in a production environment. Common software defects include incorrect calculations, erroneous data manipulation, flawed data matching, inaccurate search results, faulty processing of data relationships, incorrect implementation of business rules, and inadequate performance, among others.

5.1 Basic Software Testing

According to Kaner (2009), software testing is defined as a systematic, technical investigation conducted to provide stakeholders with information about the quality of the product or service under test. The key terms in this definition require further clarification. Systematic refers to a methodical approach derived from experimentation, experience, and observation. Technical denotes the application of specific skills and practical knowledge. Investigation signifies a detailed inquiry or systematic examination. In essence, software testing employs specialized technical skills and knowledge to provide an evidence-based assessment of a software project's quality.

5.1.1 Importance

As software applications increase in complexity, encompassing more features and supporting diverse platforms, achieving a defect-free state becomes increasingly challenging. Defects, in any form, can compromise software functionality and, in critical cases, lead to severe system malfunctions. The history of software engineering includes numerous examples of severe consequences resulting from software defects, ranging from substantial financial losses to physical harm.

For instance, five nuclear power plants in the United States were temporarily shut down due to a flaw in a simulation program used for designing earthquake-resistant nuclear reactors (Lin, 1985). The root cause was the use of an arithmetic sum of a set of numbers instead of their absolute values. An F-18 fighter aircraft crashed due to a missing else clause in a conditional statement, a scenario that was erroneously assumed to be impossible (Neumann, 1986). The Ariane 5 spacecraft exploded during launch due to an integer overflow error (Le Lann, 1997). More recently, Toyota was required to recall over four million vehicles due to unintended acceleration caused by software defects (Austen-Smith et al., 2017). These cases underscore the potential for rigorous testing to have prevented such incidents.

The optimal level of effort and expenditure on testing has been a subject of debate. The justification for a given level of testing often depends on the potential cost savings achieved by preventing critical defects. In other words, the decision is determined by the potential impact of a failure relative to the cost of testing (i.e.,

the risk-to-impact ratio). Furthermore, intangible consequences, such as customer dissatisfaction and damage to brand reputation, can be prohibitively expensive.

A fundamental principle in software engineering is that defects found later in the development cycle are more expensive to fix. The cost to resolve a defect increases substantially, often exponentially, as it progresses through the development lifecycle. For example, the effort required to correct a defect in the requirements specification that could be fixed in one person-hour during the design phase may increase exponentially if it is not discovered until after the software has been deployed. These increased costs are attributable to rework in development, additional meetings, re-testing, and other related activities.

5.1.2 Sources of bugs

As previously established, the cost to correct a defect is significantly lower the earlier it is identified and resolved in the development lifecycle. A study by Perry and Stieg (1993) revealed that defects introduced during the implementation and testing phases constitute approximately 50 percent of all defects found. This finding emphasizes the importance of defect detection in the earlier phases of development, as the cost and complexity of correction increase significantly for defects that persist into later stages.

During the requirements phase, defects can be introduced through an inadequate specification process. These often manifest as erroneous, incomplete, or inconsistent requirements, frequently attributable to insufficient diligence during the requirements gathering and elicitation stages. During the design phase, defects can arise from flaws in the system's architecture or detailed design. A notable example is a design flaw discovered in the communication protocol of some Bluetooth-enabled devices, which created a security vulnerability (Zuo et al., 2019). In the implementation and testing phases, defects can be introduced directly during coding or can arise from issues with the configuration of the testing environment.

5.1.3 Testing methods

The V-Model, depicted in Figure 5.1, is a widely recognized conceptual model that illustrates the software testing process in relation to development phases (Java T

Point, 2020). It outlines two primary phases: verification and validation. Verification is the process of evaluating work products to determine if they meet their specified requirements, often involving static analysis of components. Validation, by contrast, is the process of evaluating the final software product during or at the end of development to determine if it satisfies customer expectations. In essence, verification asks, *Are we building the product right?*, while validation inquires, *Are we building the right product?*

The left side of the V-Model represents the development phases and their corresponding work products. The right side enumerates the testing activities that correspond to each development phase. A brief explanation of the entries on the left-hand side is provided below:

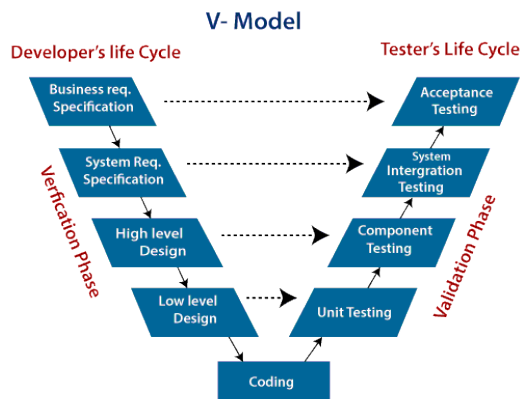


Figure 5.1: The V-Model of Software Testing (Java T Point 2020)

- **Business requirement specification:** This phase focuses on defining the high-level goals and expectations of the customer. Clear communication is crucial to ensure that the development team fully understands these requirements.
- **System requirement specification:** This stage involves detailing the specific features, functions, and behaviors of the system required to meet the business needs.

- **High-level design:** This phase defines the overall system architecture, including the relationships between major components, modules, and data systems. It describes high-level data flows and data structures.
- **Low-level design:** This phase involves the detailed design of individual software components, often including the creation of class diagrams that specify variables and methods.
- **Implementation:** This phase involves the writing of source code to build the software application according to the design specifications.

The right side of the V-Model illustrates the validation phases, where each level of testing corresponds to a level on the development (verification) side:

- **Unit testing:** Corresponding to the Low-Level Design phase, unit testing focuses on verifying the correctness of the smallest testable parts of an application, such as individual methods or classes. These tests ensure that each unit functions correctly in isolation.
- **Component testing:** This level of testing, corresponding to the High-Level Design phase, validates the functionality of individual software components, which are typically composed of multiple, related units.
- **Integration testing:** Following component testing, integration testing verifies the interaction and data flow between integrated components to ensure they function correctly as a combined group.
- **System testing:** Corresponding to the System Requirement Specification phase, system testing evaluates the complete and integrated software system to ensure it meets all specified functional and non-functional requirements. This is often conducted by a dedicated Quality Assurance (QA) team in an environment that simulates production.
- **Acceptance testing:** Corresponding to the Business Requirement Specification phase, this is a crucial final stage where the system is evaluated by end-users or client representatives to confirm that it meets their business

needs and is acceptable for deployment. It focuses on validating the system's functionality and usability in a real-world context.

5.1.4 Testing objectives

The primary objectives of software testing are to identify defects as early as possible in the development lifecycle and to verify that these defects are resolved. To achieve these goals, testers design and implement test cases, which are scripted procedures or separate programs executed with several intentions: (1) to uncover defects, (2) to verify that the system meets its specified requirements in its target environment, (3) to validate that the system is suitable for its intended purpose, and (4) to ensure the system performs as expected.

The following list outlines common testing objectives:

- **Alpha testing:** A form of internal acceptance testing conducted by in-house teams (e.g., developers, QA) to identify defects before the software is released to external users.
- **Beta testing:** A form of external acceptance testing where the software is released to a limited audience of real users outside of the development organization to obtain feedback and identify defects in a real-world environment.
- **Conformance Testing:** Verifies that a software product conforms to specific standards, conventions, or regulations.
- **Performance testing:** Evaluates a system's responsiveness, stability, and scalability under a particular workload.
- **Recovery testing:** Assesses a system's ability to recover gracefully from failures, such as hardware crashes or network outages.
- **Stress testing:** Evaluates a system's robustness by testing it at or beyond the limits of its specified operational capacity.
- **Usability testing:** Evaluates how easy and intuitive a software application is to use from the perspective of an end-user.

5.2 Unit Testing

This course will focus on unit testing as it serves as the foundation for other testing methodologies. To illustrate the concept of software unit testing, an analogy can be drawn to the process of troubleshooting electronic devices. Consider a scenario in which a stereo system is faulty, with sound produced by only one channel. The diagnostic process would involve isolating each major component (e.g., speakers, amplifier, source) and testing their interactions. This level of testing is analogous to software integration testing. If the amplifier is suspected as the source of the fault, its internal components can be tested individually, for instance, by using a multimeter to measure voltage. This systematic examination of the smallest functional parts corresponds to unit testing. Although potentially time-consuming, this granular approach enables the precise isolation of faulty components. This mirrors the practice of unit testing in software development, where individual units of code are tested in isolation to verify their correct functionality.

In a software context, a unit test is therefore a piece of code that exercises a small, specific unit of functionality, such as a single method within a class. When a unit test verifies that a code segment performs as expected, that unit can be integrated into the larger system with a higher degree of confidence in its correctness.

A primary challenge in unit testing is the management of external dependencies, such as file systems or databases, which a unit of code may rely upon. To overcome this challenge and maintain isolation, testers often use mock objects. These are simulated objects that mimic the behavior of real dependencies in a controlled manner. For example, when testing a method that makes a Hypertext Transfer Protocol (HTTP) request to an external API, the test can be isolated from the network by using a mock object. This mock would not make a real network request but would instead return a predefined response (e.g., from a local JavaScript Object Notation (JSON) file) that mimics the data from the actual API. This approach provides complete control over the test environment and allows for the predictable and repeatable verification of the method's behavior.

5.2.1 Objective of unit testing

A well-designed unit testing strategy provides developers with greater confidence in their code. It is widely recognized that as a software system grows, the likelihood of defects increases. This is due to both the increased functional scope and the complex interactions between components. A well-structured suite of unit tests facilitates the early detection and localization of these defects. By writing unit tests concurrently with feature development, the number of defects introduced into the codebase can be minimized. This, in turn, reduces the need for complex debugging efforts.

It is important to emphasize that the defining characteristic of unit tests is their isolation. A well-designed unit test must be autonomous and not rely on external dependencies such as databases, APIs, or network resources. This isolation ensures that the test exclusively verifies the behavior of the unit under test, which enhances the reliability and repeatability of the test results.

5.2.2 Example scenario

In the context of an e-commerce application, the shopping cart is a critical component that requires rigorous testing to ensure its reliability. Several key test scenarios are essential, including: adding an item (verifying correct addition and state updates), removing an item (confirming removal and state updates), modifying item quantities (validating quantity and price changes), and calculating the total price (verifying correctness across various item combinations, discounts, and updates).

Each unit test focuses on a specific method or function within the shopping cart's source code. For example, a test for the `addItemToCart(Item item, int quantity)` method would invoke it with various inputs to verify that items are added correctly and that the cart's internal state is updated accurately. Similarly, dedicated unit tests would be created for methods such as `removeItemFromCart()`, `updateItemQuantity()`, and `calculateTotalPrice()` to verify their individual functionality.

Unit testing is of significant importance in this scenario as it allows for the isolation and verification of the smallest functional units of code. By testing individual components in isolation, unit tests facilitate the rapid identification and correc-

tion of defects within the shopping cart's underlying logic. This proactive approach helps prevent runtime problems such as inaccurate price calculations or item management failures. Ultimately, a comprehensive suite of unit tests contributes to the development of a resilient and dependable shopping cart feature, enhancing the user experience and mitigating potential financial losses from operational failures.

This example emphasizes that unit testing is focused on the granular examination of individual code components to ensure their correct behavior in isolation. It is essential to recognize that unit tests, by design, do not evaluate complex interactions between different functions or modules, nor do they test the integration of business logic with the GUI. These higher-level interactions are assessed through other testing methodologies, such as integration testing and system testing.

5.2.3 JUnit 5

JUnit (2026) is the predominant framework for conducting unit testing in Java. Its library provides a comprehensive set of tools that streamline and systematize the unit testing process. By convention, for a given class named `Foo`, a corresponding test class named `FooTest` is created. This test class contains methods designed to test the public methods of `Foo` at the unit level. Each method within `FooTest` defines a specific test case that verifies an expected outcome and determines whether the test passes or fails.

JUnit provides a suite of `assert` methods that form the core of a unit test's verification logic. The underlying principle is to invoke an assertion method within the body of a test method, providing a condition that is expected to evaluate to true. For example, a test could verify that an empty `ArrayList` has a size of two after two elements have been added. This expected outcome would be expressed as a condition within an assertion. If the condition evaluates to false, the assertion fails, which in turn causes the entire test case to fail.

Script 5.1 illustrates an example of a JUnit test class structure. JUnit employs annotations, such as `@Test`, to identify test methods. When the test suite is executed, the JUnit framework automatically discovers and runs all methods marked with the `@Test` annotation, evaluating the assertions within them to determine the test results.

Script 5.1: Example JUnit 5 Test Case

NameTest.java

```
+ 1 import org.junit.jupiter.api.Test;
+ 2 import static org.junit.jupiter.api.Assertions.assertTrue;
+ 3
+ 4 class ShoppingCartTest {
+ 5
+ 6     @Test
+ 7     void addItemToCart_givenAnItemAdd_cartShouldContainItemEntity() {
+ 8         ShoppingCart cart = new ShoppingCart();
+ 9         Item item = new Item("Java Textbook", 399.00);
+10         cart.addItem(item);
+11         assertTrue(cart.getItems().contains(item), "Item not added to cart");
+12     }
+13 }
```

Assertion methods

Assertion methods are fundamental tools in unit testing frameworks used to verify the expected behavior of the code under test. Both JUnit 4 and JUnit 5 offer a comprehensive suite of static assertion methods, facilitating the evaluation of specific conditions during test execution. These methods compare the actual outcome of a variable or expression against a predefined expected value, ensuring it aligns with the expected behavior. If the assertion condition is not met, the method signals a test failure by throwing an `AssertionError` or a similar exception.

In JUnit 4, the assertion methods typically begin with the keyword `assert`. For instance, `assertTrue("message", condition)` fails the test if the provided condition evaluates to false, optionally displaying a message to provide context for the failure. JUnit 5, on the other hand, encapsulates its assertion methods within the `org.junit.jupiter.api.Assertions` class. While the functionality largely mirrors that of JUnit 4, the method names and usage show slight variations. Other common methods address a variety of assertion requirements. Commonly used JUnit 5 assertion methods include:

- **`Assertions.assertTrue(condition, message)`**: Fails if the boolean condition evaluates to false.

- **Assertions.assertFalse(condition, message):** Fails if the boolean condition evaluates to true.
- **Assertions.assertEquals(expected, actual, message):** Fails if the expected and actual values are not equal.
- **Assertions.assertSame(expected, actual, message):** Fails if the expected and actual variables do not refer to the same object.
- **Assertions.assertNotSame(expected, actual, message):** Fails if the expected and actual variables refer to the same object.
- **Assertions.assertNull(object, message):** Fails if the object is not null.
- **Assertions.assertNotNull(object, message):** Fails if the object is null.
- **Assertions.fail(message):** Unconditionally fails the test with the provided message.
- **Assertions.assertThrows(expectedType, executable, message):** Fails if the executable (e.g., a lambda expression) does not throw an exception of the expectedType.

Script 5.2 presents a simple Calculator class used for demonstrating JUnit tests. The corresponding test class, CalculatorTest, is illustrated in Script 5.3 and is designed to evaluate the Calculator's functionality. It contains two distinct test methods, each preceded by the execution of a setup method. The setUp() method, annotated to run before each test, instantiates the Calculator object. The first test method, add_givenTwoPositiveNumbers_thenSum() (Line 11), invokes the add() method with values 3 and 4 and subsequently verifies that the result is 7 using the assertion in Line 13. The second test method, subtract_givenTwoPositiveNumbers_thenDifference(), follows a similar structure, invoking the subtract() method with values 9 and 5 and using an assertion (Line 18) to confirm the result matches the expected value of 4.

Script 5.2: A Simple Calculator Class for JUnit Demonstration**Calculator.java**

```
+ 1 public class Calculator {
+ 2     public int add(int a, int b) {
+ 3         return a + b;
+ 4     }
+ 5     public int subtract(int a, int b) {
+ 6         return a - b;
+ 7     }
+ 8 }
```

JUnit naming conventions

JUnit promotes the use of descriptive naming conventions for both test classes and test methods. A common convention for test classes is to append the suffix `Test` to the name of the class being tested. For instance, the test class in Script 5.3 is named `CalculatorTest`, which clearly indicates that it is designed to test the `Calculator` class.

Script 5.3: JUnit Test Class with Two Test Methods**CalculatorTest.java**

```
+ 1 import static org.junit.jupiter.api.Assertions.assertEquals;
+ 2 import org.junit.jupiter.api.Test;
+ 3
+ 4 public class CalculatorTest {
+ 5     private Calculator calculator;
+ 6     @BeforeEach
+ 7     void setUp() {
+ 8         calculator = new Calculator();
+ 9     }
+10     @Test
+11     void add_givenTwoPositiveNumbers_thenSum() {
+12         int result = calculator.add(3, 4);
+13         assertEquals(7, result);
+14     }
+15     @Test
+16     void subtract_givenTwoPositiveNumbers_thenDifference() {
+17         int result = calculator.subtract(9, 5);
+18         assertEquals(4, result);
+19     }
+20 }
```


Test method names should be self-explanatory, allowing a developer to understand the test's purpose and the specific behavior under test without needing to examine the method's implementation. This is crucial because when a test suite is executed, the names of all executed test methods are typically listed along with their pass or fail status. A descriptive method name allows a developer to quickly identify a failed test and understand the context of the failure.

In contrast to standard Java method naming conventions, which often favor brevity, test method names benefit from being verbose and highly descriptive. Several naming conventions have emerged in JUnit testing to clearly convey a test's purpose and scope. The following are some common formats:

1. **method_givenCondition_thenExpectedResult:** This format explicitly states the method being tested, the condition or input scenario, and the expected outcome, e.g.,
 - `add_givenTwoPositiveNumbers_thenSum`
 - `subtract_givenNegativeNumber_thenDifference`
2. **testMethodName:** This format is simple and straightforward, beginning with the word *test* followed by the name of the method being tested, e.g.,
 - `testAdd`
 - `testSubtract`
3. **shouldExpectedBehavior_whenStateUnderTest:** This format emphasizes the expected behavior and the specific state or scenario under which the test is performed, e.g.,
 - `shouldReturnSum_whenTwoNumbersArePositive`
 - `shouldReturnDifference_whenSubtractingSmallerFromLarger`
4. **whenCondition_thenExpectedResult:** This format highlights the condition and the expected outcome, e.g.,
 - `whenAddingTwoPositiveNumbers_thenSumIsCorrect`
 - `whenSubtractingSmallerNumber_thenDifferenceIsPositive`

5. **givenCondition_whenAction_thenExpectedResult:** This format is highly descriptive, explicitly stating the condition, the action being taken, and the expected result, e.g.,
- `givenTwoPositiveNumbers_whenAdded_thenSumIsCorrect`
 - `givenNegativeNumber_whenSubtracted_thenDifferenceIsCorrect`

Script 5.4 demonstrates the implementation of test methods that follow several of these naming conventions. These descriptive conventions effectively communicate the expected behavior being tested, often obviating the need to examine the implementation details of the test itself. For clarity and consistency, this coursebook will adopt the `method_givenCondition_thenExpectedResult` format for all subsequent test method examples.

Unit testing tips

A commonly recommended practice in unit testing is to design tests that are trustworthy and focused. To achieve this, complex test cases are often decomposed into multiple, smaller tests, with each test case ideally containing only a single assertion. This approach ensures that each test method verifies a single, specific aspect of the unit's functionality, which makes failures easier to diagnose.

Furthermore, to maintain test clarity, it is advisable to avoid complex programmatic logic within test cases. The use of control flow statements such as `if-else`, `switch`, `for`, and `try-catch` can introduce complexity that obscures the test's primary objective. By minimizing such constructs, tests remain concise and focused, which improves their readability and maintainability.

Exhaustive testing of every possible input is often impractical, particularly for methods that accept a large or infinite range of values (e.g., real numbers). Therefore, a strategic approach is to select a limited set of test cases that are most likely to uncover defects. A common technique for this is boundary value analysis, which prioritizes the testing of values at the edges of an input domain. For example, when testing a method that operates on a numeric range, the boundary values (e.g., minimum, -1, 0, 1, and maximum) are more likely to expose defects than values in the middle of the range. Thorough testing of these boundary conditions often pro-

Script 5.4: Demonstration of Various JUnit Test Method Naming Conventions
CalculatorTest.java

```
1 //Imports are omitted
2 public class CalculatorTest {
3     private Calculator calculator;
4     ...
+ 23 // 1. method_givenCondition_thenExpectedResult
+ 24 @Test
+ 25 void add_givenTwoPositiveNumbers_thenSum() {
+ 26     assertEquals(7, calculator.add(3, 4));
+ 27 }
+ 28 // 2. testMethodName
+ 29 @Test
+ 30 void testAdd() {
+ 31     assertEquals(7, calculator.add(3, 4));
+ 32 }
+ 33 // 3. shouldExpectedBehavior_whenStateUnderTest
+ 34 @Test
+ 35 void shouldReturnSum_whenTwoNumbersArePositive() {
+ 36     assertEquals(7, calculator.add(3, 4));
+ 37 }
+ 38 // 4. whenCondition_thenExpectedResult
+ 39 @Test
+ 40 void whenAddingTwoPositiveNumbers_thenSumIsCorrect() {
+ 41     assertEquals(7, calculator.add(3, 4));
+ 42 }
+ 43 // 5. givenCondition_whenAction_thenExpectedResult
+ 44 @Test
+ 45 void givenTwoPositiveNumbers_whenAdded_thenSumIsCorrect() {
+ 46     assertEquals(7, calculator.add(3, 4));
+ 47 }
+ 48 }
```

vides sufficient confidence in the unit's correctness without requiring the testing of numerous other values within the range.

Other scenarios prone to off-by-one errors and other defects include operations on the boundaries of arrays or collections (e.g., the first and last elements). This can be a common source of error, particularly for developers accustomed to different programming languages that may use 1-based instead of 0-based indexing. Additionally, inconsistent handling of different representations for empty or invalid values (e.g., `0`, `-1`, `null`, `NaN`, or empty collections) can also introduce defects.

Consider a scenario where the applicable age range for an insurance package is 18 to 65, inclusive. A concise unit test can be constructed using `JUnit` to verify if a given age falls within this specified range. An efficient and comprehensive testing approach involves selecting test cases from three partitions: ages below 18, ages within the 18-65 range, and ages above 65. If time and resources permit, testing the exact boundary values of 18 and 65 can provide additional confidence in the implementation's correctness. Testing additional values within these partitions is generally considered redundant, as they are not expected to reveal new types of defects.

For this insurance eligibility scenario, using exceptions and the `assertThrows` method provides several advantages over returning a simple boolean value. By throwing an `IllegalArgumentException` for invalid ages, the error condition is made explicit, which offers greater insight into the nature of a test failure. This approach also allows for distinguishing between different invalid input cases (e.g., age too young vs. age too old), which enhances the diagnostic value of the tests.

Furthermore, using `assertThrows` strengthens test assertions. It verifies not only that an exception is thrown but also that the thrown exception is of the expected type and optionally contains a specific message. This approach creates more robust tests that can be adapted as the eligibility logic evolves, facilitating the testing of various invalid input scenarios.

Additionally, modifying the `isEligibleForInsurance()` method to throw exceptions rather than returning boolean values follows the design principle of explicit error signaling, which enhances the overall robustness and maintainability of the codebase.

Script 5.5: Unit Test Using Exception Assertions**InsuranceEligibilityTest.java**

```
+ 1 //Imports are omitted
+ 2 class InsuranceEligibilityTest {
+ 3
+ 4     @Test
+ 5     void isEligible_givenAgeUnder18_thenFalse() {
+ 6         int age = 17;
+ 7         assertFalse(isEligibleForInsurance(age), "Age under 18 should be ineligible");
+ 8     }
+ 9
+ 10    @Test
+ 11    void isEligible_givenAgeBetween18And65_thenTrue() {
+ 12        int age = 35;
+ 13        assertTrue(isEligibleForInsurance(age), "Age within 18-65 should be eligible");
+ 14    }
+ 15
+ 16    @Test
+ 17    void isEligible_givenAgeOver65_thenFalse() {
+ 18        int age = 70;
+ 19        assertFalse(isEligibleForInsurance(age), "Age over 65 should be ineligible");
+ 20    }
+ 21
+ 22    // Additional boundary tests for completeness (optional)
+ 23    @Test
+ 24    void isEligible_givenAgeExactly18_thenTrue() {
+ 25        int age = 18;
+ 26        assertTrue(isEligibleForInsurance(age), "Age 18 should be eligible");
+ 27    }
+ 28
+ 29    // Additional boundary tests for completeness (optional)
+ 30    @Test
+ 31    void isEligible_givenAgeExactly65_thenTrue() {
+ 32        int age = 65;
+ 33        assertTrue(isEligibleForInsurance(age), "Age 65 should be eligible");
+ 34    }
+ 35
+ 36    private boolean isEligibleForInsurance(int age) {
+ 37        return age >= 18 && age <= 65;
+ 38    }
+ 39 }
```

The implementation shown in Script 5.5 modifies the approach from Script 5.6 by changing the handling of invalid cases. Specifically, test methods such as `isEligible_givenAgeUnder18_thenFalse` are refactored to verify that an `IllegalArgumentException` is thrown when the input age falls outside the valid range (18-65). To support this, the `isEligibleForInsurance()` method is modified to explicitly throw this exception with an informative message indicating whether the age is below or above the acceptable limits. This enhancement provides more precise feedback during testing and facilitates debugging.

Script 5.6: Unit Test Using Boolean Assertions**InsuranceEligibilityTest.java**

```

1 //Imports are omitted
2 class InsuranceEligibilityTest {
3
4     @Test
5     void isEligible_givenAgeUnder18_thenFalse() {
6         int age = 17;
7         assertFalse(isEligibleForInsurance(age), "Age under 18 should be ineligible");
+ 8         Exception exception = assertThrows(IllegalArgumentException.class, () -> {
+           isEligibleForInsurance(age); });
+ 9         assertEquals("Age must be 18 or older", exception.getMessage());
10     }
11     ...
12
13     @Test
14     void isEligible_givenAgeOver65_thenFalse() {
15         int age = 70;
16         assertFalse(isEligibleForInsurance(age), "Age over 65 should be ineligible");
+ 17         Exception exception = assertThrows(IllegalArgumentException.class, () -> {
+           isEligibleForInsurance(age); });
+ 18         assertEquals("Age must be 65 or younger", exception.getMessage());
19     }
20     ...
21
22     private boolean isEligibleForInsurance(int age) {
23         return age >= 18 && age <= 65;
+ 24         if (age < 18) {
+ 25             throw new IllegalArgumentException("Age must be 18 or older");
+ 26         } else if (age > 65) {
+ 27             throw new IllegalArgumentException("Age must be 65 or younger");
+ 28         }
+ 29         return true;
30     }
31 }

```

JUnit test suites

To assess the comprehensive functionality of an application, multiple test classes are typically executed together. JUnit provides a test suite mechanism to facilitate this process, which allows for the aggregation of multiple test classes into a single executable suite. Executing the test suite then invokes all tests from all included classes in a single, unified run.

In JUnit 5, creating a test suite requires the `org.junit.platform:junit-platform-suite-engine` dependency. It can be integrated into the `pom.xml` file as demonstrated in Script 5.7. After this dependency is added and the Maven project is refreshed, a test suite class can be implemented, as shown in Script 5.8. This script demonstrates a JUnit 5 test suite that aggregates tests from the `CalculatorTest` and `InsuranceEligibilityTest` classes. This aggregation is specified in Line 6 using the `@SelectClasses` annotation.

Script 5.7: pom.xml

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.
    apache.org/xsd/maven-4.0.0.xsd">
5      <modelVersion>4.0.0</modelVersion>
        ...
36     <version>${junit.version}</version>
37     <scope>test</scope>
38 </dependency>
+ 39 <dependency>
+ 40     <groupId>org.junit.platform</groupId>
+ 41     <artifactId>junit-platform-suite-engine</artifactId>
+ 42     <version>1.12.2</version>
+ 43     <scope>test</scope>
44 </dependency>
45 </dependencies>
    ...

```

The `@TestInstance(TestInstance.Lifecycle.PER_CLASS)` annotation on Line 7 configures the test instance lifecycle. By default, JUnit 5 creates a new instance of a test class for each test method executed. This annotation, however, modifies that behavior, instructing JUnit to create only a single instance of the test class to be shared across all test methods within it.

Upon executing this test suite, the results for all aggregated test methods are displayed, as illustrated in Figure 5.2. The descriptive method names provide a clear overview of the behavior tested, the conditions applied, and the expected outcomes.

Script 5.8: Example Test Suite

JUnitTestSuite.java

```
+ 1 import org.junit.jupiter.api.TestInstance;
+ 2 import org.junit.platform.suite.api.SelectClasses;
+ 3 import org.junit.platform.suite.api.Suite;
+ 4
+ 5 @Suite
+ 6 @SelectClasses({CalculatorTest.class, InsuranceEligibilityTest.class})
+ 7 @TestInstance(TestInstance.Lifecycle.PER_CLASS)
+ 8 public class JUnitTestSuite {
+ 9
+ 10 }
```

JUnitTestSuite	13 ms	Tests passed: 12 of 12 tests – 13 ms
CalculatorTest	12 ms	/Library/Java/JavaVirtualMachin
shouldReturnSum_whenTwoNumbersArePositive()	12 ms	
givenTwoPositiveNumbers_whenAdded_thenSumIsCorrect()		
testAdd()		
whenAddingTwoPositiveNumbers_thenSumIsCorrect()		
subtract_givenTwoNumbers_thenDifference()		
add_givenTwoPositiveNumbers_thenSum()		
add_givenTwoNumbers_thenSum()		
InsuranceEligibilityTest	1 ms	
isEligible_givenAgeBetween18And65_thenTrue()		
ageAboveRange_shouldBeInvalid()	1 ms	
ageBelowRange_shouldBeInvalid()		
isEligible_givenAgeExactly18_thenTrue()		
isEligible_givenAgeExactly65_thenTrue()		

Process finished with exit code

Figure 5.2: Execution Results of an Aggregated Test Suite

Mocking

Mocking is a technique used in unit testing to simulate the behavior of real objects, particularly complex external dependencies such as web services, databases, or file systems. Because unit testing requires the isolation of the component under test, the use of live external dependencies would violate this principle. Therefore, mocking is an indispensable technique for testing code that interacts with these external systems. This approach mitigates the impact of factors such as network latency or external service availability, ensuring that unit tests are reliable, fast, and focused exclusively on the functionality of the unit under test. Mocking is particularly useful in the following scenarios:

- When the code under test has dependencies that are difficult or impractical to set up in a test environment.
- When specific scenarios, such as error conditions or particular data responses, must be simulated, which may be difficult to reproduce with a live dependency.
- When the code under test must be completely isolated from external factors to ensure that test failures are attributable only to defects within the unit itself.

A widely used framework for creating mock objects in Java is *Mockito* (2026). It enables the isolation of code by allowing developers to substitute real dependencies with mock objects that have a defined, controlled behavior. This facilitates focused testing by ensuring the predictable behavior of dependencies and allowing for the verification of interactions between the code under test and its collaborators. Its intuitive API simplifies the process of defining the behavior of mock objects, which promotes the creation of clear and maintainable unit tests.

Script 5.9 presents the `FetchData` class and its `fetchRange()` method, which is a modified version of the method implemented in the case study of Chapter 2. The purpose of this `fetchRange()` method is to retrieve currency data from an external API for a given currency symbol and a specified number of days. The method is expected to return an array of date-rate pairs in JSON format. Because this method

Script 5.9: The Modified FetchData Class to Be Tested Using Mocking

```
+ 1 //Imports are omitted
+ 2 public class FetchData {
+ 3     public static final DateTimeFormatter formatter = DateTimeFormatter.ofPattern
+ 4         ("yyyy-MM-dd");
+ 5     public static List<CurrencyEntity> fetchRange(String symbol, int N) throws
+ 6         IOException {
+ 7         String dateEnd = LocalDate.now().format(formatter);
+ 8         String dateStart = LocalDate.now().minusDays(N).format(formatter);
+ 9         String urlStr = String.format("https://cmu.to/SE233currencyapi?base=THB&
+10             symbol=%s&start_date=%s&end_date=%s", symbol, dateStart, dateEnd);
+11         List<CurrencyEntity> histList = new ArrayList<>();
+12         String retrievedJson = IOUtils.toString(new URL(urlStr), Charset.
+13             defaultCharset());
+14         JSONObject jsonObj = new JSONObject(retrievedJson).getJSONObject("rates");
+15         Iterator keysToCopyIterator = jsonObj.keys();
+16         while(keysToCopyIterator.hasNext()) {
+17             String key = (String) keysToCopyIterator.next();
+18             Double rate = Double.parseDouble(jsonObj.get(key).toString());
+19             histList.add(new CurrencyEntity(rate, key));
+20         }
+21         histList.sort(Comparator.comparing(CurrencyEntity::getDate));
+22         return histList;
+23     }
```

depends on an external API, its unit test requires the use of mocking to maintain isolation.

Script 5.10 presents the `FetchDataTest` class, which uses the Mockito framework to test the `fetchRange()` method in isolation from its external dependencies. This method is designed to fetch currency exchange rates over a specified date range. To verify its correct operation without depending on a live API, the test uses a mock object to simulate the API response. This isolation improves the reliability and execution speed of the test.

The `setUp()` method, annotated with `@BeforeEach`, prepares the test environ-

ment before each test case. It initializes a mock for the static `IOUtils.toString()` method from the Apache Commons IO library. It defines `mockJsonData`, a JSON string that simulates the API response, containing exchange rates for three days (January 1, 2024 to January 3, 2024). `Mockito.mockStatic()` is used to create a mock of the `IOUtils` class. This mock is configured to return the predefined `mockJsonData` string whenever the `IOUtils.toString()` method is invoked, regardless of the URL or charset arguments. Within the test method `fetchRange_givenValidSymbolAndN_returnsExpectedCurrencyEntities()` (annotated with `@Test`), a list of expected `CurrencyEntity` objects is defined. This expected list corresponds to the data within the `mockJsonData` string. The test then invokes the `fetchRange()` method of the `FetchData` class, requesting three days of data. Because of the mock configured in the `setUp()` method, this invocation does not make a network request but instead receives the `mockJsonData`. Finally, an assertion is used to compare the size of the list returned by `fetchRange()` against the size of the expected list to verify the outcome.

The use of mocking ensures that this unit test verifies only the logic within the `fetchRange()` method itself, isolating it from external factors like network latency or API availability. This approach enhances the robustness and portability of the test, providing high confidence in the method's logical correctness.

Reflection

Reflection is a feature that enables the examination and modification of an application's runtime behavior. It allows a program to inspect its own objects, classes, and methods at runtime and, if necessary, modify their behavior. A common use case for reflection in Java is the unit testing of `private` methods, which are not otherwise accessible from outside the class. Without reflection, the alternative would be to change the method's visibility to `public` for testing purposes, an approach that is considered an anti-pattern as it violates the principle of encapsulation.

Reflection, by contrast, provides a mechanism to bypass standard access control, allowing `private` methods to be invoked as if they were `public`. This eliminates the need to modify the source code of the class under test. Specifically, the reflection API enables a program to perform the following actions at runtime:

Script 5.10: Unit Test for the Modified FetchData Class Using Mockito

```
+ 1 //Imports are omitted
+ 2 import static org.junit.jupiter.api.Assertions.assertEquals;
+ 3
+ 4 class FetchDataTest {
+ 5     private MockedStatic<IOUtils> mockIOUtils;
+ 6     private String mockJsonData;
+ 7
+ 8     @BeforeEach
+ 9     void setUp() {
+10         mockJsonData = "{\n" +
+11             "  \"rates\": {\n" +
+12             "    \"2024-01-01\": 37.983211517333984,\n" +
+13             "    \"2024-01-02\": 37.49863052368164,\n" +
+14             "    \"2024-01-03\": 37.66517639160156\n" +
+15             "  }\n" +
+16             "}";
+17         mockIOUtils = Mockito.mockStatic(IOUtils.class);
+18         mockIOUtils.when(() -> IOUtils.toString(Mockito.any(URL.class), Mockito.
+           any(Charset.class))).thenReturn(mockJsonData);
+19     }
+20
+21     @Test
+22     void fetchRange_givenValidSymbolAndN_returnsExpectedCurrencyEntities() throws
+       IOException {
+23         List<CurrencyEntity> expectedList = new ArrayList<>();
+24         expectedList.add(new CurrencyEntity(37.983211517333984, "2024-01-01"));
+25         expectedList.add(new CurrencyEntity(37.49863052368164, "2024-01-02"));
+26         expectedList.add(new CurrencyEntity(37.66517639160156, "2024-01-03"));
+27         expectedList.sort(Comparator.comparing(CurrencyEntity::getDate));
+28         List<CurrencyEntity> actualList = FetchData.fetchRange("EUR", 3); // Fetch
+           3 days of data
+29         assertEquals(actualList.size(), actualList.size());
+30     }
+31 }
```

- Examine the class of an object.
- Create an instance of a class whose name is not known until runtime.
- Inspect the fields and methods of a class.
- Invoke any method on an object.
- Modify the accessibility of constructors, methods, and fields.

Suppose the `Calculator` class contains a private method, `magicFunction()`, that computes the hypotenuse of a right triangle using the Pythagorean theorem, as shown in Script 5.11. In such scenarios, a direct test of this private method may be necessary to verify its internal logic without altering its visibility. Java's reflection API facilitates this process.

Script 5.11: Adding a Private Method to the Calculator Class**Calculator.java**

```
1 public class Calculator {  
    ...  
+ 8     private double magicFunction(int a, int b) {  
+ 9         return Math.sqrt(Math.pow(a, 2) + Math.pow(b, 2));  
+ 10    }  
11 }
```

Script 5.12 provides the unit test for this `magicFunction()`. Using the reflection API, a `Method` object representing `magicFunction()` is first obtained (Line 22). The method's accessibility is then modified to bypass Java's access control checks (Line 23). By invoking this method with specific arguments (e.g., 3 and 4), the output can be directly tested. In this case, the expected result of 5.0 verifies the correct implementation of the Pythagorean theorem.

While reflection is a powerful tool, it should be used with caution. Its improper use can lead to code that is less readable, less maintainable, and more difficult to debug. Furthermore, reflection typically introduces performance overhead and can create security vulnerabilities if used improperly. Therefore, a careful evaluation of its trade-offs is necessary before incorporating reflection into an application.

Script 5.12: Unit Testing a Private Method Using Reflection `CalculatorTest.java`

```
1 import static org.junit.jupiter.api.Assertions.assertEquals;
2 import org.junit.jupiter.api.Test;
3
4 public class CalculatorTest {
5     private Calculator calculator;
6     ...
+ 20     @Test
+ 21     void testMagicFunction() throws Exception {
+ 22         Method magicMethod = Calculator.class.getDeclaredMethod("magicFunction",
+                                     int.class, int.class);
+ 23         magicMethod.setAccessible(true);
+ 24         double result = (double) magicMethod.invoke(calculator, 3, 4);
+ 25         assertEquals(5.0, result, 0.0001);
+ 26     }
```

5.3 Test-Driven Development (TDD)

TDD is a fundamental methodology in modern software engineering. This methodology reverses the traditional development sequence by requiring the creation of automated tests prior to the implementation of the corresponding production code. This paradigm enforces the writing of the minimal amount of production code necessary to satisfy specific test requirements, resulting in a more robust, tightly focused architectural design. The standard TDD process executes through the following sequence:

1. **Decompose tasks:** The overall system requirements are partitioned into smaller, prioritized logical units.
2. **Create a failing test:** A single unit is selected, and a test case is authored to verify its intended functionality. Because no production code exists, this new test must fail. This deliberate failure confirms that the test itself functions correctly and accurately targets the unimplemented behavior.

3. **Write production code:** The minimal amount of production code is implemented exclusively to satisfy the new test.
4. **Run all tests:** The comprehensive test suite is executed to verify that the new implementation satisfies the immediate requirement without introducing regressions into existing functionality.
5. **Refactor and repeat:** The implementation is restructured to improve architectural design without altering observable behavior. This cycle repeats sequentially until all system requirements are fulfilled.

Authoring test cases before production code enforces strict architectural discipline. Developers must explicitly define the design of the solution, the flow of information, expected output parameters, and potential edge cases before any functional logic is implemented.

The essence of this discipline is formalized in the three rules of TDD established by Robert C. Martin (Martin, 2007):

1. Production code may only be written to make a failing unit test pass.
2. A unit test may only be written up to the point of failure, with compilation failures qualifying as legitimate test failures.
3. Production code may only be written up to the point sufficient to make the currently failing unit test pass.

These three constraints impose a rigorous micro-cycle on the development process. The first rule strictly prohibits speculative implementation, ensuring production code exists solely to satisfy a concrete test expectation. The second rule prevents the test from over-specifying behavior in a single step, forcing incremental growth. The third rule guards against over-engineering by limiting the implementation to the absolute minimum required to satisfy the immediate test. Collectively, these rules enforce a highly structured cycle wherein test code and production code evolve in short, verifiable increments.

Although TDD necessitates a substantial initial time investment, this methodology produces software characterized by higher quality, fewer runtime defects,

and significantly improved long-term maintainability. Furthermore, the rigorous test-first process ensures a precise translation of system requirements into functional code.

5.3.1 Benefits

TDD offers several distinct advantages in the software development process. By writing tests before the corresponding production code, developers receive immediate feedback on their implementation. This feedback is provided by a comprehensive suite of tests that covers both new and existing functionality. This allows for continuous regression testing, ensuring that new additions do not negatively impact existing functionality.

The TDD workflow naturally encourages the decomposition of complex problems into smaller, testable units. This provides a structured context for making low-level design decisions. By focusing on writing only the code necessary to pass a specific test, developers are guided away from overengineering, resulting in a more focused and efficient codebase.

Because each unit test has a limited and well-defined scope, the required rework to fix a failing test is typically less complex compared to debugging failures in larger, more integrated tests. Furthermore, TDD produces a comprehensive suite of automated tests as a natural byproduct of the development process. This test suite can be executed at any time to validate the correctness of the application and detect regressions introduced by recent changes.

In summary, development teams that follow TDD principles can potentially achieve higher code quality, increased productivity, and a comprehensive test suite with greater overall coverage. A concrete demonstration of TDD will be presented in the second case study of this chapter.

5.3.2 Red-Green-Refactor: A Worked Example

The TDD cycle is commonly referred to as the *red-green-refactor* loop (Beck, 2003). To make this cycle concrete, consider a minimal `Calculator` class that must provide an `add` method accepting two integers and returning their sum. The following

three steps trace the full loop from a failing test through a passing implementation to a refactored design.

Step 1. Red: Write a Failing Test. Before any production code exists, a JUnit test class is created that describes the expected behaviour of the add method. Because the Calculator class has not yet been implemented, the project will not compile, and any attempt to run the test suite will produce a failure. This deliberate failure is the *red* phase: it confirms that the test itself is functional and that the expected behaviour is not yet satisfied.

Script 5.13: Failing Test for Calculator.add()**CalculatorTest.java**

```
+ 1 import org.junit.jupiter.api.Test;
+ 2 import static org.junit.jupiter.api.Assertions.assertEquals;
+ 3
+ 4 class CalculatorTest {
+ 5
+ 6     @Test
+ 7     void add_givenTwoIntegers_shouldReturnTheirSum() {
+ 8         Calculator calculator = new Calculator();
+ 9         int result = calculator.add(3, 7);
+10         assertEquals(10, result);
+11     }
+12
+13     @Test
+14     void add_givenNegativeOperands_shouldReturnCorrectSum() {
+15         Calculator calculator = new Calculator();
+16         assertEquals(-5, calculator.add(-2, -3));
+17     }
+18
+19     @Test
+20     void add_givenZero_shouldReturnTheOtherOperand() {
+21         Calculator calculator = new Calculator();
+22         assertEquals(4, calculator.add(0, 4));
+23         assertEquals(4, calculator.add(4, 0));
+24     }
+25 }
```

Script 5.13 contains three test methods. The first verifies the straightforward addition of two positive integers. The second covers the boundary condition of negative operands. The third checks the identity property of zero. Together, these tests establish a specification for the add method before any implementation exists. Attempting to compile at this point fails because the Calculator class is undefined.

Step 2. Green: Write the Minimum Code to Pass. The objective of the *green* phase is to make all tests pass with the least amount of production code. Over-engineering is intentionally avoided; the focus is on satisfying the test assertions and nothing more.

Script 5.14: Minimal Calculator Implementation

Calculator.java

```
+ 1 public class Calculator {  
+ 2  
+ 3     public int add(int a, int b) {  
+ 4         return a + b;  
+ 5     }  
+ 6 }
```

Script 5.14 provides the simplest implementation that satisfies all three tests. Running the test suite at this point produces three passing tests, confirming that the specification is met. The project is now in the *green* state.

Step 3. Refactor: Improve the Design Without Changing Behaviour. With a passing test suite acting as a safety net, the code can be restructured to improve readability, reduce duplication, or prepare for future extension. In this example, the test class contains a repeated instantiation of Calculator in every test method. This duplication can be eliminated by extracting the object creation into a @BeforeEach setup method, as demonstrated in Script 5.15.

After refactoring, the test suite is executed once more to verify that all tests still pass. This confirmation is essential: the refactoring must not alter the observable behaviour of the code. The @BeforeEach annotation causes the setUp() method to execute before each test method, guaranteeing that every test operates on a fresh Calculator instance. This pattern becomes increasingly valuable as the number

of test methods grows, as it centralises object construction and reduces the maintenance burden of modifying initialisation logic.

The red-green-refactor cycle then repeats for the next piece of functionality. For instance, a subtract method would begin with a new failing test (red), proceed through the minimal implementation (green), and conclude with any necessary restructuring (refactor). Over successive iterations, the production code and the test suite grow incrementally, each addition validated by the full suite of existing tests.

Script 5.15: Refactored Test with @BeforeEach**CalculatorTest.java**

```
+ 1 import org.junit.jupiter.api.BeforeEach;
+ 2 import org.junit.jupiter.api.Test;
+ 3 import static org.junit.jupiter.api.Assertions.assertEquals;
+ 4
+ 5 class CalculatorTest {
+ 6     private Calculator calculator;
+ 7
+ 8     @BeforeEach
+ 9     void setUp() {
+10         calculator = new Calculator();
+11     }
+12
+13     @Test
+14     void add_givenTwoIntegers_shouldReturnTheirSum() {
+15         assertEquals(10, calculator.add(3, 7));
+16     }
+17
+18     @Test
+19     void add_givenNegativeOperands_shouldReturnCorrectSum() {
+20         assertEquals(-5, calculator.add(-2, -3));
+21     }
+22
+23     @Test
+24     void add_givenZero_shouldReturnTheOtherOperand() {
+25         assertEquals(4, calculator.add(0, 4));
+26         assertEquals(4, calculator.add(4, 0));
+27     }
+28 }
```

5.4 Case Study (Part 1)

AI Advisory

AI tools can generate unit tests rapidly, but tests generated without explicit comprehension tend to verify trivial logic and overlook meaningful edge cases. Generating a passing test suite is fundamentally distinct from designing a suite that guards against regressions, documents contractual behavior, and reveals boundary-condition defects. Test design is a rigorous reasoning task that requires domain-specific knowledge beyond the capability of statistical language models.

Recommended Approach

AI assistance is highly appropriate for boilerplate construction, such as generating a JUnit 5 test class skeleton, producing the `@BeforeEach` setup method, or scaffolding a standard `assertEquals` pattern, provided that the expected behavior is already completely understood. However, deciding *what* to test, identifying which inputs constitute critical boundary cases, and determining which execution paths are most prone to runtime errors must remain an analytical, human-driven design activity. Following the independent design of a test, an AI tool may be consulted to suggest additional edge cases for a pure function, but each suggestion must be critically evaluated against the actual contract of the function.

Crucially, the V-Model presented earlier in this chapter establishes that every unit test must trace its origin directly to a specific requirement on the left-hand side of the V. A test that merely confirms an implementation detail, such as verifying that a method returns the exact value it was previously assigned, provides no assurance that the underlying requirement is satisfied. AI-generated tests frequently exhibit this specific weakness, as they tend to replicate the syntax of the production code rather than the specification the code was authored to fulfill. Effective test design necessitates examining the core requirement, explicitly identifying the acceptance criteria, and subse-

quently constructing test cases designed to *fail* if those criteria are violated. This requirement-driven practice fundamentally distinguishes a robust test suite from an ineffective suite that achieves high coverage metrics while failing to guard against unintended side-effects.

Appropriate Uses

- ✓ Utilize an AI tool to generate a JUnit 5 test class skeleton once the class under test and its expected behavior are fully comprehended.
- ✓ Consult an AI tool for Mockito syntax regarding a specific mocking scenario after the test architecture has been designed independently.
- ✓ Request an AI tool to suggest edge cases for a pure function, subsequently evaluating each suggestion against the documented contract of the function prior to adoption.
- ✓ Employ an AI tool to generate repetitive assertion patterns for data-driven tests where the underlying assertion logic has already been validated manually.

Practices to Avoid

- ✗ Do not request an AI tool to “write tests for this class” without explicitly specifying the behavior to be verified. The resulting tests will frequently pass without validating any meaningful functionality.
- ✗ Do not delegate test-coverage targets or test-design decisions to an AI tool. Coverage is a metric of thoroughness, not correctness; only the developer can accurately determine which execution paths carry architectural risk.
- ✗ Do not accept AI-generated Mockito stubs without verifying that the mock behavior precisely matches the actual interface contract of the dependency being replaced.

- ✗ Do not use an AI tool to interpret a failing test before reading the failure message independently and formulating a concrete hypothesis regarding the root cause.

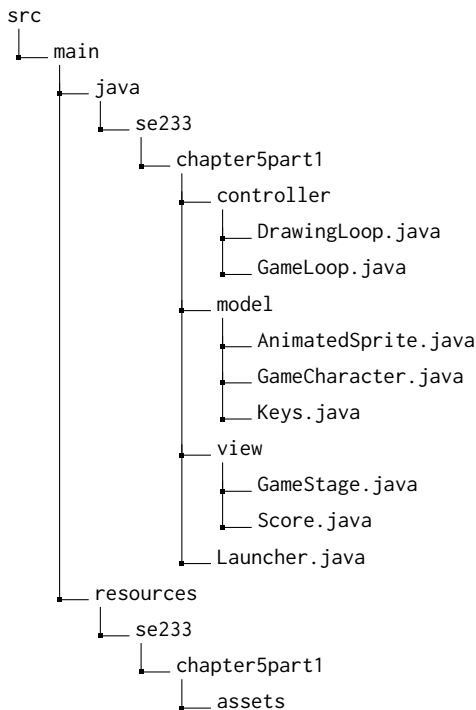
The 2D platformer game developed in Chapter 4 serves as the foundation for this case study. The sprite animation pipeline, collision detection, and logging integration implemented previously are retained, utilizing the `GameCharacter`, `AnimatedSprite`, and `DrawingLoop` classes. This continuity demonstrates how unit testing complements, rather than replaces, the debugging techniques established in the prior chapter by applying tests to a known, stable codebase.

Alongside the introduction of unit testing, this case study expands the platformer game by implementing three additional core functionalities. To ensure a consistent starting environment, a baseline project snapshot is provided at <https://cmu.to/SE233CH5P1>. This archive contains the completed modifications from the case study and exercises in Chapter 4.



Figure 5.3: GUI of the Initialized Project for the Chapter 5 (Part 1) Case Study

To configure the project, replace the existing `src` directory within the workspace with the `src` directory extracted from the downloaded archive. Once the configuration is complete and a new `Score.java` file is added to the view package, the initial project GUI will render as shown in Figure 5.3. The resulting project structure is organized as follows:



In the current implementation, two distinct characters are animated using separate sprite sheets and are controlled by different sets of keys. To develop this into a playable game, several features must be implemented, including collision detection, game objective logic, and a scoring system.

Collisions detected along the x-axis are resolved by reverting both characters to their positions from the preceding frame. This logic prevents the characters from overlapping or passing through each other horizontally. For collisions along the y-axis, the method determines which character is positioned above the other. This differentiation allows for distinct outcomes based on the nature of the vertical collision. For instance, a future implementation might award points to the character on top, while the character below enters a “defeated” state and is subsequently respawned at its initial coordinates.

The `respawn()` method (Lines 134-144) is responsible for resetting a character’s state variables to their initial values. Several new variables are introduced to support this respawn mechanism. Specifically, the `startX` and `startY` variables store the character’s initial coordinates, allowing the character to be returned to its original position by reassigning these values to the current `x` and `y` coordinates.

The updated `DrawingLoop` class, presented in Script 5.17, incorporates the logic for character collision detection. The conditional statement in Line 27 uses the `getBoundsInParent().intersects()` method to determine if the bounding boxes of the two character sprites are overlapping. If an overlap is detected, the corresponding `GameCharacter` objects invoke their respective `collided()` methods to handle the collision, as shown in Lines 28 and 29.

5.4.1 Character collision

Character-to-character collision logic is managed by the `collided()` method within the `GameCharacter` class (Script 5.16). This method incorporates two distinct logical checks to handle collisions along the x-axis and y-axis differently. This logic is detailed in Lines 120-133 of the script.

Script 5.16: model/GameCharacter.java

```

1  //Imports are omitted
  ...
9  public class GameCharacter extends Pane {
    ...
12  private int y;
+ 13  private int startX;
+ 14  private int startY;
    ...
33  public GameCharacter(int id, int x, int y, String imgName, int count, int
      column, int row, int width, int height, KeyCode leftKey, KeyCode rightKey,
      KeyCode upKey) {
+ 34      this.startX = x;
+ 35      this.startY = y;
    ...
+ 120  public boolean collided(GameCharacter c) {
+ 121      if (this.isMoveLeft && this.x > c.getX()) {
+ 122          this.x = Math.max(this.x, c.getX() + c.getCharacterWidth());
+ 123          this.stop();
+ 124      } else if (this.isMoveRight && this.x < c.getX()) {
+ 125          this.x = Math.min(this.x, c.getX() - this.characterWidth);
+ 126          this.stop();
+ 127      }
+ 128      if (this.isFalling && this.y < c.getY()) {
+ 129          c.respawn();
+ 130          return true;
+ 131      }
+ 132      return false;
+ 133  }
+ 134  public void respawn() {
+ 135      this.x = this.startX;
+ 136      this.y = this.startY;
+ 137      this.imageView.setFitWidth(this.characterWidth);
+ 138      this.imageView.setFitHeight(this.characterHeight);
+ 139      this.isMoveLeft = false;
+ 140      this.isMoveRight = false;
+ 141      this.isFalling = true;
+ 142      this.canJump = false;
+ 143      this.isJumping = false;
+ 144  }
145  public KeyCode getLeftKey() {
    ...

```

Script 5.17: controller/DrawingLoop.java

```
1 //Imports are omitted
  ...
8 public class DrawingLoop implements Runnable {
  ...
19 private void checkDrawCollisions(List<GameCharacter> gameCharacterList) {
20     for (GameCharacter gameCharacter : gameCharacterList) {
21         gameCharacter.checkReachGameWall();
22         gameCharacter.checkReachHighest();
23         gameCharacter.checkReachFloor();
24     }
+25     GameCharacter cA = gameCharacterList.get(0);
+26     GameCharacter cB = gameCharacterList.get(1);
+27     if (cA.getBoundsInParent().intersects(cB.getBoundsInParent())) {
+28         if (cA.collided(cB) == false) {
+29             cB.collided(cA);
+30         }
+31     }
32 }
  ...
```

5.4.2 Death scene rendering

With the current implementation, the application provides a more complete game-play experience. However, further refinements can be made to improve the overall presentation. The next update introduces a simple animation for the character’s “defeated” state, as the current instantaneous respawn upon being stomped can be visually jarring and disrupt gameplay.

Script 5.18 introduces modifications to the `collided()` method of the `GameCharacter` class and adds a new method named `collapsed()`. Specifically, when a character is stomped, it will now enter a “collapsed” animation state. After a brief delay of 300 milliseconds, the character will be respawned at its initial coordinates. This enhancement adds a clear visual indicator for a character’s defeat, which makes the gameplay more intuitive and visually polished.

Script 5.18: model/GameCharacter.java

```

1  //Imports are omitted
  ...
9  public class GameCharacter extends Pane {
    ...
120 public void collided(GameCharacter c) {
    ...
128     if (this.isFalling && this.y < c.getY()) {
+129         this.y = Math.min(GameStage.GROUND - this.characterHeight, c.getY());
+130         this.repaint();
+131         c.collapsed();
132         c.respawn();
133         return true;
134     }
135 }
+136 public void collapsed() {
+137     this.imageView.setFitHeight(5);
+138     this.y = this.y + this.characterHeight - 5;
+139     this.repaint();
+140     try {
+141         TimeUnit.MILLISECONDS.sleep(300);
+142     } catch (InterruptedException e) {
+143         e.printStackTrace();
+144     }
145 }
    ...

```

5.4.3 Scoreboard

The final feature to be implemented for this platformer game is the scoreboard. The scoring logic requires that each character begins with a score of zero, and their score is incremented by one each time they successfully stomp on the opposing character. Unlike the previous enhancement, implementing the scoreboard requires the creation of a new view class to display the players' scores. This new view component is then integrated into the main GameStage. Script 5.19 presents the implementation of this scoreboard component.

The required modifications to the application logic are detailed in Scripts 5.20, 5.21, and 5.22. The `GameStage` class (Script 5.20) is updated to include instances of the new `Score` view component. The `GameCharacter` class is updated with a new instance variable, `score`, which is initialized to zero and incremented each time the character successfully stomps on an opponent.

The `GameLoop` acts as the controller for this scoreboard functionality. On each tick of the game loop, it synchronizes the score displayed in the view with the current score value stored in each `GameCharacter` instance. With the implementation of these final updates, the case study application is complete.

Figure 5.4 illustrates the completed version of the game from this case study. Common implementation errors that may be encountered include the absence of required accessor methods (Getters), missing exception handling logic, or incomplete dependency configurations.

Script 5.19: view/Score.java

```
+ 1 //Imports are omitted
+ 2 public class Score extends Pane {
+ 3     Label point;
+ 4     public Score(int x, int y) {
+ 5         point = new Label("0");
+ 6         setTranslateX(x);
+ 7         setTranslateY(y);
+ 8         point.setFont(Font.font("Verdana", FontWeight.BOLD, 30));
+ 9         point.setTextFill(Color.web("#FFF"));
+10         getChildren().addAll(point);
+11     }
+12     public void setPoint(int score) {
+13         this.point.setText(Integer.toString(score));
+14     }
+15 }
```

Script 5.20: view/GameStage.java

```

1  //Imports are omitted
  ...
14 public class GameStage extends Pane {
    ...
18     private Image gameStageImg;
- 19     private List<GameCharacter> gameCharacterList = new ArrayList();
+ 20     private List<GameCharacter> gameCharacterList;
+ 21     private List<Score> scoreList;
22     private Keys keys;
23     public GameStage() {
+ 24         gameCharacterList = new ArrayList<>();
+ 25         scoreList = new ArrayList();
26         keys = new Keys();
    ...
32         gameCharacterList.add(new GameCharacter(1, GameStage.WIDTH-60, 30, "assets/
            Character2.png", 4, 4, 1, 129, 66, KeyCode.LEFT, KeyCode.RIGHT, KeyCode.
                UP));
+ 33         scoreList.add(new Score(30, GROUND + 30));
+ 34         scoreList.add(new Score(GameStage.WIDTH-60, GROUND + 30));
35         getChildren().add(backgroundImg);
36         getChildren().addAll(gameCharacterList);
+ 37         getChildren().addAll(scoreList);
    ...

```

Script 5.21: model/GameCharacter.java

```

1  //Imports are omitted
  ...
11 public class GameCharacter extends Pane {
    ...
+ 20     private int score=0;
21     private KeyCode leftKey;
    ...
123     public void collided(GameCharacter c) {
    ...
131         if (this.isFalling && this.y < c.getY()) {
+ 132             score++;
133             this.y = Math.min(GameStage.GROUND - this.characterHeight, c.getY());
134             this.repaint();
    ...

```

Script 5.22: controller/GameLoop.java

```

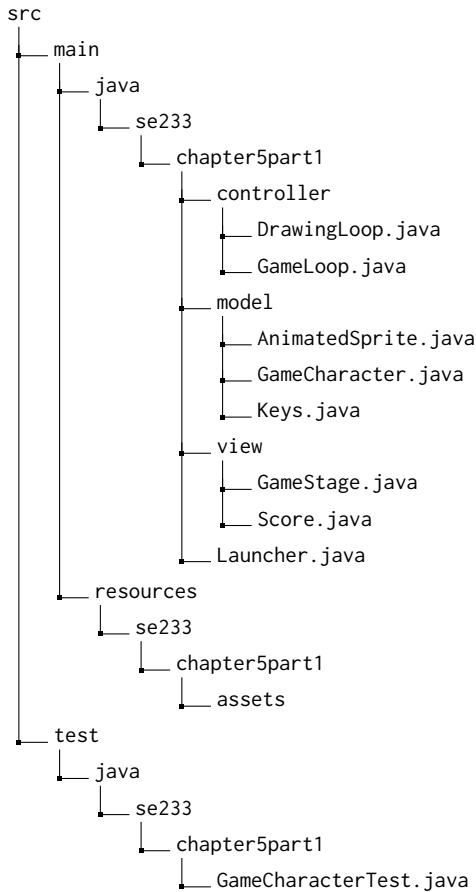
1  //Imports are omitted
  ...
8  public class GameLoop implements Runnable {
    ...
+ 43  private void updateScore(List<Score> scoreList, List<GameCharacter>
      gameCharacterList) {
+ 44      javafx.application.Platform.runLater(() -> {
+ 45          for (int i=0 ; i<scoreList.size() ; i++) {
+ 46              scoreList.get(i).setPoint(gameCharacterList.get(i).getScore());
+ 47          }
+ 48      });
+ 49  }
50  @Override
51  public void run() {
52      while (running) {
53          float time = System.currentTimeMillis();
54          update(gameStage.getGameCharacterList());
+ 55          updateScore(gameStage.getScoreList(), gameStage.getGameCharacterList());
      ...

```

**Figure 5.4: The Completed 2D Platformer Game Case Study**

5.4.4 Unit testing

A demonstration of unit testing for the platformer game is now presented. The initial step is to establish the following file structure for the test sources:



Script 5.23 presents an example unit test for the `GameCharacter` class using JUnit 5. Following standard convention, the `GameCharacterTest` class is created to serve as the test class.

Script 5.23: test/java/se233/chapter5part1/GameCharacterTest.java

```
+ 1 package se233.chapter5part1;
+ 2
+ 3 import javafx.scene.input.KeyCode;
+ 4 import org.junit.jupiter.api.BeforeAll;
+ 5 import org.junit.jupiter.api.BeforeEach;
+ 6 import org.junit.jupiter.api.Test;
+ 7 import se233.chapter5part1.model.GameCharacter;
+ 8
+ 9 import static org.junit.jupiter.api.Assertions.*;
+10
+11 public class GameCharacterTest {
+12     private GameCharacter gameCharacter;
+13
+14     @BeforeAll
+15     public static void initJfxRuntime() {
+16         javafx.application.Platform.startup(() -> {});
+17     }
+18     @BeforeEach
+19     public void setUp() throws NoSuchFieldException {
+20         gameCharacter = new GameCharacter(0,30, 30,"assets/Character1.png", 4, 3
+21             ,2, 111,97, KeyCode.A, KeyCode.D, KeyCode.W);
+22     }
+23     @Test
+24     public void respawn_givenNewGameCharacter_thenCoordinatesAre30_30() {
+25         gameCharacter.respawn();
+26         assertEquals(30, gameCharacter.getX(), "Initial x");
+27         assertEquals(30, gameCharacter.getY(), "Initial y");
+28     }
+29     @Test
+30     public void respawn_givenNewGameCharacter_thenScoreIs0() {
+31         gameCharacter.respawn();
+32         assertEquals(0, gameCharacter.getScore(), "Initial score");
+33     }
+34 }
```


The `@BeforeAll` annotation on Line 14 designates a method that is executed once before any tests in this class are run. Similarly, the `@BeforeEach` annotation on Line 18 designates a method that is executed before each individual test method. These annotations are useful for preparing shared resources, such as initializing an object instance, prior to the execution of tests. The lambda expression on Line 16 initializes the JavaFX toolkit, which is a prerequisite for testing JavaFX components.

In this example, the `setUp()` method creates a `GameCharacter` instance that is used by the test method. The single test case in this script (Lines 22-32) verifies that the values passed to the `GameCharacter` constructor are correctly assigned to their corresponding instance variables. Upon execution, this test should pass.

5.4.5 Unit testing with reflection

The following section demonstrates the testing of the character movement logic. The primary challenge in testing this functionality is the need to verify the state of private fields, which are not directly accessible from an object instance. As previously discussed, the reflection API provides a mechanism to address this situation.

The updated `GameCharacterTest` class, shown in Script 5.24, utilizes reflection to inspect the internal state of the `GameCharacter` object. By declaring `Field` objects for `xVelocity`, `yVelocity`, and `yAcceleration`, the test class prepares to access these private fields. The `setUp()` method initializes the `GameCharacter` instance and then uses reflection to make these private fields accessible for the duration of the test. The method is declared to throw `NoSuchFieldException` to handle potential discrepancies between the test code and the implementation, ensuring that the test setup fails gracefully if a specified field does not exist.

The test methods, located at Lines 43, 50, and 59, each verify a specific behavior of the `GameCharacter` class. They are declared to throw `IllegalAccessExcept` to handle potential issues when accessing the private fields. For example, the `moveX_givenMoveRightOnce_thenXCoordinateIncreasedByXVelocity()` test (Line 43) verifies that invoking the character's `move` method correctly updates its x-coordinate based on its x-velocity. Similarly, the other tests verify that vertical velocity increases as expected (Line 50) and that vertical acceleration remains constant (Line 59). Collectively, these tests verify that the internal motion logic of the `GameCharacter` class functions as expected.

Script 5.24: test/java/se233/chapter5part1/GameCharacterTest.java

```

1  //Imports are omitted
  ...
11 public class GameCharacterTest {
+ 12     Field xVelocityField, yVelocityField, yAccelerationField;
      ...
18     @BeforeEach
- 19     public void setup() {
+ 20     public void setup() throws NoSuchFieldException {
21         gameCharacter = new GameCharacter(0,30, 30,"assets/Character1.png", 4, 3 ,2,
            111,97, KeyCode.A, KeyCode.D, KeyCode.W);
+ 22         xVelocityField = gameCharacter.getClass().getDeclaredField("xVelocity");
+ 23         yVelocityField = gameCharacter.getClass().getDeclaredField("yVelocity");
+ 24         yAccelerationField = gameCharacter.getClass().getDeclaredField("yAcceleration");
+ 25         xVelocityField.setAccessible(true);
+ 26         yVelocityField.setAccessible(true);
+ 27         yAccelerationField.setAccessible(true);
      ...
+ 42     @Test
+ 43     public void moveX_givenMoveRightOnce_thenXCoordinateIncreasedByXVelocity() throws
        IllegalAccessException {
+ 44         gameCharacter.respawn();
+ 45         gameCharacter.moveRight();
+ 46         gameCharacter.moveX();
+ 47         assertEquals(30 + xVelocityField.getInt(gameCharacter), gameCharacter.getX(), "
            Move right x");
+ 48     }
+ 49     @Test
+ 50     public void moveY_givenTwoConsecutiveCalls_thenYVelocityIncreases() throws
        IllegalAccessException {
+ 51         gameCharacter.respawn();
+ 52         gameCharacter.moveY();
+ 53         int yVelocity1 = yVelocityField.getInt(gameCharacter);
+ 54         gameCharacter.moveY();
+ 55         int yVelocity2 = yVelocityField.getInt(gameCharacter);
+ 56         assertTrue(yVelocity2 > yVelocity1, "Velocity is increasing");
+ 57     }
+ 58     @Test
+ 59     public void moveY_givenTwoConsecutiveCalls_thenYAccelerationUnchanged() throws
        IllegalAccessException {
+ 60         gameCharacter.respawn();
+ 61         gameCharacter.moveY();
+ 62         int yAcceleration1 = yAccelerationField.getInt(gameCharacter);
+ 63         gameCharacter.moveY();
+ 64         int yAcceleration2 = yAccelerationField.getInt(gameCharacter);
+ 65         assertTrue(yAcceleration1 == yAcceleration2, "Acceleration is not change");
+ 66     }
+ 67 }

```

Figure 5.5 presents an example of the test execution results for the `GameCharacterTest`.

✓ GameCharacterTest (se233.chapter5part1)	82 ms
✓ respawn_givenNewGameCharacter_thenCoordinatesAre30_30()	66 ms
✓ moveY_givenTwoConsecutiveCalls_thenYAccelerationUnchanged()	7 ms
✓ respawn_givenNewGameCharacter_thenScoreIs0()	3 ms
✓ moveY_givenTwoConsecutiveCalls_thenYVelocityIncreases()	3 ms
✓ moveX_givenMoveRightOnce_thenXCoordinateIncreasedByXVelocity()	3 ms

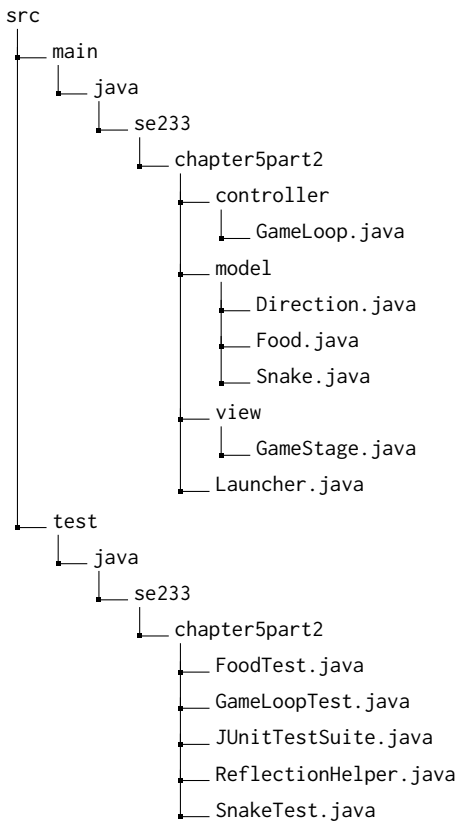
Figure 5.5: Test Results for Chapter5part1 Project

5.5 Case Study (Part 2)

The second case study of this chapter demonstrates the TDD principle through the implementation of a simplified variation of the classic snake game. Following the TDD process, development begins with the decomposition of the project into its essential features. The primary functional requirements for the snake game are as follows:

- The snake must move one unit in its current direction per game tick.
- The snake’s direction of movement must be controllable across four axes (up, down, left, right).
- The snake must be able to collide with food items.
- Upon colliding with a food item, the snake’s length must increase.
- After a food item is consumed via collision, a new food item must be respawned at a different location.
- A collision between the snake’s head and any part of its own body must result in a “game over” state.
- A collision between the snake’s head and the game boundaries must also result in a “game over” state.

The initial step in implementing these requirements is to establish the following project story structure:



5.5.1 The snake's head, body, and tail

The `SnakeTest` class (Script 5.25) is created first to serve as the test class for the `Snake` class. It is important to note that this test code is not expected to compile or execute successfully at this stage; rather, it serves as a specification to guide the implementation of the production code. The corresponding production code will be implemented subsequently to satisfy these tests. The initial tests target the fol-

lowing functionalities:

- The correct instantiation of a Snake object at a specific coordinate (e.g., 0,0).
- The ability to set the snake's direction of movement.
- The representation of the four cardinal directions using an enumeration (Direction).
- The correct update of the snake's position after a single game tick, where it is expected to move one unit in its current direction.

Script 5.25: test/java/se233/chapter5part2/SnakeTest.java

```
+ 1 //Imports are omitted
+ 2 public class SnakeTest {
+ 3     private Snake snake;
+ 4
+ 5     @BeforeAll
+ 6     public static void initJfxRuntime() {
+ 7         javafx.application.Platform.startup(() -> {});
+ 8     }
+ 9     @BeforeEach
+10     public void setup() {
+11         snake = new Snake(new Point2D(0, 0));
+12     }
+13     @Test
+14     public void initialPosition_shouldBe_atOrigin() {
+15         assertEquals(snake.getHead(), new Point2D(0, 0));
+16     }
+17     @Test
+18     public void move_afterInitialized_headShouldBeInDownwardDirection() {
+19         snake.setDirection(Direction.DOWN);
+20         snake.move();
+21         assertEquals(snake.getHead(), new Point2D(0, 1));
+22     }
+23 }
```

Script 5.26 presents an enum that represents the four cardinal directions of the snake's movement. This `Direction` enumeration is utilized by the `Snake` class. The initial version of the `Snake` class, presented in Script 5.27, is implemented with the minimal code required to pass the existing test cases. A snake is represented by a `List` of 2D coordinates, where each coordinate in the `List` corresponds to a segment of the snake's body.

Movement is calculated based on the snake's head. Therefore, a dedicated variable is required to track the head's position in addition to the `List` representing the body. Furthermore, the coordinates of the snake's tail are also tracked in a separate variable. This variable is used to determine which coordinate to remove from the body `List` each time the snake moves. The `update()` method implements the movement logic according to the following steps:

- Determine the new coordinate to which the snake's head will move.
- Remove the coordinate that currently represents the snake's tail from the body `List`.
- After the move, add the new head coordinate to the body `List`.

The other methods included in the initial implementation are accessor methods (Getters) for the variables required by the `SnakeTest` class.

Script 5.26: model/Direction.java

```
+ 1 //Imports are omitted
+ 2 public enum Direction {
+ 3     RIGHT(new Point2D(1, 0)),
+ 4     LEFT(new Point2D(-1, 0)),
+ 5     UP(new Point2D(0, -1)),
+ 6     DOWN(new Point2D(0, 1));
+ 7     public final Point2D current;
+ 8     Direction(Point2D current) { this.current = current; }
+ 9 }
```

Script 5.27: model/Snake.java

```
+ 1 //Imports are omitted
+ 2 public class Snake {
+ 3     private Direction direction;
+ 4     private Point2D head;
+ 5     private Point2D prev_tail;
+ 6     private List<Point2D> body;
+ 7     public Snake(Point2D position) {
+ 8         direction = Direction.DOWN;
+ 9         body = new ArrayList<>();
+10         this.head = position;
+11         this.body.add(this.head);
+12     }
+13     public void move() {
+14         head = head.add(direction.current);
+15         prev_tail = body.remove(body.size() - 1);
+16         body.add(0, head);
+17     }
+18     public void setDirection(Direction direction) { this.direction = direction; }
+19     public Direction getDirection() { return this.direction; }
+20     public Point2D getHead() { return head; }
+21 }
```

5.5.2 Food collision and snake growth

Script 5.28 augments the SnakeTest class with three additional test cases. These test cases verify three aspects of the snake's behavior: its ability to detect collisions with food items, the subsequent increase in its length by one unit, and the correct addition of its previous head coordinate to its body. The three newly added methods are `grow_shouldIncreaseLengthByOne`, `grow_shouldAddPreviousHeadToBody`, and `collided_withSnake_shouldBeDetected`, respectively.

The FoodTest class is introduced to verify that a food item respawns at a new location after it has been consumed. This test class (Script 5.29) contains a single test method, `respawn_shouldBeOnNewPosition()`. This method invokes the food item's `respawn()` method and then asserts that its new coordinates are different from its original coordinates.

Script 5.28: test/java/se233/chapter5part2/SnakeTest.java

```
1  //Imports are omitted
   ...
12 public class SnakeTest {
   ...
+ 33 @Test
+ 34 public void grow_shouldIncreaseLengthByOne() {
+ 35     snake.grow();
+ 36     assertEquals(snake.getLength(), 2);
+ 37 }
+ 38 @Test
+ 39 public void grow_shouldAddPreviousHeadToBody() {
+ 40     Point2D cur_head = snake.getHead();
+ 41     snake.move();
+ 42     snake.grow();
+ 43     assertTrue(snake.getBody().contains(cur_head));
+ 44 }
+ 45 @Test
+ 46 public void collided_withSnake_shouldBeDetected() {
+ 47     Food food = new Food(new Point2D(0,0));
+ 48     assertTrue(snake.collided(food));
+ 49 }
50 }
```

Script 5.29: test/java/se233/chapter5part2/FoodTest.java

```
+ 1 //Imports are omitted
+ 2 public class FoodTest {
+ 3     private Food food;
+ 4
+ 5     @BeforeAll
+ 6     public static void initJfxRuntime() {
+ 7         javafx.application.Platform.startup(() -> {});
+ 8     }
+ 9     @BeforeEach
+10     public void setup() {
+11         food = new Food(new Point2D(0, 0));
+12     }
+13     @Test
+14     public void respawn_shouldBeOnNewPosition() {
+15         food.respawn();
+16         assertNotSame(food.getPosition(), new Point2D(0, 0));
+17     }
+18 }
```


To create the minimal production code necessary to pass these new tests, two additional classes are required: `Food` and `GameStage`. The `Food` class (Script 5.30) encapsulates the coordinates of a food item as a `Point2D` object. It uses a `Random` object to determine a new, random position for the food item after it is consumed by the snake. The `GameStage` class (Script 5.31) defines the game's playable area with fixed dimensions of 30x30 tiles, where each tile is 10x10 pixels.

Script 5.30: model/Food.java

```
+ 1 //Imports are omitted
+ 2 public class Food {
+ 3     private Point2D position;
+ 4     private Random rn;
+ 5
+ 6     public Food(Point2D position) {
+ 7         this.rn = new Random();
+ 8         this.position = position;
+ 9     }
+10     public Food() {
+11         this.rn = new Random();
+12         this.position = new Point2D(rn.nextInt(GameStage.WIDTH), rn.nextInt(
+13             GameStage.HEIGHT));
+14     }
+15     public void respawn() {
+16         Point2D prev_position = this.position;
+17         do {
+18             this.position = new Point2D(rn.nextInt(GameStage.WIDTH), rn.nextInt(
+19                 GameStage.HEIGHT));
+20         } while (prev_position == this.position);
+21     }
+22     public Point2D getPosition() {
+23         return position;
+24     }
+25 }
```

Script 5.31: view/GameStage.java

```
+ 1 //Imports are omitted
+ 2 public class GameStage extends Pane {
+ 3     public static final int WIDTH = 30;
+ 4     public static final int HEIGHT = 30;
+ 5     public static final int TILE_SIZE = 10;
+ 6     private Canvas canvas;
+ 7     private KeyCode key;
+ 8
+ 9     public GameStage() {
+10         this.setHeight(TILE_SIZE * HEIGHT);
+11         this.setWidth(TILE_SIZE * WIDTH);
+12         canvas = new Canvas(TILE_SIZE * WIDTH, TILE_SIZE * HEIGHT);
+13         this.getChildren().add(canvas);
+14     }
+15     public void render(Snake snake, Food food) {
+16         GraphicsContext gc = canvas.getGraphicsContext2D();
+17         gc.clearRect(0,0,WIDTH*TILE_SIZE,HEIGHT*TILE_SIZE);
+18         gc.setFill(Color.BLUE);
+19         snake.getBody().forEach(p -> {
+20             gc.fillRect(p.getX() * TILE_SIZE, p.getY() * TILE_SIZE, TILE_SIZE,
+21                         TILE_SIZE);
+22         });
+23         gc.setFill(Color.RED);
+24         gc.fillRect(food.getPosition().getX() * TILE_SIZE, food.getPosition().getY()
+25                     * TILE_SIZE, TILE_SIZE, TILE_SIZE);
+26     }
+27     public KeyCode getKey() { return key; }
+28     public void setKey(KeyCode key) { this.key = key; }
+29 }
```

Furthermore, the Snake class is updated with four new methods, as presented in Script 5.32. The `collided()` method determines if the snake's head intersects with the coordinates of another game entity. The `grow()` method increases the snake's length by one segment at its tail. The `getLength()` accessor method returns the current size of the body list, which represents the snake's length. Lastly, the `getBody()` method returns the list of coordinates that constitute the snake's body.

Script 5.32: model/Snake.java

```
1 //Imports are omitted
  ...
8 public class Snake {
    ...
24 public void setDirection(Direction direction) { this.direction = direction; }
25 public Direction getDirection() { return this.direction; }
26 public Point2D getHead() { return head; }
+ 27 public boolean collided(Food food) { return head.equals(food.getPosition()); }
+ 28 public void grow() { body.add(prev_tail); }
+ 29 public int getLength() { return body.size(); }
+ 30 public List<Point2D> getBody() { return body; }
31 }
```

5.5.3 Implementing snake death condition

The initial requirements specify two terminal conditions for the snake: collision with the game boundaries and collision with its own body. Script 5.33 addresses these scenarios through the implementation of two test methods within the SnakeTest class: `checkDead_ifHitGameBorder_snakeWillDie` and `checkDead_ifHitItself_snakeWillDie`.

To satisfy these test cases, a new method named `checkDead()` is introduced to the Snake class, as shown in Script 5.34. The `checkDead_ifHitGameBorder_snakeWillDie()` test case positions the snake near a boundary and sets its direction towards that boundary. After a single update, a collision is guaranteed, and the test then asserts that the snake's state is correctly set to dead.

For the `checkDead_ifHitItself_snakeWillDie()` test case, the snake's length is first increased to a point where a self-collision is possible. The test logic then directs the snake to turn back on itself, ensuring a collision with its own body. Subsequently, the test asserts that the snake's state is updated to dead as expected.

With these modifications implemented, the entire test suite can be executed to verify that all eight test cases now pass.

Script 5.33: test/java/se233/chapter5part2/SnakeTest.java

```

1  //Imports are omitted
   ...
14 public class SnakeTest {
   ...
+ 52  @Test
+ 53  public void checkDead_ifHitGameBorder_snakeWillDie() {
+ 54      snake = new Snake(new Point2D(30,30));
+ 55      snake.setDirection(Direction.RIGHT);
+ 56      snake.move();
+ 57      assertTrue(snake.checkDead());
+ 58  }
+ 59  @Test
+ 60  public void checkDead_ifHitItself_snakeWillDie() {
+ 61      snake = new Snake(new Point2D(0,0));
+ 62      snake.setDirection(Direction.DOWN);
+ 63      snake.move();
+ 64      snake.grow();
+ 65      snake.setDirection(Direction.LEFT);
+ 66      snake.move();
+ 67      snake.grow();
+ 68      snake.setDirection(Direction.UP);
+ 69      snake.move();
+ 70      snake.grow();
+ 71      snake.setDirection(Direction.RIGHT);
+ 72      snake.move();
+ 73      snake.grow();
+ 74      assertTrue(snake.checkDead());
+ 75  }
76 }

```

Script 5.34: model/Snake.java

```

1  //Imports are omitted
   ...
8  public class Snake {
   ...
+ 24  public boolean checkDead() {
+ 25      boolean isOutOfBounds = head.getX() < 0 || head.getY() < 0 || head.getX() >
          GameState.WIDTH || head.getY() > GameState.HEIGHT;
+ 26      boolean isHitBody = body.lastIndexOf(head) > 0;
+ 27      return isOutOfBounds || isHitBody;
+ 28  }
   ...

```

5.5.4 The game loop

Testing the game loop presents a significant challenge due to its tight coupling with the `GameStage`, `Snake`, and `Food` classes, which requires the interaction of multiple components. This interdependence violates the principle of isolation required for a true unit test and is more characteristic of an integration test. However, with careful design, specific elements within the game loop can still be subjected to unit testing.

Script 5.35: `test/java/se233/chapter5part2/ReflectionHelper.java`

```
+ 1 package se233.chapter5part2;
+ 2
+ 3 import java.lang.reflect.Field;
+ 4 import java.lang.reflect.Method;
+ 5
+ 6 public class ReflectionHelper {
+ 7     public static void setField(Object object, String fieldName, Object value)
+ 8         throws Exception {
+ 9         Field field = object.getClass().getDeclaredField(fieldName);
+10         field.setAccessible(true);
+11         field.set(object, value);
+12     }
+13     public static Object getField(Object object, String fieldName) throws
+14         Exception {
+15         Field field = object.getClass().getDeclaredField(fieldName);
+16         field.setAccessible(true);
+17         return field.get(object);
+18     }
+19     public static Object invokeMethod(Object object, String methodName, Class<?>[]
+20         parameterTypes, Object... args) throws Exception {
+21         Method method = object.getClass().getDeclaredMethod(methodName,
+22             parameterTypes);
+23         method.setAccessible(true);
+24         return method.invoke(object, args);
+25     }
+26 }
```

Suitable candidates for unit testing include key processing logic, collision detection algorithms, and rendering logic. These can be tested by focusing on the `GameLoop` class while using mock objects to simulate the behavior of its collaborators. For instance, key press events can be mocked to verify that the snake's direction is updated correctly. Similarly, collision logic can be tested by using mock `Snake` and `Food` objects to verify outcomes such as snake growth, food respawning, and the triggering of a “game over” state. The rendering logic can be verified by inspecting the state of internal variables to confirm that the snake and food components have been correctly updated for the next frame.

To facilitate the assignment and retrieval of private variable values, as well as the invocation of private methods during testing, a helper class named `ReflectionHelper` is introduced (Script 5.35).

Script 5.36 illustrates the `GameLoopTest` class, designed to test the `GameLoop` class. The `GameLoop` implementation follows the structure from the previous case study and consists of three primary private methods: `keyProcess()`, `checkCollision()`, and `redraw()`. Due to their private visibility, the reflection technique is employed to test them. Within `GameLoopTest`, a helper method named `clockTickHelper()` is implemented. This method simulates a single game tick by sequentially invoking the three main private methods of the `GameLoop` class, which improves the organization and maintainability of the test code.

Script 5.36: test/java/se233/chapter5part2/GameLoopTest.java

```

+ 1 //Imports are omitted
+ 2 public class GameLoopTest {
+ 3     private GameState gameStage;
+ 4     private Snake snake;
+ 5     private Food food;
+ 6     private GameLoop gameLoop;
+ 7     @BeforeAll
+ 8     public static void initJfxRuntime() {
+ 9         javafx.application.Platform.startup(() -> {
+10             });
+11     }
+12     @BeforeEach
+13     public void setUp() {
+14         gameStage = new GameState();
+15         snake = new Snake(new Point2D(0, 0));
+16         food = new Food(new Point2D(0, 1));
+17         gameLoop = new GameLoop(gameStage, snake, food);
+18     }
+19     private void clockTickHelper() throws Exception {
+20         ReflectionHelper.invokeMethod(gameLoop, "keyProcess", new Class<?>[0]);
+21         ReflectionHelper.invokeMethod(gameLoop, "checkCollision", new Class<?>[0]);
+22         ReflectionHelper.invokeMethod(gameLoop, "redraw", new Class<?>[0]);
+23     }
+24     @Test
+25     public void keyProcess_pressRight_snakeTurnRight() throws Exception {
+26         ReflectionHelper.setField(gameStage, "key", KeyCode.RIGHT);
+27         ReflectionHelper.setField(snake, "direction", Direction.DOWN);
+28         clockTickHelper();
+29         Direction currentDirection = (Direction) ReflectionHelper.getField(snake, "
+30             direction");
+31         assertEquals(Direction.RIGHT, currentDirection);
+32     }
+33     @Test
+34     public void collided_snakeEatFood_shouldGrow() throws Exception {
+35         clockTickHelper();
+36         assertTrue(snake.getLength() > 1);
+37         clockTickHelper();
+38         assertNotSame(food.getPosition(), new Point2D(0, 1));
+39     }
+40     @Test
+41     public void collided_snakeHitBorder_shouldDie() throws Exception {
+42         ReflectionHelper.setField(gameStage, "key", KeyCode.LEFT);
+43         clockTickHelper();
+44         Boolean running = (Boolean) ReflectionHelper.getField(gameLoop, "running");
+45         assertFalse(running);
+46     }
+47 }

```

The test class in Script 5.36 contains three test methods that verify the key processing and collision logic. However, directly testing the `redraw()` method with JUnit alone is challenging, as it requires verifying that a method was invoked on a collaborator object rather than checking a return value. For this type of interaction testing, a mocking framework such as Mockito is the recommended tool. Script 5.37 provides an example of how mocking can be utilized for this purpose.

Script 5.37: test/java/se233/chapter5part2/GameLoopTest.java

```
1 //Imports are omitted
  ...
16 public class GameLoopTest {
    ...
+ 60     @Test
+ 61     public void redraw_calledThreeTimes_snakeAndFoodShouldRenderThreeTimes() throws
        Exception {
+ 62         GameState mockGameState = Mockito.mock(GameState.class);
+ 63         Snake mockSnake = Mockito.mock(Snake.class);
+ 64         Food mockFood = Mockito.mock(Food.class);
+ 65         GameLoop gameLoop = new GameLoop(mockGameState, mockSnake, mockFood);
+ 66         ReflectionHelper.invokeMethod(gameLoop, "redraw", new Class<?>[0]);
+ 67         ReflectionHelper.invokeMethod(gameLoop, "redraw", new Class<?>[0]);
+ 68         ReflectionHelper.invokeMethod(gameLoop, "redraw", new Class<?>[0]);
+ 69         verify(mockGameState, times(3)).render(mockSnake, mockFood);
+ 70     }
71 }
```

The corresponding production code to be implemented is the `GameLoop` class. An examination of the test classes (Scripts 5.36 and 5.37) indicates that the `GameLoop` class must contain instances of the `GameState`, `Snake`, and `Food` classes. Script 5.38 presents the full implementation of the `GameLoop` class, including the `update()`, `checkCollision()`, and `redraw()` methods, which is designed to pass all test cases defined in the `GameLoopTest` class.

To use Mockito, the `mockito-core` library must be added as dependencies; the required `pom.xml` configuration is shown in Script 5.39.

Script 5.38: controller/GameLoop.java

```

+ 1  //Imports are omitted
+ 2  public class GameLoop implements Runnable {
+ 3      private GameState gameStage;
+ 4      private Snake snake;
+ 5      private Food food;
+ 6      private float interval = 1000.0f / 10;
+ 7      private boolean running;
+ 8      public GameLoop(GameState gameStage, Snake snake, Food food) {
+ 9          this.snake = snake;
+10          this.gameStage = gameStage;
+11          this.food = food;
+12          running = true;
+13      }
+14      private void keyProcess() {
+15          KeyCode curKey = gameStage.getKey();
+16          Direction curDirection = snake.getDirection();
+17          if (curKey == KeyCode.UP && curDirection != Direction.DOWN)
+18              snake.setDirection(Direction.UP);
+19          else if (curKey == KeyCode.DOWN && curDirection != Direction.UP)
+20              snake.setDirection(Direction.DOWN);
+21          else if (curKey == KeyCode.LEFT && curDirection != Direction.RIGHT)
+22              snake.setDirection(Direction.LEFT);
+23          else if (curKey == KeyCode.RIGHT && curDirection != Direction.LEFT)
+24              snake.setDirection(Direction.RIGHT);
+25          snake.move();
+26      }
+27      private void checkCollision() {
+28          if (snake.collided(food)) {
+29              snake.grow();
+30              food.respawn();
+31          }
+32          if (snake.checkDead()) { running = false; }
+33      }
+34      private void redraw() { gameStage.render(snake, food); }
+35      @Override
+36      public void run() {
+37          while (running) {
+38              keyProcess();
+39              checkCollision();
+40              redraw();
+41              try {
+42                  Thread.sleep((long)interval);
+43              } catch (InterruptedException e) {
+44                  e.printStackTrace();
+45              }
+46          }
+47      }
+48  }

```

Script 5.39: pom.xml

```

1  <?xml version="1.0" encoding="UTF-8"?>
    ...
38      <version>${junit.version}</version>
39      <scope>test</scope>
40  </dependency>
+ 41  <dependency>
+ 42      <groupId>org.mockito</groupId>
+ 43      <artifactId>mockito-core</artifactId>
+ 44      <version>5.23.0</version>
+ 45      <scope>test</scope>
+ 46  </dependency>
47  </dependencies>
    ...

```

Script 5.40: Launcher.java

```

+ 1  //Imports are omitted
+ 2  public class Launcher extends Application {
+ 3      public static void main(String[] args) { launch(args); }
+ 4      @Override
+ 5      public void start(Stage stage) {
+ 6          GameState gameState = new GameState();
+ 7          Snake snake = new Snake(new Point2D(gameState.WIDTH/2,gameState.HEIGHT/2));
+ 8          Food food = new Food();
+ 9          GameLoop gameLoop = new GameLoop(gameStage, snake, food);
+ 10         Scene scene = new Scene(gameStage,gameStage.WIDTH*gameStage.TILE_SIZE,
            gameState.HEIGHT * gameState.TILE_SIZE);
+ 11         scene.setOnKeyPressed(event-> gameState.setKey(event.getCode()));
+ 12         scene.setOnKeyReleased(event -> gameState.setKey(null));
+ 13         stage.setTitle("Snake Game");
+ 14         stage.setScene(scene);
+ 15         stage.setResizable(false);
+ 16         stage.show();
+ 17         (new Thread(gameLoop)).start();
+ 18     }
+ 19 }

```

The successful execution of the expanding suite of test cases indicates incremental progress in the implementation. The final step is to integrate all components by implementing the main Launcher class. An example of this Launcher class is provided in Script 5.40. Upon execution, the game window appears as depicted in Figure 5.6 after several seconds of gameplay.

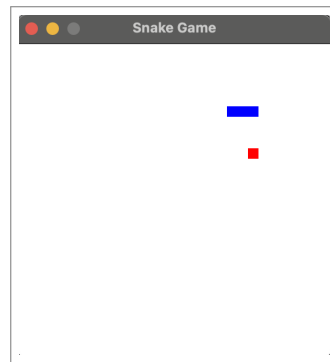


Figure 5.6: Rendered Snake Game

5.5.5 The JUnit test suite

Script 5.41 demonstrates the integration of the `org.junit.platform:junit-platform-suite-engine` dependency into the `pom.xml` file. Subsequently, a class named `JUnitTestSuite` is implemented, as shown in Script 5.42. Executing this suite class will invoke all test cases from the test classes it aggregates. Figure 5.7 depicts the example test results after executing this `TestSuite` class within the IntelliJ IDEA environment.

Furthermore, since the JavaFX toolkit only needs to be initialized once for the entire test run, the `initJfxRuntime()` method is removed from the individual `FoodTest`, `SnakeTest`, and `GameLoopTest` classes. This refactoring is demonstrated in Scripts 5.43, 5.44, and 5.45, respectively.

Script 5.41: pom.xml

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.
         apache.org/xsd/maven-4.0.0.xsd">
5      <modelVersion>4.0.0</modelVersion>
6      ...
35     <dependency>
36         <groupId>org.mockito</groupId>
37         <artifactId>mockito-core</artifactId>
38         <version>5.23.0</version>
39         <scope>test</scope>
40     </dependency>
+ 41     <dependency>
+ 42         <groupId>org.junit.platform</groupId>
+ 43         <artifactId>junit-platform-suite-engine</artifactId>
+ 44         <version>1.12.2</version>
+ 45         <scope>test</scope>
+ 46     </dependency>
47 </dependencies>
...

```

Script 5.42: test/java/se233/chapter5part2/JUnitTestSuite.java

```

+ 1 package se233.chapter5part2;
+ 2
+ 3 import org.junit.jupiter.api.BeforeAll;
+ 4 import org.junit.jupiter.api.TestInstance;
+ 5 import org.junit.platform.suite.api.SelectClasses;
+ 6 import org.junit.platform.suite.api.Suite;
+ 7
+ 8 @Suite
+ 9 @SelectClasses({FoodTest.class, GameLoopTest.class, SnakeTest.class})
+10 @TestInstance(TestInstance.Lifecycle.PER_CLASS)
+11 public class JUnitTestSuite {
+12     @BeforeAll
+13     public static void initJfxRuntime() {
+14         javafx.application.Platform.startup(() -> {});
+15     }
+16 }

```

Script 5.43: test/java/se233/chapter5part2/FoodTest.java

```
1 //Imports are omitted
...
11 public class FoodTest {
12     private Food food;
13
14     @BeforeAll
15     public static void initJfxRuntime() {
16         javafx.application.Platform.startup(() -> {});
17     }
...

```

Script 5.44: test/java/se233/chapter5part2/SnakeTest.java

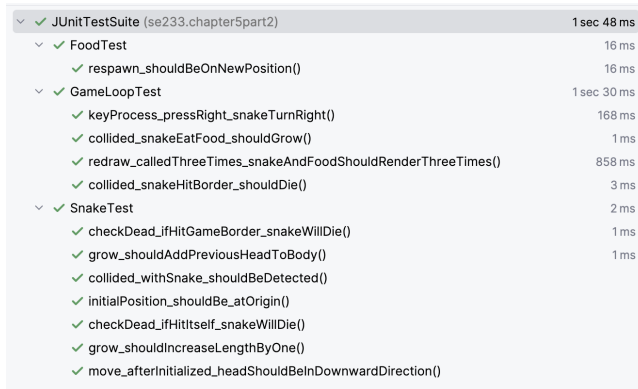
```
1 //Imports are omitted
...
14 public class SnakeTest {
15     private Snake snake;
16
17     @BeforeAll
18     public static void initJfxRuntime() {
19         javafx.application.Platform.startup(() -> {});
20     }
...

```

Script 5.45: test/java/se233/chapter5part2/GameLoopTest.java

```
1 //Imports are omitted
...
19 public class GameLoopTest {
...
24     @BeforeAll
25     public static void initJfxRuntime() {
26         javafx.application.Platform.startup(() -> {});
27     }
...

```



Test Suite	Execution Time
JUnitTestSuite (se233.chapter5part2)	1 sec 48 ms
FoodTest	16 ms
respawn_shouldBeOnNewPosition()	16 ms
GameLoopTest	1 sec 30 ms
keyProcess_pressRight_snakeTurnRight()	168 ms
collided_snakeEatFood_shouldGrow()	1 ms
redraw_calledThreeTimes_snakeAndFoodShouldRenderThreeTimes()	858 ms
collided_snakeHitBorder_shouldDie()	3 ms
SnakeTest	2 ms
checkDead_ifHitGameBorder_snakeWillDie()	1 ms
grow_shouldAddPreviousHeadToBody()	1 ms
collided_withSnake_shouldBeDetected()	
initialPosition_shouldBe_atOrigin()	
checkDead_ifHitItself_snakeWillDie()	
grow_shouldIncreaseLengthByOne()	
move_afterInitialized_headShouldBeInDownwardDirection()	

Figure 5.7: GUI of the Completed Snake Game Application

5.6 Exercise (2 parts)

5.6.1 Part 1

1. Implement two test cases for the `checkReachGameWall()` method in the `GameCharacter` class. The first test should verify the collision logic for the game boundary on the left side, and the second should verify the logic for the game boundary on the right side.
2. Implement two test cases for the `jump()` method in the `GameCharacter` class. One test must verify the case where a jump can be successfully initiated, and the other must verify the case where a jump is not permitted (e.g., the character is already airborne).
3. Implement two test cases for the `collided()` method in the `GameCharacter` class. The first should verify a horizontal (x-axis) collision, and the second should verify a vertical (y-axis) collision. These tests will require the instantiation of a second `GameCharacter` object to act as the collision target.
4. Create a new test class for the `Keys` class to verify its key-press handling logic. This class must include at least two test cases: one to verify the state change

for a single key press and another to verify the correct state when multiple keys are pressed simultaneously.

5.6.2 Part 2

1. The current implementation contains a defect where the game does not terminate immediately upon a boundary collision. Use the debugger to identify and correct this issue. After resolving the defect, implement a feature that displays a “Game Over” message in a popup window when the snake’s death condition is met.
2. Following the TDD process, first create a test method, then implement the corresponding production code for a scoring mechanism where consuming a standard food item awards one point.
3. Following the TDD process, create a test and then the corresponding implementation for a special food item. This special food item should be visually distinct (e.g., colored green) and should award five points when consumed.
4. Following the TDD process, develop a test and then the corresponding implementation to enforce a new movement rule: the snake must be prevented from reversing its direction (e.g., changing from moving right to moving left without an intermediate turn).

5.7 Discussion and Summary

This chapter has presented software testing as a systematic, evidence-based discipline and has progressively built the technical foundation required to apply it in practice. The discussion began with the V-Model, which established the correspondence between each development phase and its associated testing activity, thereby framing unit testing within the broader quality assurance lifecycle. The chapter then introduced JUnit 5 as the primary testing framework, covering annotations, assertion methods, naming conventions, test suites, and the mocking of external dependencies using Mockito. Reflection was examined as a mechanism for testing

private methods without compromising encapsulation. The treatment of TDD formalized the *red-green-refactor* cycle through Martin's three rules (Martin, 2007), reinforcing the principle that production code should exist only to satisfy a concrete, failing test. Two case studies applied these techniques to interactive JavaFX applications: the first extended the platformer game from Chapter 4 with collision, death-scene, and scoreboard logic validated by JUnit tests and reflection; the second constructed a Snake game entirely through the TDD process.

Knowledge Outcomes

Upon the completion of this chapter, a student should demonstrate the ability to:

- Explain the V-Model of software testing, identifying how unit testing, integration testing, system testing, and acceptance testing each correspond to a specific development phase.
- Describe the purpose of JUnit 5 annotations (@Test, @BeforeEach, @AfterEach, @TestInstance) and explain how they control test lifecycle and instance management.
- Justify the use of descriptive test-method naming conventions, such as the Given-When-Then pattern, and explain how such names improve diagnostic clarity when a test fails.
- Explain the role of mocking in unit testing, specifically how substituting live dependencies with controlled mock objects preserves test isolation and repeatability.
- Describe how Java reflection bypasses access-control restrictions to invoke private methods, and analyze the trade-offs in readability, performance, and security that this technique introduces.
- State the three rules of TDD and explain how each rule constrains the relationship between test code and production code during the *red-green-refactor* cycle.

Points for Discussion

1. The V-Model pairs each development phase with a corresponding testing activity, yet many agile teams iterate through multiple phases within a single sprint. How can the principles of the V-Model be reconciled with an iterative development workflow, and what risks arise if the correspondence between development and testing phases is not maintained?
2. Reflection enables the testing of private methods without altering their visibility, but this approach creates a dependency on internal implementation details that may change across refactoring cycles. Under what circumstances does the diagnostic value of testing a private method outweigh the maintenance cost of a reflection-based test, and when is it preferable to test the private logic indirectly through public methods?
3. The third rule of TDD restricts production code to the minimum required to pass the current failing test. In practice, developers frequently anticipate future requirements and write code beyond what the immediate test demands. What are the concrete consequences of violating this rule, and how does strict adherence influence the long-term design quality of a codebase?

Suggested Extensions and Project Ideas

1. Integrate a code-coverage tool, such as JaCoCo, into the Maven build configuration and generate a coverage report for the platformer test suite. Identify untested branches in the `collided()` method and write additional test cases to achieve full branch coverage.
2. Implement a parameterized test class using `@ParameterizedTest` and `@CsvSource` to replace repetitive assertion patterns in the `CalculatorTest` class, and compare the resulting test readability and maintenance burden with the original approach.
3. Extend the Snake game with a persistent high-score feature, developing the entire implementation through the TDD *red-green-refactor* cycle. Begin with

a failing test that asserts the correct serialization and deserialization of score data, then build the minimal production code to satisfy each successive test.

Progress Checklist

- ☐ Explain the V-Model of software testing and identify the testing activity that corresponds to each development phase.
- ☐ Distinguish unit testing from integration testing and system testing in terms of scope, isolation, and the types of defects each level targets.
- ☐ Write a JUnit 5 test class using `@Test`, `@BeforeEach`, and `@AfterEach` annotations with descriptive method names that follow the Given-When-Then convention.
- ☐ Select and apply appropriate assertion methods (`assertEquals`, `assertTrue`, `assertThrows`) to verify expected outcomes and exception conditions.
- ☐ Aggregate multiple test classes into a JUnit test suite using the `@Suite` and `@SelectClasses` annotations.
- ☐ Create a Mockito mock object to isolate a class under test from an external dependency and verify interaction behavior.
- ☐ Access and invoke a private method using the Java Reflection API to validate internal logic without modifying the visibility of the method under test.
- ☐ Apply the TDD *red-green-refactor* cycle to implement a feature, progressing from a failing test through minimal production code to a refactored design.

References

- Austen-Smith, D., Diermeier, D., & Zemel, E. (2017). Unintended acceleration: Toyota's recall crisis. *Kellogg School of Management Cases*, 1–16.
- Beck, K. (2003). *Test-driven development: By example*. Addison-Wesley Professional.
- Java T Point. (2020). V-Model. Retrieved May, 2026, from <https://www.javatpoint.com/software-engineering-v-model>
- JUnit. (2026). JUnit5: The 5th major version of the programmer-friendly testing framework for Java and the JVM. Retrieved May, 2026, from <https://junit.org/junit5/>
- Kaner, C. (2009). Software testing as a quality improvement activity. *IEEE Computer Society Webinar Series*, 1–86.
- Le Lann, G. (1997). An analysis of the Ariane 5 flight 501 failure—A system engineering perspective. In *Proc. of Intl. Conf. and Workshop on Engineering of Computer-Based Systems* (pp. 339–346).
- Lin, H. (1985). The development of software for ballistic-missile defense. *Scientific American*, 253(6), 46–53.
- Martin, R. C. (2007). The three rules of TDD. Object Mentor.
- Mockito. (2026). Mockito framework site. Retrieved May, 2026, from <https://site.mockito.org/>
- Neumann, P. G. (1986). Risks to the public in computer systems. *ACM SIGSOFT Software Engineering Notes*, 11(2), 2–14.
- Perry, D. E., & Stieg, C. S. (1993). Software faults in evolving a large, real-time system: A case study. In *Proc. of the European Software Engineering Conference* (pp. 48–67).
- Zuo, C., Wen, H., Lin, Z., & Zhang, Y. (2019). Automatic fingerprinting of vulnerable IoT devices with static UUIDs from mobile apps. In *Proc. of the ACM SIGSAC Conf. on Computer and Communications Security* (pp. 1469–1483).

CHAPTER 6

Build Management System

Prerequisites

- Completion of Chapters 1 through 5, encompassing event-driven graphical user interface (GUI) development, exception handling, concurrency, debugging, and unit testing.
- Familiarity with the Project Object Model (POM) file (`pom.xml`), as dependencies such as Log4j, JUnit, and Mockito have been declared in earlier chapters without formal treatment of the underlying build system.
- Experience with the JUnit testing framework and the Maven test lifecycle phase, as the Continuous Integration (CI) pipeline discussed in this chapter automates the execution of test suites configured in previous projects.
- Access to an integrated development environment (IDE), such as IntelliJ IDEA, with a functional internet connection for downloading Maven dependencies from remote repositories.

Expected Outcomes

- Distinguish between a build tool and a build management system, and explain the role each plays in automating the software development lifecycle.
- Describe the Apache Maven standard directory layout and explain how convention-over-configuration reduces project setup overhead.
- Identify the principal phases of the Maven default build lifecycle (validate, compile, test, package, install, deploy) and explain the purpose of each phase.
- Configure a POM file to declare project metadata, manage dependencies, and specify build plugins for a multi-module Java project.
- Explain the purpose of Continuous Integration in a team-based development workflow and describe how a CI pipeline automates build, test, and reporting tasks.
- Evaluate the appropriate use of Artificial Intelligence (AI)-assisted development tools within the build and configuration workflow, applying the critical review habits established throughout this course.

SOFTWARE build management systems are essential components of Software Configuration Management (SCM), a critical support discipline in the software development process. SCM is primarily concerned with managing changes, corrections, extensions, and adaptations made to a software application during its lifecycle. In practice, SCM is implemented through a variety of tools, such as Git (Git, 2026) for version control, Apache Maven (Apache Maven, 2026) for build automation, and Docker (Docker, 2026) for containerization. Key SCM functionalities include build and release management, workspace management, versioning, dependency management, and change management.

This chapter will primarily focus on two of these functionalities: software building and dependency management. Other SCM concepts will be introduced in more advanced courses within the software engineering curriculum.

6.1 Built Tool and Dependency Management

6.1.1 Build tool

A build tool is a software utility designed to automate the process of converting source code files into a standalone, executable artifact. It acts as a central orchestrator, managing tasks such as compilation, linking, packaging, and testing. This automation not only streamlines the development workflow but also ensures consistency and reproducibility across different environments. At its core, a build tool comprises two fundamental components:

- **Configuration:** A build script or configuration file that defines the project's structure, dependencies, and the sequence of tasks required to build the software. This is often written in a declarative, domain-specific language (DSL).
- **Execution engine:** An executable program that parses the configuration file and executes the defined tasks in the correct order. It invokes external tools such as compilers and linkers to produce the final build artifact.

One of the defining characteristics of modern build tools is their platform independence. They are designed to function seamlessly across various operating systems, minimizing the need for manual configuration and adaptation. This portability is essential in collaborative development environments where team members may use different operating systems. The key responsibilities of a modern build tool include the following:

- **Incremental builds:** The tool intelligently identifies which parts of the source code have been modified since the last build and recompiles only those affected files and their dependents. This incremental approach significantly reduces build times, especially for large projects.
- **Dependency management:** The tool automates the process of downloading and managing external libraries and frameworks (dependencies), ensuring that the correct versions are available during the build process.

- **Build profiles:** Build tools allow developers to define different build configurations or profiles. For example, a project might have separate profiles for development, testing, and production environments, each with its own set of compiler flags, optimization settings, and resource configurations.
- **Adaptability:** As software projects evolve, so do their requirements and dependencies. A build tool should be flexible enough to accommodate these changes with minimal manual intervention. It should provide mechanisms for easily updating dependencies, adding new build steps, or modifying existing ones.
- **Automation:** The primary goal of a build tool is to automate repetitive and error-prone tasks. By automating the build process, developers can focus on writing code and implementing features, rather than dealing with the intricacies of compilation and deployment.

6.1.2 Dependency management

In practical software development, individual projects rarely exist in complete isolation. They frequently rely on external libraries or components, creating a network of dependencies. These dependencies, such as the libraries used for PDF extraction or logging, require careful management to ensure compatibility and avoid version conflicts. The manual management of these dependencies (a process that includes downloading libraries, adding them to a project, and verifying compatibility) is often cumbersome and error-prone, particularly in large-scale projects.

Dependency management tools automate this process. They can automatically identify, download, and resolve the versions of required dependencies, ensuring the seamless integration of external components. Modern build tools often incorporate dependency management as a core feature. This level of automation streamlines the development workflow and allows developers to focus on core application logic rather than on managing third-party libraries.

Furthermore, modern build tools often provide comprehensive, automated build pipelines that encompass tasks beyond dependency management, such as compilation, testing, packaging, documentation generation, and deployment. These au-

tomated pipelines collectively enhance the efficiency and reliability of the entire software development lifecycle.

6.2 Java Build Management System

There are three prominent build management systems for the Java platform: Apache ANT with Ivy (Apache ANT+Ivy, 2017), Apache Maven (Apache Maven, 2026), and Gradle (Gradle, 2026).

Introduced in 1999, ANT (Another Neat Tool) was the first widely adopted Java build tool, recognized for its compact, extensible design and platform independence. It provided users with fine-grained control over the build process. However, ANT's reliance on Extensible Markup Language (XML) for its configuration files, initially considered a strength, became a limitation as more concise notations like domain-specific languages (DSLs) gained popularity. Additionally, the lack of a predefined build lifecycle and integrated dependency management hindered ANT's scalability in larger projects. While Apache later introduced Ivy to address dependency management, this occurred after the release of Apache Maven, which had already achieved significant adoption due to its native dependency management capabilities.

Maven, which means accumulator of knowledge in Yiddish, is currently one of the most popular Java build tools. Despite also using XML for configuration, Maven's success can be attributed to two key features: its robust dependency management and its standardized build process. Maven simplifies the process of packaging software and adding required dependencies to a project. Prior to Maven, developers primarily relied on ANT, which was often criticized for requiring the repetition of common build steps across different projects. Maven addressed this by introducing the concept of a build lifecycle, which is composed of predefined phases (e.g., compile, package) common to most development projects.

Gradle, released in 2007, is the most recent of the three build tools discussed. It is designed with a focus on multi-project builds, a common requirement for modern, large-scale systems. Unlike ANT and Maven, Gradle employs a Groovy or Kotlin-based DSL for its build scripts, which are typically more compact and readable than their XML-based counterparts.

This course focuses on Maven to demonstrate the components of a Java build management system. Its structured, convention-based approach is well-suited for introducing these concepts.

6.2.1 Apache Maven

Apache Maven is widely recognized for its standardized build lifecycle and its streamlined dependency management. A key advantage of Maven is that managing dependencies can often be accomplished with minimal effort. For example, Maven's dependency management eliminates the need to manually download and configure project libraries. Instead, a developer simply declares the required dependency coordinates in the project's configuration file, and Maven automatically handles the process of downloading and linking the appropriate library.

Maven build lifecycle

The Maven build lifecycle is defined as a sequence of named phases, where each phase represents a specific stage in the build process. Each phase consists of one or more goals that are executed in order. For example, the package phase has the goal of bundling the compiled code into a distributable format, such as a Java Archive (JAR) file. Maven defines a default lifecycle that includes 23 phases, though only a subset is commonly used for a typical project. The following list outlines common phases along with their descriptions:

- **clean:** Deletes all artifacts generated by previous builds.
- **validate:** Validates that the project is correct and all necessary information is available.
- **process-resources:** Copies and processes resource files into the target directory.
- **compile:** Compiles the project's source code.
- **process-test-resources:** Copies and processes test resource files into the test target directory.
- **test-compile:** Compiles the test source code.

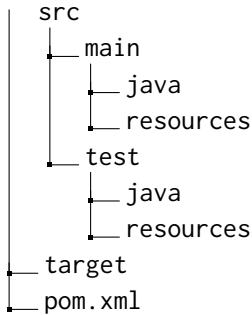
- **test:** Executes unit tests.
- **package:** Packages the compiled code into its distributable format (e.g., a JAR file).
- **verify:** Runs any checks to verify the package is valid and meets quality criteria.
- **install:** Installs the package into the local Maven repository for use as a dependency in other local projects.
- **deploy:** Copies the final package to a remote repository for sharing with other developers and projects.

In practice, a developer invokes a single phase, and Maven executes that phase along with all preceding phases in the defined lifecycle. For example, executing the package phase will trigger validate, compile, test, and finally package. A common command used to build and install a typical Java project is `mvn install`, which executes the entire lifecycle up to the install phase (McIntosh, Adams, & Hassan, 2012). The deploy phase is typically invoked separately to publish the final artifact to a shared repository.

Maven standard directory layout

Maven employs a standard directory layout to enforce a consistent project structure, which facilitates automated builds. Without this convention, projects could adopt disparate folder structures, complicating navigation and the onboarding process for new developers.

Because this standard layout is independent of any specific IDE, Maven projects can be seamlessly imported into various IDEs, such as IntelliJ IDEA and Eclipse, without compatibility issues. A newly created Maven project adheres to the following standard directory layout:



The root directory contains two primary subdirectories: `src` and `target`. The `src` directory holds all source materials for the project, including source code and resource files. It is further subdivided into `main` for the primary application code and `test` for the testing code. Both `main` and `test` contain a `java` directory for source code and a `resources` directory for other assets. The `java` directories follow the standard package structure, while the `resources` directories contain files that are bundled into the final build artifact.

A crucial component of this structure is the `pom.xml` file, which is the POM for the project. This file serves as the central configuration point where developers define project metadata, dependencies, and build instructions for Maven.

Project object model (POM)

POM is an XML file located at the root directory of a Maven project. This file contains essential information about the project and the configuration settings used for building it. When initiating a build, Maven reads the POM file to understand the developer's intent and configure the build process accordingly. The information commonly specified in a POM file includes project properties, dependencies, plugins, goals, build profiles, project version, and project descriptions.

Before creating a POM file, it is necessary to define the project's group, name, and version. Maven designates these identifiers as `groupId`, `artifactId`, and `version`, respectively. These identifiers, along with the specified Maven model version, constitute the minimum requirements for a valid POM file. An example of a minimal POM file is illustrated in Script 6.1.

Script 6.1: Minimum Requirements for a pom.xml File

pom.xml

```
+ 1 <project>
+ 2   <modelVersion>4.0.0</modelVersion>
+ 3
+ 4   <groupId>se233</groupId>
+ 5   <artifactId>chapter6</artifactId>
+ 6   <version>1.0-SNAPSHOT</version>
+ 7   <name>chapter6</name>
+ 8 </project>
```

The following provides detailed explanations for each tag in Script 6.1:

- **<project>**: The root element of the POM file.
- **<modelVersion>**: The version of the POM model itself, which should be set to 4.0.0 for Maven 2 and later.
- **<groupId>**: A unique identifier for the project's group, typically representing the organization or project collection.
- **<artifactId>**: The identifier for the project, corresponding to the artifact (e.g., JAR, WAR) name.
- **<version>**: The specific version of the project. This identifier is used in an artifact repository to distinguish between different releases. It is conventional to append *-SNAPSHOT* to the version to indicate that the project is in an active development phase.

Script 6.2 expands upon the minimal POM file by incorporating additional details concerning project packaging, properties, and dependencies. In this example, the project specifies its source file encoding as UTF-8. Moreover, it declares project dependencies, which are enumerated under the `<dependencies>` tag. Each dependency follows the `<groupId>: <artifactId>: <version>` coordinate system, mirroring the format used to identify the project itself. In this instance, the project incorporates the `org.openjfx:javafx-controls` and `org.openjfx:javafx-fxml` libraries, both at version 26.0.1.

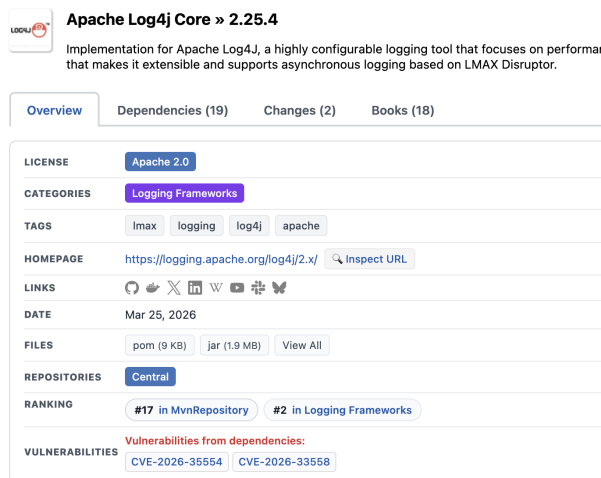
Script 6.2: pom.xml File with Properties and Dependencies**pom.xml**

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.
5      apache.org/xsd/maven-4.0.0.xsd">
6
7      <groupId>se233</groupId>
8      <artifactId>chapter6</artifactId>
9      <version>1.0-SNAPSHOT</version>
10     <name>chapter6</name>
11
12     <properties>
13         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
14         <junit.version>5.10.0</junit.version>
15     </properties>
16
+ 17     <dependencies>
+ 18         <dependency>
+ 19             <groupId>org.openjfx</groupId>
+ 20             <artifactId>javafx-controls</artifactId>
+ 21             <version>26.0.1</version>
+ 22         </dependency>
+ 23         <dependency>
+ 24             <groupId>org.openjfx</groupId>
+ 25             <artifactId>javafx-fxml</artifactId>
+ 26             <version>26.0.1</version>
+ 27         </dependency>
+ 28     </dependencies>
29 </project>

```

A common question regarding dependency management is how to determine the appropriate XML snippet to add to the `pom.xml` file for a required library. A straightforward approach is to search for the library's name followed by the term “Maven dependency” or “pom” in a search engine. Results often lead to a repository website, such as MVNRepository (<https://mvnrepository.com/>). Figure 6.1 illustrates an example of such a webpage. The required XML block to be added to the `pom.xml` file is typically provided under a “Maven” tab. Script 6.3 showcases the updated `pom.xml` file with the Log4j library included in the project's dependency list.



Apache Log4j Core » 2.25.4
Implementation for Apache Log4J, a highly configurable logging tool that focuses on performance that makes it extensible and supports asynchronous logging based on LMAX Disruptor.

Overview Dependencies (19) Changes (2) Books (18)

LICENSE Apache 2.0

CATEGORIES Logging Frameworks

TAGS lmax logging log4j apache

HOMEPAGE <https://logging.apache.org/log4j/2.x/> [Inspect URL](#)

LINKS [GitHub](#) [Twitter](#) [LinkedIn](#) [YouTube](#) [Facebook](#) [Instagram](#)

DATE Mar 25, 2026

FILES pom (9 KB) jar (1.9 MB) [View All](#)

REPOSITORIES Central

RANKING #17 in MvnRepository #2 in Logging Frameworks

VULNERABILITIES Vulnerabilities from dependencies: CVE-2026-35554 CVE-2026-33558

Figure 6.1: Example Query Results from a Maven Dependency Search

6.3 Continuous Integration (CI)

CI is a fundamental practice in modern software development that involves the frequent merging of code changes into a shared repository, with a recommended frequency of at least once per day. The primary objective of CI is to maintain a consistently integratable codebase, ensuring that the software can be successfully built, executed, and deployed at any time while passing all automated tests. By in-

Script 6.3: pom.xml File with Properties, Dependencies and Plugins

pom.xml

```
1 <project>
2   <modelVersion>4.0.0</modelVersion>
3   ...
11  <dependencies>
12    <dependency>
13      <groupId>org.openjfx</groupId>
14      <artifactId>javafx-controls</artifactId>
15      <version>26.0.1</version>
16    </dependency>
+ 17    <dependency>
+ 18      <groupId>org.apache.logging.log4j</groupId>
+ 19      <artifactId>log4j-core</artifactId>
+ 20      <version>2.25.4</version>
+ 21    </dependency>
22  </dependencies>
23 </project>
```

tegrating changes early and often, development teams can proactively identify and resolve integration problems before they escalate, thereby fostering collaboration and continuous improvement.

CI serves as a preventive measure against the accumulation of integration issues that arise when developers work in isolation for extended periods. Without CI, these issues may remain hidden until later stages, leading to a scenario commonly termed *integration hell*, where merging code from multiple developers becomes an exceptionally complex and time-consuming undertaking. Frequent integration allows developers to identify conflicts and incompatibilities quickly, resolving them while the context of the changes is still recent.

Beyond preventing integration problems, CI facilitates faster feedback loops through automated tests that execute with each integration, providing immediate insights into the impact of any changes. This rapid feedback enables the early detection and remediation of bugs, which is typically more efficient and economical. Additionally, CI enhances collaboration by encouraging developers to share their work regularly and communicate potential issues promptly.

Figure 6.2 depicts a typical CI workflow, wherein code commits to a version control system (e.g., Git) trigger an automated pipeline. This pipeline builds the code, runs a suite of automated tests, and, if successful, can trigger a continuous delivery (CD) pipeline for deployment. This automated process ensures that each code change is thoroughly tested and integrated, reducing the risk of introducing errors into the main codebase.

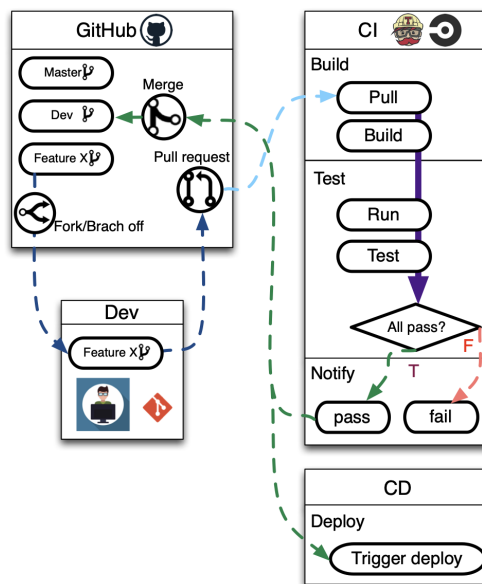


Figure 6.2: Example CI Workflow

Build tools play a crucial role in CI by automating the build process, which ensures consistency and repeatability. They manage tasks such as dependency resolution, code compilation, test execution, and software packaging. Popular build tools for Java, such as Ant, Maven, and Gradle, offer features like predefined build lifecycles, customizable build scripts, and integration with testing frameworks, making them invaluable assets in CI pipelines.

CI is often coupled with CD, which automates the release of software to production environments. By automating the entire software delivery pipeline, in-

cluding code integration, testing, and final deployment, CD enables organizations to achieve a rapid time-to-market. This streamlined process eliminates manual interventions, which reduces the risk of errors during deployments. Consequently, new features, enhancements, and bug fixes can be promptly delivered to end-users. The adoption of CD empowers development teams to respond swiftly to market demands, thereby fostering a competitive advantage.

6.4 Case Study

AI Advisory

This chapter demonstrates how the deliberate, informed use of AI assistance transitions from cautious experimentation to professional practice. Build configuration is declarative, compositional, and immediately verifiable: a correct POM compiles, whereas an incorrect one does not. These properties make dependency declarations, plugin configurations, and CI pipeline definitions well-suited to AI-assisted completion, provided that every generated suggestion is treated as a preliminary draft requiring verification rather than as a finalized artifact.

Recommended Approach

When adding a dependency or configuring a CI workflow, begin by understanding the structural requirements of the task: the minimum POM elements, the applicable Maven lifecycle phase, or the GitHub Actions workflow syntax. CI platforms such as GitHub Actions use YAML (YAML Ain't Markup Language), a human-readable data-serialization format, to define pipeline configurations. Use GitHub Copilot to accelerate the mechanical aspects of the task, such as auto-completing coordinate triples, suggesting plugin configuration blocks, or scaffolding a CI workflow definition. Subsequently, verify every generated value against an authoritative source before committing it to the repository.

Appropriate Uses

- ✓ Use Copilot to auto-complete repetitive POM dependency blocks after the groupId, artifactId, and version structure is fully understood.
- ✓ Use Copilot to generate a starter GitHub Actions workflow for a Maven project. Because CI workflow files utilize YAML syntax that may be unfamiliar, every line of the generated file must be reviewed and comprehended before it is committed to a repository.
- ✓ Use the Copilot chat panel to query quick-reference information, such as “what Maven lifecycle phase runs unit tests?”, as a supplement to official documentation.
- ✓ Use an AI tool to compare Maven and Gradle for a specific project scenario after the Maven lifecycle has been studied independently.
- ✓ Treat every AI-generated version number as a suggestion; always verify the version against the Maven Central Repository.

Practices to Avoid

- ✗ Do not accept an AI-suggested <version> for a security-sensitive library without verifying the Common Vulnerabilities and Exposures (CVE) status of that specific release.
- ✗ Do not allow Copilot to generate an entire pom.xml file from scratch without first comprehending the minimum POM requirements detailed in this chapter.
- ✗ Do not use AI tools to bypass an understanding of the Maven build lifecycle; the phases and their execution order constitute foundational knowledge for professional Java development.

- × Do not assume that a Copilot-generated CI workflow file will function correctly for a specific project structure without systematically reading and verifying every line of the configuration.

Reflecting across the six chapters, the AI advisory progression of this coursebook follows the same instructional logic as the chapters themselves: mastering the fundamentals, understanding the failure modes, and subsequently introducing tools that amplify existing capabilities. A developer who has manually traced an MVC event flow, deliberately handled a network exception, diagnosed a thread-safety issue using a breakpoint, and authored a robust unit test suite is fully equipped to utilize GitHub Copilot productively. This exact profile of technical competence defines the intended state of the reader upon concluding this coursebook.

6.4.1 Managing Maven project

The case study from the previous chapter, chapter5part1, demonstrated the ease with which a Maven project can be reused. For instance, the project was provided as a zip file via <https://cmu.to/SE233CH5P1>. Because the project did not require modifications to its `pom.xml` file, the only step needed to reuse it on a different machine was to copy the `src` directory. The project's existing configuration, including its properties, dependencies, and plugins, facilitates this portability. Once the project is opened in an IDE like IntelliJ IDEA and Maven is refreshed, the build tool automatically configures the project based on the `pom.xml` file, eliminating the need for further manual setup.

To initiate the present case study, a new project, chapter6, is created by cloning the chapter5part2 (snake game) project. This is accomplished by copying the entire `src` directory and the `pom.xml` file from chapter5part2 into the new chapter6 project directory. The `pom.xml` file is included to simulate a real-world scenario where specific dependency versions are required to avoid package conflicts. In this case, `org.junit.platform:junit-platform-suite-engine:1.14.4` was added as a project dependency.

However, this action alone is insufficient for Maven to configure and run the project. First, the project's coordinates, `<groupId>:<artifactId>:<version>`, must be updated to match the new project's identity. Script 6.4 provides the initial update for the `pom.xml` file, setting the coordinates to `se233:chapter6:1.0-SNAPSHOT`. To rename the project's packages, the IDE's refactoring function can be utilized, as illustrated in Figures 6.3. By right-clicking on the `se233.chapter5part2` package, selecting *Refactor* and then *Rename*, choosing the option to rename *All directories*, and entering the new package name `se233.chapter6`, all package declarations are updated accordingly. An additional manual edit is required for the `module-info.java` file, as shown in Script 6.5. After completing these modifications, the project should build and run successfully without errors.

Suppose logging functionality is to be integrated into the project. In prior case studies, the instructor provided the `<groupId>:<artifactId>:<version>` coordinates for stable versions of the `log4j-core`, `log4j-api`, and `log4j-slf4j-impl` libraries. However, retrieving these coordinates is a straightforward task. To do so, one can search for these libraries on a public repository such as <https://mvnrepository.com/> and copy the provided Maven dependency XML for the latest stable version into the project's `pom.xml` file.

Script 6.6 provides an example of the updated `pom.xml` file demonstrating the inclusion of these logging dependencies. It is recommended that users verify and select the most recent stable versions available at the time of implementation.

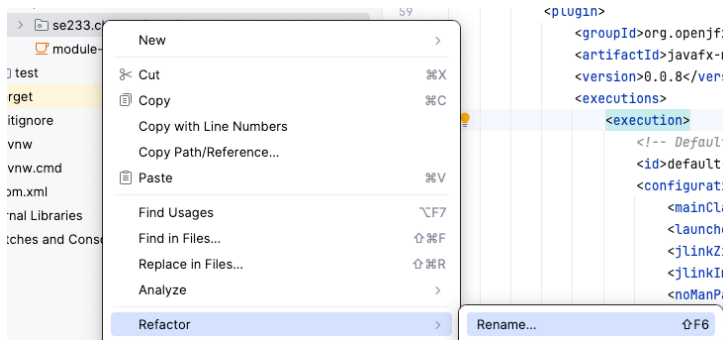


Figure 6.3: Renaming Packages in a Project (Step 1)

Script 6.4: pom.xml

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.
    apache.org/xsd/maven-4.0.0.xsd">
5      <modelVersion>4.0.0</modelVersion>
6
7      <groupId>se233</groupId>
8      <artifactId>chapter6</artifactId>
9      <version>1.0-SNAPSHOT</version>
+ 10     <name>chapter5part2</name>
11     <name>chapter6</name>
12
13     <properties>
14         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15         ...
58         <executions>
59             <execution>
60                 <!-- Default configuration for running with: mvn clean
                    javafx:run -->
61                 <id>default-cli</id>
62                 <configuration>
- 63                     <mainClass>se233.chapter6/se233.chapter6.
                        HelloApplication</mainClass>
+ 64                     <mainClass>se233.chapter6.Launcher</mainClass>
65                     <launcher>app</launcher>
66                     <jlinkZipName>app</jlinkZipName>
67                     <jlinkImageName>app</jlinkImageName>
68                     <noManPages>true</noManPages>
69                     <stripDebug>true</stripDebug>
70                     <noHeaderFiles>true</noHeaderFiles>
71                 </configuration>
72             </execution>
73         </executions>
74     </plugin>
75 </plugins>
76 </build>
77 </project>

```

Script 6.5: module-info.java

```
- 1 module se233.chapter5part2 {
+ 2 module se233.chapter6 {
  3     requires javafx.controls;
  4     requires javafx.fxml;
  5
  6     opens se233.chapter6 to javafx.fxml;
  7     exports se233.chapter6;
  8 }
```

Script 6.6: pom.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.
5     apache.org/xsd/maven-4.0.0.xsd">
6     <modelVersion>4.0.0</modelVersion>
7     ...
49     <version>1.11.0</version>
50     <scope>test</scope>
51     </dependency>
+ 52     <dependency>
+ 53         <groupId>org.apache.logging.log4j</groupId>
+ 54         <artifactId>log4j-core</artifactId>
+ 55         <version>SET THIS TO THE LATEST STABLE VERSION</version>
+ 56     </dependency>
+ 57     <dependency>
+ 58         <groupId>org.apache.logging.log4j</groupId>
+ 59         <artifactId>log4j-api</artifactId>
+ 60         <version>SET THIS TO THE LATEST STABLE VERSION</version>
+ 61     </dependency>
+ 62     <dependency>
+ 63         <groupId>org.apache.logging.log4j</groupId>
+ 64         <artifactId>log4j-slf4j-impl</artifactId>
+ 65         <version>SET THIS TO THE LATEST STABLE VERSION</version>
+ 66     </dependency>
67 </dependencies>
...

```

Next, the `module-info.java` file must be modified to require the new logging modules, as shown in Script 6.7. Additionally, a `log4j.properties` configuration file must be created in the resources directory with the content provided in Script 6.8. Upon completion of these steps, the integration of logging into the `Food` class, as depicted in Script 6.9, should proceed without compilation errors.

Script 6.7: module-info.java

```
1 module se233.chapter6 {
2     requires javafx.controls;
3     requires javafx.fxml;
+ 4     requires org.apache.logging.log4j;
5
6     opens se233.chapter6 to javafx.fxml;
7     exports se233.chapter6;
8 }
```

Script 6.8: /src/resources/log4j2.properties

```
+ 1 name = PropertiesConfig
+ 2 appender.console.type = Console
+ 3 appender.console.name = consoleLogger
+ 4 appender.console.layout.type = PatternLayout
+ 5 appender.console.layout.pattern = [%5p] %d{yyyy-MM-dd HH:mm:ss} [time %r ms] [%t]
    %c{1} - %msg%n
+ 6
+ 7 rootLogger.level = info
+ 8 rootLogger.appenderRef.stderr.ref = consoleLogger
```

Script 6.9: model/Food.java

```

1  //Imports are omitted
  ...
8  public class Food {
9
+10     private static final Logger logger = LogManager.getLogger(Food.class);
    ...
23     public void respawn() {
24         Point2D prev_position = this.position;
25         do {
26             this.position = new Point2D(rn.nextInt(Platform.WIDTH), rn.nextInt(
                Platform.HEIGHT));
27         } while(prev_position == this.position);
+28         logger.info("food: x:{} y:{}",this.position.getX(), this.position.getY());
29     }
30     public Point2D getPosition() { return position; }
31 }

```

6.4.2 Packaging with Maven

Packaging a project with Maven can be accomplished through the Maven toolset menu within an IDE such as IntelliJ, as shown in Figure 6.4. Double-clicking the “package” option under “Lifecycle” initiates the compilation, testing, and packaging of the project into a JAR file compatible with any Java Virtual Machine environment. The resulting JAR file is placed in the target folder, specifically at `target/chapter6-1.0-SNAPSHOT.jar`.

Ordinarily, this JAR file can be executed by double-clicking it or by invoking it from a command-line interface. However, an error message like *no main*

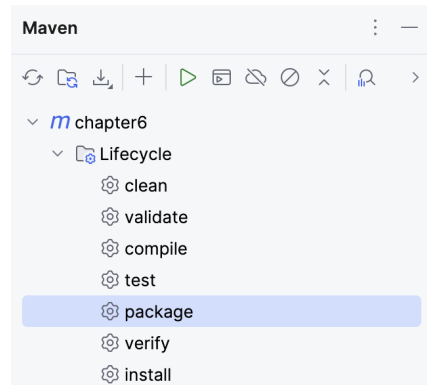


Figure 6.4: Packaging a Project using IntelliJ’s Maven Toolset

manifest attribute in chapter6-1.0-SNAPSHOT.jar (see Figure 6.5) indicates a configuration issue that must be resolved. To address this error, two actions are required:

- **Inclusion of Dependencies:** All project dependencies must be bundled within the JAR file.
- **Specification of Main Class:** The entry point, or main class, that is called upon execution must be explicitly designated.

These requirements can be met by modifying the `pom.xml` file to include a build plugin, such as the Maven shade plugin or the Maven assembly plugin. These plugins automate the process of packaging dependencies and specifying the main class, ensuring that the final JAR file is executable.

```
(base) kong@mba target % java -jar chapter6-1.0-SNAPSHOT.jar
no main manifest attribute, in chapter6-1.0-SNAPSHOT.jar
(base) kong@mba target %
```

Figure 6.5: Error Message Encountered When Executing `chapter6-1.0-SNAPSHOT.jar`

An example configuration for creating an executable JAR using the Maven shade plugin is available on the plugin's official website: <https://maven.apache.org/plugins/maven-shade-plugin/examples/executable-jar.html>. The instructions on this page provide the necessary configuration to be added to the `pom.xml` file.

Script 6.10: `JarLauncher.java`

```
+ 1 //Imports are omitted
+ 2 public class JarLauncher {
+ 3     public static void main(String[] args) {
+ 4         Launcher.main(args);
+ 5     }
+ 6 }
```

Script 6.11: pom.xml

```

1  <?xml version="1.0" encoding="UTF-8"?>
    ...
80      <plugin>
81          <groupId>org.openjfx</groupId>
82          <artifactId>javafx-maven-plugin</artifactId>
83          <version>0.0.8</version>
84          <executions>
85              <execution>
86                  <id>default-cli</id>
+ 87                  <goals>
+ 88                      <goal>run</goal>
+ 89                  </goals>
90                  <configuration>
- 91                      <mainClass>se233.chapter6.Launcher</mainClass>
+ 92                      <mainClass>se233.chapter6.JarLauncher</mainClass>
- 93                      <launcher>app</launcher>
- 94                      <jlinkZipName>app</jlinkZipName>
- 95                      <jlinkImageName>app</jlinkImageName>
- 96                      <noManPages>true</noManPages>
- 97                      <stripDebug>true</stripDebug>
- 98                      <noHeaderFiles>true</noHeaderFiles>
99                  </configuration>
100              </execution>
101          </executions>
102      </plugin>
+ 103      <plugin>
+ 104          <groupId>org.apache.maven.plugins</groupId>
+ 105          <artifactId>maven-shade-plugin</artifactId>
+ 106          <version>3.6.2</version>
+ 107          <executions>
+ 108              <execution>
+ 109                  <phase>package</phase>
+ 110                  <goals>
+ 111                      <goal>shade</goal>
+ 112                  </goals>
+ 113                  <configuration>
+ 114                      <transformers>
+ 115                          <transformer implementation="org.apache.maven.plugins.
+                               shade.resource.ManifestResourceTransformer">
+ 116                              <mainClass>se233.chapter6.JarLauncher</mainClass>
+ 117                          </transformer>
+ 118                      </transformers>
+ 119                  </configuration>
+ 120              </execution>
+ 121          </executions>
+ 122      </plugin>
    ...

```

After rebuilding the project with the updated configuration, attempting to execute the JAR file may result in a different error: “*Error: JavaFX runtime components are missing and are required to run this application*”. This issue, specific to JavaFX applications, requires additional steps to resolve. A potential solution, outlined on Stack Overflow at <https://stackoverflow.com/questions/52653836>, indicates a class loading problem and recommends creating a separate launcher class as a workaround.

Following this suggestion, a new Java class named `JarLauncher` must be created with the content provided in Script 6.10. The `pom.xml` file is then modified to designate this new `JarLauncher` class as the main class for the executable JAR, rather than the original `Launcher` class. Script 6.11 illustrates the updated `pom.xml` configuration reflecting this change.

After implementing these modifications and re-executing the Maven “clean” and “package” lifecycle goals, the game should be launchable. The application can be started either by double-clicking the `chapter6-1.0-SNAPSHOT.jar` file or by running the command `java -jar chapter6-1.0-SNAPSHOT.jar` from the project’s target directory.

6.5 AI-Assisted Development and GitHub Copilot

Throughout the preceding chapters, each AI Advisory section has identified a specific category of task in which uncritical reliance on AI-generated code would prove counterproductive: untraceable MVC integration in Chapter 1, silenced exceptions in Chapter 2, unsafe threading in Chapter 3, bypassed debugging in Chapter 4, and ineffective tests in Chapter 5. In each of these instances, the inherent risk arose from delegating tasks where *context and reasoning* are strictly irreplaceable. Build configuration, however, occupies a fundamentally different position on this spectrum.

POM declarations, Maven dependency blocks, and CI pipeline definitions are largely *declarative and compositional*. The correct parameters are retrievable directly from official documentation, the structure is highly standardized, and the failure mode, such as a build that fails to compile or a pipeline that aborts, is immediate and unambiguous. Consequently, this is precisely the category of task in

which AI-assisted development tools provide high-value assistance with comparatively low risk.

6.5.1 What is GitHub Copilot?

GitHub Copilot is an AI-powered code-completion tool developed by GitHub in collaboration with OpenAI. It operates as an integrated plugin within the IDE, analyzing the surrounding code context, including open files, comments, and function signatures, to generate inline suggestions that range from single-line completions to multi-line code blocks. Copilot is trained on a large corpus of publicly available source code and documentation, which enables it to produce syntactically correct suggestions for a wide variety of languages, frameworks, and configuration formats.

Unlike a general-purpose chatbot, Copilot is designed for in-editor use: it responds to the immediate editing context rather than to a conversational prompt. This contextual awareness makes it particularly effective for repetitive, pattern-based tasks such as writing boilerplate code, completing structured configuration files, and generating standard code templates. However, the same mechanism that enables rapid suggestion also introduces the risk of hallucinated dependencies, outdated version numbers, and configurations that appear plausible but do not match the specific requirements of the project.

6.5.2 Integration with IntelliJ IDEA

Copilot integrates into IntelliJ IDEA as a plugin installable through the IDE plugin marketplace. Once installed and authenticated with a GitHub account, the tool provides real-time inline suggestions as development progresses. Suggestions appear as grayed-out text that can be accepted with a single keystroke or dismissed to continue manual entry. The plugin also provides a chat panel for querying specific details about the current file or project without requiring the developer to navigate outside the IDE environment.

The integration is context-sensitive: when editing a `pom.xml` file, Copilot utilizes its knowledge of Maven conventions to suggest dependency blocks, plugin configurations, and property declarations. When editing a CI pipeline configu-

ration file written in YAML (a widely used data-serialization format for DevOps tooling), Copilot can suggest workflow structures, job definitions, and common step configurations tailored for Java and Maven projects. This specificity distinguishes Copilot from general search engines, as it generates suggestions that are syntactically aligned with the specific file currently active in the editor.

6.5.3 Appropriate Use Cases in Build Configuration

The declarative nature of build and CI configuration files makes them well-suited to AI-assisted completion. The following scenarios represent cases in which Copilot can meaningfully accelerate the development workflow:

- **Dependency declarations.** When initiating a `<dependency>` block in a POM file, Copilot can suggest the correct `<groupId>`, `<artifactId>`, and `<version>` for widely used libraries such as JUnit, Log4j, or Mockito. The suggested version must be verified against the Maven Central Repository (`mvnrepository.com`) prior to acceptance, as the tool may generate an out-dated or deprecated release.
- **CI workflow scaffolding.** For projects hosted on GitHub, Copilot can generate a starter GitHub Actions workflow file that checks out the repository, configures a Java environment, and executes `mvn test`. The generated YAML typically adheres to the standard structure documented in the official GitHub workflow templates, but must be reviewed to ensure compatibility with the specific project layout and Java version.
- **Plugin configuration.** Maven plugins, such as the `maven-compiler-plugin`, `maven-surefire-plugin`, and `maven-shade-plugin`, require specific configuration blocks. Copilot can suggest these blocks based on the plugin name and the surrounding POM context, minimizing the necessity to consult documentation for routine configurations.
- **Repetitive structural patterns.** In multi-module projects, the parent POM and child POM structures follow a well-defined architectural pattern. Copilot can replicate this pattern across modules, significantly reducing manual duplication and adaptation efforts.

6.5.4 Limitations and Verification Habits

Despite its effectiveness in declarative contexts, Copilot remains a statistical model that generates suggestions based on patterns in its training data rather than on verified, domain-specific knowledge. The following limitations require active mitigation:

- **Version accuracy.** Copilot may suggest a library version that was current at the time of its training but has since been superseded or found to contain a security vulnerability. Every suggested `<version>` tag must be verified against the Maven Central Repository, and for security-sensitive libraries, the CVE database should be consulted.
- **Project-specific configuration.** Copilot lacks awareness of the specific module structure, Java version, or custom plugin requirements of the project currently under development. A suggestion that compiles in isolation may fail within the context of a multi-module build or conflict with existing plugin configurations.
- **CI environment assumptions.** A generated CI workflow may assume a default operating system, Java distribution, or caching strategy that does not align with the project's actual deployment environment. Each line of a generated CI configuration file must be reviewed and comprehended prior to repository integration.
- **Licensing and compliance.** Code suggestions derived from publicly available repositories may inadvertently introduce code patterns that carry licensing obligations. For production projects, the provenance of AI-generated code segments should be reviewed against organizational compliance policies.

6.5.5 Lessons from This Course

The AI advisory progression of this coursebook follows the same structural logic as the chapters themselves: mastering the fundamentals, understanding the failure modes, and subsequently introducing tools that amplify existing capabilities. A

developer who has manually traced an MVC event flow (Chapter 1), deliberately handled a network exception (Chapter 2), diagnosed a thread-safety issue using a breakpoint (Chapter 4), and authored a robust unit test suite (Chapter 5) possesses the requisite domain knowledge to evaluate whether an AI-generated suggestion is correct, adequate, or misleading.

This foundational competence transforms the dynamic with AI assistance from one of dependency to one of technical collaboration. Build configuration, as demonstrated in this chapter, represents a clear example of a domain where AI tools operate optimally: the tasks are highly structured, the correctness criteria are strictly verifiable, and the consequence of an error is a failed build rather than a silent runtime defect. The critical competency lies not in the mere operation of Copilot, but rather in the ability to determine *when* the generated output is trustworthy and *when* manual reasoning remains strictly necessary.

6.6 Exercise

1. For the case study presented in Chapter 1, use Maven to add a logger that records which item is equipped each time the user releases the mouse after a drag-and-drop operation. Package the final project into an executable JAR file.
2. For the case study presented in Chapter 2, use Maven to implement a logger that records which currency is added or deleted when the user clicks the *Add* or *Delete* button, respectively. Package the final project into an executable JAR file.
3. For the case study presented in Chapter 3, use Maven to integrate a logger that records the list of all file names when the *Start Indexing* button is clicked. Package the final project into an executable JAR file.
4. For the first case study presented in Chapter 4, use Maven to package the project into an executable JAR file.
5. For the case study presented in Chapter 5 (Part 1), use Maven to package the project into an executable JAR file.

6.7 Discussion and Summary

This chapter has positioned build management as a foundational practice within SCM, progressing from the manual compilation of individual source files to the fully automated build-test-deploy pipelines that define professional Java development. The discussion began by distinguishing build tools from Build Management Systems, establishing that the latter provide dependency resolution, lifecycle management, and plugin extensibility beyond simple compilation. Apache Maven was examined in detail: its convention-over-configuration philosophy, the standard directory layout, the sequential phases of the default build lifecycle, and the structure of the Project Object Model that governs every Maven project. Continuous Integration was then introduced as the practice of merging and verifying code changes at least daily, utilizing automated pipelines that execute the full Maven lifecycle upon each commit. The chapter concluded with an examination of AI-assisted development through GitHub Copilot, demonstrating that declarative, pattern-based configuration tasks, such as POM authoring, dependency declaration, and CI workflow scaffolding, represent the domain in which AI tools deliver optimal value, provided that every generated suggestion is verified against authoritative sources.

Knowledge Outcomes

Upon completion of this chapter, a student should demonstrate the ability to:

- Explain the role of Software Configuration Management in the software development lifecycle and identify how build management, version control, and dependency management contribute to project integrity.
- Distinguish between a build tool that automates compilation and a Build Management System that additionally manages dependencies, enforces a standard project structure, and provides an extensible plugin architecture.
- Describe the Maven standard directory layout and explain how the convention-over-configuration principle minimizes the requirement for explicit build instructions.

- Trace the execution order of the Maven default build lifecycle phases and explain how each phase contributes to producing a tested, packaged artifact.
- Specify the purpose of Continuous Integration in a team-based workflow and explain how automated build and test execution upon each commit prevents the accumulation of integration defects.
- Evaluate the suitability of AI-assisted code completion for declarative configuration tasks and explain why verification against official repositories and documentation remains strictly necessary.

Points for Discussion

1. Maven enforces a single standard directory layout and a fixed lifecycle phase order, while Gradle offers a flexible, script-based approach that allows developers to define custom build logic. What are the trade-offs between convention-driven rigidity and script-driven flexibility, and under what project conditions does each approach provide a net advantage?
2. Continuous Integration requires that all developers integrate their changes into a shared repository at least once per day. In practice, developers sometimes delay integration to avoid exposing incomplete work. What strategies can a team adopt to reconcile the CI requirement of frequent integration with the reality that not every feature can be completed within a single day?
3. The chapter argues that build configuration is a domain where AI-assisted tools provide high-value assistance with low risk, in contrast to the cautions raised in earlier chapters. Under what circumstances could this assessment prove incorrect; that is, in what scenarios might an AI-generated POM or CI configuration introduce a defect that is not immediately caught by a failed build?

Suggested Extensions and Project Ideas

1. Create a multi-module Maven project that separates a JavaFX application into distinct modules for the model, view, and controller layers. Configure

the parent POM to manage shared dependencies and plugin versions centrally, and verify that the `mvn package` command produces a deployable artifact from the aggregated modules.

2. Set up a GitHub Actions CI pipeline for one of the earlier coursebook projects that automatically executes the full JUnit test suite on every push to the main branch. Configure the pipeline to report test results as a build status check and investigate the behavior when a deliberately failing test is committed.
3. Compare the Maven and Gradle build systems by migrating the case study project from this chapter to a Gradle-based configuration. Document the structural differences in project layout, dependency declaration syntax, and build lifecycle, and measure the build execution time for both systems under identical conditions.
4. Investigate the Maven dependency tree for a project that includes transitive dependencies by executing `mvn dependency:tree`. Identify at least one case of a transitive dependency that could introduce a version conflict, and configure a `<dependencyManagement>` block to resolve it.

Progress Checklist

- ☐ Explain the distinction between a build tool and a build management system, and identify the additional capabilities that a build management system provides over manual compilation.
- ☐ Describe the Maven standard directory layout and locate the correct placement for source files, test files, and resource files within a Maven project.
- ☐ List the phases of the Maven default build lifecycle in sequential order and explain the purpose of the compile, test, package, and install phases.
- ☐ Configure a `pom.xml` file with the minimum required elements and add a dependency declaration that specifies the correct `groupId`, `artifactId`, and `version`.
- ☐ Explain the purpose of Continuous Integration (CI) and describe how automated build and test execution upon each commit prevents integration problems.
- ☐ Package a Maven project into an executable JAR file using the appropriate Maven lifecycle command and plugin configuration.
- ☐ Use GitHub Copilot to auto-complete a POM dependency block or generate a starter CI workflow, and verify the suggested output against official documentation.
- ☐ Evaluate an AI-generated build configuration by identifying potential risks, such as outdated version numbers, missing project-specific settings, and incorrect CI environment assumptions.

References

- Apache ANT+Ivy. (2017). Apache Ivy. Retrieved May, 2026, from <https://ant.apache.org/ivy/>
- Apache Maven. (2026). Apache Maven. Retrieved May, 2026, from <https://maven.apache.org/>
- Docker. (2026). Docker: Accelerated Container Application Development. Retrieved May, 2026, from <https://www.docker.com/>
- Git. (2026). Git. Retrieved May, 2026, from <https://git-scm.com/>
- Gradle. (2026). Gradle Build Tool. Retrieved May, 2026, from <https://gradle.org/>
- McIntosh, S., Adams, B., & Hassan, A. E. (2012). The evolution of Java build systems. *Empirical Software Engineering*, 17(4–5), 578–608.

Appendix A

THIS appendix presents a tutorial for setting up the development environment used throughout this textbook. Specifically, the tutorial covers the installation and configuration of the current versions of the Java SE Development Kit (JDK), the IntelliJ IDEA IDE, and the JavaFX framework.

Installation

JDK

For Java development, the official Java Development Kit (JDK) provided by Oracle is the recommended implementation. It is advisable to use a version with Long-Term Support (LTS) to ensure stability and extended maintenance. At the time of this writing, JDK 24 is the version that supports the latest update of JavaFX. The JDK can be obtained from the official Oracle website: <https://www.oracle.com/java/technologies/downloads/#java24>. On the download page, navigate to the JDK Download section and select the appropriate installer for your operating system (e.g., Windows, macOS, or Linux).

After the download is complete, locate the installer file and execute it. Follow the prompts provided by the installation wizard to complete the JDK setup.

IntelliJ Idea

The IntelliJ IDEA Integrated Development Environment (IDE) is utilized for all demonstrations and practical exercises in this course. The IDE is provided by JetBrains at <https://www.jetbrains.com/idea/>. Prior to downloading the IDE, it is highly recommended that students apply for an educational license, which is available at <https://www.jetbrains.com/community/education/#students>. This license typically provides access to the Ultimate edition of the IDE, which includes a more comprehensive feature set.

A detailed comparison of the features available in the different editions of IntelliJ IDEA is provided at https://www.jetbrains.com/idea/features/editions_comparison_matrix.html.

After the installer has been downloaded, execute it and follow the on-screen instructions to complete the installation.

JavaFX

The process for configuring a JavaFX project in recent versions of IntelliJ IDEA has been significantly streamlined. This section provides a walkthrough of the setup process.

The process begins by selecting *Create a New Project* from the main IntelliJ IDEA window. In the project setup wizard, choose the *JavaFX* option and specify a directory for the course's lab work. Select *Java* as the language and *Maven* as the build system. The project should be named `HelloWorld` with the group ID set to "se233". It is important to ensure that the selected JDK version corresponds to the version installed previously (e.g., Oracle OpenJDK 24). Proceed by clicking *Next* and then *Create* without modifying any further options. An example of this configuration is shown in Figure A.1. If a prompt regarding Maven appears, select the "Load Maven Project" option. A detailed discussion of Maven is provided in Chapter 6.

The IDE will initialize a Java project with the following structure:

```
src
├── main
│   ├── java
│   │   ├── se233
│   │   │   ├── helloworld
│   │   │   │   ├── HelloApplication
│   │   │   │   └── HelloController
│   │   └── module-info.java
│   └── resources
│       ├── se233
│       │   └── helloworld
│       │       └── hello-view.fxml
```

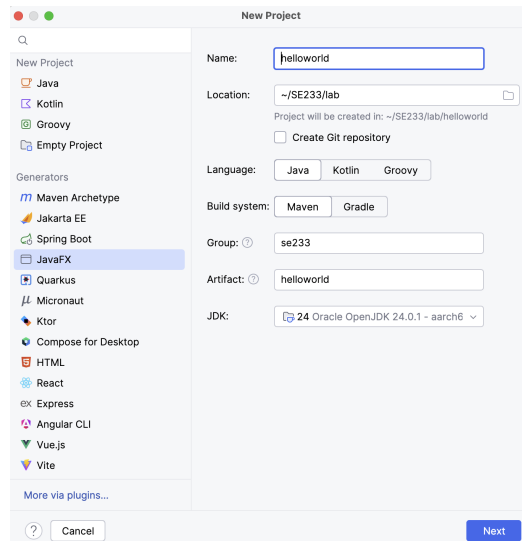


Figure A.1: New Project Configuration in IntelliJ IDEA

Although the initial setup procedures in recent versions of IntelliJ IDEA are largely automated, the default configuration generated by the IDE may require manual adjustments to accommodate the most recent updates to the JDK and JavaFX. Before executing the application, it is necessary to modify the Project Object Model (`pom.xml`) file located in the project's root directory.

First, the JavaFX dependency versions must be explicitly updated to ensure compatibility. Within the `pom.xml` file, locate the `<dependencies>` section and modify the `<version>` elements for both the `javafx-controls` and `javafx-fxml` artifacts to `26.0.1`. Furthermore, the build system must be instructed to utilize the newly installed JDK. Navigate to the `maven-compiler-plugin` within the `<plugins>` section of the file and update its configuration so that the `<source>` and `<target>` elements are both explicitly set to `24` to match the environment setup. These required modifications are illustrated in Script A.1:

After applying these modifications, ensure that the Maven changes are reloaded by the IDE (often prompted via a floating action button or accessible through the Maven tool window). Once the configuration is synchronized, executing the application becomes straightforward. The `HelloApplication` class can be executed by selecting the *Run* option from its context menu within the IDE. This action will compile and run the project, launching an example graphical user interface (GUI) application as shown in Figure A.2.

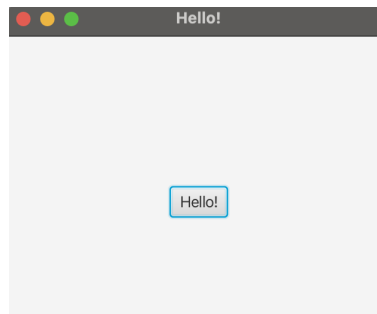


Figure A.2: Default GUI of the Generated JavaFX Application

Script A.1: pom.xml

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.
5         apache.org/xsd/maven-4.0.0.xsd">
6      <modelVersion>4.0.0</modelVersion>
7      ...
17     <dependencies>
18         <dependency>
19             <groupId>org.openjfx</groupId>
20             <artifactId>javafx-controls</artifactId>
21             <version>21.0.6</version>
+ 22             <version>26.0.1</version>
23         </dependency>
24         <dependency>
25             <groupId>org.openjfx</groupId>
26             <artifactId>javafx-fxml</artifactId>
27             <version>21.0.6</version>
+ 28             <version>26.0.1</version>
29         </dependency>
30     ...
43     <build>
44         <plugins>
45             <plugin>
46                 <groupId>org.apache.maven.plugins</groupId>
47                 <artifactId>maven-compiler-plugin</artifactId>
48                 <version>3.13.0</version>
49                 <configuration>
50                     <source>8</source>
+ 51                     <source>24</source>
52                     <target>8</target>
+ 53                     <target>24</target>
54                 </configuration>
55             ...

```


Appendix B

THIS chapter outlines the creation of the Currency Exchange Application Programming Interface (API) developed for the case study in Chapter 2. This API application is constructed by integrating several services from Amazon Web Services (AWS, 2026). Specifically, the API is composed of two distinct applications: the Exchange Rate Processor and the API Endpoint.

Exchange Rate Processor

The primary function of this module is to fetch and prepare historical currency data from external sources. The architecture consists of a scheduled AWS Lambda function coupled with Amazon S3 for storage. The architecture diagram, presented in Figure B.1, comprises four main components: Amazon EventBridge, AWS Lambda, the internet, and an AWS S3 bucket.

Amazon EventBridge is configured with a rule that triggers an AWS Lambda function on a schedule, such as daily, shortly after midnight Coordinated Universal Time (UTC). The AWS Lambda function serves as the core engine of this application, executing code that retrieves currency data from a data provider over the internet. In the past, developers could easily access various commercial finan-

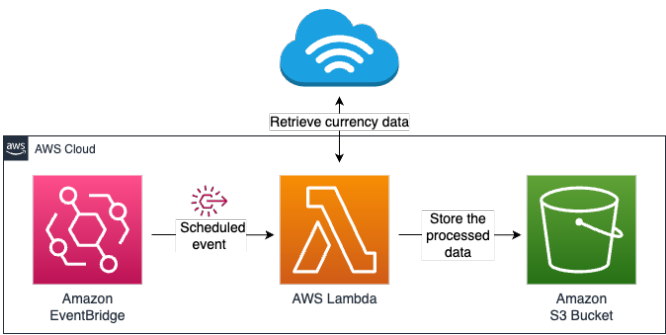


Figure B.1: Data Processing Architecture Diagram

cial APIs to fetch historical and live currency exchange rates. However, in recent years, these commercial data providers have become increasingly expensive, often imposing high monthly subscription fees and strict limitations on the number of requests allowed. This makes relying on commercial APIs highly impractical for an academic project like that of this coursebook.

To overcome this limitation, the Bank of Thailand (BOT) API (Bank of Thailand, 2026) serves as an excellent alternative. Although the BOT API provides daily average exchange rates rather than real-time data, this frequency is more than adequate to demonstrate the system architecture and serve the educational purposes of this book. It should be noted that while the BOT provides this data for public accessibility, users must agree to their terms of service, which state that the BOT does not guarantee the complete accuracy or timeliness of the data, and users assume all responsibility for its use in their applications.

To utilize the BOT API, registration on the portal is required. Once registered, developers must create a new application. For example, an application named SE233Exchange was created for this case study. This would generate an authentication key (BOT_DATA_API_KEY) that will be later used in the source code to access this API. The application must then be subscribed to the Exchange Rates Plan. While this plan offers an unlimited quota, it strictly enforces a rate limit of 200 calls per hour. Additionally, the API restricts data retrieval to a maximum of 31 days per single request.

Once the exchange data are retrieved, the Lambda function selects and refor-

mats this data into a structure suitable for the use case. This processed data acts as a cache for the API Endpoint application, which prevents end-users from directly interacting with the BOT API and minimizes unnecessary processing of irrelevant information. After preprocessing, the data is stored as a binary file in an Amazon S3 bucket. While numerous storage options exist, ranging from flat files to specialized low-latency databases, S3 was chosen for its simplicity and cost-effectiveness. With this architecture, the application can serve query responses in approximately 300 milliseconds, a timeframe deemed sufficiently fast for this educational context. A more detailed discussion on storage choices is provided later in this chapter.

The BOT API's Average Exchange Rate endpoint provides daily historical data for a wide range of currencies, inherently using the Thai Baht (THB) as the base currency. Although the API supports many more options, this case study intentionally limits the retrieval to a specific subset of 13 major foreign currencies (AUD, CAD, CNY, EUR, GBP, HKD, IDR, INR, JPY, MYR, PHP, SGD, and USD). Because the 31-day data limit requires multiple queries per currency to compile a multi-month history, restricting the list of currencies minimizes the total number of requests. This ensures the application stays well below the strict rate limit of 200 calls per hour. The data refreshes at the end of every business day. When queried, the API returns a JavaScript Object Notation (JSON) structure containing a nested data_detail array, with each daily record formatted similarly to the following example:

```
[
  {
    "period": "2024-01-04",
    "currency_id": "USD",
    "currency_name_th": "สหรัฐอเมริกา",
    "currency_name_eng": "USA",
    "buying_sight": 34.12,
    "buying_transfer": 34.25,
    "selling": 34.50,
    "mid_rate": 34.375
  },
  {
    "period": "2024-01-05",
    "currency_id": "USD",
    "currency_name_th": "สหรัฐอเมริกา",
    "currency_name_eng": "USA",
```

```
    "buying_sight":34.15,  
    "buying_transfer":34.28,  
    "selling":34.55,  
    "mid_rate":34.415  
  }  
]
```

This JSON response contains information that is not strictly required for the application's use case. For instance, the case study in Chapter 2 only requires a single daily value. While the API provides distinct buying and selling rates, the `mid_rate` serves as a sensible and balanced choice for this purpose.

Furthermore, as mentioned earlier, the BOT API imposes a strict limitation: each API request can only fetch a maximum of 31 days of historical data. Because the case study requires data spanning over several months, a single API call is insufficient. The application must dynamically break the required timeframe into smaller, 31-day chunks and execute multiple sequential queries to compile the full dataset.

Since the source data requires substantial pruning and multiple queries to construct, it is highly beneficial to store the processed result in a format optimized for its intended use. After benchmarking various data formats available in Python, the Apache Arrow Inter-Process Communication (IPC) format was selected due to its efficiency in both storage size and data access speed. The Arrow IPC format is known for its efficient serialization and is designed for the high-performance transfer of data between processes or systems. This facilitates effective data sharing in data-intensive applications. For further details on Apache IPC, refer to the official documentation (Apache Arrow, 2026).

Therefore, the Lambda function's role is to execute shortly after midnight, determine the necessary date ranges, execute multiple queries to the BOT API to retrieve the nested JSON data, extract the `mid_rate` for each date, convert the data into the cached Arrow IPC format, and store the resulting IPC file in an Amazon S3 bucket for universal access.

To create a Lambda function, navigate to <https://aws.amazon.com/compute/lambda> and select "Sign in to the console". If an account does not already exist, registration is required. Within the AWS Management Console, locate the Lambda service. On the Lambda page, select "Create function". Figure B.2 outlines the ba-

Create function [Info](#)

Choose one of the following options to create your function.

☒ **Author from scratch**
Start with a simple Hello World example.

☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name
Enter a name that describes the purpose of your function.

Use only letters, numbers, hyphens, or underscores with no spaces.

Runtime [Info](#)
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.
 [Refresh](#)

Architecture [Info](#)
Choose the instruction set architecture you want for your function code.
☐ x86_64
☒ arm64

Figure B.2: AWS Lambda Function Creation

sic setup configuration. Specify the function name, runtime, and architecture. The architecture selection depends on the development environment’s hardware; for instance, arm64 is chosen for systems using Apple Silicon, while x86_64 is appropriate for most other cases. After entering this information, click “Create function”.

The main page for the Lambda function will then be displayed. The code editor under the `lambda_function` tab is where the function’s logic is implemented, with various configurable options available in the menus. To test the function, click the “Test” button, name the test case, click “Save”, and then click “Test” again. The execution results will appear, including the Test Event Name, Response, Function Logs, and Request ID. Particular attention should be paid to the Response and Function Logs. In the Response, verify that the status code is 200 and that the body matches the expected output. In the Function Logs, the Billed Duration and Memory Size are important metrics, as these factors determine the execution cost and responsiveness of the function.

Scripts [B.1](#) and [B.2](#) provides the complete program for this purpose. The script contains methods such as `put_ipc_to_s3`, `get_bot_data`, and `lambda_handler`. The `put_ipc_to_s3` method handles data storage in Amazon S3. It takes a Po-

lars DataFrame (to_write_object) as input, serializes it into the efficient Apache Arrow IPC format, and uploads it to the designated S3 bucket (se233) with the specified filename and .ipc extension.

The aws_access_key_id and aws_secret_access_key can be configured and retrieved from the AWS Identity and Access Management (IAM) service page. Under “Access management,” navigate to “Users.” From there, locate the “Access keys” menu under the “Security credentials” tab, as illustrated in Figure B.3, and create an access key for programmatic access to AWS services. Once created, the key information should be set as environment variables in the Lambda function’s “Configuration” tab, as shown in Figure B.4. This allows the key to be accessed within the Lambda code using the os.environ variable, as demonstrated in Line 8 of the script.

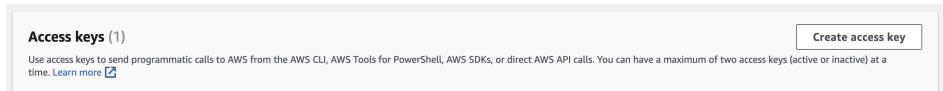


Figure B.3: Creating an Access Key for S3

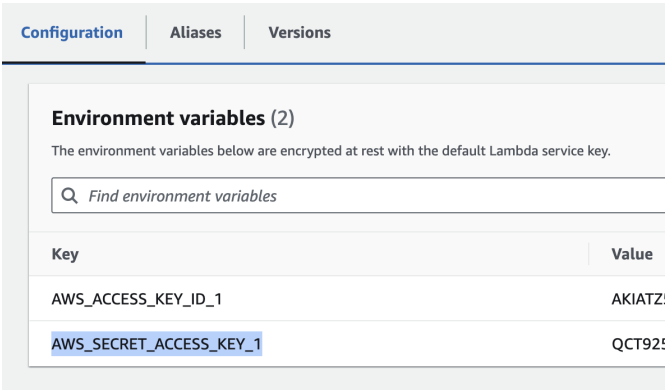


Figure B.4: Binding Access Keys with Environment Variables

Script B.1: Python Lambda Function for Retrieving and Caching BOT Currency Data (Part 1)

```

+ 1 import boto3
+ 2 import requests
+ 3 import polars as pl
+ 4 import datetime
+ 5 import os
+ 6 from io import StringIO, BytesIO
+ 7
+ 8 BOT_DATA_API_URL = 'https://gateway.api.bot.or.th/Stat-ExchangeRate/v2/
    DAILY_AVG_EXG_RATE/'
+ 9 BOT_DATA_API_KEY = 'Your_BOT_API_KEY_Here'
+ 10
+ 11 def put_ipc_to_s3(to_write_object, filename):
+ 12     session = get_aws_session()
+ 13     to_write_object.write_ipc(buffer := BytesIO())
+ 14     message = session.resource('s3').Object('se233', f"{filename}.ipc").put(Body=
        buffer.getvalue())
+ 15     return message
+ 16
+ 17 def get_aws_session():
+ 18     session = boto3.Session(
+ 19         aws_access_key_id=os.environ['AWS_ACCESS_KEY_ID_1'],
+ 20         aws_secret_access_key=os.environ['AWS_SECRET_ACCESS_KEY_1']
+ 21     )
+ 22     return session
+ 23
+ 24 def get_support_symbols():
+ 25     return ['AUD', 'CAD', 'CNY', 'EUR', 'GBP', 'HKD', 'IDR', 'INR', 'JPY', 'MYR', '
        PHP', 'SGD', 'USD']
+ 26
+ 27 def get_bot_data(symbol):
+ 28     headers = {
+ 29         'Accept': 'application/json',
+ 30         'Authorization': BOT_DATA_API_KEY,
+ 31     }
+ 32     today = datetime.date.today() - datetime.timedelta(1)
+ 33     m1 = today - datetime.timedelta(31)
+ 34     m2 = m1 - datetime.timedelta(32)
+ 35     m3 = m2 - datetime.timedelta(32)
+ 36     periods = [
+ 37         {'start_period': m1.strftime('%Y-%m-%d'), 'end_period': today.strftime('%Y-%m
            -%d')},
+ 38         {'start_period': m2.strftime('%Y-%m-%d'), 'end_period': (m1 - datetime.
            timedelta(1)).strftime('%Y-%m-%d')},
+ 39         {'start_period': m3.strftime('%Y-%m-%d'), 'end_period': (m2 - datetime.
            timedelta(1)).strftime('%Y-%m-%d')}
+ 40     ]
+ 41     dfs = []

```

Script B.2: Python Lambda Function for Retrieving and Caching BOT Currency Data (Part 2)

```

1  #Imports are omitted
...
+ 41 for period in periods:
+ 42     querystring = {"start_period": period['start_period'], "end_period": period['
+ 43         end_period'], "currency": symbol}
+ 44     response = requests.get(BOT_DATA_API_URL, headers=headers, params=querystring
+ 45         )
+ 46     data = response.json()
+ 47     df = pl.DataFrame(data['result']['data']['data_detail']) \
+ 48         .select(
+ 49             pl.col('period').str.strptime(pl.Date, '%Y-%m-%d').alias('Date'),
+ 50             pl.col('mid_rate').cast(pl.Float64).alias(symbol)
+ 51         )
+ 52     dfs.append(df)
+ 53     df = pl.concat(dfs, how="vertical")
+ 54     return df
+ 55
+ 56 def lambda_handler(event, context):
+ 57     all_symbols = get_support_symbols()
+ 58     all_df = []
+ 59     for s in all_symbols:
+ 60         all_df.append(get_bot_data(s))
+ 61
+ 62     all_df_x = [df.reverse() for df in all_df]
+ 63     full_date = [all_df_x[0].select('Date').min().item() + datetime.timedelta(days=x)
+ 64         for x in range((datetime.date.today() - all_df_x[0].select('Date').min().
+ 65             item()).days + 1)]
+ 66     full_date = pl.DataFrame({'Date': full_date})
+ 67
+ 68     all_df_x = pl.concat(all_df_x + [full_date], how="align").select(pl.all().
+ 69         forward_fill()).drop_nulls()
+ 70     all_df_x = all_df_x.with_columns(pl.lit(1).alias('THB'))
+ 71
+ 72     all_df_x = all_df_x.with_columns(
+ 73         [(pl.col('USD') / pl.col(col)).alias(col) for col in all_df_x.columns if col
+ 74             not in ['Date']]
+ 75     )
+ 76     all_df_x = all_df_x.cast({pl.selectors.numeric(): pl.Float32})
+
+ 77     put_ipc_to_s3(all_df_x, 'cached_list')
+ 78     return {
+ 79         'statusCode': 200,
+ 80         'TITLE': put_ipc_to_s3(all_df_x.filter(pl.col('Date') >= pl.date(2024, 5, 1))
+ 81             , 'short_list')
+ 82     }

```

The `get_bot_data` method fetches currency exchange rate data for a specific symbol (e.g., USD) from the BOT API. Because the API restricts data fetching to a maximum of 31 days per request, this method dynamically calculates date boundaries and iterates through a list of periods. It makes a GET request to the provider's API endpoint, passing the `BOT_DATA_API_KEY` securely via headers. It then parses the JSON response into a Polars DataFrame, extracts the `mid_rate`, and builds a consolidated DataFrame.

The `lambda_handler` method is the main entry point executed by AWS Lambda, orchestrating the data retrieval and processing pipeline. It defines a list of currency symbols and fetches historical data for each symbol using the `get_bot_data` method. It then combines the resulting DataFrames, cleans the data by filling any missing dates (forward fill), and adds a baseline currency column (THB). Because the original data represents currencies in relation to THB, the script recalibrates the data using the USD column, allowing USD to act as the base currency for the final cached file. It then casts numeric columns to a more memory-efficient data type, `Float32`. Finally, it saves the processed data to S3 as `short_list.ipc` using the `put_ipc_to_s3` method and returns a success status code.

The next configuration step involves managing dependencies. Numpy, Pandas, and requests are the primary dependencies for this application. While some tutorials suggest using the `pip` command for installation, libraries like Numpy are closely tied to the system architecture. Therefore, it is recommended to build these dependencies manually. This can be done by downloading the appropriate wheel files directly from a repository like pypi.org (e.g., from <https://pypi.org/project/numpy/#files> for Numpy). Once all dependency files are collected, they should be placed in a directory named `python` and compressed into a zip file. For convenience, a pre-built layer file is available for download at <https://cmu.to/SE233/lambda-layer>.

After preparing the dependencies zip file, add it to the function as an AWS Lambda Layer. Click the “Add a layer” button at the bottom of the function's page, choose “Create a new layer”, and configure it as shown in Figure B.5. Note that the provided file is prepared for the arm64 architecture, so ensure the configuration matches Figure B.5. Once the layer is created, confirm its addition to the function by clicking “Add a layer” again and configuring it as shown in Figure B.6.

The final configuration step involves setting up the S3 bucket. Navigate to the Amazon S3 service page, click “Create bucket”, provide a unique name, uncheck “Block *all* public access”, and confirm by clicking “Create bucket”.

Layer configuration

Name
SE233-Dependencies

Description - *optional*
Description

☒ Upload a .zip file
☐ Upload a file from Amazon S3

Upload

python_w_np_pl_req.zip
43.19 MB

For files larger than 10 MB, consider uploading using Amazon S3.

Compatible architectures - *optional* [Info](#)
Choose the compatible instruction set architectures for your layer.

☐ x86_64
☒ arm64

Compatible runtimes - *optional* [Info](#)
Choose up to 15 runtimes.

Runtimes

Python 3.10

Figure B.5: Creating Layers for Lambda Dependencies

Additionally, by default, AWS Lambda functions have a timeout of 3 seconds. Since this data-fetching task can be time-consuming, the timeout should be increased. In the function’s general configuration, set the timeout to 2 minutes and the memory limit to 256 MB, as illustrated in Figure B.7. The application should now be ready. Return to the Lambda function page and click “Test”. Upon successful execution (indicated by a response code of 200), the `cached_list.ipc` file should be present in the configured S3 bucket. The bucket name can be changed (if not `se233` as in the example) on Line 14 of Script B.1.

Choose a layer

Layer source [Info](#)

Choose from layers with a compatible runtime and instruction set architecture or specify the Amazon Resource Name (ARN) of a layer version. You can also [create a new layer](#).

☐ **AWS layers**
Choose a layer from a list of layers provided by AWS.

☒ **Custom layers**
Choose a layer from a list of layers created by your AWS account or organization.

☐ **Specify an ARN**
Specify a layer by providing the ARN.

Custom layers

Layers created by your AWS account or organization that are compatible with your function's runtime.

SE233-Dependencies

▼

Version

1

▼

Figure B.6: Adding Layers to the Lambda Function

General configuration

AWS Region

Asia Pacific (Mumbai) ap-south-1

Bucket name [Info](#)

SE233

Bucket name must be unique within the global namespace and follow the bucket naming rules. [See rules for bucket naming](#)

Copy settings from existing bucket - optional

Only the bucket settings in the following configuration are copied.

Choose bucket

Format: s3://bucket/prefix

Object Ownership [Info](#)

Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.

☒ **ACLs disabled (recommended)**
All objects in this bucket are owned by this account. Access to this bucket and its objects is specified using only policies.

☐ **ACLs enabled**
Objects in this bucket can be owned by other AWS accounts. Access to this bucket and its objects can be specified using ACLs.

Object Ownership

Bucket owner enforced

Block Public Access settings for this bucket

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to this bucket and its objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to this bucket or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

☐ **Block all public access**
Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

☐ Block public access to bucket and objects created through console and the AWS CLI

Figure B.7: Lambda Function General Configuration

To finalize the application, the data processing function must be scheduled for automatic execution. The chosen service for this task is Amazon EventBridge. After navigating to the EventBridge service page, select “Schedules” under the “Scheduler” section and click “Create schedule”. Then, provide a name for the schedule (e.g., *SE233-Scheduler-RetrieveAndCache Currency*). For the schedule type, choose “Recurring schedule” with a “Cron-based” configuration. To execute the function daily at 10 minutes past midnight UTC, use the cron expression `10 7 * * ? *`, as depicted in Figure B.8. For the target details, select “AWS Lambda Invoke”, choose the Lambda function to execute (i.e., *SE233-RetrieveAndCacheCurrency*), and proceed through the remaining steps by clicking “Next” and then “Create schedule”.

Schedule pattern

Occurrence [Info](#)

You can define an one-time or recurrent schedule.

☐ One-time schedule

☒ Recurring schedule

Time zone

The time zone for the schedule.

(UTC+07:00) Asia/Bangkok

Schedule type

Choose the schedule type that best meets your needs.

☒ Cron-based schedule

☐ Rate-based schedule

A schedule set using a cron expression that runs at a specific time, such as 8:00 a.m. PST on the first Monday of every month.

A schedule that runs at a regular rate, such as every 10 minutes.

Cron expression [Info](#)

Define the cron expression for the schedule

Copy

Clear

cron (

10

Minutes

0

Hours

*

Day of month

*

Month

?

Day of the week

*

Year

)

Next 10 trigger dates

Date and time are displayed in your current time zone in UTC format, e.g. "Wed, Nov 9, 2022 09:00 (UTC - 08:00)" for Pacific time

Mon, 03 Jun 2024 00:10:00 (UTC+07:00)

Tue, 04 Jun 2024 00:10:00 (UTC+07:00)

Wed, 05 Jun 2024 00:10:00 (UTC+07:00)

Thu, 06 Jun 2024 00:10:00 (UTC+07:00)

Fri, 07 Jun 2024 00:10:00 (UTC+07:00)

Sat, 08 Jun 2024 00:10:00 (UTC+07:00)

Sun, 09 Jun 2024 00:10:00 (UTC+07:00)

Mon, 10 Jun 2024 00:10:00 (UTC+07:00)

Tue, 11 Jun 2024 00:10:00 (UTC+07:00)

Wed, 12 Jun 2024 00:10:00 (UTC+07:00)

Flexible time window

If you choose a flexible time window, Scheduler invokes your schedule within the time window you specify. For example, if you choose 15 minutes, your schedule runs within 15 minutes after the schedule start time.

5 minutes

Figure B.8: Configuring Amazon EventBridge

The new schedule should now appear with an *Enabled* status, indicating that the configuration is active. To test the schedule, one can either wait for the configured time or temporarily modify the cron expression for an earlier trigger. This completes the setup of the scheduled task to automatically retrieve and cache currency data using AWS Lambda and Amazon EventBridge.

API Endpoint

The second part of the application is the API endpoint, which provides responses to user requests. The architecture for this component is shown in Figure B.9. In this application, users interact with the system via API requests that are managed by the AWS API Gateway. This gateway is responsible for receiving input queries from users and triggering a second AWS Lambda function to process them. This Lambda function loads the `cached_currency.ipc` file from Amazon S3, selects the relevant information based on the user's query, transforms the data back into JSON format, and sends it through the API Gateway as the response.

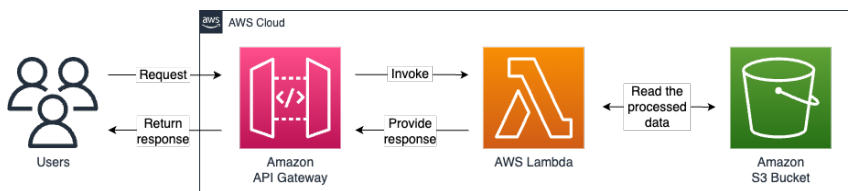


Figure B.9: API Endpoint Architecture Diagram

Scripts B.3 and B.4 provide the code for this second Lambda function. It consists of three methods: `get_ipc_from_s3`, `lambda_handler`, and `get_currency_history`. The `get_ipc_from_s3` method facilitates data retrieval from an S3 bucket. It constructs a temporary local file path, downloads the file (in Apache IPC format) from the designated S3 bucket (se233), reads the file into a Polars DataFrame, and returns this DataFrame for further processing.

Script B.3: Python Lambda Function for Providing Currency Exchange API

```
+ 1 import json
+ 2 import boto3
+ 3 import datetime
+ 4 import os
+ 5 import polars as pl
+ 6 import pytz
+ 7
+ 8 def get_ipc_from_s3(file_name):
+ 9     file_path = '/tmp/' + file_name + '.ipc'
+10     s3 = boto3.Session(
+11         aws_access_key_id=os.environ['AWS_ACCESS_KEY_ID_1'],
+12         aws_secret_access_key=os.environ['AWS_SECRET_ACCESS_KEY_1']
+13     ).client('s3').download_file('se233', file_name + '.ipc', file_path)
+14     df = pl.read_ipc(file_path)
+15     return df
+16
+17 def lambda_handler(event, context):
+18     try:
+19         base = event['queryStringParameters']['base']
+20         symbol = event['queryStringParameters']['symbol']
+21         start_date = event['queryStringParameters']['start_date']
+22         end_date = event['queryStringParameters']['end_date']
+23     except:
+24         base = event['base']
+25         symbol = event['symbol']
+26         start_date = event['start_date']
+27         end_date = event['end_date']
+28     res = get_currency_history(base, symbol, start_date, end_date)
+29     return {
+30         "statusCode": 200,
+31         "body": json.dumps(res)
+32     }
```

Script B.4: Python Lambda Function for Providing Currency Exchange API

```

1  import json
    ...
+ 32 def get_currency_history(base, symbol, start_date_str, end_date_str):
+ 33     start_date = datetime.datetime.strptime(start_date_str, "%Y-%m-%d").date()
+ 34     end_date = datetime.datetime.strptime(end_date_str, "%Y-%m-%d").date()
+ 35     sys_start_date = datetime.datetime.strptime('2022-01-01', "%Y-%m-%d").date()
+ 36     today = datetime.datetime.now(pytz.timezone('Asia/Bangkok')).date()
+ 37
+ 38
+ 39     if (end_date - start_date).days > 30:
+ 40         return {'message': 'You have exceeded the maximum day limit (30 days) for
+                                     historical currency conversion. Please request for a shorter time period.'}
+ 41     elif end_date <= start_date:
+ 42         return {'message': 'Invalid date range: End date must be later than the start
+                                     date.'}
+ 43     elif start_date < sys_start_date:
+ 44         return {'message': 'Invalid start date: The minimum start date is 2022-01-01.'}
+ 45     elif start_date > today or end_date > today:
+ 46         return {'message': 'Invalid date range: Start and end dates cannot exceed today
+                                     .' }
+ 47     all_df = get_ipc_from_s3('cached_currency')
+ 48     if base not in all_df.columns[1:]:
+ 49         return {'message': base + ' is not a supported currency.'}
+ 50     if symbol not in all_df.columns[1:]:
+ 51         return {'message': symbol + ' is not a supported currency.'}
+ 52     all_df = all_df.with_columns(pl.col("date").str.strptime(pl.Date, "%Y-%m-%d"))
+ 53     end_date = min(end_date, today)
+ 54     all_df = (
+ 55         all_df.filter((pl.col("date") >= start_date) & (pl.col("date") <= end_date))
+ 56         .with_columns(pl.col("date").dt.strftime("%Y-%m-%d"))
+ 57         .select(['date', base, symbol])
+ 58         .with_columns((pl.col(base) / pl.col(symbol)).alias("result"))
+ 59         .select('date', 'result')
+ 60     )
+ 61     return {
+ 62         'message': 'Success',
+ 63         'based': base,
+ 64         'symbol': symbol,
+ 65         'start_date': start_date_str,
+ 66         'end_date': end_date_str,
+ 67         'rates': dict(zip(all_df["date"].to_list(), all_df["result"].to_list()))
+ 68     }

```

The `lambda_handler` method serves as the main entry point for this AWS Lambda function. It parses the input event, extracting the base and target currency symbols along with the start and end dates. It then calls the `get_currency_history` function to retrieve the relevant exchange rate data. Finally, it returns a Hypertext Transfer Protocol (HTTP) 200 status code with a JSON body containing the result from `get_currency_history`. Note that the `try` and `except` block on Line 18 is designed to handle input formats from both a Lambda test event and the AWS API Gateway.

The `get_currency_history` method first validates the input parameters (base currency, target currency, and date range), ensuring they meet specific criteria such as a valid date range and supported currency symbols. It then loads the cached currency data from S3 using the `get_ipc_from_s3` method, filters the data based on the specified dates, and calculates the exchange rate between the base and target currencies for each date. The method returns a dictionary containing a success message, the base currency, the target currency, the date range, and a dictionary of exchange rates for each date within the specified range.

API details

☒ New API
Create a new REST API.

☐ Clone existing API
Create a copy of an API in this AWS account.

☐ Import API
Import an API from an OpenAPI definition.

☐ Example API
Learn about API Gateway with an example API.

API name

Description - optional

API endpoint type
Regional APIs are deployed in the current AWS Region. Edge-optimized APIs route requests to the nearest CloudFront Point of Presence. Private APIs are only accessible from VPCs.

Regional

Figure B.10: Creating a New API in AWS API Gateway

Configurations for dependencies, environment variables, timeout, and memory limits are required for this Lambda function, similar to the setup for the Exchange Rate Processor. Once configured, a test event can be created with a JSON string such as the one below to validate the function's behavior. "base": "THB", "symbol": "JPY", "start_date": "2024-01-01", "end_date": "2024-01-10"

The final component of the API is the configuration of the AWS API Gateway. To begin, navigate to "APIs" within the API Gateway service and select "Create API". On the subsequent page, choose "Build" under the "Representational State Transfer (REST) API" option. This will direct you to the API details page, as depicted in Figure B.10. Select "New API", name it (e.g., *SE233-REST_API*), and click "Create API".

On the API's configuration page, a specific resource path (e.g., *exchange*) should be created, as illustrated in Figure B.11. After creating the resource, select it from the main API page and click "Create method". For the method type, choose "GET" and select the designated Lambda function from the dropdown list. The required input query parameters (*base*, *symbol*, *start_date*, and *end_date*) should be added, as shown in Figure B.12. The Integration request type should be set to "Lambda Proxy integration" (Figure B.13).

Resource details

☒ **Proxy resource** [Info](#)
Proxy resources handle requests to all sub-resources. To create a proxy resource use a path parameter that ends with a plus sign, for example {proxy+}.

☐ **CORS (Cross Origin Resource Sharing)** [Info](#)
Create an OPTIONS method that allows all origins, all methods, and several common headers.

Resource path: / ▼

Resource name:

Figure B.11: Creating a New Resource Path in API Gateway

▼ URL query string parameters

Name	Required	Caching	
base	☒	☐	<button>Remove</button>
symbol	☒	☐	<button>Remove</button>
start_date	☒	☐	<button>Remove</button>
end_date	☒	☐	<button>Remove</button>
<button>Add query string</button>			

Figure B.12: Linking the API with the Created Lambda Function

Integration type

☒ Lambda function

Integrate your API with a Lambda function.



☐ HTTP

Integrate with an existing HTTP endpoint.



☐ Mock

Generate a response based on API Gateway mappings and transformations.



☐ AWS service

Integrate with an AWS Service.



☐ VPC link

Integrate with a resource that isn't accessible over the public internet.



☒ Lambda proxy integration

Send the request to your Lambda function as a structured event.

Figure B.13: Selecting Lambda Proxy Integration in API Gateway

After configuration, the API can be tested within the console using the query string: `base=THB&symbol=JPY&start_date=2024-01-01&end_date=2024-01-10`. A successful test will return a response with a status code of 200 and a response body in the expected format (Figure B.14). To finalize the setup, click “Deploy API”, select “New stage”, name it (e.g., *prod* for production), and click “Deploy”. This process generates an Invoke URL (e.g., <https://x9f9znds1.execute-api.ap-south-1.amazonaws.com/prod>). The deployed API can be tested by appending the query string to this Invoke URL in a web browser, and the resulting JSON response can be observed, as demonstrated in Figure B.15.

Test method

Make a test call to your method. When you make a test call, API Gateway skips authorization and directly invokes your method.

Query strings

base=THB&symbol=JPY&start_date=2024-01-01&end_date=2024-01-10

Headers

Enter a header name and value separated by a colon (:). Use a new line for each header.

header1:value1
header2:value2

Client certificate

No client certificates have been generated.

Test



/exchange - GET method test results

Request

/exchange?
base=THB&symbol=JPY&start_date=2024-01-01&end_date=2024-01-10

Status

200

Response body

{
 "message": "Success",
 "based": "THB",
 "symbol": "JPY",
 "start_date": "2024-01-01",
 "end_date": "2024-01-10",
 "rates": {
 "2024-01-01": 0.24410845339298248,
 "2024-01-02": 0.241363525390625,
 "2024-01-03": 0.24069228768348694,
 "2024-01-04": 0.23871257901191711,
 "2024-01-05": 0.239760845899592,
 "2024-01-06": 0.239785715937614,
 "2024-01-07": 0.242251962423325,
 "2024-01-08": 0.241659760475159,
 "2024-01-09": 0.240625753998756,
 "2024-01-10": 0.240625753998756
 }
}

Latency ms

1776

Figure B.14: Example Output of an API Test

←

→

↺

🔍 x9f9zndszl.execute-api.ap-south-1.amazonaws.com/prod/exchange?base=THB&symbol=JPY&start_date=2024-01-01&end_date=2024-01-10

Pretty-print

```
{  
  "message": "Success",  
  "based": "THB",  
  "symbol": "JPY",  
  "start_date": "2024-01-01",  
  "end_date": "2024-01-10",  
  "rates": {  
    "2024-01-01": 0.24410845339298248,  
    "2024-01-02": 0.241363525390625,  
    "2024-01-03": 0.24069228768348694,  
    "2024-01-04": 0.23871257901191711,  
    "2024-01-05": 0.239760845899592,  
    "2024-01-06": 0.239785715937614,  
    "2024-01-07": 0.242251962423325,  
    "2024-01-08": 0.241659760475159,  
    "2024-01-09": 0.240625753998756,  
    "2024-01-10": 0.240625753998756  
  }  
}
```

Figure B.15: Example Output in a Web Browser

This concludes the overview of the application developed for the Chapter 2 case study. While the setup may seem complex, the effort and resources required are significantly less than those for owning and configuring a physical server. Furthermore, the operational cost is remarkably low; for instance, millions of requests to a minimally configured Lambda function can be served for approximately one US dollar.

Selecting appropriate storage for an AWS Lambda function involves a trade-off between latency and cost. The primary options include:

- **Amazon S3:** Offers high availability with low latency (typically tens to hundreds of milliseconds), making it suitable for general-purpose storage. Its cost is moderate for storage, low for GET requests, and moderate for PUT requests and data transfer.
- **Amazon Glacier:** Optimized for long-term, archival storage. It features low storage costs, but retrieval times can range from minutes to hours. Retrieval and data transfer costs are moderate and vary based on retrieval speed.
- **Amazon DynamoDB:** A NoSQL database designed for single-digit millisecond latency, making it well-suited for applications requiring fast, consistent responses. Its storage cost is moderate, with data transfer costs also being moderate. Throughput pricing can be high for provisioned capacity but is moderate for on-demand mode.
- **Amazon ElastiCache:** Utilizes in-memory caching (e.g., Redis) to provide sub-millisecond latency, which is ideal for caching and real-time data processing. Costs are generally high for instances but low for data transfer within the same region.

Each storage option has distinct advantages: S3 for versatility, Glacier for cost-effective archiving, ElastiCache for low-latency caching, and DynamoDB for rapid, consistent access in high-transaction applications. Table B.1 summarizes these options. For this course, Amazon S3 was chosen to minimize the combined costs of storage, requests, and data transfer. With code optimizations, this choice satisfies the project's requirements with an acceptable latency (a few hundred milliseconds) for its educational purpose.

Service	Latency	Storage Cost	Request Cost	Data Transfer Cost
Amazon Glacier	Minutes to hours	Low	Medium (varied retrieval)	Medium
Amazon S3	Hundreds of ms	Medium	Low (GET), Medium (PUT)	Medium
DynamoDB	Single-digit ms	Medium	High (provisioned), Medium (on-demand)	Medium
Amazon ElastiCache	Sub-ms	High (by instance)	Included in instance cost	Low

Table B.1: Comparison of Storage Options for AWS Lambda

References

- Apache Arrow. (2026). Streaming, Serialization, and IPC. Retrieved May, 2026, from <https://arrow.apache.org/docs/python/ipc.html>
- AWS. (2026). Cloud Computing Services – Amazon Web Services. Retrieved May, 2026, from <https://aws.amazon.com/>
- Bank of Thailand. (2026). BOT API – Bank of Thailand. Retrieved May, 2026, from <https://portal.api.bot.or.th/>

Bibliography

- Apache ANT+Ivy. (2017). Apache Ivy. Retrieved May, 2026, from <https://ant.apache.org/ivy/>
- Apache Arrow. (2026). Streaming, Serialization, and IPC. Retrieved May, 2026, from <https://arrow.apache.org/docs/python/ipc.html>
- Apache Maven. (2026). Apache Maven. Retrieved May, 2026, from <https://maven.apache.org/>
- Arduino-er. (2015). Control Arduino + 8x8 LED matrix. Retrieved May, 2026, from <http://arduino-er.blogspot.com/2015/09/javajavafxjssc-control-arduino-8x8-led.html>
- Armstead, L. (2022). Exemplo De Spritesheet – Sprite Sheet Megaman Png, Transparent Png. Retrieved June, 2026, from https://www.kindpng.com/imgv/mwToi_exemplo-de-spritesheet-sprite-sheet-megaman-png-transparent/
- Austen-Smith, D., Diermeier, D., & Zemel, E. (2017). Unintended acceleration: Toyota's recall crisis. *Kellogg School of Management Cases*, 1–16.
- AWS. (2026). Cloud Computing Services – Amazon Web Services. Retrieved May, 2026, from <https://aws.amazon.com/>
- Bank of Thailand. (2026). BOT API – Bank of Thailand. Retrieved May, 2026, from <https://portal.api.bot.or.th/>
- Beck, K. (2003). *Test-driven development: By example*. Addison-Wesley Professional.
- Bloch, A. (2003). *Murphy's law*. Penguin.
- Docker. (2026). Docker: Accelerated Container Application Development. Retrieved May, 2026, from <https://www.docker.com/>
- Git. (2026). Git. Retrieved May, 2026, from <https://git-scm.com/>

- Gradle. (2026). Gradle Build Tool. Retrieved May, 2026, from <https://gradle.org/>
- Haase, C., & Guy, R. (2007). *Filthy rich clients: Developing animated and graphical effects for desktop Java applications*. Pearson Education.
- Java T Point. (2020). V-Model. Retrieved May, 2026, from <https://www.javatpoint.com/software-engineering-v-model>
- JUnit. (2026). JUnit5: The 5th major version of the programmer-friendly testing framework for Java and the JVM. Retrieved May, 2026, from <https://junit.org/junit5/>
- Kaner, C. (2009). Software testing as a quality improvement activity. *IEEE Computer Society Webinar Series*, 1–86.
- Le Lann, G. (1997). An analysis of the Ariane 5 flight 501 failure—A system engineering perspective. In *Proc. of Intl. Conf. and Workshop on Engineering of Computer-Based Systems* (pp. 339–346).
- Lin, H. (1985). The development of software for ballistic-missile defense. *Scientific American*, 253(6), 46–53.
- Log4j. (2026). Log4j – Apache Log4j 2. Retrieved May, 2026, from <https://logging.apache.org/log4j/>
- Martin, R. C. (2007). The three rules of TDD. Object Mentor.
- McIntosh, S., Adams, B., & Hassan, A. E. (2012). The evolution of Java build systems. *Empirical Software Engineering*, 17(4–5), 578–608.
- Mockito. (2026). Mockito framework site. Retrieved May, 2026, from <https://site.mockito.org/>
- Neumann, P. G. (1986). Risks to the public in computer systems. *ACM SIGSOFT Software Engineering Notes*, 11(2), 2–14.
- OpenJFX. (2026a). JavaFX. Retrieved May, 2026, from <https://openjfx.io>
- OpenJFX. (2026b). Overview (JavaFX 26). Retrieved May, 2026, from <https://openjfx.io/javadoc/26/index.html>
- Oracle. (2026a). FXCollections (JavaFX 26). Retrieved May, 2026, from <https://docs.oracle.com/en/java/java-components/javafx/26/docs/javafx.base/javafx/collections/FXCollections.html>
- Oracle. (2026b). Package java.awt. Retrieved May, 2026, from <https://docs.oracle.com/en/java/javase/26/docs/api/java.desktop/java/awt/package-summary.html>
- Oracle. (2026c). Package javax.swing. Retrieved May, 2026, from <https://docs.oracle.com/en/java/javase/26/docs/api/java.desktop/javafx/swing/package-summary.html>

- Oracle. (2026d). Exception (Java SE 26). Retrieved May, 2026, from <https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/lang/Exception.html>
- Oracle. (2026e). java.util.stream (Java SE 26). Retrieved May, 2026, from <https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/util/stream/package-summary.html>
- Perry, D. E., & Stieg, C. S. (1993). Software faults in evolving a large, real-time system: A case study. In *Proc. of the European Software Engineering Conference* (pp. 48–67).
- Phannachitta, P. (2020). On an optimal analogy-based software effort estimation. *Information and Software Technology*, 125, 106330.
- Phannachitta, P., & Matsumoto, K. (2019). Model-based software effort estimation—A robust comparison of 14 algorithms widely used in the data science community. *International Journal of Innovative Computing, Information and Control*, 15(2), 569–589.
- SLF4J. (2026). SLF4J. Retrieved May, 2026, from <http://www.slf4j.org/>
- Wong, S. (2017). Java GUI programming primer. Retrieved May, 2026, from <https://www.clear.rice.edu/comp310/JavaResources/GUI/>
- Zuo, C., Wen, H., Lin, Z., & Zhang, Y. (2019). Automatic fingerprinting of vulnerable IoT devices with static UUIDs from mobile apps. In *Proc. of the ACM SIGSAC Conf. on Computer and Communications Security* (pp. 1469–1483).

